

# R-node — Repository dump (Repomix)

72 file(s) — generated 2026-08-12T21:10:10.378Z

## Summary

This file is a merged representation of the entire codebase, combined into a single document by Repomix.

<file\_summary>

This section contains a summary of this file.

<purpose>

This file contains a packed representation of the entire repository's contents. It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

</purpose>

<file\_format>

The content is organized as follows:

1. This summary section
2. Repository information
3. Directory structure
4. Repository files (if enabled)
5. Multiple file entries, each consisting of:
  - File path as an attribute
  - Full contents of the file

</file\_format>

<usage\_guidelines>

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

</usage\_guidelines>

<notes>

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)

</notes>

</file\_summary>

<directory\_structure>

```
.claude/
  launch.json
dev/
  parity.html
  parity.ts
docs/
  archive/
    PIANO-IMMAGINI.md
    ADR-001-immagini-e-piattaforma.md
    AGENT_GUIDE.md
    ARCHITECTURE.md
    ELEMENTI_VISUALI.md
    RICH_TEXT_EDITOR.md
    ROADMAP.md
public/
  apple-touch-icon.png
  favicon-16.png
  favicon-32.png
  favicon-64.png
  favicon.svg
scripts/
  bundle_layout.cjs
src/
  core/
```

- doc.ts
- history.ts
- ops.ts
- text.ts
- tree.ts
- types.ts
- dev/
  - trace.ts
- editor/
  - context.ts
  - exportBridge.ts
  - externalImage.ts
  - imageImport.ts
  - imageImport.worker.ts
  - shortcuts.ts
  - store.ts
  - view.ts
- layout/
  - measure.ts
  - mindmap.ts
- persist/
  - assets.ts
  - storage.ts
- render/
  - renderer.ts
  - theme.ts
  - viewport.ts
- ui/
  - CanvasView.tsx
  - Inspector.tsx
  - lexicalRuns.ts
  - Outliner.tsx
  - Palette.tsx
  - pasteSanitizer.ts
  - RichEditor.tsx
  - Sidebar.tsx
  - TopBar.tsx
- App.tsx
- main.tsx
- styles.css
- src-tauri/
  - capabilities/
    - default.json
  - icons/
    - 128x128.png
    - 128x128@2x.png
    - 32x32.png
    - icon.icns
    - icon.ico
    - icon.png
- src/
  - lib.rs
  - main.rs
- build.rs
- Cargo.toml
- tauri.conf.json
- tests/
  - assets.test.ts
  - exportBridge.test.ts
  - externalImage.test.ts
  - imageImport.test.ts
  - layout.test.ts
  - lexicalRuns.test.ts
  - measure.test.ts
  - ops.test.ts
  - pasteSanitizer.test.ts
  - perf.test.ts
  - store.test.ts
  - tauriAdapter.test.ts
  - viewport.test.ts
- .gitignore
- CLAUDE.md
- index.html
- package.json
- README.md
- tsconfig.json
- vite.config.ts

```
</directory_structure>

<files>
This section contains the contents of the repository's files.
```

## Index

- docs/ELEMENTI\_VISUALI.md
- .claude/launch.json
- dev/parity.html
- dev/parity.ts
- docs/archive/PIANO-IMMAGINI.md
- public/favicon.svg
- scripts/bundle\_layout.cjs
- src/core/history.ts
- src/core/text.ts
- src/core/tree.ts
- src/editor/context.ts
- src/editor/externalImage.ts
- src/editor/imageImport.ts
- src/editor/imageImport.worker.ts
- src/editor/view.ts
- src/render/viewport.ts
- src/ui/pasteSanitizer.ts
- src-tauri/build.rs
- tests/externalImage.test.ts
- tests/imageImport.test.ts
- tests/layout.test.ts
- tests/pasteSanitizer.test.ts
- tests/perf.test.ts
- tests/viewport.test.ts
- CLAUDE.md
- tsconfig.json
- vite.config.ts
- src/core/doc.ts
- src/layout/mindmap.ts
- src/render/theme.ts
- src/ui/Outliner.tsx
- src-tauri/src/main.rs
- tests/exportBridge.test.ts
- tests/lexicalRuns.test.ts
- .gitignore
- docs/RICH\_TEXT\_EDITOR.md
- src/editor/exportBridge.ts
- src/ui/lexicalRuns.ts
- src/ui/Sidebar.tsx
- src/App.tsx
- src-tauri/capabilities/default.json
- tests/tauriAdapter.test.ts
- index.html
- src/core/ops.ts
- src/dev/trace.ts
- src/main.tsx
- tests/assets.test.ts
- tests/ops.test.ts
- docs/ARCHITECTURE.md
- src/layout/measure.ts
- src/persist/storage.ts
- src/ui/Palette.tsx
- src-tauri/Cargo.toml
- src-tauri/tauri.conf.json
- tests/measure.test.ts
- docs/AGENT\_GUIDE.md
- src/editor/shortcuts.ts
- src/ui/Inspector.tsx
- src/ui/TopBar.tsx
- package.json
- docs/ADR-001-immagini-e-piattaforma.md
- src/persist/assets.ts
- src-tauri/src/lib.rs
- docs/ROADMAP.md
- src/core/types.ts
- src/ui/RichEditor.tsx
- README.md
- src/styles.css
- tests/store.test.ts
- src/render/renderer.ts
- src/ui/CanvasView.tsx
- src/editor/store.ts

## 1. docs/ELEMENTI\_VISUALI.md

```
# R-node – Catalogo degli elementi visuali e della GUI
```

```
Foglio di riferimento di tutto ciò che è visibile e modificabile in R-node:
tema del canvas, design system della GUI, elementi per nodo, per foglio e per
testo, e asset di branding. I valori riportati sono quelli reali del codice
(`src/render/theme.ts`, `src/styles.css`, `src/core/types.ts`,
`src/ui/Inspector.tsx`).
```

```
> Fonti da aggiornare insieme a questo file: `src/render/theme.ts` (tokens del
> canvas), `src/styles.css` (tokens e componenti GUI), `src/core/types.ts`
> (modello `Style`, `TaskInfo`, `StructureConfig`, `TopicShape`, `TextRun`).
```

```
---
```

```
## 1. Logo e branding
```

```
| Asset | File | Note |
|---|---|---|
| Logo principale | `public/favicon.svg` | SVG 256x256 con PNG base64 incorporato (23 KB). Marchio rosso (`#e10100`) su sfondo trasparente:
mappa mentale stilizzata – due nodi collegati a un corpo arrotondato. È l'icona usata dal web. |
| Favicon | `public/favicon-16.png` / `favicon-32.png` / `favicon-64.png` | Rendering del logo a 16 / 32 / 64 px. |
| Apple touch | `public/apple-touch-icon.png` | Icona per home screen iOS. |
| Icone desktop (Tauri) | `src-tauri/icons/icon.png`, `icon.ico`, `icon.icns`, `32x32.png`, `128x128.png`, `128x128@2x.png` | Icone dell'installer
desktop (Windows/macOS/Linux). |
| Punto del brand (sidebar) | CSS .brand-dot | Cerchio 14 px, conic-gradient: indigo → rosa → verde → indigo (`#4f46e5` → #db2777 → #16a34a). |
```

```
Colori del brand (unica fonte informale): indigo #4f46e5 (accento,
selezione, pill), rosso logo #e10100, rosa #db2777, verde #16a34a.
Non esiste un logo file dedicato diverso da favicon.svg: il marchio è
condiviso tra web e desktop.
```

```
---
```

## ## 2. Tema del canvas – design tokens (`src/render/theme.ts`)

Un solo tema oggi: ```light``` (il tema scuro è stato rimosso di proposito; aggiungerlo è una palette nuova, non un refactoring). Ogni token vive in ``RenderTheme`` e governa il disegno del canvas:

| Token                            | Valore light                        | Cosa colora                                     |
|----------------------------------|-------------------------------------|-------------------------------------------------|
| <code>background`</code>         | <code>`#ffffff`</code>              | sfondo della mappa                              |
| <code>grid`</code>               | <code>`#f5f7fa`</code>              | griglia di sfondo                               |
| <code>text`</code>               | <code>`#111827`</code>              | testo dei nodi (default)                        |
| <code>textMuted`</code>          | <code>`#6b7280`</code>              | testi secondari                                 |
| <code>selection`</code>          | <code>`#4f46e5`</code>              | selezione, drop indicator, relazioni            |
| <code>selectionFill`</code>      | <code>`rgba(79,70,229,0.10)`</code> | riempimento selezione/marquee                   |
| <code>dropIndicator`</code>      | <code>`#4f46e5`</code>              | indicatore di drop durante il drag              |
| <code>rootFill`</code>           | <code>`#ffffff`</code>              | riempimento del nodo centrale                   |
| <code>rootText`</code>           | <code>`#111827`</code>              | testo del nodo centrale                         |
| <code>branch[]`</code>           | 8 colori (vedi sotto)               | palette dei rami principali, ruotata per indice |
| <code>branchSoft[]`</code>       | 8 colori pastello                   | riempimento dei discendenti di ciascun ramo     |
| <code>connector`</code>          | <code>`#9aa3b2`</code>              | connettori struttura                            |
| <code>collapsedBadge`</code>     | <code>`#e5e7eb`</code>              | cerchio "collapsed" sui rami chiusi             |
| <code>collapsedBadgeText`</code> | <code>`#374151`</code>              | numero di nodi nascosti nel badge               |
| <code>nodeBorder`</code>         | <code>`rgba(0,0,0,0.06)`</code>     | bordo dei nodi                                  |
| <code>shadow`</code>             | <code>`rgba(28,35,51,0.14)`</code>  | ombre                                           |

**\*\*Palette dei rami\*\*** (``branch``, ordine di rotazione):

```
---
#ff646b #ff9a66 #4eb5e8 #55c9bd #a7d9bb #d979e5 #70b9e8 #f0bd62
---
```

**\*\*Pastelli dei discendenti\*\*** (``branchSoft``, stesso ordine):

```
---
#ffdfef #ffe8dc #dff4ff #dff8f5 #e5f6ec #f6e1f9 #deeffb #fff0d3
---
```

Il nodo centrale usa ``rootFill`/`rootText``; i rami ``main`` prendono ``branch[i]``; i discendenti ``branchSoft[i]`` – finché lo stile del nodo non sovrascrive il riempimento (vedi §5).

---

## ## 3. Design system CSS – GUI (`src/styles.css`)

### ### 3.1 Variabili globali (`:root`)

| Variabile                   | Valore                                     | Uso                                       |
|-----------------------------|--------------------------------------------|-------------------------------------------|
| <code>--accent`</code>      | <code>`#4f46e5`</code>                     | indigo: azioni primarie, focus, selezione |
| <code>--accent-soft`</code> | <code>`rgba(79,70,229,0.14)`</code>        | fondi selezionati (attivo)                |
| <code>--radius`</code>      | <code>`8px`</code>                         | angoli grandi (menu, palette, toast)      |
| <code>--radius-sm`</code>   | <code>`6px`</code>                         | angoli piccoli (bottoni, input, righe)    |
| <code>--font`</code>        | <code>system-ui stack</code>               | tipografia GUI                            |
| <code>--mono`</code>        | <code>SF Mono / Cascadia / Consolas</code> | hint e testi mono (palette)               |

### ### 3.2 Variabili tema chiaro (`:root[data-theme="light"]`)

| Variabile                               | Valore                                         | Uso                           |
|-----------------------------------------|------------------------------------------------|-------------------------------|
| <code>--bg`</code>                      | <code>`#ffffff`</code>                         | sfondo app                    |
| <code>--surface`</code>                 | <code>`#ffffff`</code>                         | superfici (pannelli, barre)   |
| <code>--surface-2`</code>               | <code>`#f8f9fc`</code>                         | hover/righe                   |
| <code>--border`</code>                  | <code>`#e5e7eb`</code>                         | bordi e separatori            |
| <code>--text`</code>                    | <code>`#1c2333`</code>                         | testo principale              |
| <code>--text-muted`</code>              | <code>`#6b7280`</code>                         | etichette, hint               |
| <code>--accent` / --accent-soft`</code> | come sopra                                     | coerenza con il canvas        |
| <code>--danger`</code>                  | <code>`#dc2626`</code>                         | azioni distruttive            |
| <code>--shadow`</code>                  | <code>`0 10px 30px rgba(28,35,51,0.16)`</code> | ombre di menu/palette/overlay |

Il selettore ``[data-theme="light"]`` è il punto d'innesto per un tema scuro futuro: basta aggiungere ``:root[data-theme="dark"]``.

---

## ## 4. Elementi della GUI – inventario per componente

### ### 4.1 Sidebar (colonna sinistra, 264 px)

- **\*\*Brand\*\***: punto ``conic-gradient`` + nome "R-node".
- **\*\*Ricerca\*\*** documenti (``Ctrl/Cmd+F`` dal canvas porta qui).
- **\*\*Azioni\*\***: crea documento, apre file (``Open``).
- **\*\*Lista documenti\*\***: righe (``doc-row``) con titolo, hover mostra azioni (rinomina, duplica, archivia, elimina); riga attiva con fondo `accent-soft`.
- **\*\*Footer\*\***: metadati (conteggio, storage).

### ### 4.2 TopBar

- **\*\*Titolo documento\*\*** (input inline, rename on blur/Enter).
- **\*\*Gruppo storico\*\***: Undo `↶` / Redo `↷` (disabilitati se vuoti).
- **\*\*Save\*\*** (primario) + **\*\*Open\*\*** + stato salvataggio (`Saved` / `Unsaved` ambra quando sporco).
- **\*\*Gruppo zoom\*\***: `↻` / `%` (click = fit view) / `+`.
- **\*\*Pannelli\*\***: Outliner `≡`, Inspector `🔍`, Zen `@`, Palette `🎨`.
- **\*\*Trace\*\*** `●` (solo dev, compilato fuori dalle build di produzione).
- **\*\*Menu Export\*\***: JSON (`.rnode.json`), Markdown, PNG, `.rnode.zip` (completo / compatto, solo se ci sono immagini), voci placeholder SVG·PDF·DOCX.

### 4.3 Canvas

- **\*\*Canvas unico\*\*** a piena area, viewport culling.
- **\*\*Marquee\*\*** (selezione a riquadro): bordo 1.5px accent + fondo `color-mix(accent 12%)`, overlay DOM temporaneo.
- **\*\*Ghost del drag esterno\*\*** (immagini da Explorer/browser): anteprima 150x110 con bordo tratteggiato accent che segue il cursore.
- **\*\*Overlay di editing rich text\*\*** (uno solo): box posizionato in world unit, scala via `transform: scale(zoom)`, bordo 2px accent, riempimento e colore testo del nodo; **\*\*toolbar flottante\*\*** sopra (`top: -32px`) con B / I / U / colore / clear-color.
- **\*\*Pill relazione pendente\*\*** (`.rel-pending`): pillola accent in alto al centro ("Click target node...") mentre si crea un collegamento.

### 4.4 Inspector (colonna destra, 292 px)  
Sezioni mostrate a seconda della selezione:

- **\*\*Topic\*\***: titolo, azioni (+Child, +Sibling, ↑Promote, ↓Demote, 🔄Duplicate, Delete, 🔗Link..., 🗑️Group, (Summary, Collapse/Expand), checkbox \*Free positioning branch\* (solo nodi `main`).
- **\*\*Style\*\***: Fill, Text, Font size, Node width (slider + Auto), Image size (solo se c'è immagine), Bold, Shape (dropdown), Reset to branch color.
- **\*\*Task\*\***: Status, Priority, Progress (slider), Assignee.
- **\*\*Notes\*\***: textarea testo semplice (rich notes in Phase 4).
- **\*\*Sheet\*\***: titolo, Structure (9 tipi), Orientation, Level spacing, Branch spacing, 🔄 Auto layout, conteggio nodi visibili.
- **\*\*Relationship\*\*** (se selezionata): etichetta, bidirezionale, stile linea (solid/dashed/dotted), elimina.
- **\*\*Group\*\***: numero di membri, elimina.
- **\*\*Summary\*\***: etichetta, numero di membri, elimina.

### 4.5 Outliner (pannello inferiore, 38% di altezza)

- Header (etichetta), righe indentate: caret `⤵`/`⤴`, checkbox task, **\*\*pallino priorità\*\*** (8 px: verde `#a3e635`, ambra `#fbbf24`, arancio `#fb923c`, rosso `#f87171`), titolo editabile; riga selezionata con fondo accent-soft.

### 4.6 Palette comandi (Ctrl/Cmd+K)

- Overlay scuro (35%), pannello 520 px, input, lista con righe etichetta+hint mono, footer con scorciatoie.

### 4.7 StatusBar

- Hints contestuali (accent), zoom %, info selezione, separatori.

### 4.8 Toast

- Pillola centrata in basso (border-radius 999px, ombra), entrata animata (`.toast-in` 0.16s), cliccabile per chiudere.

### 4.9 Testo rich (dentro il canvas)

- Blocchi con **\*\*gap condiviso\*\*** (`.--rnode-block-gap`), liste senza marker UA: pallini disegnati `•` (livello 1), `◦` (2), `▪` (3) in colonna **\*\*`--rnode-bullet-w`**.

---

## ## 5. Elementi modificabili per NODO

### ### 5.1 Modello `Style` (`.src/core/types.ts`) – cosa esiste e cosa è esposto

| Campo                              | Valori / default                              | Nell'Inspector?          |
|------------------------------------|-----------------------------------------------|--------------------------|
| fill                               | hex; default undefined → palette ramo         | ✅ color picker           |
| textColor                          | hex; default → tema                           | ✅ color picker           |
| fontSize                           | 10-48; default 14                             | ✅ numero                 |
| fontWeight                         | 400 / 600 (bold)                              | ✅ checkbox Bold          |
| shape                              | 8 forme (vedi sotto); default rounded         | ✅ dropdown               |
| width                              | 90-640, default auto (max 640); text re-wraps | ✅ slider + Auto          |
| image + imageWidth                 | id asset; larghezza 48-640 (naturale ≤ 240)   | ✅ slider + Natural       |
| stroke                             | default transparent                           | 🚫 nel modello, non in UI |
| borderWidth                        | default 0                                     | 🚫                        |
| borderStyle                        | solid / dashed / dotted                       | 🚫                        |
| cornerRadius                       | default 10                                    | 🚫 (usato dalle forme)    |
| fontFamily                         | default stack di sistema                      | 🚫                        |
| italic / underline / strikethrough | booleani                                      | 🚫 (via editor di testo)  |
| opacity                            | default 1                                     | 🚫                        |
| shadow                             | booleano                                      | 🚫                        |
| icon                               | stringa (id icona)                            | 🚫 (non implementato)     |
| link                               | stringa URL                                   | 🚫 (Phase 4, roadmap)     |

```
| `padding` | default 10 | 🚫 |
| `align` | `left`/`center`; default center | 🚫 (solo testo) |
| `height` | numero | 🚫 |
| `rotation` | gradi | 🚫 |
| `locked` / `hidden` | booleani | 🚫 |

**Forme** (`TopicShape`): `rounded` (default) · `rect` · `capsule` ·
`circle` · `diamond` · `hexagon` · `underline` · `none`.
`cloud` esiste nel tipo ma **non** è né disegnato né nel dropdown (8 voci).

### 5.2 Task
| Campo | Valori |
|---|---|
| `status` | `not-started` · `in-progress` · `blocked` · `completed` · `cancelled` |
| `priority` | `none` · `low` · `medium` · `high` · `urgent` |
| `progress` | 0-100 (al 100% → completed) |
| `assignee` | testo |

Rendering sul canvas: completato → **ribbon**; priorità → pallino colorato
(high/urgent `#f87171`, medium `#fbbf24`, low `#a3e635`).

### 5.3 Altro
- **Notes**: testo semplice (rich notes in Phase 4).
- **Collapsed/expand**: badge con conteggio nascosti (`collapsedBadge`).
- **Posizione manuale**: `position.manual` (branch a posizionamento libero).
- **Immagine**: sopra il testo, proporzioni bloccate, livelli original /
  1024px / 256px, decodifica al livello giusto per lo zoom.

---

## 6. Elementi della MAPPA / foglio

| Elemento | Valori / default | Esposto? |
|---|---|---|
| `structureType` | `mindmap` (default) · `logic` · `tree` · `org` · `timeline` · `fishbone` · `matrix` · `treetable` · `freeform` | ✅ dropdown Sheet |
| `orientation` | `horizontal`/`vertical` (disabilitato per mindmap/freeform) | ✅ |
| `spacing` (livelli) | 80-400, default 180 | ✅ slider |
| `branchSpacing` (fratelli) | 4-80, default 14 | ✅ slider |
| `connectorStyle` | `curved` (default) · `straight` · `elbow` | 🚫 nel modello, non in UI |
| `padding` / `compactMode` / `autoBalance` | nel modello | 🚫 |
| `freePositioningBranches` | booleano | via checkbox per-nodo |
| `allowManualPositioning` | booleano | 🚫 |

**Relazioni** (frece indipendenti dall'albero): etichetta, `bidirectional`,
`lineStyle` (`dashed` default, `solid`, `dotted`), colore (default =
selezione), stato selezionato evidenziato. **Group** (confine): membri.
**Summary**: etichetta + intervallo di membri.

---

## 7. Elementi di TESTO (`TextRun[]`)

| Proprietà | Valori |
|---|---|
| `bold` / `italic` / `underline` / `strikethrough` | booleani |
| `color` | hex (default: colore tema/nodo) |
| `fontSize` | heading h1=26 ... h6=14 (px world), usato come `font-size` inline |
| `paraGap` | inizio nuovo blocco → gap verticale |
| `listIndent` | 1+ → item di lista (pallini `•` `` `•` `) |

Invariante: `node.title === runsToPlain(node.titleRuns)`.

---

## 8. Nota sulla personalizzazione

- **Tema**: solo `light` oggi; il tema scuro è un cambio di palette
  (`:root[data-theme="dark"]` + nuovo token in `THEMES`), non un refactoring.
- **Font**: stack di sistema, nessun font bundled.
- **Icone nodi** (`Style.icon`) e **link** (`Style.link`) sono nel modello ma
  non ancora implementati/visibili.
- Le voci GUI marcate 🚫 esistono nel modello dati ma non hanno controllo UI:
  sono i candidati naturali per l'Inspector in fasi successive.
```

## 2. .claude/launch.json

```
{
  "version": "0.0.1",
  "configurations": [
    {
      "name": "r-node",
      "runtimeExecutable": "npm",
      "runtimeArgs": ["run", "dev"],
      "port": 5173
    }
  ]
}
```

```
}
]
}
```

### 3. dev/parity.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>R-node – editor/canvas parity harness</title>
  </head>
  <body>
    <div id="parity-out"></div>
    <script type="module" src="./parity.ts"></script>
  </body>
</html>
```

### 4. dev/parity.ts

```
/**
 * Editor ↔ canvas parity harness (dev only – never imported by the app).
 *
 * The canvas reproduces the browser's text layout by hand (wrapRunLines +
 * renderTextBitmap). "Pixel for pixel" is therefore an *imitation*, and every
 * imitation drifts. This harness measures the drift instead of reasoning about
 * it:
 *
 * 1. it renders a corpus of TextRun[] cases into the REAL .topic-rich-editable
 *    DOM (same classes, same CSS, same padding maths as RichEditor);
 * 2. it extracts the browser's TRUE line boxes by walking every character with
 *    a Range and grouping the rects that overlap vertically;
 * 3. it runs wrapRunLines() on the same runs at the same wrap width;
 * 4. it diffs break points, line tops (cumulative vertical rhythm), line
 *    heights and line lefts.
 *
 * Results land on `window.__parity` as JSON and render as a table.
 * Run: npm run dev → http://localhost:5173/dev/parity.html
 *
 * The font family is forced to the canvas stack on both sides so the harness
 * isolates *layout* divergence; the fact that the live overlay inherits --font
 * instead is a separate (P0) defect, not something to drown these numbers in.
 */
import "../src/styles.css";
import {
  BLOCK_GAP_FACTOR,
  BULLET_WIDTH_EM,
  FONT_STACK,
  LINE_HEIGHT_FACTOR,
  MAX_TOPIC_W,
  TEXT_INSET,
  createCanvasTextMeasurer,
  measureTopic,
  wrapRunLines,
} from "../src/layout/measure";
import type { Style, TextRun } from "../src/core/types";

/** The one stack both sides use (measure.FONT_STACK === styles.css --font). */
const CANVAS_FONT = FONT_STACK;
const EPS = 0.5; // px tolerance below which a delta is not a divergence

// -----
// Corpus – one case per suspected divergence
// -----

interface Case {
  name: string;
  what: string;
  runs: TextRun[];
  style?: Partial<Style>;
  boxW?: number;
}

const LOREM =
  "Struttura condivisa fra editor e canvas con parole abbastanza lunghe da mandare a capo il paragrafo almeno tre volte";

const CASES: Case[] = [
  {
    name: "plain-short",
```

```

    what: "single line, no wrap – the trivial baseline",
    runs: [{ text: "Central topic" }],
  },
  {
    name: "plain-wrap",
    what: "long paragraph, several wraps – core wrap parity",
    runs: [{ text: LOREM }],
  },
  {
    name: "trailing-spaces",
    what: "wrap points landing on spaces – canvas counts the trailing space in line width, CSS hangs it",
    runs: [{ text: "alfa beta gamma delta epsilon zeta eta theta iota kappa lambda" }],
  },
  {
    name: "two-paragraphs",
    what: "paraGap block gap between two paragraphs",
    runs: [{ text: "Primo blocco di testo" }, { text: "Secondo blocco di testo", paraGap: true }],
  },
  {
    name: "heading-then-body",
    what: "h1 run (26px) + body – block gap uses em of the <p> (14px) but the canvas uses the line height (26px)",
    runs: [{ text: "Titolo grande", fontSize: 26 }, { text: "Corpo del testo normale", paraGap: true }],
  },
  {
    name: "body-then-heading",
    what: "body then h1 – the block gap is em of the FOLLOWING block, so the canvas must size it from the following line too",
    runs: [{ text: "Corpo del testo normale" }, { text: "Titolo grande", fontSize: 26, paraGap: true }],
  },
  {
    name: "mixed-size-line",
    what: "26px and 14px runs on the SAME line – per-inline-box half-leading vs one half-leading per line",
    runs: [{ text: "Grande ", fontSize: 26 }, { text: "e piccolo insieme" }],
  },
  {
    name: "small-run",
    what: "10px run on a 14px base – CSS keeps the 14px strut, the canvas does not",
    runs: [{ text: "minuscolo testo di prova", fontSize: 10 }],
  },
  {
    name: "bullets-flat",
    what: "three flat list items – bullet glyph width + inter-item gap",
    runs: [
      { text: "Primo elemento", listIndent: 1 },
      { text: "\n" },
      { text: "Secondo elemento", listIndent: 1 },
      { text: "\n" },
      { text: "Terzo elemento", listIndent: 1 },
      { text: "\n" },
    ],
  },
  {
    name: "bullets-wrapped",
    what: "list item long enough to wrap – hanging indent on continuation lines",
    runs: [{ text: LOREM, listIndent: 1 }, { text: "\n" }],
  },
  {
    name: "bullets-nested",
    what: "nested list – canvas indents via listGlyph spaces, CSS has padding-left: 0",
    runs: [
      { text: "Livello uno", listIndent: 1 },
      { text: "\n" },
      { text: "Livello due", listIndent: 2 },
      { text: "\n" },
      { text: "Ancora due", listIndent: 2 },
      { text: "\n" },
    ],
  },
  {
    name: "empty-paragraph",
    what: "blank line between paragraphs – the canvas collapses it, the editor shows it",
    runs: [{ text: "sopra" }, { text: "\n" }, { text: "\n" }, { text: "sotto" }],
  },
  {
    name: "long-word",
    what: "token wider than the box – the canvas bitmap clips it, the editor overflows",
    runs: [{ text: "https://example.com/un/percorso/molto/lungo/che/non/entra/nella/box" }],
  },
  {
    name: "bold-across-word",
    what: "emphasis split mid-word – the token must not break between runs",
    runs: [{ text: "para" }, { text: "grafo", bold: true }, { text: "misto con altre parole che vanno a capo" }],
  },
  {
    // Reproduces a real central topic (fontSize 22, padding 4, box 275) where
    // the canvas wrapped to one line MORE than the editor: token widths summed
    // one by one drift from the browser's shaping of the whole line.
    name: "real-topic-22px",
  },

```

```

what: "central-topic metrics – accumulated token widths vs whole-line shaping",
runs: [
  { text: "Titolo grande", fontSize: 26 },
  { text: "Corpo del testo che va a capo perche e abbastanza lungo", paraGap: true },
  { text: "Primo elemento", listIndent: 1, paraGap: true },
  { text: "\n" },
  { text: "Secondo elemento piu lungo che deve andare a capo da solo", listIndent: 1 },
  { text: "\n" },
  { text: "Annidato", listIndent: 2 },
  { text: "\n" },
],
style: { fontSize: 22, padding: 4 },
boxW: 275,
},
{
  name: "left-align-bullets",
  what: "align:left + bullets – hanging indent is only visible when left aligned",
  runs: [{ text: LOREM, listIndent: 1 }, { text: "\n" }],
  style: { align: "left" },
},
];

// -----
// runs → DOM (mirrors lexicalRuns.runsToParagraphNodes / buildList)
// -----

function groupRuns(runs: TextRun[]): TextRun[][] {
  const groups: TextRun[][] = [];
  for (const run of runs) {
    if (run.paraGap) groups.push([]);
    if (groups.length === 0) groups.push([]);
    groups[groups.length - 1].push({ ...run, paraGap: false });
  }
  if (groups.length === 0) groups.push([]);
  return groups;
}

/** One TextNode as Lexical renders it: <span> + theme classes + inline style. */
function runSpan(text: string, run: TextRun): HTMLElement {
  const el = document.createElement("span");
  const cls: string[] = [];
  if (run.bold) cls.push("rnode-text-bold");
  if (run.italic) cls.push("rnode-text-italic");
  if (run.underline) cls.push("rnode-text-underline");
  if (cls.length > 0) el.className = cls.join(" ");
  const styles: string[] = [];
  if (run.color) styles.push(`color: ${run.color}`);
  if (run.fontSize) styles.push(`font-size: ${run.fontSize}px`);
  if (styles.length > 0) el.setAttribute("style", styles.join("; "));
  el.textContent = text;
  return el;
}

/** Mirrors lexicalRuns.trimBlockNewline: a block's closing \n is not a <br>. */
function trimBlockNewline(runs: TextRun[]): TextRun[] {
  const out = runs.map((r) => ({ ...r }));
  while (out.length > 0) {
    const last = out[out.length - 1];
    if (last.text === "") {
      out.pop();
      continue;
    }
    if (last.text.endsWith("\n")) {
      last.text = last.text.slice(0, -1);
      if (last.text === "") out.pop();
    }
    break;
  }
  return out;
}

function inlineChildrenDom(runsIn: TextRun[]): Node[] {
  const out: Node[] = [];
  for (const run of trimBlockNewline(runsIn)) {
    const parts = run.text.split("\n");
    parts.forEach((seg, i) => {
      if (i > 0) out.push(document.createElement("br"));
      if (seg.length === 0) return;
      out.push(runSpan(seg, run));
    });
  }
  return out;
}

function buildParagraphDom(runs: TextRun[]): HTMLElement {
  const p = document.createElement("p");
  for (const child of inlineChildrenDom(runs)) p.appendChild(child);
}

```

```

    return p;
}

/** Mirrors lexicalRuns.buildList – including the <p> Lexical nests inside <li>. */
function buildListDom(group: TextRun[]): HTMLElement {
    const root = document.createElement("ul");
    const items: { indent: number; runs: TextRun[] }[] = [];
    let cur: { indent: number; runs: TextRun[] } | null = null;
    for (let i = 0; i < group.length; i++) {
        const run = group[i];
        if (run.listIndent !== undefined) {
            if (cur) items.push(cur);
            cur = { indent: run.listIndent, runs: [run] };
            continue;
        }
        if (!cur) continue;
        if (run.text === "\n" || run.text === "\n\n") {
            let j = i + 1;
            while (j < group.length && (group[j].text === "\n" || group[j].text.trim() === "")) j++;
            if (j < group.length && group[j].listIndent !== undefined) {
                items.push(cur);
                cur = null;
                continue;
            }
        }
        cur.runs.push(run);
    }
    if (cur) items.push(cur);

    const stack: { list: HTMLElement; indent: number }[] = [{ list: root, indent: 0 }];
    for (const item of items) {
        while (stack.length > 1 && stack[stack.length - 1].indent >= item.indent) stack.pop();
        const li = document.createElement("li");
        const clean = item.runs.map((r) => ({ ...r, listIndent: undefined, paraGap: undefined }));
        // inline children, never a nested <p> – mirrors lexicalRuns.buildList
        for (const child of inlineChildrenDom(clean)) li.appendChild(child);
        const childList = document.createElement("ul");
        li.appendChild(childList);
        stack[stack.length - 1].list.appendChild(li);
        stack.push({ list: childList, indent: item.indent });
    }
    stripEmptyLists(root);
    return root;
}

/** Mirrors lexicalRuns.stripEmptyNestedLists: the sub-lists hang off the ITEMS. */
function stripEmptyLists(list: HTMLElement): void {
    for (const item of Array.from(list.children)) {
        if (item.tagName !== "LI") continue;
        for (const child of Array.from(item.children)) {
            if (child.tagName !== "UL") continue;
            if (child.children.length === 0) child.remove();
            else stripEmptyLists(child as HTMLElement);
        }
    }
}

function runsToDom(runs: TextRun[]): HTMLElement[] {
    return groupRuns(runs).map((g) => (g.some((r) => r.listIndent !== undefined) ? buildListDom(g) : buildParagraphDom(g)));
}

// -----
// Browser line boxes: walk every character, group rects that overlap vertically
// -----

interface Line {
    text: string;
    top: number;
    /** absolute baseline y – the reference both sides are compared on */
    baseline: number;
    /**
     * Distance to the next line's top (line height + any block gap), NOT the
     * inline box height: a character's client rect is the glyph box
     * (ascent+descent = 19px at 14px), which is TALLER than the 17.5px line box
     * and would add a constant fake +1.5 to every case. The advance is what the
     * canvas actually accumulates, so it is what must match.
     */
    advance: number;
    left: number;
    width: number;
    /** canvas side only: a blank line, invisible to the browser walker */
    empty?: boolean;
}

function browserLines(editable: HTMLElement): Line[] {
    const walker = document.createTreeWalker(editable, NodeFilter.SHOW_TEXT);
    interface Ch {

```

```

ch: string;
top: number;
bottom: number;
left: number;
right: number;
/** the inline box's own font metrics – needed to recover the baseline */
size: number;
ascent: number;
}

const measurer = createCanvasTextMeasurer();
const fontCache = new Map<Element, { size: number; ascent: number }>();
const fontOf = (el: Element): { size: number; ascent: number } => {
  const hit = fontCache.get(el);
  if (hit) return hit;
  const cs = getComputedStyle(el);
  const size = parseFloat(cs.fontSize) || 14;
  const met = measurer.metrics?({
    fontSize: size,
    fontFamily: cs.fontFamily,
    fontWeight: parseInt(cs.fontWeight, 10) || 400,
    italic: cs.fontStyle === "italic",
  }) ?? { ascent: size * 0.8, descent: size * 0.2 };
  const out = { size, ascent: met.ascent };
  fontCache.set(el, out);
  return out;
};

const chars: Ch[] = [];
let node: Node | null;
while ((node = walker.nextNode())) {
  const data = (node as Text).data;
  const font = fontOf((node as Text).parentElement!);
  for (let i = 0; i < data.length; i++) {
    const range = document.createRange();
    range.setStart(node, i);
    range.setEnd(node, i + 1);
    const r = range.getBoundingClientRect();
    if (r.height === 0) continue; // collapsed whitespace at a wrap point
    chars.push({ ch: data[i], top: r.top, bottom: r.bottom, left: r.left, right: r.right, size: font.size, ascent: font.ascent });
  }
}

// Characters arrive in reading order. Vertical overlap is NOT a usable
// signal: an inline box (ascent+descent = 19px at 14px) is taller than the
// 17.5px line box, so consecutive lines overlap. The reliable signal in LTR
// text is the horizontal reset – x grows monotonically inside a line and
// jumps back left at every wrap, in centered text too (a line's first glyph
// always starts left of the previous line's last glyph).
const lines: { chars: Ch[]; top: number; bottom: number }[] = [];
let prev: Ch | null = null;
for (const c of chars) {
  const cur = lines[lines.length - 1];
  if (cur && prev && c.left + 0.5 >= prev.left) {
    cur.chars.push(c);
    cur.top = Math.min(cur.top, c.top);
    cur.bottom = Math.max(cur.bottom, c.bottom);
  } else {
    lines.push({ chars: [c], top: c.top, bottom: c.bottom });
  }
  prev = c;
}

const box = editable.getBoundingClientRect();
const cs = getComputedStyle(editable);
const padLeft = parseFloat(cs.paddingLeft) || 0;
const padBottom = parseFloat(cs.paddingBottom) || 0;
const originX = box.left + padLeft;
const contentBottom = box.bottom - padBottom;

return lines.map((l, i) => {
  // trailing/leading spaces do not count towards the visible extent
  let a = 0;
  let b = l.chars.length - 1;
  while (a <= b && /\s/.test(l.chars[a].ch)) a++;
  while (b >= a && /\s/.test(l.chars[b].ch)) b--;
  const vis = l.chars.slice(a, b + 1);
  const left = vis.length > 0 ? Math.min(...vis.map((c) => c.left)) : l.chars[0].left;
  const right = vis.length > 0 ? Math.max(...vis.map((c) => c.right)) : l.chars[0].right;
  // A character's client rect is its inline box: [baseline – ascent,
  // baseline + descent]. So the baseline is recoverable, and comparing
  // BASELINES is both exact and the thing that actually matters – glyph-box
  // tops sit at a different offset from the line box for every font size,
  // which showed as a phantom 1px on any line carrying a heading.
  const ref = l.chars.reduce((a, c) => (c.size > a.size ? c : a), l.chars[0]);
  const baseline = ref.top + ref.ascent;
  const next = lines[i + 1];
  const nextRef = next ? next.chars.reduce((a, c) => (c.size > a.size ? c : a), next.chars[0]) : null;

```

```

    return {
      text: l.chars.map((c) => c.ch).join(""),
      top: l.top,
      baseline,
      advance: nextRef ? nextRef.top + nextRef.ascent - baseline : contentBottom - l.top,
      left: left - originX,
      width: right - left,
    };
  });
}

// -----
// Canvas side: wrapRunLines + the cumulative geometry renderTextBitmap applies
// -----

function canvasLines(runs: TextRun[], maxW: number, style: Style): { lines: Line[]; totalH: number } {
  const measurer = createCanvasTextMeasurer();
  const base = style.fontSize ?? 14;
  const wrapped = wrapRunLines(runs, maxW, measurer, style);
  const out: Line[] = [];
  let y = 0;
  for (const l of wrapped) {
    const lh = l.height ?? base * LINE_HEIGHT_FACTOR;
    y += l.gapPx ?? 0;
    const text = l.segments.map((s) => s.text).join("");
    // mirrors renderer.drawText: list items are always left-aligned
    const indent = l.indent ?? 0;
    const isList = indent > 0 || !!l.bullet;
    const left = style.align === "left" || isList ? indent : (maxW - l.width) / 2;
    out.push({ text, top: y, baseline: y + (l.baseline ?? lh * 0.8), advance: lh, left, width: l.width, empty: l.segments.length === 0 });
    y += lh;
  }
  // A blank line has no characters, so the browser walker cannot see it – drop
  // it from the line-by-line comparison (totalH still covers it: if the canvas
  // and the editor disagreed about a blank line, the heights would diverge).
  const visible = out.filter((l) => !l.empty);
  // advance = distance to the next VISIBLE line's top, exactly like the
  // browser side (computing this before the filter compared a canvas advance
  // that stopped at the blank line against a browser advance that skipped it).
  for (let i = 0; i < visible.length - 1; i++) visible[i].advance = visible[i + 1].baseline - visible[i].baseline;
  return { lines: visible, totalH: y };
}

// -----
// Compare
// -----

/** Break-point comparison ignores the bullet glyph (a ::marker in the browser). */
function normText(s: string): string {
  return s.replace(/[^\s]/g, " ").replace(/\s+/g, " ").trim();
}

interface LineDiff {
  i: number;
  canvas: string;
  browser: string;
  sameText: boolean;
  dTop: number;
  dAdv: number;
  dLeft: number;
  dWidth: number;
}

interface CaseResult {
  name: string;
  what: string;
  boxW: number;
  wrapW: number;
  linesCanvas: number;
  linesBrowser: number;
  firstBreakMismatch: number | null;
  maxDTop: number;
  maxDLeft: number;
  maxDAdv: number;
  totalHCanvas: number;
  totalHBrowser: number;
  dTotalH: number;
  lines: LineDiff[];
  verdict: "OK" | "DRIFT" | "BREAKS";
}

function runCase(c: Case, host: HTMLElement): CaseResult {
  const style: Style = { fontSize: 14, padding: 10, align: "center", ...c.style } as Style;
  const boxW = c.boxW ?? 280;
  const pad = style.padding ?? 10;
  const wrapW = boxW - pad * 2 - TEXT_INSET;

```

```

// --- DOM exactly as RichEditor builds it (minus the transform: scale) ---
const inner = document.createElement("div");
inner.className = "topic-rich-inner";
Object.assign(inner.style, {
  width: `${boxW}px`,
  height: "auto",
  fontSize: `${style.fontSize ?? 14}px`,
  lineHeight: String(LINE_HEIGHT_FACTOR),
  textAlign: style.align === "left" ? "left" : "center",
  fontFamily: style.fontFamily ?? CANVAS_FONT,
});
inner.style.setProperty("--rnode-block-gap", `calc(${BLOCK_GAP_FACTOR} * ${LINE_HEIGHT_FACTOR}em)`);
inner.style.setProperty("--rnode-bullet-w", `${BULLET_WIDTH_EM}em`);

const editable = document.createElement("div");
editable.className = "topic-rich-editable";
Object.assign(editable.style, {
  paddingTop: `${pad + 2}px`,
  paddingBottom: `${pad + 2}px`,
  paddingLeft: `${pad - 2}px`,
  paddingRight: `${pad + TEXT_INSET - 2}px`,
  overflow: "visible",
});
for (const el of runsToDom(c.runs)) editable.appendChild(el);
inner.appendChild(editable);
host.appendChild(inner);

// --- measure both sides ---
const bLines = browserLines(editable);
const { lines: cLines, totalH: totalHCanvas } = canvasLines(c.runs, wrapW, style);
const eBox = editable.getBoundingClientRect();
const eCs = getComputedStyle(editable);
const totalHBrowser = eBox.height - (parseFloat(eCs.paddingTop) || 0) - (parseFloat(eCs.paddingBottom) || 0);

const b0 = bLines.length > 0 ? bLines[0].baseline : 0;
const c0 = cLines.length > 0 ? cLines[0].baseline : 0;
const n = Math.max(bLines.length, cLines.length);
const lines: LineDiff[] = [];
let firstBreakMismatch: number | null = null;
let maxDTop = 0;
let maxDLeft = 0;
let maxDAdv = 0;
for (let i = 0; i < n; i++) {
  const cl = cLines[i];
  const bl = bLines[i];
  const sameText = !!cl && !!bl && normText(cl.text) === normText(bl.text);
  if (!sameText && firstBreakMismatch === null) firstBreakMismatch = i;
  const last = i === n - 1;
  // baseline drift, both sides measured from their own first baseline
  const dTop = cl && bl ? cl.baseline - c0 - (bl.baseline - b0) : NaN;
  // the last "advance" is not an inter-line distance on either side
  const dAdv = cl && bl && !last ? cl.advance - bl.advance : NaN;
  const dLeft = cl && bl ? cl.left - bl.left : NaN;
  const dWidth = cl && bl ? cl.width - bl.width : NaN;
  if (Number.isFinite(dTop)) maxDTop = Math.max(maxDTop, Math.abs(dTop));
  if (Number.isFinite(dLeft)) maxDLeft = Math.max(maxDLeft, Math.abs(dLeft));
  if (Number.isFinite(dAdv)) maxDAdv = Math.max(maxDAdv, Math.abs(dAdv));
  lines.push({
    i,
    canvas: cl ? cl.text : "-",
    browser: bl ? bl.text : "-",
    sameText,
    dTop: round(dTop),
    dAdv: round(dAdv),
    dLeft: round(dLeft),
    dWidth: round(dWidth),
  });
}

const dTotalH = totalHCanvas - totalHBrowser;
const verdict =
  cLines.length !== bLines.length || firstBreakMismatch !== null
    ? "BREAKS"
    : maxDTop > EPS || maxDLeft > EPS || maxDAdv > EPS || Math.abs(dTotalH) > EPS
      ? "DRIFT"
      : "OK";

return {
  name: c.name,
  what: c.what,
  boxW,
  wrapW,
  linesCanvas: cLines.length,
  linesBrowser: bLines.length,
  firstBreakMismatch,
  maxDTop: round(maxDTop),
  maxDLeft: round(maxDLeft),

```

```

    maxDAdv: round(maxDAdv),
    totalHCanvas: round(totalHCanvas),
    totalHBrowser: round(totalHBrowser),
    dTotalH: round(dTotalH),
    lines,
    verdict,
  });
}

function round(n: number): number {
  return Number.isFinite(n) ? Math.round(n * 100) / 100 : NaN;
}

// -----
// Report
// -----

function render(results: CaseResult[]): void {
  const out = document.getElementById("parity-out")!;
  const bad = results.filter((r) => r.verdict !== "OK").length;
  const esc = (s: string): string => s.replace(/[<&]/g, (m) => (m === "<" ? "&lt;" : "&amp;"));
  out.innerHTML = `
<h1 style="font:600 18px system-ui;margin:0 0 4px">Editor + canvas parity</h1>
<p style="font:13px system-ui;margin:0 0 16px;color:#555">
  ${results.length} cases · <b>${bad}</b> diverging<b>·
  BREAKS = different line count or different break points · DRIFT = same breaks, geometry off by &gt; ${EPS}px
</p>
${results
  .map(
    (r) => `
<section style="margin:0 0 20px;border:1px solid #ddd;border-radius:6px;overflow:hidden">
  <header style="padding:6px 10px;background:${r.verdict === "OK" ? "#e8f5e9" : r.verdict === "DRIFT" ? "#fff8e1" : "#ffebee"};font:600 13px
system-ui">
    ${r.verdict} · ${r.name}
    <span style="font-weight:400;color:#666"> — ${esc(r.what)}</span>
  </header>
  <div style="padding:6px 10px;font:12px system-ui;color:#444">
    lines canvas/browser: <b>${r.linesCanvas}/${r.linesBrowser}</b> ·
    height canvas/browser: <b>${r.totalHCanvas}/${r.totalHBrowser}</b> (Δ ${r.dTotalH}) ·
    maxΔtop <b>${r.maxDTop}</b> · maxΔleft <b>${r.maxDLeft}</b> · maxΔadv <b>${r.maxDAdv}</b>
  </div>
  <table style="width:100%;border-collapse:collapse;font:11px/1.4 ui-monospace,monospace">
    <tr style="background:#fafafa"><th>#</th><th style="text-align:left">canvas</th><th style="text-align:left">browser</th><th>Δtop</th>
<th>Δadv</th><th>Δleft</th><th>Δw</th></tr>
    <tr>
      <th>${r.lines}</th>
      <td>
        <table>
          <tr>
            <td>
              <tr style="background:${l.sameText ? "transparent" : "#ffebee"}">
                <td style="text-align:center;color:#999">${l.i}</td>
                <td>${esc(l.canvas.slice(0, 46))}</td>
                <td>${esc(l.browser.slice(0, 46))}</td>
                <td style="text-align:right;${Math.abs(l.dTop) > EPS ? "color:#c62828;font-weight:700" : ""}">${l.dTop}</td>
                <td style="text-align:right;${Math.abs(l.dAdv) > EPS ? "color:#c62828;font-weight:700" : ""}">${l.dAdv}</td>
                <td style="text-align:right;${Math.abs(l.dLeft) > EPS ? "color:#c62828;font-weight:700" : ""}">${l.dLeft}</td>
                <td style="text-align:right;color:#999">${l.dWidth}</td>
            </tr>
          </tr>
        </table>
      </td>
    </tr>
  </table>
  </section>`
  ).join("")}`;
}

const stage = document.createElement("div");
// off-screen but really laid out (display:none would kill getClientRects)
Object.assign(stage.style, { position: "absolute", left: "-10000px", top: "0", visibility: "hidden" });
document.body.appendChild(stage);

const results = CASES.map((c) => runCase(c, stage));
render(results);
(window as unknown as { __parity: CaseResult[] }).__parity = results;
// debug handle: lets the console interrogate the measurement side directly
(window as unknown as { __measure: unknown }).__measure = {
  wrapRunLines,
  measureTopic,
  MAX_TOPIC_W,
  TEXT_INSET,
  measurer: createCanvasTextMeasurer(),
};
// eslint-disable-next-line no-console
console.log(["parity"], JSON.stringify(results.map(({ lines, ...r }) => r)));

```

## 5. docs/archive/PIANO-IMMAGINI.md

```
# Piano di esecuzione – immagini nei nodi

> **ARCHIVIATO (2026-08-12)** – piano completamente eseguito (passi 0–15,
> task T12a–T20). Non descrive più lavoro da fare; resta come record storico.
> Le decisioni sono registrate in [ADR-001](../ADR-001-immagini-e-piattaforma.md)
> §12, e i risultati delle misure in ADR-001 §12 e qui §5.

Sequenza completa, dall'archivio degli asset fino al desktop e alla
condivisione. Pensata per essere consegnata **una volta** a un esecutore che
la percorre da solo.

Le specifiche di dettaglio stanno in [ROADMAP.md](../ROADMAP.md); qui c'è
**l'ordine, i cancelli e le condizioni di arresto**. Le decisioni che stanno
sotto tutto sono in [ADR-001](../ADR-001-immagini-e-piattaforma.md) §12.

---

## 1. Protocollo di esecuzione – leggere prima di iniziare

Questa è la parte che rende sicura una sequenza lunga. Senza, un errore al
passo 2 arriva mascherato al passo 6.

### Regole

1. **Un passo alla volta, nell'ordine della tabella.** Non anticipare, non
accorpare, non "già che ci sono".
2. **Un commit per passo.** Messaggio in inglese all'imperativo che spiega il
`perché`. Serve a poter isolare un guasto con `git bisect`.
3. **Dopo ogni passo, esegui il suo cancello.** Se non passa: **fermati**,
non passare al successivo, e scrivi cosa non torna.
4. **Non disattivare test, non aggiungere `skip`, non alzare soglie** per far
passare un cancello. Un cancello che fallisce è informazione, non un
ostacolo da aggirare.
5. **Non modificare file fuori dalla lista del passo.** Se credi che serva,
fermati e spiega perché.

### I cancelli

| Sigla | Cosa eseguire | Passa se |
| --- | --- | --- |
| **G-tsc** | `npm run typecheck` | 0 errori |
| **G-test** | `npm test` | tutti verdi, nessuno saltato |
| **G-par** | `npm run dev`, poi `http://localhost:5173/dev/parity.html` | riporta **0 diverging / 16** |
| **G-mem** | vedi passo 11 | budget rispettato |

**G-par è obbligatorio** per ogni passo che tocca uno di questi file:
`src/layout/measure.ts`, la parte testo di `src/render/renderer.ts`,
`src/ui/lexicalRuns.ts`, le regole CSS `.topic-rich-*`,
`src/ui/RichEditor.tsx`.

`npm test` **non** copre la parità: l'harness gira in un browser vero perché
jsdom non implementa il layout. Vedi `AGENT_GUIDE.md` §3.

### Fermati e chiedi se

- un cancello fallisce e non capisci perché entro un tentativo ragionevole;
- serve una dipendenza non elencata nel passo;
- serve cambiare una costante condivisa (`LINE_HEIGHT_FACTOR`,
`BLOCK_GAP_FACTOR`, `BULLET_WIDTH_EM`, `FONT_STACK`, `TEXT_INSET`);
- serve toccare `wrapRunLines`;
- la parità non torna a 0 dopo un tuo cambiamento;
- un passo marcato **UMANO** nella tabella.

---

## 2. La sequenza

| # | Passo | Dipende | Chi | Cancelli |
| --- | --- | --- | --- | --- |
| 0 | **T0** – bring-up di Tauri | – | **UMANO** | app si apre, documento sopravvive a riapertura |
| 1 | **T12a** – archivio asset (IndexedDB) | – | ☒ **fatto** (`01f29bd`) | G-tsc, G-test |
| 2 | **T12b** – correzioni a T12a | 1 | agente | G-tsc, G-test |
| 3 | **T12-1** – modello e misura | 2 | agente | G-tsc, G-test, **G-par** |
| 4 | **T12-2** – disegno sul canvas | 3 | agente | G-tsc, G-test, **G-par** |
| 4b | **T12-2b** – decodificare alla dimensione resa | 4 | agente | G-tsc, G-test, **G-par** |
| 5 | **T12-3** – spazio riservato nell'overlay | 4 | agente | G-tsc, G-test, **G-par** |
| 6 | **T12-4** – op `setNodeImage` e undo | 5 | agente | G-tsc, G-test |
| 7 | **T13-1** – pipeline di import in un Worker | 6 | agente | G-tsc, G-test |
| 8 | **T13-2** – drop sul nodo e paste | 7 | agente | G-tsc, G-test |
| 9 | **T14** – slider e maniglia di ridimensionamento | 8 | agente | G-tsc, G-test, **G-par** |
| 10 | **T16** – `TauriAssetStore` e comandi Rust | 0, 8 | agente | G-tsc, G-test, app desktop funzionante |
| 11 | **T17** – misura della memoria e confronto | 10 | agente | G-mem |
| 12 | **T15** – `rnode.zip` | 10 | agente | G-tsc, G-test |
```

I passi 3-6 sono la scomposizione di **T12**, che in ROADMAP.md è scritto come un task unico: tocca sei aree e tre file governati dalla parità, quindi va eseguito a pezzi con un cancello ciascuno.

---

## 3. I passi non ancora specificati altrove

Per 2, 7 (parziale), 8, 9, 12 vale quanto scritto in ROADMAP.md. Qui i pezzi che lì non ci sono.

### Passo 3 – T12-1: modello e misura

```
/*Fa*/: `Style.imageWidth?: number`; costanti `MAX_IMAGE_W = 240` e  
`IMAGE_GAP = 6`; `measureTopic` accetta  
`resolveImage?: (id) => { w: number; h: number } | null`; helper  
`imageResolver(sheet)`; geometria come in ROADMAP T12 passo 2.
```

**/\*Non fa\*/: nessun disegno, nessuna cache, nessuna modifica all'overlay.**

**/\*Il punto su cui si rompe\*/: `measureTopic` è puro e non può leggere  
`sheet.attachments`. Il resolver va passato in **tutti** e **tre** i punti che  
misurano – `layoutSheet`/`applyLayout`, il renderer, `overlayMeasurer` in  
CanvasView. Se anche uno solo non lo passa, quel chiamante misura i nodi senza  
immagine, il layout li piazza a una dimensione e il renderer ne disegna  
un'altra. È l'invariante I9. `imageResolver(sheet)` esiste perché sia  
costruito in un punto solo.**

```
/*File*/: `src/core/types.ts`, `src/layout/measure.ts`, `src/layout/mindmap.ts`,  
`src/render/renderer.ts`, `src/ui/CanvasView.tsx`, `tests/measure.test.ts`.
```

### Passo 4 – T12-2: disegno sul canvas

**/\*Fa\*/: l'immagine disegnata **fra** la forma e il testo, orizzontalmente  
centrata, nel rettangolo calcolato al passo 3.**

Cache delle bitmap secondo ADR-001 §12:

- chiave `\${assetId}@\${livello}`, dove il livello segue lo zoom corrente con lo stesso schema a bucket già usato per le bitmap del testo in  
`renderer.ts` (`Math.ceil(curScale \* dpr)`, limitato);
- decodifica con `createImageBitmap(blob, { resizeWidth })` – fuori dal thread principale e senza materializzare la piena risoluzione;
- **solo** per i nodi visibili (il renderer culla già);
- **budget** in byte, non in numero: `w \* h \* 4` per bitmap, tetto 128MB, sfratto LRU finché si rientra;
- allo sfratto **bitmap.close()**: la memoria di un `ImageBitmap` non sta nello heap JS e il GC non la libera in modo prevedibile;
- **tetto** duro: mai decodificare sopra 1024px. È la pixelatura accettata in ADR-001 §12, resa regola;
- decodifiche in volo limitate (4-6), e se una finisce per un nodo non più visibile va chiusa subito invece di entrare in cache;
- al termine di una decodifica **richiama** un repaint (callback passata al Renderer da CanvasView).

**/\*Trappole\*/: niente `await` dentro il disegno, il renderer resta sincrono.**

Non sfrattare **durante** il disegno di un frame – una bitmap chiusa lancia: sfratta a fine frame. Non toccare mai il livello `original`.

**/\*Aggiungi al tracer\*/ (`src/dev/trace.ts`) i contatori per frame:  
immagini visibili, bitmap in cache, byte occupati, decodifiche in volo. Servono  
al passo 11 e alla regola §4bis di AGENT\_GUIDE.**

```
/*File*/: `src/render/renderer.ts`, `src/ui/CanvasView.tsx`,  
`src/dev/trace.ts`.
```

### Passo 4b – T12-2b: decodificare alla dimensione resa

Correzione al passo 4. Lì `createImageBitmap(blob)` viene chiamato **senza** `resizeWidth`, quindi la bitmap si materializza alla dimensione di archiviazione invece che a quella che serve.

Il tetto a 1024px regge lo stesso – lo impone il livello memorizzato – quindi non c'è rischio di esaurire la memoria. Ma il costo sì: su schermo retina (`dpr = 2`) la soglia `res <= 1` manda su `large` già sopra zoom 0.5, cioè bitmap da 1024x768 → **3MB** l'una. A zoom 0.8 con 50 nodi con immagine visibili sono 150MB: sopra il budget, quindi sfratto e ri-decodifica continui.

**Va chiuso prima del passo 11, altrimenti quella misura registra numeri gonfiati da una riga mancante e sembrerebbe che a non reggere sia la decisione di ADR-001 §12.**

**/\*Il cambiamento\*/: in `renderer.ts`:**

```
``ts  
// pixel fisici realmente occupati DA QUESTA immagine, non lo zoom globale  
const neededPx = imgW * this.curScale * this.dpr;
```

```
// quantizza a potenze di due: senza, ogni micro-variazione di zoom
// produrrebbe una chiave nuova e la cache non colpirebbe mai
const bucket = clamp(pow2ceil(neededPx), 128, 1024);
const level: AssetLevel = bucket <= 256 ? "small" : "large";
const key = `${imageId}@${bucket}`;
...
const bitmap = await createImageBitmap(blob, { resizeWidth: bucket, resizeQuality: "high" });
...
```

Quattro punti da non sbagliare:

1. **La chiave deve contenere il bucket**, non il livello. Con dimensioni di decodifica variabili e chiave per livello, una bitmap decodificata a 384px verrebbe riusata quando ne servono 900 e l'immagine resterebbe sfocata per sempre.
2. **Mai ingrandire**: ``bucket`` non supera 1024, che è la larghezza del livello ``large`` memorizzato. Decodificare più grande della sorgente spreca memoria senza aggiungere qualità.
3. **Passa solo ``resizeWidth``**: l'altezza viene calcolata mantenendo le proporzioni. Passarle entrambe rischia di deformare.
4. Il conteggio dei byte (``bitmap.width * bitmap.height * 4``) e lo sfratto restano com'erano: si adeguano da soli.

**Effetto atteso**: a zoom 0.8 su retina, ``neededPx = 240 × 0.8 × 2 = 384`` → bucket 512 → ~786KB invece di 3MB. **Circa quattro volte meno**, e le cifre tornano a coincidere con la tabella di ADR-001 §12.

**Mentre ci sei**, ``imageResolver(store.sheet)`` viene ricostruito a ogni render in ``CanvasView.tsx`` (riga ~634): memoizzalo su ``store.sheet``.

**File**: ``src/render/renderer.ts``, ``src/ui/CanvasView.tsx``.

**Cancelli**: G-tsc, G-test, **G-par**.

### Passo 5 – T12-3: spazio riservato nell'overlay

**Fa**: mentre si edita, ``RichEditor`` mostra sopra l'editabile un blocco non editabile (``contentEditable={false}``) alto ``imgH`` e largo ``imgW``, con lo stesso ``IMAGE_GAP`` sotto.

**Perché**: senza, la box salta al doppio click. È la classe di bug che la sessione di parità ha chiuso, e G-par la intercetta.

**File**: ``src/ui/RichEditor.tsx``, eventualmente ``src/styles.css``.

### Passo 6 – T12-4: l'operazione

**Fa**: op ``setNodeImage { nodeId, imageId: string | null, prevImageId }`` con inverso, più la scheda ``AttachmentInfo`` aggiunta a ``sheet.attachments``.

**Regola**: l'op porta **solo l'id**, mai i byte. La history tiene 400 voci.

**File**: ``src/core/ops.ts``, ``src/editor/store.ts``, ``tests/ops.test.ts``.

### Passo 7 – T13-1: pipeline di import in un Worker

Come ROADMAP T13 passo 1, **ma dentro un Web Worker**: lettura del file, SHA-256 e generazione dei tre livelli (originale intatto, 1024px, 256px).

``AssetStore.put`` legge l'originale intero in un ``ArrayBuffer`` per l'hash: con originali ad alta risoluzione sono decine di MB copiati sul thread principale a ogni import, e l'interfaccia si blocca. Venti immagini in un colpo passano da fastidiose a inaccettabili.

Nel worker si usa ``OffscreenCanvas`` per il ridimensionamento.

**File**: ``src/editor/imageImport.ts``, ``src/editor/imageImport.worker.ts``, ``tests/imageImport.test.ts``.

### Passo 10 – T16: ``TauriAssetStore`` e comandi Rust

**Perché**: ADR-001 §12 sceglie B1. Centinaia di originali insostituibili non possono vivere in IndexedDB, che Safari cancella dopo ~7 giorni di inattività e Chrome può sfrattare. Su desktop diventano file veri.

**Rust** (``src-tauri/src/lib.rs``), accanto ai quattro comandi documenti già presenti:

```
``rust
put_asset(id: String, level: String, bytes: Vec) -> Result<(), String>
get_asset(id: String, level: String) -> Result<Vec

```

I file vanno nella cartella dati dell'app, come ``assets/<id>/<level>``. I metadati possono stare in SQLite accanto ai documenti.

```
**TypeScript**: `TauriAssetStore implements AssetStore` in  
`src/persist/assets.ts` (o un file accanto), selezionato a runtime quando  
l'ambiente Tauri è presente, con `IndexedDbAssetStore` come ripiego per il web.
```

```
**Non fare**: non rimuovere `IndexedDbAssetStore`. Serve al target web e ai  
test.
```

```
**File**: `src-tauri/src/lib.rs`, `src/persist/assets.ts`,  
`src/persist/storage.ts`.
```

```
### Passo 11 – T17: misura, invece di fidarsi
```

```
**Fa**: verifica sul campo i numeri su cui poggia ADR-001 §12.
```

1. Genera una mappa con **\*\*300 immagini\*\*** attaccate ai nodi.
2. Con il tracer (bottone  o `Ctrl+Shift+D`), cattura in quattro regimi:  
vista d'insieme, zoom medio, zoom ravvicinato, zoom massimo.
3. Per ciascuno annota: immagini visibili, bitmap in cache, **\*\*byte occupati\*\***,  
ms per frame.

```
**G-mem passa se**: in ogni regime i byte delle bitmap restano **sotto  
128MB**, e il frame time resta comparabile a quello senza immagini.
```

```
**Se non passa**: fermati e riporta i numeri. ADR-001 §12 contiene la  
condizione che riaprirebbe la decisione (il caso "moodboard"), e questi sono i  
dati che servono per valutarla.
```

```
---
```

```
## 4. Al termine
```

Alla fine dei dodici passi, la sequenza va rivista da capo: un commit per  
passo rende possibile leggerla come storia. I punti da guardare per primi sono  
i cancelli **\*\*G-par\*\*** dei passi 3, 4, 5 e 9 – se la parità è tornata a 0 senza  
che nessuno abbia toccato `wrapRunLines`, il grosso è andato bene.

```
---
```

```
## 5. Risultati del passo 11 (T17) – misurato il 2026-08-12
```

```
Mappa reale: **344 nodi, 300 immagini** da 1600×1200 attaccate, viewport  
1280×720, **devicePixelRatio = 1**.
```

```
### Caso mappa normale (fit)
```

| Regime   Immagini visibili   Bitmap in cache   RAM bitmap   ms/frame |
|----------------------------------------------------------------------|
| --- --- --- --- ---                                                  |
| fit (zoom 0.4)   22   54   <b>**5,6 MB**</b>   2,5                   |

```
### Caso moodboard – 300 immagini impacchettate in griglia
```

È la **\*\*condizione di riapertura\*\*** dichiarata in ADR-001 §12: molte immagini  
grandi tutte visibili insieme. Forzata spostando i 300 nodi con immagine in una  
griglia fitta.

| Zoom   Immagini visibili   Bitmap in cache   RAM bitmap   ms/frame        |
|---------------------------------------------------------------------------|
| --- --- --- --- ---                                                       |
| 0,25   <b>**300**</b>   66   6,2 MB   37,2 (burst iniziale di decodifica) |
| 0,5   90   306   <b>**17,4 MB**</b>   1,9                                 |
| 1,0   30   306   17,4 MB   1,6                                            |

```
### Verdetto: **G-mem passa**
```

- Picco misurato **\*\*17,4 MB\*\*** contro un budget di 128 MB: **\*\*7 volte di margine\*\***,  
e nel caso peggiore previsto dall'ADR, non in quello favorevole.
- Media **\*\*~57 KB** per bitmap\*\*. Senza la correzione del passo 4b sarebbero stati  
~3 MB l'una: le stesse 306 bitmap avrebbero fatto **\*\*~900 MB\*\***, cioè sfratto  
continuo. La correzione vale un fattore ~50 su questo carico.
- Il frame time resta **\*\*1,6–1,9 ms\*\*** a regime. Il picco di 37 ms si presenta una  
volta sola, quando 300 immagini entrano insieme nel viewport e la coda di  
decodifica è satura; si riassorbe da solo.
- **\*\*La condizione di riapertura di ADR-001 §12 non scatta.\*\*** Il caso moodboard  
è stato provato e regge.

```
### Avvertenza sulla misura
```

`devicePixelRatio` era **\*\*1\*\***. Su schermo retina i bucket salgono di un passo e i  
byte crescono di circa 4x: il picco stimato diventa **\*\*~70 MB\*\***, ancora sotto il  
budget ma con margine 1,8x invece di 7x. **\*\*Da rimisurare su un monitor a dpr 2\*\***  
prima di considerare chiusa la questione.

```
---
```

```
## 6. Passo 13 – T18: chiudere i difetti emersi dalla revisione
```

Tre punti trovati rivedendo i passi 10 e 12. **\*\*A** va chiuso prima che qualcuno

salvi lavoro vero.\*\*

```
**File**: `src/editor/store.ts` · `src/editor/exportBridge.ts` ·  
`src/core/types.ts` · `src/persist/assets.ts` · `src/render/renderer.ts` ·  
`src/ui/RichEditor.tsx` · i rispettivi test.
```

### A – il salvataggio può scrivere uno zip dentro un `.rnode.json`

`writePortableBytes` riusa la maniglia di file memorizzata e la chiave è **solo** il `docId`. Quindi: salvi come `mappa.rnode.json`, alleggi un'immagine, premi Ctrl+S, e `hasImages` fa scrivere **byte** di zip dentro il file `.json`, in silenzio, sulla stessa maniglia. Vale anche al contrario.

R-node lo riapre lo stesso (sniffa i magic bytes `PK\x03\x04` – quella parte è giusta e va tenuta), ma il file mente al sistema operativo, alle cartelle sincronizzate e a chiunque lo riceva.

- [ ] la chiave della maniglia diventa `\${docId}:\${format}` con `format = "json" | "zip"`, **sia** nella mappa in memoria **sia** in IndexedDB;
- [ ] estrai la derivazione della chiave in una funzione pura e testala;
- [ ] quando per il formato richiesto non c'è maniglia, si apre il selettore (comportamento già esistente) **con** un toast che spiega perché, es. «Il documento ora contiene immagini: scegli dove salvare il `.rnode.zip`»;
- [ ] non cancellare la vecchia maniglia dell'altro formato: se le immagini vengono rimosse, il salvataggio torna al file originale.

### B – dopo un round-trip compact l'originale è perso e nulla lo dice

Un export `compact` non porta gli originali: all'import il livello 1024px diventa anche `original`. È corretto e voluto. Il problema è che **niente** lo segna, quindi un successivo export `complete` produce un file che dichiara di contenere gli originali e contiene ridimensionamenti.

- [ ] `AttachmentInfo.originalLost?: boolean`, impostato all'import di un container `compact`;
- [ ] `buildRnodeZip(..., "complete")` rileva gli asset con quel flag: lo registra nel `manifest.json` (es. degraded: true) e avvisa l'utente prima di generare;`
- [ ] l'avviso deve dire quanti asset sono coinvolti, non solo che ce ne sono.

### C – l'invariante dell'indirizzamento per contenuto ora ha un'eccezione

`putUnderId` è necessario (un container compact non porta l'originale, quindi l'id non può essere ri-derivato) ma rompe `id === sha256(original)`.

- [ ] scrivilo accanto a `putUnderId` in `assets.ts`;
- [ ] aggiungilo agli invarianti in `AGENT_GUIDE.md` §2: l'id è lo SHA-256 dell'originale **tranne** per gli asset ripristinati da un container compact, che portano `originalLost`.

### D – la factory si fissa troppo presto

`getAssetStore()` è chiamato **a** livello di modulo in `renderer.ts:48` e `RichEditor.tsx:166`. Il singleton si fissa alla prima chiamata: se l'inizializzazione dei moduli girasse prima che Tauri inietti `window.__TAURI__`, resterebbe su IndexedDB per sempre. Empiricamente funziona, ma dipende da un ordine che non controlliamo.

- [ ] chiama `getAssetStore()` **al** momento dell'uso, non allo scope di modulo (nel `Renderer` basta spostarlo nel costruttore).

### Fatto quando

- Un test sulla derivazione della chiave: stesso `docId` e formati diversi → chiavi diverse.
- Un test: importare un container compact marca gli asset con `originalLost`; costruirne uno `complete` da un documento che li contiene lo registra nel manifest.
- Nessuna chiamata a `getAssetStore()` allo scope di modulo.
- G-tsc, G-test verdi.

---

## 7. Passo 14 – T19: il documento è una cartella (solo desktop)

**Obiettivo.** Su desktop un documento diventa una cartella che contiene tutto:

```
...  
MiaMappa.rnode/  
├─ document.json  
└─ assets/<id>/ {original, large, small, meta}  
...
```

**Perché.** Oggi il desktop è a metà: le immagini stanno su file veri (passo 10) ma il **documento** resta nel localStorage della webview – `TauriStorageAdapter` è uno stub e nessuno lo istanzia. Conseguenze: il cap da

~5MB di localStorage vale ancora su desktop, e se quello storage viene ripulito restano file immagine orfani sul disco senza il documento che li referenziava.

**\*\*Perché una cartella e non uno zip.\*\*** Uno zip va riscritto per intero a ogni salvataggio: con centinaia di originali sono GB ricompresi a ogni Ctrl+S. È la struttura di ``.xmind`` senza il suo costo di salvataggio – qui ``.document.json`` si riscrive da solo, poche centinaia di KB, e le immagini si scrivono una volta all'import e non si toccano più.

``.rnode.zip`` **\*\*resta\*\*** il formato di scambio (passo 12). Cartella mentre lavori, file singolo quando mandi.

**\*\*File\*\*:** ``.src-tauri/src/lib.rs`` · ``.src/persist/assets.ts`` · ``.src/persist/storage.ts`` · ``.src/editor/store.ts`` · ``.src/main.tsx``

### Passi

1. **\*\*Rust\*\*:** i comandi asset prendono una **\*\*radice esplicita\*\*** invece di usare la cartella dati dell'app: ``.put_asset(root, id, level, bytes)`` e simili. Senza stato nascosto: una radice sbagliata è un bug visibile nella chiamata, non uno stato desincronizzato. Mantieni la guardia anti path-traversal.
2. **\*\*Rust\*\*:** ``.pick_document_folder()`` → ``.Option<String>`` con un dialogo nativo (es. ``.crate `rfd``), così non serve una dipendenza npm – coerente con come è stato fatto il passo 10.
3. **\*\*TauriAssetStore\*\*:** tiene la radice corrente come **\*\*stato mutabile interno\*\***, non catturata alla costruzione.

> **\*\*Trappola.\*\*** Il ``.Renderer`` cattura il suo ``.assetStore`` nel costruttore.  
> Se aprire un altro documento creasse un'istanza nuova, il renderer > continuerebbe a leggere dalla vecchia e mostrerebbe le immagini del > documento precedente. L'istanza deve restare la stessa e cambiare radice.

4. **\*\*TauriStorageAdapter\*\*:** legge e scrive ``.document.json`` sotto la radice corrente. Niente più localStorage sul desktop.
5. **\*\*main.tsx\*\*:** costruisce ``.EditorStore`` con ``.TauriStorageAdapter`` quando ``.window.__TAURI__`` è presente, ``.LocalStorageAdapter`` altrimenti – stesso schema della factory degli asset.
6. **\*\*Apri / Salva con nome\*\*:** selettore cartella → imposta la radice → scrive ``.document.json`` **\*\*e garantisce che ogni asset referenziato esista sotto ``.<root>/assets``, copiandolo dallo store corrente se manca.**

> Senza il passo 6b il documento salvato referenzia id che nella sua > cartella non ci sono, e riaprendolo le immagini mancano. È la stessa > logica di ``.buildRnodeZip``, che scrive in una cartella invece che in un > archivio: riusala, non riscriverla.

7. **\*\*saveNow\*\*** su desktop scrive **\*\*solo\*\*** ``.document.json``.

### Fuori scope, di proposito

- **\*\*Non toccare la sidebar / la lista documenti.\*\*** Con il documento-cartella la "libreria" diventa un elenco di cartelle recenti: è un cambiamento di prodotto e va progettato a parte. Questo task apre e salva **\*\*un\*\*** documento.
- **\*\*Non migrare\*\*** gli asset già in ``.<app-data>/assets`` dai test del passo 10.
- **\*\*Non toccare il percorso web\*\*:** IndexedDB resta com'è.

### Fatto quando

- G-tsc, G-test verdi.
- Sul desktop: crei una mappa, alleggi un'immagine, salvi in una cartella scelta tu, **\*\*chiudi l'app\*\***, la riapri da quella cartella → l'immagine c'è.
- La cartella si ispeziona da Esplora risorse: ``.document.json`` leggibile e ``.assets/<id>/`` con i quattro file.
- Salvando un documento con immagini in una cartella **\*\*nuova\*\***, le immagini vengono copiate: riaprendolo da lì non manca niente.

---

## 8. Passo 15 – T20: ``.rnode`` è un file SQLite

**\*\*Sostituisce il backend di T19.\*\*** Il documento passa da cartella a **\*\*un solo file\*\*** con estensione propria: ``.MiaMappa.rnode``, che dentro è un database SQLite contenente il documento **\*\*e\*\*** le immagini.

**\*\*Perché.\*\*** Il proprietario vuole una cosa singola, e uno zip non può esserlo: non si aggiorna in posto, quindi ogni salvataggio ricomprimerebbe centinaia di originali. SQLite dà file singolo **\*\*e\*\*** scrittura incrementale **\*\*e\*\*** transazioni atomiche, e ``.rusqlite`` è già fra le dipendenze.

**\*\*Perché adesso.\*\*** Il formato è la cosa più costosa da cambiare dopo: il codice si riscrive, i file degli utenti no. Oggi gli unici file esistenti sono quelli di test – è l'ultimo momento in cui la decisione è gratis.

**\*\*Le interfacce non cambiano.\*\*** ``.AssetStore`` e ``.StorageAdapter`` restano identiche: cambia il **\*\*corpo\*\*** dei comandi Rust, da filesystem a SQL. È il motivo per cui i seam esistono.

[illegible]

vDkL+8bqFq3gH8CWfP4WUroYvjCgGghjEMTQAGjzThGk74DQK2hHmM2hWvXfHbbBTrgIu7get3BvdntOGNlVpvtO8PrIZzLdp+ZaFfTe/5KnywdHl/oklyTh32gub08X3hmw0Kzob+INAGOMHvhvDU  
 HFVfHGFCY2GXUAl/NtfaHrJFyucRb5Y1JgkGvDh1FGNKsQpNst7r9u82t5bmg3CserIEVNDZ2G9SdB+F0a0F40Uwv1uYdFaIK10SCsqQ786evSCmWvXQpJqQ6m5SV4z1kVfSUA0sJwoq50SNe1Pz8r  
 VFd/VN1JA12GfXkHdAmXhB8eP/cPjNtHGG0CNBfMfMmYmJaeJrTcdFXLI5/3Jd9/v/3m3u0s3UDN/1z4a775rhlPr05L0G3uK1CKE0UsiDcSGvXqF4E4nv27+z1zVswGNa3p3It3y3r3wlvw+X  
 W9f6lDKA3Z7sZ2s15HyesRoy6aW4u9q57dtb+ua1JvXr15Vu2Y2a8eJEP+D77BKup6LMoHwNj1lTndu30a3WnYS17ZiUvNnHhuJ5IXQFtejjjoyR0DFU53xgV9ambP/izw01j0OD7B1K1cTDht0X21  
 D285p3f9/VxzXkX9pQ/95c39R6uqJkVXkE/qY1Y1L06XtE7F9HmWdUq7QE0G6+MoZ286gZdFxsGL78hW2zsfakQ9R3HqGfMCRBFAVS+2+TfFqG0W6C3jY1j6NgvYwTAWEsA08sB+C0q0Y1Zxtk  
 o2CN4+tsaL08CB2FKctf9rEmMLu61vUkY6VChSVhsbpq8tN6AE41FMVdN+dxt3J2t3v4yG6pN2LaootI/TeJ3j2ojsaykblfMfCjntZ/TSvE1Xnd6XEsZs0uQ3C84RFtMcEbSvfyY26KuXqAMCPw0F3  
 rCBYyWihsMRD7QU6fIuFp/s/1TZ5YV8fVf/u6yWKjDogP9tgjsufChZm6T0Mph557J+JVCKAXd0hJCjIjXjJoqMP+r+tTz//1DYS8gxHyN1555fB1D/71o0hLuy5j9WCou7r7D3ur33mz71wc++/TTSh  
 bkyUl1uz4cMWH/9N1w/ePZ/nrkB085EjX8PnZ/MKCY1LgxoC2oZq4/I86J0TBZmwNrcHEuQ1S12EiIsocvKxhE2p4qCTCj/04yY8G3DwgMbCfWdQjvdkPfgfmF3y+VAH+dudTUX/SbQhNlt5y/sb  
 Z/k+zg2D5CDuqF9Uw+1e1ZnKdCBWz8k5C5M2jYg3m75vSwPANCd6p236S5Pmbf3N2F20RVf5509smnk6E8B8ANOpfd13/nxbTe122AbayNqm+ZjYMPehZve+EPy+r/K/RFuP0SCSYDv4a+18K5D  
 B2ZPad41XCXyKmf+P669WH06cnNt1yy2u1B46f1b7j75w+uAD1Qp7RwXfFjYX33j11b54z/27d16qK0qTDC56Ad2BXXJqBv/twoe1w/r7aTeCMhtK7HG0MT6m3jZqYDYMSuWdVlnY7x23  
 vDPw88VhKx+ddkxmmM5S2wC12WTG/0yNQ0EbfSbqCT+Y1M0kbba41g5Idve/Dzn1a47Yx06w17XYLZXKB2D7C2a4JhdJ+TB9m5yUfQA353PpK95JfA+e/1SLG9Y5JBTShwFhJPNYFXd5Daxr/Zz+8W  
 14+54+WymyMaDKUvB6MhQX0UUVET42Cxv6n71+Vg/iuupdor/DBXDN6xfj5uMP6Yf/YdVhYh8fQpPvOwti560zljThPdW2oERF135T3AFIsEqAFqTWHSPzURH75xmRjR5/VoLZpwJjOG1  
 FtH5EACsq6dkaF5Y93/VUF03bT1N/B1OWALFas0uSk5seB81NJcPryhKf1c1uPpEXuX2DnQJBKUBQ0F5W0ZlUd+nHd21WmYpJ3sNy07UonnXrAp131SLm7B8ZRH7e4tT48sJBuq01GrgHkM  
 K4X5b83bc+NR/C70pVJ3Jv1z1KbXnVr7F1K7meWuq43ky5nMLC725Kpgena9smnNwETNetxQCUnmwrtY3D9t34Gd5NDnCbazaayVEHD0EzPto/HUMhKYK61aFdrMqZfXmLq/FMV0P/N+Vp9kNzGrH9Ur  
 t5sXwYAp1inXmyr0aXsLkPwCUIXU+XQ16G8FQTD06/75q15eAKHAg1eRQWkK1c1uPpEXuX2DnQJBKUBQ0F5W0ZlUd+nHd21WmYpJ3sNy07UonnXrAp131SLm7B8ZRH7e4tT48sJBuq01GrgHkM  
 /ssjpmjKAMCmVuuUj1t084cz3y8u9vG9mCvXAGAGvPmk+GOEE1H3Zc1A91cEq7Y1MwqmwuhAJ6GYWqJdKwCBZXFuOCXvAB1QmQpQb505Y41H46v0a+YJNBwv4e/LmC1L0  
 6fi+Jkt7avleEEVJDFVPZFCRRR4PytdbjfFGKTN03JoSoKwievK07J/s/qZ21S0Loa/TqNwDrAFw+uS2tMME4CChFXAIR1z/s0LTKX3jJqF4o80sohUhZf1Z151+kf/x9JPPxsuaez4HiKu1G8R  
 9K8stQNG0tAY6GHZy130H87H1B7JAEzuuQwb9Y/NIhG60R20W0p3zJDXmJ3MwFPd301345+afdkJr19fURN2E0P9Y19y5IME9J29PzLk1ZHG5Q05U1gveG5WbYBjBLPMhke1Y0LT1YcK5MeqJ12aUv  
 gscKnyssYf2G51Z5m3Zld1FduBuqV717Faqps+T89YTW3LrCtYh20eY72fC38akGf3azno52TnEcGkCNoeejpk1uWmTkhACpAra+Lm5FEgRBSwkT5G5Gyab+KYKNTC21Unsh51X5Cz46Aq0Y153G  
 TCTeePNC+tpOuG3TE9cVuzsQ8m/suzkw10zn/q339JYNeW4PugxmJUK7JFBFHSUJiUcwa5sEMS2PNB5eAUhKzIndxgipNvKC0W6pw6mm//j6TV0MSAACfcpQDM04sesHH94Yu+GGvVD0H9208Gus  
 +Rh5jCoyK8Cf8x+J6012CTtssXqhmYrCpghPNhK3n81zKqE1CodyZn2t4y8LabYOHkMRTc18gpmnhpAxGkZFCqrgyNMm1a121Ch2agRgP78Y1p1l1n1Uz1KMLCewz2EWEt90RT7qCwdK1  
 huolQYikK5IsW5g+XoJ2ETtXjIscjIrcpYgShX+o8bAe5IMCJNtCtYJdXUaIW3D3jP9+/cZ2JE1w+K1m6B2W1ZlCwLFA51894cpR+MXnn6JLntCv1PLeIQJw7EUPYXCxM7P+IgyzSB7  
 YpDKUFPBCt8eTSN5R59d8st3j/6+rSadtRKC023a0iOialrkkjFnZkz1MaqpnZaba1IK0Y5APaf4wafkPlKwTlTq6n14XNam+3a4VnVjQ137+nJGVCrRSF8jjghyvhV5HdK5m9jCuLZ  
 LLVkw0tgn1e3j1Z1tfn0isXKCK1u0NFVqT43j1WkXdc06K1ZiW0hSv2jPqkX7muFP90PKRfG5z7RDVNDAL04V5h0HnASHAOPUjYcvtPh/7B84qECYt4MRT7P7VZWV7e5vP0ga63391wAm4G0r  
 OFD0AXKXC0A087R708e75553PtvLV9q5vq+jjC5JkR4BiMPXf14gUwrr16UyU1V1+ocYPHdIG7BQ0GKcRb4b+PzZtX0R0DASHAOPUjYcvtPh/7B84qECYt4MRT7P7VZWV7e5vP0ga63391wAm4G0r  
 dHd8W1C18d/19KZD5KFK8pZ1KtLgVfLqNm5+YosS0UXb51Bge1/W2AQ4D73DE6WGA392QTcyotZPMCR0ZYCu8d+Yn+1fW/jfM46d7410wWQJ50rPe9qSKpM9h6m7CD+v102E53PpKQLO9Y/gqLT  
 BKGGKJbreieffB+gw8+SMvX0A0G0I+t7T+xbIr/8kuPK0bGaAZwfuBJNM/mY01j6V79VhSffe6x3b+7hnn0N566aZd+Znb6xdNzqdnZ18Zx0rKhkeTXXV7rMuvh7Rj3j58adtYpCIAESdSVBiGo  
 0AS0n6QL4wt4rtw2FLVZCV7J3j3HwG0Pj1M/NIHUR0tMn6vGcwQ02Fb/JmrJX29MB82SKqcvEDDbabhK5/ZEG+fo2o6r1DMKU24dqnm0U8jyJ609uGrQPSDN0GtTtzyFCBkq5F966gkYrFEUTuq68  
 mpZ1FIInUs18r1PNu5AKoqjEiHEW06e9695J3181BM1VqG7j0dT4qYU139PFPjggZtY/05K8548enB6++/1ZC07G03FmaT7X48UoyfElaP4/cFnsititOkRngv/R809j01Z5k2cfbkt/HEUTfluvk  
 AuUBEDC10ZSSMmr9trvdPrNs1/FN64bkvZ9FDH94JcQGV8NmGmrhLgYXjVvDeDLc9655TW5PuRoZcs8i1YvM5d5FEXPE9KvkZSznrDn4MeiF5YQUTJ6xtim+Q2xyqpddwRLTgUpd4VjX2UpJjCC  
 +amK5Xb9kbn15e5W8yuzV661GzCIDZNTw+wxS0eQNEEwOLk7oZuqRq6tLW8Pdz1fyz0REuCFvA1MLC1Lqwm/Hn0l1f+mv+vrnRCc/jzxk41hr10P3wTSSHjJw1/f0B040a0yPvAvrBw5  
 cbrTeUxL8Yv8EgTZfjbhktDKgHb+gMYX3Pyds98E79r/7B82P9yK610PFxASgmpYd1ZP1fAgyeykvfVMVNT05fVQ5Z0M7F0Jr3hr5ZRaocrP23K4QkVt5f5kfrQDmX2Jntkw+7yuu  
 iFyhW0QjNNkhzg8TCMEF6XsbVmsuLrIgeIuEh1Ct1xglmk0k1vY4TbCUAJe0R7ZU27Tr8AKh9nPqJ7YU1t1MkyCZLmMuncCtMpxvYr1Z6YxvF7A813h4rXXH0g2EmIMW71hFec94IdK5YUae43403Y  
 f54511+1EemVVR/PZ0MBfMMPNf5IbNuW0l10agLQpu1wvDD13T9aeFvR7f5sQ0Yd5p00P78pJy1PVMggtCMceWG

[illegible]

## 7. scripts/bundle\_layout.cjs

```
for (const f of chosen) {
  parts.push(`<h2>${escapeHtml(f.path)}</h2>`);
  parts.push(`<pre><code>`);
```

```

    parts.push(escapeHtml(f.content));
    parts.push('</code></pre>');
  }

  parts.push('</body></html>');

  await fs.promises.writeFile(OUT_FILE, parts.join('\n'), 'utf8');
  console.log(`Wrote ${OUT_FILE} with ${chosen.length} files.`);
})();

```

## 8. src/core/history.ts

```

/**
 * History – undo/redo stack.
 *
 * Each entry is a BATCH of ops (a single user gesture may produce several
 * ops, e.g. deleting multiple selected topics). The entry stores the forward
 * ops and their inverses in undo order, so undo/redo are pure replays.
 */
import type { Op } from "./ops";

export interface HistoryEntry {
  ops: Op[];
  /** Inverses in the order they must be applied to undo the batch. */
  inverse: Op[];
}

export class History {
  private undoStack: HistoryEntry[] = [];
  private redoStack: HistoryEntry[] = [];
  private readonly max = 400;

  /**
   * Record an applied batch.
   * @param ops forward ops, already applied to the sheet
   * @param inverses per-op inverse lists, in forward order
   */
  push(ops: Op[], inverses: Op[][]): HistoryEntry {
    const entry: HistoryEntry = {
      ops,
      // undo applies each batch's inverses in reverse order
      inverse: [...inverses].reverse().flat(),
    };
    this.undoStack.push(entry);
    if (this.undoStack.length > this.max) this.undoStack.shift();
    this.redoStack.length = 0;
    return entry;
  }

  /** Ops to apply for undo (already in correct order). */
  undo(): Op[] | null {
    const entry = this.undoStack.pop();
    if (!entry) return null;
    this.redoStack.push(entry);
    return entry.inverse;
  }

  redo(): Op[] | null {
    const entry = this.redoStack.pop();
    if (!entry) return null;
    this.undoStack.push(entry);
    return entry.ops;
  }

  get canUndo(): boolean {
    return this.undoStack.length > 0;
  }

  get canRedo(): boolean {
    return this.redoStack.length > 0;
  }

  clear(): void {
    this.undoStack = [];
    this.redoStack = [];
  }
}

```

## 9. src/core/text.ts

```
/**
 * TextRun helpers – the bridge between the plain-string title (legacy and
 * derived consumers: search, export, outliner, tests) and the styled
 * titleRuns array (the rich-text source of truth).
 *
 * Invariant maintained by every writer: node.title === runsToPlain(node.titleRuns)
 * and titleRuns is normalized (no empty runs, adjacent runs with identical
 * formatting merged).
 */
import type { TextRun } from "../types";

export function runsToPlain(runs: TextRun[]): string {
  return runs.map((r) => r.text).join("");
}

export function plainToRuns(text: string): TextRun[] {
  return text.length > 0 ? [{ text }] : [];
}

/** Drop empty runs and merge adjacent runs that share the same formatting. */
export function normalizeRuns(runs: TextRun[]): TextRun[] {
  const out: TextRun[] = [];
  for (const run of runs) {
    if (!run || run.text.length === 0) continue;
    const prev = out[out.length - 1];
    if (prev && sameFormat(prev, run)) {
      prev.text += run.text;
    } else {
      out.push({ ...run });
    }
  }
  return out;
}

function sameFormat(a: TextRun, b: TextRun): boolean {
  return (
    (a.bold ?? false) === (b.bold ?? false) &&
    (a.italic ?? false) === (b.italic ?? false) &&
    (a.underline ?? false) === (b.underline ?? false) &&
    (a.color ?? null) === (b.color ?? null) &&
    (a.fontSize ?? null) === (b.fontSize ?? null) &&
    (a.paraGap ?? false) === (b.paraGap ?? false) &&
    (a.listIndent ?? null) === (b.listIndent ?? null)
  );
}

export function runsEqual(a: TextRun[] | undefined, b: TextRun[] | undefined): boolean {
  if (!a || !b) return !a && !b;
  if (a.length !== b.length) return false;
  for (let i = 0; i < a.length; i++) {
    const ra = a[i];
    const rb = b[i];
    if (ra.text !== rb.text || !sameFormat(ra, rb)) return false;
  }
  return true;
}

/** The styled runs of a node: explicit titleRuns or the whole title as one plain run. */
export function nodeRuns(title: string, titleRuns?: TextRun[]): TextRun[] {
  if (titleRuns && titleRuns.length > 0) return titleRuns;
  return plainToRuns(title);
}

export function isEmptyRuns(runs: TextRun[]): boolean {
  return runs.every((r) => r.text.trim().length === 0);
}

/**
 * Trim the plain text of a run sequence: leading whitespace of the first
 * run and trailing whitespace of the last run only – internal spacing
 * between runs is preserved.
 */
export function trimRuns(runs: TextRun[]): TextRun[] {
  const out = runs.map((r) => ({ ...r }));
  if (out.length === 0) return out;
  const first = out[0];
  const leading = first.text.match(/^\s*/)?.[0] ?? "";
  first.text = first.text.slice(leading.length);
  const last = out[out.length - 1];
  const trailing = last.text.match(/\s*/)?.[0] ?? "";
  last.text = last.text.slice(0, last.text.length - trailing.length);
  return normalizeRuns(out);
}
```

## 10. src/core/tree.ts

```
import type { DocumentModel } from "./doc";

/** Is `id` a descendant of `ancestorId`? (strictly below) */
export function isDescendantOf(doc: DocumentModel, ancestorId: string, id: string): boolean {
  if (ancestorId === id) return false;
  let cur = doc.node(id)?.parentId ?? null;
  while (cur) {
    if (cur === ancestorId) return true;
    cur = doc.node(cur)?.parentId ?? null;
  }
  return false;
}
```

## 11. src/editor/context.ts

```
import { createContext, useContext } from "react";
import type { EditorStore } from "./store";

export const StoreContext = createContext<EditorStore | null>(null);

export function useStore(): EditorStore {
  const store = useContext(StoreContext);
  if (!store) throw new Error("StoreContext missing – wrap the app in <StoreContext.Provider>");
  return store;
}
```

## 12. src/editor/externalImage.ts

```
/**
 * External image drag-drop sources (T-extension): an image dragged from the
 * OS file explorer or from a browser page lands on the canvas as one of two
 * shapes – a `File` in `dataTransfer.files`, or, when the browser only hands
 * over the image URL (common for cross-origin images), a `text/uri-list`
 * string. This module turns either into a `File` the import pipeline already
 * knows how to handle (`attachImageFile`).
 *
 * The fetch of a dropped URL runs through `tauri-plugin-http` inside the
 * desktop webview (CORS-free, Rust-side) and through plain `window.fetch`
 * in the web build, where the page's own origin rules apply.
 */
import { fetch as tauriFetch } from "@tauri-apps/plugin-http";

export type FetchFn = (url: string) => Promise<Response>;

/** The fetch implementation for the current runtime (Tauri webview vs browser). */
export function resolveFetcher(): FetchFn {
  const tauri = typeof window !== "undefined" && !!window.__TAURI__;
  return tauri ? tauriFetch : window.fetch.bind(window);
}

/** First `image/*` entry in the dropped file list, or null. */
export function firstImageFile(files: FileList | File[]): File | null {
  for (const f of files) {
    if (f.type.startsWith("image/")) return f;
  }
  return null;
}

/**
 * First URL in a `text/uri-list` payload. Comment lines start with `#`;
 * the browser also appends the URL of the page the image came from on a
 * later line – the FIRST non-comment line is the image itself.
 */
export function firstUriFromList(urilist: string): string | null {
  for (const line of urilist.split("\n")) {
    const url = line.trim();
    if (url && !url.startsWith("#")) return url;
  }
  return null;
}

/** A dropped URL that is definitely an image (so the URL→bytes direction is a
 * download, never a re-upload of local data). */
const URL_BYTE_CAP = 25 * 1024 * 1024; // same ceiling as MAX_SOURCE_BYTES

/**
 * Download the image at `url` as a `File`, or null when the response is not
```

```

* an image / too large / unreachable. `fetcher` is injectable for tests.
*/
export async function fetchImageAsFile(
  url: string,
  fetcher: FetchFn = resolveFetcher()
): Promise<File | null> {
  let res: Response;
  try {
    res = await fetcher(url);
  } catch {
    return null; // network error / CORS / plugin unavailable
  }
  if (!res.ok) return null;
  const announced = Number(res.headers.get("content-length"));
  if (Number.isFinite(announced) && announced > URL_BYTE_CAP) return null;
  let blob: Blob;
  try {
    blob = await res.blob();
  } catch {
    return null;
  }
  if (!blob.type.startsWith("image/")) return null;
  if (blob.size > URL_BYTE_CAP) return null;
  const name = url.split("/").pop()?.split("?")[0] || "dropped-image";
  return new File([blob], name, { type: blob.type });
}

```

### 13. src/editor/imageImport.ts

```

/**
 * Image import pipeline (T13-1).
 *
 * The heavy work – reading the file, SHA-256, decoding, and generating the
 * three levels – runs inside a Web Worker (imageImport.worker.ts) so a
 * multi-megabyte original never stalls the main thread. This module holds
 * the pure, testable contract: what is accepted, how the derived levels are
 * sized, and how the worker is dispatched. The bytes are stored through
 * AssetStore.put (ADR-001 §12): the original intact plus 1024px and 256px
 * levels on the long side.
 */
import type { AssetBlob, AssetLevel, AssetMeta } from "../persist/assets";

/** Raster formats only. SVG is an executable document: excluded (XSS hardening is deferred). */
export const IMAGE_MIME_ALLOWLIST = ["image/png", "image/jpeg", "image/gif", "image/webp"];

export const MAX_SOURCE_BYTES = 25 * 1024 * 1024;

/** Long side of each derived level, in pixels. */
export const LEVEL_LONG_SIDE: Record<Exclude<AssetLevel, "original">, number> = {
  large: 1024,
  small: 256,
};

export function isAllowedImageMime(mime: string): boolean {
  return IMAGE_MIME_ALLOWLIST.includes(mime);
}

export type ImageSourceValidation = { ok: true } | { ok: false; reason: string };

/** Reject unsupported formats and oversized files BEFORE any decoding. */
export function validateImageSource(mime: string, bytes: number): ImageSourceValidation {
  if (!isAllowedImageMime(mime)) return { ok: false, reason: `unsupported mime ${mime}` };
  if (bytes > MAX_SOURCE_BYTES) return { ok: false, reason: `too large ${bytes} bytes` };
  return { ok: true };
}

/**
 * Size a level so its long side is `targetLong` px, keeping the aspect
 * ratio. Images already smaller than the target are kept at natural size
 * (upscaling adds bytes, not information).
 */
export function computeLevelDims(w: number, h: number, targetLong: number): { w: number; h: number } {
  if (w <= 0 || h <= 0) return { w: 1, h: 1 };
  const scale = targetLong / Math.max(w, h);
  if (scale >= 1) return { w, h };
  return { w: Math.max(1, Math.round(w * scale)), h: Math.max(1, Math.round(h * scale)) };
}

/** What the import pipeline produces: a content-addressed id + the three levels + metadata. */
export interface ImportedImage {
  id: string;
  levels: Record<AssetLevel, AssetBlob>;
  meta: Omit<AssetMeta, "id">;
}

```

```

/**
 * Run the import in a Web Worker. Resolves with the three levels ready for
 * AssetStore.put; rejects with a message when the source is unusable.
 */
export function importImageFile(file: Blob & { name?: string }): Promise<ImportedImage> {
  return new Promise((resolve, reject) => {
    const worker = new Worker(new URL("./imageImport.worker.ts", import.meta.url), { type: "module" });
    worker.onmessage = (e: MessageEvent) => {
      const data = e.data as { ok: boolean; result?: Image; error?: string };
      worker.terminate();
      if (data.ok && data.result) resolve(data.result);
      else reject(new Error(data.error ?? "image import failed"));
    };
    worker.onerror = (e: ErrorEvent) => {
      worker.terminate();
      reject(new Error(e.message ?? "image import worker crashed"));
    };
    worker.postMessage({ blob: file, mime: file.type, name: file.name });
  });
}

```

## 14. src/editor/imageImport.worker.ts

```

/**
 * Image import worker (T13-1).
 *
 * Runs off the main thread: reads the original bytes, computes the content
 * address (SHA-256), decodes the image and draws the 1024px and 256px levels
 * onto OffscreenCanvases. The main thread receives { id, levels, meta } and
 * hands it to AssetStore.put – nothing image-sized ever blocks the UI.
 *
 * The worker keeps the original blob INTACT: it is the source of truth for
 * the T14 slider and high-quality export, never recompressed.
 */

type WorkerScope = {
  onmessage: ((e: MessageEvent) => void) | null;
  postMessage(msg: unknown): void;
};

const scope = self as unknown as WorkerScope;

function hex(buffer: ArrayBuffer): string {
  return Array.from(new Uint8Array(buffer), (b) => b.toString(16).padStart(2, "0")).join("");
}

/** Level blobs are PNG when the source can carry alpha, JPEG otherwise. */
function levelMime(sourceMime: string): string {
  return sourceMime === "image/png" || sourceMime === "image/webp" || sourceMime === "image/gif"
    ? "image/png"
    : "image/jpeg";
}

scope.onmessage = (e: MessageEvent) => {
  void (async () => {
    const { blob, mime, name } = e.data as { blob: Blob; mime: string; name?: string };
    try {
      // Content address: SHA-256 of the ORIGINAL bytes – the same image
      // attached to N nodes occupies space exactly once.
      const originalBytes = await blob.arrayBuffer();
      const digest = await crypto.subtle.digest("SHA-256", originalBytes);
      const id = hex(digest);

      const source = await createImageBitmap(blob);
      const srcW = source.width;
      const srcH = source.height;
      const levels = {
        original: { blob, w: srcW, h: srcH },
      } as Record<"original" | "large" | "small", { blob: Blob; w: number; h: number }>;

      const outMime = levelMime(mime);
      for (const [key, targetLong] of [
        ["large", 1024],
        ["small", 256],
      ] as const) {
        const scale = targetLong / Math.max(srcW, srcH);
        if (scale >= 1) {
          // Already smaller than the target: the original IS the level.
          levels[key] = { blob, w: srcW, h: srcH };
          continue;
        }
        const w = Math.max(1, Math.round(srcW * scale));
        const h = Math.max(1, Math.round(srcH * scale));

```

```

    const canvas = new OffscreenCanvas(w, h);
    const ctx = canvas.getContext("2d");
    if (!ctx) throw new Error("offscreen 2d context unavailable");
    ctx.drawImage(source, 0, 0, w, h);
    levels[key] = { blob: await canvas.convertToBlob({ type: outMime, quality: 0.85 }), w, h };
  }
  source.close();

  scope.postMessage({
    ok: true,
    result: {
      id,
      levels,
      meta: { mime, w: srcW, h: srcH, bytes: originalBytes.byteLength, name },
    },
  });
} catch (err) {
  scope.postMessage({ ok: false, error: err instanceof Error ? err.message : String(err) });
}
})();
};

```

## 15. src/editor/view.ts

```

/** Latest canvas viewport size (CSS px), kept by CanvasView for global consumers. */
export const viewSize = { w: 1200, h: 800 };

```

## 16. src/render/viewport.ts

```

/**
 * Camera / viewport transforms.
 *
 * camera.x/y = the world coordinate at the center of the viewport.
 * screen = (world - camera) * scale + viewportSize / 2
 */
export interface Camera {
  x: number;
  y: number;
  scale: number;
}

export const MIN_SCALE = 0.1;
export const MAX_SCALE = 4.0;

export const HOME_CAMERA: Camera = { x: 0, y: 0, scale: 1 };

export function worldToScreen(c: Camera, vw: number, vh: number, wx: number, wy: number): { x: number; y: number } {
  return { x: (wx - c.x) * c.scale + vw / 2, y: (wy - c.y) * c.scale + vh / 2 };
}

export function screenToWorld(c: Camera, vw: number, vh: number, sx: number, sy: number): { x: number; y: number } {
  return { x: (sx - vw / 2) / c.scale + c.x, y: (sy - vh / 2) / c.scale + c.y };
}

/** Zoom by `factor` keeping the world point under (sx, sy) fixed. */
export function zoomAt(c: Camera, vw: number, vh: number, sx: number, sy: number, factor: number): Camera {
  const anchor = screenToWorld(c, vw, vh, sx, sy);
  const scale = Math.min(MAX_SCALE, Math.max(MIN_SCALE, c.scale * factor));
  // Keep the world point currently under the cursor at the same screen
  // coordinate after changing scale. The camera center is not the anchor
  // point unless the cursor happens to be in the middle of the viewport.
  return {
    scale,
    x: anchor.x - (sx - vw / 2) / scale,
    y: anchor.y - (sy - vh / 2) / scale,
  };
}

export function panBy(c: Camera, dx: number, dy: number): Camera {
  return { ...c, x: c.x - dx / c.scale, y: c.y - dy / c.scale };
}

export function centerOn(c: Camera, wx: number, wy: number, scale = c.scale): Camera {
  return { x: wx, y: wy, scale };
}

/** Fit the given world bounds (with padding) into a viewport. */
export function fitBounds(_c: Camera, vw: number, vh: number, bounds: { minX: number; minY: number; maxX: number; maxY: number }): Camera {
  const pad = 60;
  const bw = Math.max(bounds.maxX - bounds.minX, 1);

```

```

const bh = Math.max(bounds.maxY - bounds.minY, 1);
const scale = Math.min(MAX_SCALE, Math.max(MIN_SCALE, Math.min((vw - pad * 2) / bw, (vh - pad * 2) / bh)));
const cx = (bounds.minX + bounds.maxX) / 2;
const cy = (bounds.minY + bounds.maxY) / 2;
return { x: cx, y: cy, scale: Math.min(scale, 1.25) };
}

```

## 17. src/ui/pasteSanitizer.ts

```

/**
 * Paste sanitizer – turns clipboard HTML from Word, Google Docs, Draw.io and
 * the web into the subset R-node understands, then into TextRun[].
 *
 * Two known pitfalls are handled explicitly:
 * 1. Word/Google Docs express emphasis as CSS on <span> (font-weight: 700,
 *    font-style: italic), NOT as <b><i>. We read the computed inline styles
 *    BEFORE stripping them and wrap the content in <strong><em>.
 * 2. Word exports lists as paragraphs with mso-list markers and margin-left
 *    indentation instead of <ul><li>. We detect those markers and rebuild
 *    real (possibly nested) <ul><li> before the generic cleanup.
 *
 * Colors survive as <span data-rnode-color="#hex">, read back by htmlToRuns.
 * Everything else (scripts, images, hrefs, junk styles, layout CSS) is dropped.
 */
import type { TextRun } from "../core/types";
import { normalizeRuns } from "../core/text";

const COLOR_RE = /^#[0-9a-fA-F]{3,8}$/;

function cssColor(cssColor: string | null): string | null {
  if (!cssColor) return null;
  const t = cssColor.trim();
  if (COLOR_RE.test(t)) return t.toLowerCase();
  const m = t.match(/^rgba?\(\s*(\d{1,3})[\s,]+(\d{1,3})[\s,]+(\d{1,3})\s*\)/);
  if (m) {
    const hex = (v: string): string => Math.max(0, Math.min(255, parseInt(v, 10))).toString(16).padStart(2, "0");
    return `#${hex(m[1])}${hex(m[2])}${hex(m[3])}`;
  }
  return null;
}

function boldWeight(w: string | null): boolean {
  if (!w) return false;
  const t = w.trim();
  const n = parseInt(t, 10);
  return t === "bold" || t === "bolder" || (Number.isFinite(n) && n >= 600);
}

interface Ctx {
  bold: boolean;
  italic: boolean;
  underline: boolean;
  color?: string;
  fontSize?: number;
}

/** Heading level → font size in px (kept as-is so pasted structure survives). */
const HEADING_SIZES: Record<string, number> = { h1: 26, h2: 21, h3: 18, h4: 16, h5: 15, h6: 14 };

const DROP_TAGS = new Set([
  "script", "style", "iframe", "noscript", "link", "meta", "title", "template", "svg", "img", "video", "audio", "object", "embed", "form", "input",
  "button", "select", "textarea",
]);

const BLOCK_TAGS = new Set(["p", "div", "h1", "h2", "h3", "h4", "h5", "h6", "blockquote", "pre", "ul", "ol", "li"]);

// -----
// Word list normalization: mso-list markers / MsoListParagraph + margin-left
// → real (nested) <ul><li>. Runs before the generic cleanup.
// -----

function isWordListMarker(el: Element): boolean {
  const cls = el.getAttribute("class") ?? "";
  if (cls.includes("MsoListParagraph") || cls.includes("MsoList")) return true;
  if (el.hasAttribute("mso-list")) return true;
  const st = el.getAttribute("style") ?? "";
  return st.includes("mso-list");
}

function liIndent(el: Element): number {
  const st = (el as HTMLElement).style;
  const mso = el.getAttribute("mso-list");
  if (mso) {
    const lvl = mso.match(/level(\d+)/i);

```

```

    if (lvl) return Math.min(8, Math.max(1, parseInt(lvl[1], 10)));
  }
  const ml = st.marginLeft;
  if (ml) {
    const n = parseFloat(ml);
    if (Number.isFinite(n)) return Math.min(8, Math.max(1, Math.ceil(n / 36)));
  }
  return 1;
}

function normalizeWordLists(doc: Document): void {
  const body = doc.body;
  const markers = Array.from(body.querySelectorAll("*")).filter(isWordListMarker);
  if (markers.length === 0) return;

  // Replace every marker with a plain <li> carrying its indent as data.
  const items: { li: HTMLElement; indent: number }[] = [];
  for (const m of markers) {
    const li = doc.createElement("li");
    li.setAttribute("data-rnode-indent", String(liIndent(m)));
    while (m.firstChild) li.appendChild(m.firstChild);
    m.replaceWith(li);
    items.push({ li, indent: liIndent(m) });
  }

  // Group consecutive items into (nested) <ul>.
  let start = 0;
  while (start < items.length) {
    let end = start + 1;
    while (end < items.length && items[end].li.previousSibling === items[end - 1].li) end++;
    const group = items.slice(start, end);
    start = end;
    if (group.length === 0) continue;

    const rootUl = doc.createElement("ul");
    // Insert the list BEFORE the first item's current position (the items are
    // still in the body at this point) – replacing the first item instead
    // would be invalid once the items move inside the list we just created.
    group[0].li.parentNode?.insertBefore(rootUl, group[0].li);
    // stack of {ul, indent}
    const stack: { ul: HTMLElement; indent: number }[] = [{ ul: rootUl, indent: 0 }];
    for (const { li, indent } of group) {
      while (stack.length > 1 && stack[stack.length - 1].indent >= indent) stack.pop();
      const cur = stack[stack.length - 1];
      cur.ul.appendChild(li);
      const childUl = doc.createElement("ul");
      li.appendChild(childUl);
      stack.push({ ul: childUl, indent });
    }
    // remove dangling empty <ul> containers (recursively)
    const dropEmptyUl = (el: HTMLElement): void => {
      for (const child of Array.from(el.children)) {
        if (child.tagName === "UL" && child.childElementCount === 0) child.remove();
        else dropEmptyUl(child as HTMLElement);
      }
    };
    dropEmptyUl(rootUl);
  }
}

// -----
// Generic cleanup
// -----

function applyInline(el: HTMLElement, ctx: Ctx): Ctx {
  const s = el.style;
  const next: Ctx = { ...ctx };
  if (boldWeight(s.fontWeight)) next.bold = true;
  if (s.fontStyle === "italic") next.italic = true;
  if (s.textDecorationLine?.includes("underline") || s.textDecoration?.includes("underline")) next.underline = true;
  // sanitizeHtml() re-emits colors as <span data-rnode-color="#hex"> – read
  // that marker here so htmlToRuns() keeps them (style is stripped by then).
  const marker = el.getAttribute("data-rnode-color");
  if (marker && COLOR_RE.test(marker)) next.color = marker.toLowerCase();
  else {
    const c = cssColor(s.color);
    if (c) next.color = c;
  }
  return next;
}

function tagCtx(tag: string, ctx: Ctx): Ctx {
  const next = { ...ctx };
  if (tag === "b" || tag === "strong") next.bold = true;
  if (tag === "i" || tag === "em") next.italic = true;
  if (tag === "u" || tag === "ins") next.underline = true;
  return next;
}

```

```

}

/** Wrap a text node per the accumulated context (strong/em/color). */
function wrapText(doc: Document, text: string, ctx: Ctx): Node {
  let node: Node = doc.createTextNode(text);
  const wrap = (content: Node, tag: string): Node => {
    const el = doc.createElement(tag);
    el.appendChild(content);
    return el;
  };
  if (ctx.color) {
    const span = doc.createElement("span");
    span.setAttribute("data-rnode-color", ctx.color);
    span.appendChild(node);
    node = span;
  }
  if (ctx.italic) node = wrap(node, "em");
  if (ctx.bold) node = wrap(node, "strong");
  if (ctx.underline) {
    const u = doc.createElement("u");
    u.appendChild(node);
    node = u;
  }
  return node;
}

function buildClean(doc: Document, el: Element, ctx: Ctx, out: Node[]): void {
  const tag = el.tagName.toLowerCase();
  if (DROP_TAGS.has(tag)) return;

  if (tag === "br") {
    out.push(doc.createElement("br"));
    return;
  }

  const ownCtx = applyInline(el as HTMLElement, tagCtx(tag, ctx));

  // Inline formatting elements: keep text runs only.
  if (["span", "strong", "b", "em", "i", "u", "ins", "a", "font", "small", "sub", "sup", "code", "mark", "s", "strike", "del"].includes(tag)) {
    for (const child of Array.from(el.childNodes)) {
      if (child.nodeType === Node.TEXT_NODE) out.push(wrapText(doc, child.textContent ?? "", ownCtx));
      else if (child.nodeType === Node.ELEMENT_NODE) buildClean(doc, child as Element, ownCtx, out);
    }
    return;
  }

  // A bare <li> emits its inner content only – its parent list provides the
  // wrapper. (Appending an <li> inside an <li> is a DOM HierarchyRequestError.)
  if (tag === "li") {
    for (const child of Array.from(el.childNodes)) {
      if (child.nodeType === Node.TEXT_NODE) out.push(wrapText(doc, child.textContent ?? "", ownCtx));
      else if (child.nodeType === Node.ELEMENT_NODE) buildClean(doc, child as Element, ownCtx, out);
    }
    return;
  }

  // Block elements / lists. Headings keep their tag so the heading level
  // (and therefore its larger font size) survives the sanitization.
  if (BLOCK_TAGS.has(tag)) {
    const block = doc.createElement(tag === "div" ? "p" : tag);
    if (tag === "ul" || tag === "ol") {
      for (const child of Array.from(el.childNodes)) {
        if (child.nodeType === Node.ELEMENT_NODE && child.nodeName.toLowerCase() === "li") {
          const li = doc.createElement("li");
          const itemOut: Node[] = [];
          buildClean(doc, child as Element, ownCtx, itemOut);
          for (const n of itemOut) li.appendChild(n);
          block.appendChild(li);
        } else if (child.nodeType === Node.TEXT_NODE && (child.textContent ?? "").trim() !== "") {
          // stray text inside a list -> treat as an item
          const li = doc.createElement("li");
          li.appendChild(wrapText(doc, child.textContent ?? "", ownCtx));
          block.appendChild(li);
        } else if (child.nodeType === Node.ELEMENT_NODE) {
          buildClean(doc, child as Element, ownCtx, [block]);
        }
      }
    }
    out.push(block);
    return;
  }

  const inner: Node[] = [];
  for (const child of Array.from(el.childNodes)) {
    if (child.nodeType === Node.TEXT_NODE) inner.push(wrapText(doc, child.textContent ?? "", ownCtx));
    else if (child.nodeType === Node.ELEMENT_NODE) buildClean(doc, child as Element, ownCtx, inner);
  }
  for (const n of inner) block.appendChild(n);
  // Merge adjacent empty blocks and drop fully-empty paragraphs.

```

```

    if (block.textContent?.trim() !== "" || block.childNodes.length === 0) out.push(block);
    return;
}

// Unknown element: recurse into children.
for (const child of Array.from(el.childNodes)) {
    if (child.nodeType === Node.TEXT_NODE) out.push(wrapText(doc, child.textContent ?? "", ownCtx));
    else if (child.nodeType === Node.ELEMENT_NODE) buildClean(doc, child as Element, ownCtx, out);
}
}

/** Sanitize clipboard HTML into clean semantic HTML (strong/em/u, ul/ol/li, p, br, data-rnode-color spans). */
export function sanitizeHtml(html: string): string {
    const doc = new DOMParser().parseFromString(html, "text/html");
    normalizeWordLists(doc);
    const out: Node[] = [];
    buildClean(doc, doc.body, { bold: false, italic: false, underline: false }, out);
    const container = doc.createElement("div");
    for (const n of out) container.appendChild(n);
    return container.innerHTML;
}

// -----
// Clean HTML → TextRun[] (also used directly by the Lexical paste plugin)
// -----

export function htmlToRuns(html: string): TextRun[] {
    const doc = new DOMParser().parseFromString(html, "text/html");
    const runs: TextRun[] = [];

    const walkInline = (el: Element, ctx: Ctx, out: TextRun[]): void => {
        const own = applyInline(el as HTMLElement, tagCtx(el.tagName.toLowerCase(), ctx));
        const tag = el.tagName.toLowerCase();
        // headings carry their own (larger) font size
        if (!own.fontSize && /^h[1-6]$/.test(tag)) own.fontSize = HEADING_SIZES[tag];
        for (const child of Array.from(el.childNodes)) {
            if (child.nodeType === Node.TEXT_NODE) pushInto(out, child.textContent ?? "", own);
            else if (child.nodeType === Node.ELEMENT_NODE) {
                const c = child as Element;
                const t = c.tagName.toLowerCase();
                if (t === "br") pushInto(out, "\n", own);
                else if (t === "ul" || t === "ol" || t === "li") {
                    /* block containers handled by walkList/walkBlocks */
                } else walkInline(c, own, out);
            }
        }
    };

    // Collect one block's runs, then walk the body merging blocks with a
    // paragraph gap (line break + extra spacing) between them.
    const walkBlock = (el: Element): TextRun[] => {
        const out: TextRun[] = [];
        const tag = el.tagName.toLowerCase();
        if (tag === "ul" || tag === "ol" || tag === "li") walkList(el, 0, out);
        else walkInline(el, { bold: false, italic: false, underline: false }, out);
        return out;
    };

    const walkBlocks = (parent: Element): void => {
        let first = true;
        const children = Array.from(parent.childNodes);
        for (const child of children) {
            if (child.nodeType === Node.TEXT_NODE) {
                if ((child.textContent ?? "").trim() !== "") {
                    if (!first) {
                        pushInto(runs, "\n", { bold: false, italic: false, underline: false });
                        first = false;
                    }
                    pushInto(runs, child.textContent ?? "", { bold: false, italic: false, underline: false });
                    first = false;
                }
            }
            continue;
        }
        if (child.nodeType !== Node.ELEMENT_NODE) continue;
        const block = walkBlock(child as Element);
        if (block.length === 0) continue;
        if (!first && block[0].text !== "\n") block[0].paraGap = true;
        if (!first) pushInto(runs, "\n", { bold: false, italic: false, underline: false });
        for (const r of block) runs.push(r);
        first = false;
    }
};

const walkList = (liOrUl: Element, depth: number, out: TextRun[]): void => {
    const walkItems = (el: Element, d: number, out: TextRun[]): void => {
        for (const child of Array.from(el.childNodes)) {
            if (child.nodeType !== Node.ELEMENT_NODE) continue;

```

```

const c = child as Element;
const tag = c.tagName.toLowerCase();
if (tag === "li") {
  const itemRuns: TextRun[] = [];
  walkInline(c, { bold: false, italic: false, underline: false }, itemRuns);
  // nested lists inside the item start on their own line
  for (const nested of Array.from(c.childNodes)) {
    if (nested.nodeType === Node.ELEMENT_NODE && ["ul", "ol"].includes((nested as Element).tagName.toLowerCase())) {
      if (itemRuns.length > 0) pushInto(itemRuns, "\n", { bold: false, italic: false, underline: false });
      walkItems(nested as Element, d + 1, itemRuns);
    }
  }
  // the item's first run carries the bullet depth marker
  if (itemRuns.length > 0 && itemRuns[0].listIndent === undefined) itemRuns[0].listIndent = Math.min(8, d + 1);
  for (const r of itemRuns) out.push(r);
  pushInto(out, "\n", { bold: false, italic: false, underline: false });
} else if (tag === "ul" || tag === "ol") {
  walkItems(c, d, out);
}
}
};
walkItems(liOrUl, depth, out);
};

walkBlocks(doc.body);
return normalizeRuns(runs);
}

function pushInto(out: TextRun[], text: string, ctx: Ctx): void {
  if (text.length === 0) return;
  const run: TextRun = { text };
  if (ctx.bold) run.bold = true;
  if (ctx.italic) run.italic = true;
  if (ctx.underline) run.underline = true;
  if (ctx.color) run.color = ctx.color;
  if (ctx.fontSize) run.fontSize = ctx.fontSize;
  out.push(run);
}

```

## 18. src-tauri/build.rs

```

fn main() {
  tauri_build::build()
}

```

## 19. tests/externalImage.test.ts

```

import { describe, expect, it, vi } from "vitest";
import { fetchImageAsFile, firstImageFile, firstUriFromList } from "../src/editor/externalImage";

describe("firstImageFile", () => {
  it("returns the first image/* entry and skips non-image files", () => {
    const files = [
      new File(["a"], "notes.txt", { type: "text/plain" }),
      new File(["b"], "pic.png", { type: "image/png" }),
      new File(["c"], "photo.jpg", { type: "image/jpeg" }),
    ];
    expect(firstImageFile(files)?.name).toBe("pic.png");
  });

  it("returns null when no image is present", () => {
    expect(firstImageFile([new File(["a"], "doc.pdf", { type: "application/pdf" }]))).toBeNull();
    expect(firstImageFile([])).toBeNull();
  });
});

describe("firstUriFromList", () => {
  it("skips comment lines and returns the first URL", () => {
    const list = "# dropped from https://example.com/page\nhttps://example.com/img/cat.png\nhttps://example.com/page";
    expect(firstUriFromList(list)).toBe("https://example.com/img/cat.png");
  });

  it("handles a bare single-URL payload and empty input", () => {
    expect(firstUriFromList("https://example.com/a.png")).toBe("https://example.com/a.png");
    expect(firstUriFromList("")).toBeNull();
    expect(firstUriFromList("# only a comment\n")).toBeNull();
  });
});

describe("fetchImageAsFile", () => {

```

```

function okResponse(type: string, body: string, headers: Record<string, string> = {}): Response {
    return new Response(new Blob([body], { type }), { status: 200, headers });
}

it("downloads an image URL into a File named after the URL", async () => {
    const fetcher = vi.fn(async () => okResponse("image/png", "png-bytes"));
    const file = await fetchImageAsFile("https://example.com/img/cat.png?x=1", fetcher);
    expect(fetcher).toHaveBeenCalledWith("https://example.com/img/cat.png?x=1");
    expect(file).not.toBeNull();
    expect(file!.name).toBe("cat.png");
    expect(file!.type).toBe("image/png");
    expect(await file!.text()).toBe("png-bytes");
});

it("returns null for a non-image response, an HTTP error, or a network failure", async () => {
    expect(await fetchImageAsFile("https://example.com/a.txt", vi.fn(async () => okResponse("text/plain", "x")))).toBeNull();
    expect(await fetchImageAsFile("https://example.com/404.png", vi.fn(async () => new Response(null, { status: 404 }))))).toBeNull();
    expect(await fetchImageAsFile("https://example.com/down.png", vi.fn(async () => { throw new Error("net"); }))).toBeNull();
});

it("rejects downloads that are too large (content-length or actual size)", async () => {
    const big = "x".repeat(30 * 1024 * 1024);
    const announced = vi.fn(async () => okResponse("image/png", "small", { "content-length": String(30 * 1024 * 1024) }));
    expect(await fetchImageAsFile("https://example.com/big.png", announced)).toBeNull();

    const actual = vi.fn(async () => okResponse("image/png", big));
    expect(await fetchImageAsFile("https://example.com/big.png", actual)).toBeNull();
});
});

```

## 20. tests/imagelImport.test.ts

```

import { describe, expect, it } from "vitest";
import {
    computeLevelDims,
    IMAGE_MIME_ALLOWLIST,
    isAllowedImageMime,
    MAX_SOURCE_BYTES,
    validateImageSource,
} from "../src/editor/imageImport";

describe("image source validation (T13-1)", () => {
    it("accepts the allowed raster formats", () => {
        for (const mime of IMAGE_MIME_ALLOWLIST) {
            expect(isAllowedImageMime(mime)).toBe(true);
            expect(validateImageSource(mime, 100)).toEqual({ ok: true });
        }
    });

    it("rejects SVG – an executable document, not a raster", () => {
        expect(isAllowedImageMime("image/svg+xml")).toBe(false);
        expect(validateImageSource("image/svg+xml", 100)).toEqual({
            ok: false,
            reason: "unsupported mime image/svg+xml",
        });
        expect(isAllowedImageMime("text/html")).toBe(false);
    });

    it("rejects files over MAX_SOURCE_BYTES with a message", () => {
        expect(validateImageSource("image/png", MAX_SOURCE_BYTES + 1)).toEqual({
            ok: false,
            reason: `too large ${MAX_SOURCE_BYTES + 1} bytes`,
        });
        expect(validateImageSource("image/png", MAX_SOURCE_BYTES)).toEqual({ ok: true });
    });
});

describe("computeLevelDims (T13-1)", () => {
    it("scales the long side to the target keeping proportions", () => {
        expect(computeLevelDims(4000, 3000, 1024)).toEqual({ w: 1024, h: 768 });
        expect(computeLevelDims(4000, 3000, 256)).toEqual({ w: 256, h: 192 });
    });

    it("keeps the natural size when the image is already below the target", () => {
        expect(computeLevelDims(200, 100, 1024)).toEqual({ w: 200, h: 100 });
        expect(computeLevelDims(1024, 683, 1024)).toEqual({ w: 1024, h: 683 });
    });

    it("handles portrait images (the height is the long side)", () => {
        expect(computeLevelDims(3000, 4000, 1024)).toEqual({ w: 768, h: 1024 });
        expect(computeLevelDims(3000, 4000, 256)).toEqual({ w: 192, h: 256 });
    });

    it("never returns zero-size levels", () => {

```

```

    expect(computeLevelDims(0, 0, 1024)).toEqual({ w: 1, h: 1 });
    expect(computeLevelDims(1, 1, 256)).toEqual({ w: 1, h: 1 });
  });
});

```

## 21. tests/layout.test.ts

```

import { describe, expect, it } from "vitest";
import { DocumentModel, uid } from "../src/core/doc";
import { applyOp, makeOp, type Op } from "../src/core/ops";
import { applyLayout, layoutSheet, measureNode } from "../src/layout/mindmap";

function makeMap(): DocumentModel {
  const model = new DocumentModel(DocumentModel.blank("Layout test"));
  return model;
}

function addChild(model: DocumentModel, parentId: string, title: string): string {
  const id = uid("n");
  applyOp(model.sheet, makeOp<Op & { type: "createNode" }>("createNode", { id, nodeType: "subtopic", parentId, index:
model.node(parentId)?.childrenIds.length, title }));
  return id;
}

describe("mindmap layout", () => {
  it("positions the root at the origin and alternates branch sides", () => {
    const model = makeMap();
    const sheet = model.sheet;
    const root = model.rootNode;
    const a = addChild(model, root.id, "Alpha");
    const b = addChild(model, root.id, "Beta");
    const c = addChild(model, root.id, "Gamma");

    const res = layoutSheet(sheet);
    const pRoot = res.positions.get(root.id!);
    const pA = res.positions.get(a!);
    const pB = res.positions.get(b!);
    const pC = res.positions.get(c!);

    // root centered at origin (top-left of root box at -w/2,-h/2)
    expect(pRoot.x).toBeCloseTo(-measureNode(root).w / 2, 5);
    expect(pRoot.y).toBeCloseTo(-measureNode(root).h / 2, 5);

    // autoBalance splits by subtree height: equal branches → 1 right, 2 left
    expect(pA.x).toBeGreaterThan(pRoot.x + measureNode(root).w);
    expect(pB.x).toBeLessThan(pRoot.x);
    expect(pC.x).toBeLessThan(pRoot.x);
  });

  it("never overlaps siblings on the same side", () => {
    const model = makeMap();
    const sheet = model.sheet;
    const root = model.rootNode;
    const ids: string[] = [];
    for (let i = 0; i < 8; i++) ids.push(addChild(model, root.id, `Topic ${i}`));
    for (const id of ids) {
      addChild(model, id, `${id}-c1`);
      addChild(model, id, `${id}-c2`);
    }
    const res = layoutSheet(sheet);
    const rootBox = measureNode(root);
    const sideOf = (id: string) => (res.positions.get(id)!.x > res.positions.get(root.id)!.x + rootBox.w ? 1 : -1);
    const bySide = (s: number) => ids.filter((id) => sideOf(id) === s);
    const checkNoVerticalOverlap = (group: string[]) => {
      const rects = group.map((id) => {
        const p = res.positions.get(id)!;
        const m = measureNode(model.node(id)!);
        return { y0: p.y, y1: p.y + m.h };
      });
      for (let i = 0; i < rects.length; i++) {
        for (let j = i + 1; j < rects.length; j++) {
          expect(rects[i].y0 < rects[j].y1 && rects[j].y0 < rects[i].y1).toBe(false);
        }
      }
    };
    checkNoVerticalOverlap(bySide(1));
    checkNoVerticalOverlap(bySide(-1));
  });

  it("collapsed nodes hide their subtree from layout", () => {
    const model = makeMap();
    const sheet = model.sheet;
    const root = model.rootNode;
    const big = addChild(model, root.id, "Big branch");

```

```

const hidden = [0, 1, 2, 3, 4].map((i) => addChild(model, big, `child-${i}`));
const other = addChild(model, root.id, "Other");
model.node(big)!.collapsed = true;

const res = layoutSheet(sheet);
// hidden children are not placed at all
for (const h of hidden) expect(res.positions.has(h)).toBe(false);
// and the collapsed branch's block is exactly its own height
const pBig = res.positions.get(big)!;
const hBig = measureNode(model.node(big)!).h;
const pOther = res.positions.get(other)!;
expect(Math.abs(pBig.y - pOther.y)).toBeLessThan(hBig + measureNode(model.node(other)!).h + 40);
});

it("applyLayout preserves manual positions", () => {
  const model = makeMap();
  const sheet = model.sheet;
  const floatingId = uid("n");
  applyOp(sheet, makeOp<Op & { type: "createNode" }>("createNode", { id: floatingId, nodeType: "floating", parentId: null, index: 0, title: "Free",
position: { x: 1234, y: 567, manual: true } }));
  applyLayout(sheet, false);
  const f = model.node(floatingId)!;
  expect(f.position.x).toBe(1234);
  expect(f.position.y).toBe(567);
});

it("includes manual main topics and separates automatic siblings around them", () => {
  const model = makeMap();
  const root = model.rootNode;
  const anchored = addChild(model, root.id, "Anchored");
  const automatic = addChild(model, root.id, "Automatic");
  model.node(anchored)!.position = { x: 160, y: 30, manual: true };

  const res = layoutSheet(model.sheet);
  expect(res.positions.has(anchored)).toBe(true);

  const a = res.positions.get(anchored)!;
  const b = res.positions.get(automatic)!;
  const ah = measureNode(model.node(anchored)!).h;
  expect(a.y + ah + model.sheet.structure.branchSpacing).toBeLessThanOrEqual(b.y + 0.001);
});

it("preserves major side assignment for manual root children when an auto subtopic is added", () => {
  const model = makeMap();
  const root = model.rootNode;
  const main1 = addChild(model, root.id, "Main 1");
  const main2 = addChild(model, root.id, "Main 2");
  const main3 = addChild(model, root.id, "Main 3");
  model.node(main2)!.position = { x: 240, y: 70, manual: true };
  model.node(main3)!.position = { x: -260, y: 80, manual: true };
  model.node(main2)!.style.fill = "#ff646b";
  model.node(main3)!.style.fill = "#4eb5e8";

  const before = layoutSheet(model.sheet);
  const side2Before = before.positions.get(main2)!.x > before.positions.get(root.id)!.x + measureNode(root).w ? 1 : -1;
  const side3Before = before.positions.get(main3)!.x > before.positions.get(root.id)!.x + measureNode(root).w ? 1 : -1;

  addChild(model, main1, "Subtopic 1");
  const after = layoutSheet(model.sheet);
  const side2After = after.positions.get(main2)!.x > after.positions.get(root.id)!.x + measureNode(root).w ? 1 : -1;
  const side3After = after.positions.get(main3)!.x > after.positions.get(root.id)!.x + measureNode(root).w ? 1 : -1;

  expect(side2After).toBe(side2Before);
  expect(side3After).toBe(side3Before);
  expect(model.node(main2)!.style.fill).toBe("#ff646b");
  expect(model.node(main3)!.style.fill).toBe("#4eb5e8");
});

it("keeps subtopic colors stable when a different main topic is moved", () => {
  const model = makeMap();
  const root = model.rootNode;
  const main1 = addChild(model, root.id, "Main 1");
  const main2 = addChild(model, root.id, "Main 2");
  const sub = addChild(model, main2, "Subtopic A");
  model.node(main2)!.style.fill = "#ff646b";
  model.node(sub)!.style.fill = "#ffe8dc";

  const before = model.node(sub)!.style.fill;
  model.node(main1)!.position = { x: 200, y: 0, manual: true };
  applyLayout(model.sheet, false);
  expect(model.node(sub)!.style.fill).toBe(before);
});

it("keeps a manually positioned branch fixed while laying out its descendants", () => {
  const model = makeMap();
  const root = model.rootNode;
  const branch = addChild(model, root.id, "Dragged branch");

```

```

const child = addChild(model, branch, "Child");
model.node(branch)!.position = {
  x: 240,
  y: 70,
  manual: true,
};
applyLayout(model.sheet, false);

const firstChildPosition = { ...model.node(child)!.position };
model.node(branch)!.title = "Dragged branch with a much longer title that changes its measured width";
applyLayout(model.sheet, false);

const branchPosition = model.node(branch)!.position;
const childPosition = model.node(child)!.position;
expect(branchPosition.x).toBe(240);
expect(branchPosition.y).toBe(70);
expect(childPosition.x).not.toBe(firstChildPosition.x);
});

it("moves auto subtopics relative to a manually pinned main topic", () => {
  const model = makeMap();
  const root = model.rootNode;
  const main = addChild(model, root.id, "Main branch");
  const child = addChild(model, main, "Subtopic");
  model.node(main)!.position = {
    x: 240,
    y: 70,
    manual: true,
  };
  applyLayout(model.sheet, false);

  const before = { ...model.node(child)!.position };
  model.node(main)!.position = {
    x: 340,
    y: 170,
    manual: true,
  };
  applyLayout(model.sheet, false);

  const after = model.node(child)!.position;
  expect(model.node(main)!.position.x).toBe(340);
  expect(model.node(main)!.position.y).toBe(170);
  expect(after.x - before.x).toBeCloseTo(100, 1);
  expect(after.y - before.y).toBeCloseTo(100, 1);
});

it("force layout also moves manual nodes", () => {
  const model = makeMap();
  const sheet = model.sheet;
  const root = model.rootNode;
  root.position = { x: 5000, y: 5000, manual: true };
  applyLayout(sheet, true);
  const r = model.rootNode;
  expect(r.position.x).not.toBe(5000);
});

it("children straddle the parent box and never overlap it", () => {
  const model = makeMap();
  const sheet = model.sheet;
  const root = model.rootNode;
  const a = addChild(model, root.id, "A");
  const b = addChild(model, root.id, "B");
  const c = addChild(model, root.id, "C");
  const res = layoutSheet(sheet);
  const gap = Math.max(6, sheet.structure.branchSpacing);
  const centerY = (id: string) => {
    const p = res.positions.get(id)!;
    return p.y + measureNode(model.node(id)!.h) / 2;
  };
  const yA = centerY(a);
  const yB = centerY(b);
  const yC = centerY(c);
  const centers = [yA, yB, yC];
  // balanced straddle: one child above the root, two below
  const above = centers.filter((y) => y < 0).length;
  const below = centers.filter((y) => y > 0).length;
  expect(above).toBe(1);
  expect(below).toBe(2);
  // no child box may intersect the root box [-rootH/2, +rootH/2] minus the gap
  for (const y of centers) {
    const nearestEdge = Math.abs(y) - measureNode(root).h / 2; // approx
    expect(nearestEdge).toBeGreaterThan(gap - 1);
  }
});
});

describe("size-aware balance", () => {

```

```

it("splits root children by subtree height, not raw count", () => {
  const model = makeMap();
  const sheet = model.sheet;
  const root = model.rootNode;
  // A is much taller than B, C, D
  const a = addChild(model, root.id, "A tall branch");
  for (let i = 0; i < 30; i++) addChild(model, a, `deep ${i} with a long wrapping title that spans lines`);
  const b = addChild(model, root.id, "B");
  const c = addChild(model, root.id, "C");
  const d = addChild(model, root.id, "D");

  const res = layoutSheet(sheet);
  const p = (id: string) => res.positions.get(id)!;
  const rootBox = measureNode(model.node(root.id)!);
  const isRight = (id: string) => p(id).x > p(root.id).x + rootBox.w;

  // A alone carries most of the height → it gets the right side to itself
  expect(isRight(a)).toBe(true);
  // B, C, D (small) balance the left side
  expect(isRight(b)).toBe(false);
  expect(isRight(c)).toBe(false);
  expect(isRight(d)).toBe(false);
});

it("equal-height branches split evenly", () => {
  const model = makeMap();
  const sheet = model.sheet;
  const root = model.rootNode;
  const ids = ["A", "B", "C", "D"].map((t) => addChild(model, root.id, t));
  const res = layoutSheet(sheet);
  const rootBox = measureNode(model.node(root.id)!);
  const right = ids.filter((id) => res.positions.get(id)!.x > res.positions.get(root.id)!.x + rootBox.w);
  expect(right.length).toBe(2);
});

describe("upward propagation", () => {
  it("a deep change grows the branch and redistributes siblings without overlap", () => {
    const model = makeMap();
    const sheet = model.sheet;
    const root = model.rootNode;
    const m1 = addChild(model, root.id, "M1");
    const m2 = addChild(model, root.id, "M2");
    const s1 = addChild(model, m1, "S1");
    const s2 = addChild(model, m1, "S2");

    const before = layoutSheet(sheet);
    expect(before.positions.has(m2)).toBe(true);
    const yBefore = before.positions.get(s2)!.y;

    // grow the S1 subtree (long wrapping titles make every node taller)
    for (let i = 0; i < 5; i++) addChild(model, s1, `deep ${i} ` + "word ".repeat(24));

    const after = layoutSheet(sheet);
    const yAfter = after.positions.get(s2)!.y;
    // the deep change propagated up: S2 moved (growth pushes the branch apart)
    expect(yAfter).not.toBe(yBefore);

    // and the S1 subtree never overlaps S2
    const s1BlockBottom = model.subtreeIds(s1).reduce((max, id) => {
      const p = after.positions.get(id);
      if (!p) return max;
      return Math.max(max, p.y + measureNode(model.node(id)!.h);
    }, -Infinity);
    expect(s1BlockBottom).toBeLessThanOrEqual(yAfter + 0.5);
  });
});

describe("structures", () => {
  it("tree layout stacks children horizontally below the parent", () => {
    const model = makeMap();
    const sheet = model.sheet;
    sheet.structure.structureType = "tree";
    sheet.structure.orientation = "vertical";
    const root = model.rootNode;
    const a = addChild(model, root.id, "A");
    const b = addChild(model, root.id, "B");
    const res = layoutSheet(sheet);
    const pA = res.positions.get(a)!;
    const pB = res.positions.get(b)!;
    const pRoot = res.positions.get(root.id)!;
    expect(pA.y).toBeGreaterThan(pRoot.y + measureNode(root).h); // below root
    expect(Math.abs(pA.y - pB.y)).toBeLessThan(2); // same row
    expect(pB.x).toBeGreaterThan(pA.x); // side by side
  });
});

```

## 22. tests/pasteSanitizer.test.ts

```
// @vitest-environment jsdom
import { describe, expect, it } from "vitest";
import { htmlToRuns, sanitizeHtml } from "../src/ui/pasteSanitizer";
import { runsToPlain } from "../src/core/text";

describe("sanitizeHtml – Word / Google Docs emphasis via CSS", () => {
  it("converts span font-weight:700 to <strong>", () => {
    const out = sanitizeHtml(`<span style="font-weight: 700">bold</span>`);
    expect(out).toContain("<strong>bold</strong>");
    expect(out).not.toContain("font-weight");
  });

  it("converts span font-style:italic to <em>", () => {
    const out = sanitizeHtml(`<span style="font-style: italic">slanted</span>`);
    expect(out).toContain("<em>slanted</em>");
    expect(out).not.toContain("font-style");
  });

  it("keeps both bold and italic on the same text", () => {
    const out = sanitizeHtml(`<span style="font-weight: bold; font-style: italic">combo</span>`);
    expect(out).toContain("<em>");
    expect(out).toContain("<strong>");
    expect(out).toContain("combo");
  });

  it("recognizes numeric bold weights >= 600 and ignores light weights", () => {
    expect(sanitizeHtml(`<span style="font-weight: 700">x</span>`)).toContain("<strong>x</strong>");
    expect(sanitizeHtml(`<span style="font-weight: 400">x</span>`)).not.toContain("<strong>");
  });
});

describe("sanitizeHtml – Word lists (MsolistParagraph / margin-left)", () => {
  it("rebuilds a flat Word list as <ul><li>", () => {
    const out = sanitizeHtml(
      `
```

```

});

it("handles Draw.io inline span styles (font-weight on span inside p)", () => {
  const out = sanitizeHtml(`

<span style="font-weight: bold; font-family: Arial;">draw io bold</span></p>`);
  expect(out).toContain("<strong>draw io bold</strong>");
  expect(out).not.toContain("font-family");
});
});

describe("sanitizeHtml - colors", () => {
  it("keeps hex and rgb colors as data-rnode-color spans", () => {
    const hex = sanitizeHtml(`<strong>B</strong><em>I</em><u>U</u> plain</p>`);
    const find = (s: string) => runs.find((r) => r.text.includes(s))!;
    expect(find("B").bold).toBe(true);
    expect(find("I").italic).toBe(true);
    expect(find("U").underline).toBe(true);
    expect(find("plain").bold ?? false).toBe(false);
  });

  it("reads data-rnode-color markers into run colors", () => {
    const runs = htmlToRuns(`

<span data-rnode-color="#ff0000">red text</span></p>`);
    const run = runs.find((r) => r.text.includes("red text"));
    expect(run?.color).toBe("#ff0000");
  });

  it("marks nested list items with listIndent instead of literal spaces", () => {
    const runs = htmlToRuns(`

<li>one<ul><li>one.one</li></ul></li></ul>`);
    const one = runs.find((r) => r.text.includes("one"))!;
    const nested = runs.find((r) => r.text.includes("one.one"))!;
    expect(one.listIndent).toBe(1);
    expect(nested.listIndent).toBe(2);
    // no literal space-indentation in the text anymore - the renderer draws
    // the bullet glyph and hanging indent from listIndent
    expect(one.text).toBe("one");
    expect(nested.text).toBe("one.one");
  });

  it("preserves heading levels as per-run font sizes", () => {
    const runs = htmlToRuns(`

##


```

## 23. tests/perf.test.ts

```

/**
 * Performance spike (Phase 1 validation).
 *
 * Generates balanced trees of 1k / 5k / 10k topics through the real op
 * system and measures the hot paths the editor hits on every edit:

```

```

* - op application (applyWithInverse)
* - layout (layoutSheet) and layout write-back (applyLayout)
* - tree walks (subtreeIds / visibleIds)
*
* Bounds are deliberately generous – this file exists to catch algorithmic
* blowups ( $O(n^2)$  insertions, layout regressions), not micro-timings. Run with
* `npx vitest run tests/perf.test.ts`.
*/
import { describe, expect, it } from "vitest";
import { DocumentModel, uid } from "../src/core/doc";
import { applyWithInverse, makeOp, type Op } from "../src/core/ops";
import { applyLayout, layoutSheet } from "../src/layout/mindmap";

function generate(model: DocumentModel, count: number): Op[] {
  const sheet = model.sheet;
  const rootId = sheet.rootNodeId;
  const ops: Op[] = [];
  const queue: string[] = [rootId];
  const BRANCH = 8;
  let created = 0;
  while (created < count && queue.length > 0) {
    const parentId = queue.shift()!;
    const isRoot = parentId === rootId;
    for (let i = 0; i < BRANCH && created < count; i++) {
      const id = uid("n");
      ops.push(
        makeOp<Op> & { type: "createNode" } > { "createNode", {
          id,
          nodeType: isRoot ? "main" : "subtopic",
          parentId,
          index: 0, // branch factor is 8, so sibling arrays stay tiny –  $O(1)$  inserts
          title: `Topic ${created}`,
        }}
      );
      queue.push(id);
      created++;
    }
  }
  return ops;
}

function time<T>(_label: string, fn: () => T): { result: T; ms: number } {
  const t0 = performance.now();
  const result = fn();
  return { result, ms: performance.now() - t0 };
}

describe("perf spike", () => {
  const sizes = [1_000, 5_000, 10_000];

  for (const size of sizes) {
    it(`handles ${size.toLocaleString()} topics through the op system`, { timeout: 60_000 }, () => {
      const model = new DocumentModel(DocumentModel.blank("Perf"));
      const rootId = model.sheet.rootNodeId;
      const ops = generate(model, size);
      expect(ops.length).toBe(size);
      const apply = time("apply", () => {
        for (const op of ops) applyWithInverse(model.sheet, op);
      });
      expect(model.sheet.nodes).toHaveProperty(rootId);
      expect(Object.keys(model.sheet.nodes).length).toBe(size + 1);

      const layout = time("layoutSheet", () => layoutSheet(model.sheet));
      expect(layout.result.positions.size).toBeGreaterThanOrEqual(size * 0.9);

      const writeback = time("applyLayout", () => applyLayout(model.sheet, false));

      const walks = time("tree walks", () => {
        const a = model.subtreeIds(rootId).length;
        const b = model.visibleIds(rootId).length;
        return { a, b };
      });

      // report
      console.log(
        `[perf ${size.toLocaleString()}] applyOps=${apply.ms.toFixed(1)}ms ` +
        `layout=${layout.ms.toFixed(1)}ms writeback=${writeback.ms.toFixed(1)}ms walks=${walks.ms.toFixed(1)}ms ` +
        `(${(apply.ms / size).toFixed(4)}ms/op)`
      );

      // sanity ceilings – generous, catch blowups only
      expect(apply.ms).toBeLessThan(10_000);
      expect(layout.ms).toBeLessThan(5_000);
      expect(writeback.ms).toBeLessThan(5_000);
      expect(walks.result.a).toBe(size + 1);
      expect(walks.result.b).toBe(size + 1);
    });
  }
});

```

```
}  
});
```

## 24. tests/viewport.test.ts

```
import { describe, expect, it } from "vitest";  
import { screenToWorld, worldToScreen, zoomAt } from "../src/render/viewport";  
  
describe("viewport", () => {  
  it("keeps the world point under the cursor fixed while zooming", () => {  
    const before = { x: 0, y: 0, scale: 1 };  
    const sx = 800;  
    const sy = 250;  
    const world = screenToWorld(before, 1000, 800, sx, sy);  
    const after = zoomAt(before, 1000, 800, sx, sy, 2);  
  
    expect(worldToScreen(after, 1000, 800, world.x, world.y).x).toBeCloseTo(sx, 8);  
    expect(worldToScreen(after, 1000, 800, world.x, world.y).y).toBeCloseTo(sy, 8);  
  });  
  
  it("still anchors correctly when zoom is clamped", () => {  
    const before = { x: 100, y: -40, scale: 4 };  
    const sx = 100;  
    const sy = 700;  
    const world = screenToWorld(before, 1000, 800, sx, sy);  
    const after = zoomAt(before, 1000, 800, sx, sy, 2);  
  
    expect(after.scale).toBe(4);  
    expect(worldToScreen(after, 1000, 800, world.x, world.y).x).toBeCloseTo(sx, 8);  
    expect(worldToScreen(after, 1000, 800, world.x, world.y).y).toBeCloseTo(sy, 8);  
  });  
});
```

## 25. CLAUDE.md

# R-node

Visual thinking workspace: mind maps on a single Canvas2D, keyboard-first, local-first. React + TypeScript + Vite; the engine (`src/core`, `src/layout`, `src/render`) is framework-free.

## Read before changing anything

\*\*[docs/AGENT\_GUIDE.md](docs/AGENT\_GUIDE.md) is the contract.\*\* It holds the non-negotiable invariants, the editor+canvas parity rules, the traps already paid for, and the definition of done. A change that violates it is wrong even if it compiles and the tests pass.

\*\*[docs/ROADMAP.md](docs/ROADMAP.md)\*\* is the ordered task list. Take one task at a time, in order, and do not widen its scope.

Reference: [docs/ARCHITECTURE.md](docs/ARCHITECTURE.md) (structure, phases) · [docs/RICH\_TEXT\_EDITOR.md](docs/RICH\_TEXT\_EDITOR.md) (rich text in detail).

## Commands

```
```bash  
npm test  
```
```

```
```bash  
npm run typecheck  
```
```

```
```bash  
npm run dev  
```
```

App at `http://localhost:5173/`, text parity harness at  
`http://localhost:5173/dev/parity.html` (must report 0 diverging).

## The three things people get wrong here

1. \*\*Text parity is measured, not assumed.\*\* Touching `src/layout/measure.ts`, the text path of `src/render/renderer.ts`, `src/ui/lexicalRuns.ts`, the `.topic-rich-` CSS or `src/ui/RichEditor.tsx` means re-running the harness.
2. \*\*Undo/redo lives only in the store.\*\* Lexical has no HistoryPlugin, and layout is derived data that never enters an op or the history.
3. \*\*Several "obvious improvements" are already implemented\*\* – the text measurer cache and atomic history batches among them, and there is no

```
autosave timer to fix. See AGENT_GUIDE.md §5 before acting on any
improvement list.
```

## 26. tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "useDefineForClassFields": true,
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "isolatedModules": true,
    "moduleDetection": "force",
    "noEmit": true,
    "jsx": "react-jsx",
    "esModuleInterop": true,
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src", "tests"]
}
```

## 27. vite.config.ts

```
import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    strictPort: true,
  },
  build: {
    target: "es2022",
    sourcemap: true,
  },
  test: {
    environment: "node",
    include: ["tests/**/*.test.ts"],
  },
});
```

## 28. src/core/doc.ts

```
/**
 * DocumentModel – read-side helpers over the schema.
 *
 * This class is pure data logic: no rendering, no I/O. It is the seam where
 * the Rust document engine will later plug in (same schema, same invariants).
 */
import {
  DEFAULT_STRUCTURE,
  SCHEMA_VERSION,
  type MindNode,
  type NodeType,
  type RnodeDocument,
  type Sheet,
  type Style,
  type TaskInfo,
} from "../types";

export function uid(prefix = "n"): string {
  return `${prefix}_${Date.now().toString(36)}${Math.random().toString(36).slice(2, 8)}`;
}

export function nowIso(): string {
  return new Date().toISOString();
}

export class DocumentModel {
```

```

readonly doc: RnodeDocument;

constructor(doc: RnodeDocument) {
  this.doc = doc;
  if (doc.sheets.length === 0) throw new Error("document must contain at least one sheet");
}

// -- sheet -----

get sheet(): Sheet {
  return this.doc.sheets[0];
}

get rootNode(): MindNode {
  const n = this.sheet.nodes[this.sheet.rootNodeId];
  if (!n) throw new Error("sheet root node missing");
  return n;
}

node(id: string): MindNode | undefined {
  return this.sheet.nodes[id];
}

requireNode(id: string): MindNode {
  const n = this.node(id);
  if (!n) throw new Error(`missing node ${id}`);
  return n;
}

// -- tree walks -----

/** All node ids in the subtree rooted at id, including id (BFS). */
subtreeIds(id: string): string[] {
  const out: string[] = [];
  const queue = [id];
  const seen = new Set<string>();
  while (queue.length > 0) {
    const cur = queue.shift()!;
    if (seen.has(cur)) continue;
    seen.add(cur);
    out.push(cur);
    const n = this.node(cur);
    if (n) queue.push(...n.childrenIds);
  }
  return out;
}

/** Nodes that are actually visible (not hidden by a collapsed ancestor). */
visibleIds(rootId: string): string[] {
  const out: string[] = [];
  const visit = (id: string): void => {
    out.push(id);
    const n = this.node(id);
    if (!n || n.collapsed) return;
    for (const c of n.childrenIds) visit(c);
  };
  visit(rootId);
  return out;
}

depth(id: string): number {
  let d = 0;
  let cur = this.node(id)?.parentId ?? null;
  while (cur) {
    d++;
    cur = this.node(cur)?.parentId ?? null;
  }
  return d;
}

/** 1-based index of `id` among its parent's children (-1 if root/absent). */
siblingIndex(id: string): number {
  const n = this.node(id);
  if (!n?.parentId) return -1;
  const parent = this.node(n.parentId);
  if (!parent) return -1;
  return parent.childrenIds.indexOf(id);
}

/** Id of the first "main branch" ancestor (the child of the root), for coloring. */
branchRootId(id: string): string {
  const rootId = this.sheet.rootNodeId;
  let cur: string | null = id;
  let prev: string | null = null;
  while (cur && cur !== rootId) {
    prev = cur;
    cur = this.node(cur)?.parentId ?? null;
  }

```

```

    }
    return prev ?? id;
}

get visibleNodeCount(): number {
    return this.visibleIds(this.sheet.rootNodeId).length;
}

// -- factories -----

static blank(title = "Untitled map"): RnodeDocument {
    const central: MindNode = DocumentModel.makeNode("central", null, "Central topic", {
        fontSize: 22,
        fontWeight: 700,
    });
    const sheetId = uid("s");
    const sheet: Sheet = {
        sheetId,
        title: "Map 1",
        structure: { ...DEFAULT_STRUCTURE },
        rootNodeId: central.id,
        nodes: { [central.id]: central },
        relationships: [],
        boundaries: [],
        summaries: [],
        callouts: [],
        labels: [],
        zones: [],
        attachments: [],
        comments: [],
        presentation: {},
    };
    return {
        schemaVersion: SCHEMA_VERSION,
        documentId: uid("d"),
        title,
        createdAt: nowIso(),
        updatedAt: nowIso(),
        archived: false,
        pinned: false,
        settings: { theme: "light", showOutliner: false, showInspector: true },
        themeId: "r-node-light",
        sheets: [sheet],
    };
}

/** Demo document so the first launch is not a blank canvas. */
static sample(): RnodeDocument {
    const doc = DocumentModel.blank("R-node - Roadmap");
    doc.settings.theme = "light";
    const sheet = doc.sheets[0];
    const root = sheet.nodes[sheet.rootNodeId];
    root.title = "R-node";

    const add = (parentId: string, title: string, type: NodeType = "main", task?: TaskInfo, style?: Style): MindNode => {
        const n = DocumentModel.makeNode(type, parentId, title, style);
        n.task = task ?? null;
        sheet.nodes[n.id] = n;
        sheet.nodes[parentId].childrenIds.push(n.id);
        return n;
    };

    const core = add(root.id, "Core editor", "main", undefined, { fill: "#ff646b" });
    add(core.id, "Document model + ops", "subtopic");
    add(core.id, "Undo / redo", "subtopic");
    add(core.id, "Canvas renderer", "subtopic", { status: "in-progress", priority: "high", progress: 60 });

    const structures = add(root.id, "Structures", "main", undefined, { fill: "#ff9a66" });
    add(structures.id, "Mind map", "subtopic", { status: "completed", priority: "high", progress: 100 });
    add(structures.id, "Logic chart", "subtopic");
    add(structures.id, "Tree / org chart", "subtopic");
    add(structures.id, "Timeline", "subtopic");

    const productivity = add(root.id, "Productivity", "main", undefined, { fill: "#4eb5e8" });
    add(productivity.id, "Outliner", "subtopic");
    add(productivity.id, "Tasks + Gantt", "subtopic");
    add(productivity.id, "Search", "subtopic");

    const collaboration = add(root.id, "Collaboration", "main", undefined, { fill: "#55c9bd" });
    add(collaboration.id, "CRDT sync", "subtopic");
    add(collaboration.id, "Presence", "subtopic");
    add(collaboration.id, "Permissions", "subtopic");

    const exportLayer = add(root.id, "Export", "main", undefined, { fill: "#a7d9bb" });
    add(exportLayer.id, "PNG / SVG / PDF", "subtopic");
    add(exportLayer.id, "Markdown / OPML", "subtopic");

```

```

const ai = add(root.id, "AI (later)", "main", undefined, { fill: "#d979e5" });
add(ai.id, "Map from prompt", "subtopic");
add(ai.id, "Summarize branch", "subtopic");

return doc;
}

static makeNode(type: NodeType, parentId: string | null, title: string, style?: Style): MindNode {
  const t = nowIso();
  const typeStyle: Style = type === "central"
    ? { fontSize: 22, fontWeight: 700, padding: 4, shape: "none" }
    : type === "main"
      ? { fontSize: 14, fontWeight: 600, padding: 9, cornerRadius: 8, shape: "rounded" }
      : type === "subtopic"
        ? { fontSize: 12, fontWeight: 400, padding: 7, cornerRadius: 8, shape: "rounded", align: "left" }
        : { fontSize: 14, fontWeight: 400, padding: 9, cornerRadius: 8, shape: "rounded" };
  return {
    id: uid("n"),
    type,
    parentId,
    childrenIds: [],
    title,
    position: { x: 0, y: 0, manual: false },
    style: { ...typeStyle, ...(style ?? {}) },
    collapsed: false,
    labels: [],
    markers: [],
    notes: "",
    task: null,
    metadata: { createdAt: t, updatedAt: t },
  };
}
}

```

## 29. src/layout/mindmap.ts

```

/**
 * Layout engine (pure, framework-free).
 *
 * Observable layout behavior (see docs/ARCHITECTURE.md):
 * 1. every topic has an extent (text + padding + shape) – measured with the
 *    SAME measurer the renderer uses;
 * 2. a topic + its descendants form a block;
 * 3. children STRADDLE the parent box: the block is split so some children
 *    sit above and some below the parent, always with a guaranteed gap –
 *    no child can ever overlap its parent, and growing a topic's own title
 *    pushes its neighbors apart;
 * 4. any deep change grows the block, which redistributes siblings and
 *    propagates up to the root – a full recompute on change.
 *
 * Auto positions are returned as a Map and written back by applyLayout, which
 * never overwrites manual positions unless forced.
 *
 * Strategies:
 * - mindmap: root centered; root children split LEFT/RIGHT by total subtree
 *   height (autoBalance), each side straddles the root box vertically;
 *   every branch keeps its side.
 * - logic / tree / org / timeline: children stack right (horizontal) or
 *   below (vertical).
 * - freeform: nothing – manual positions only.
 */
import type { MindNode, Sheet } from "../core/types";
import { imageResolver, measureNode, type TextMeasurer } from "../measure";

export { imageResolver, measureNode } from "../measure";
export type { Extent, TextMeasurer } from "../measure";

export interface LayoutResult {
  positions: Map<string, { x: number; y: number }>;
  bounds: { minX: number; minY: number; maxX: number; maxY: number };
}

export function layoutSheet(sheet: Sheet, force = false, measurer?: TextMeasurer): LayoutResult {
  const positions = new Map<string, { x: number; y: number }>();
  const st = sheet.structure;
  const root = sheet.nodes[sheet.rootNodeId];
  if (!root) return { positions, bounds: { minX: 0, minY: 0, maxX: 0, maxY: 0 } };

  // Nodes with images must be measured with the sheet's attachment cards or
  // the layout would place them at text-only size (invariant I9).
  const resolveImage = imageResolver(sheet);
  const size = (n: MindNode): { w: number; h: number } => measureNode(n, measurer, resolveImage);
  const gap = Math.max(6, st.branchSpacing);

```

```

const isAuto = (id: string): boolean => {
  const n = sheet.nodes[id];
  return !!n && (force || !n.position.manual);
};

const cacheH = new Map<string, number>();
const cacheW = new Map<string, number>();

/**
 * Vertical placement of `kids` around a parent box of height `parentH`.
 * Children are split into an above group and a below group (balanced by
 * total height) separated from the parent by `gap`. Returns each child's
 * center Y relative to the parent center, plus the total block height.
 */
const stacked = (kids: MindNode[], parentH: number): { centers: Map<string, number>; blockH: number } => {
  const k = kids.length;
  const blocks = kids.map((kid) => subtreeHeight(kid.id));
  const groupTotal = (from: number, to: number): number => {
    if (from >= to) return 0;
    let t = 0;
    for (let i = from; i < to; i++) t += blocks[i];
    return t + st.branchSpacing * (to - from - 1);
  };
  // choose the above-count that balances the two groups (first minimal diff
  // → children lean below the parent, Xmind-style)
  let best = 0;
  let bestDiff = Infinity;
  for (let ac = 0; ac <= k; ac++) {
    const diff = Math.abs(groupTotal(0, ac) - groupTotal(ac, k));
    if (diff < bestDiff) {
      bestDiff = diff;
      best = ac;
    }
  }
  const aboveTotal = groupTotal(0, best);
  const belowTotal = groupTotal(best, k);
  const blockH = aboveTotal + gap + parentH + gap + belowTotal;

  const centers = new Map<string, number>();
  let y = -parentH / 2 - gap - aboveTotal; // top of the above group
  for (let i = 0; i < best; i++) {
    centers.set(kids[i].id, y + blocks[i] / 2);
    y += blocks[i] + st.branchSpacing;
  }
  y = parentH / 2 + gap; // top of the below group
  for (let i = best; i < k; i++) {
    centers.set(kids[i].id, y + blocks[i] / 2);
    y += blocks[i] + st.branchSpacing;
  }
  return { centers, blockH };
};

const subtreeHeight = (id: string): number => {
  const cached = cacheH.get(id);
  if (cached !== undefined) return cached;
  const n = sheet.nodes[id];
  if (!n) return 0;
  const h = size(n).h;
  if (n.collapsed || n.childrenIds.length === 0) {
    cacheH.set(id, h);
    return h;
  }
  const kids = n.childrenIds.map((c) => sheet.nodes[c]).filter((c): c is MindNode => !!c);
  // In a mind map, descendants of a non-central topic run in a vertical
  // column beside that topic. The topic is therefore centered against the
  // child column, not surrounded by the root-style straddle gaps. Using
  // `stacked()` here overestimates every branch and makes the placement
  // code's actual geometry disagree with the block height used for balance.
  const childColumnH = kids.reduce((acc, kid) => acc + subtreeHeight(kid.id), 0) + st.branchSpacing * Math.max(0, kids.length - 1);
  const blockH = st.structureType === "mindmap" ? Math.max(h, childColumnH) : stacked(kids, h).blockH;
  cacheH.set(id, blockH);
  return blockH;
};

const subtreeWidth = (id: string): number => {
  const cached = cacheW.get(id);
  if (cached !== undefined) return cached;
  const n = sheet.nodes[id];
  if (!n) return 0;
  const w = size(n).w;
  if (n.collapsed || n.childrenIds.length === 0) {
    cacheW.set(id, w);
    return w;
  }
  const kids = n.childrenIds.map((c) => sheet.nodes[c]).filter((c): c is MindNode => !!c);
  const childWidths = kids.map((kid) => subtreeWidth(kid.id));
  const blockW = Math.max(w, childWidths.reduce((acc, v) => acc + v, 0) + st.branchSpacing * Math.max(0, kids.length - 1));

```

```

    cacheW.set(id, blockW);
    return blockW;
};

if (st.structureType === "freeform") {
  for (const n of Object.values(sheet.nodes)) {
    if (force || n.position.manual) positions.set(n.id, { x: n.position.x, y: n.position.y });
  }
  return { positions, bounds: boundsOf(positions, sheet, size) };
}

if (st.structureType === "mindmap") {
  const rootPos = isAuto(root.id)
    ? { x: 0, y: 0 }
    : { x: root.position.x + size(root).w / 2, y: root.position.y + size(root).h / 2 };
  placeMindmap(root.id, rootPos.x, rootPos.y, 0);
} else {
  placeHierarchical(root.id, 0, 0, st.orientation === "horizontal" ? "right" : "down");
}

if (st.structureType === "mindmap") resolveIntersections(root.id);

return { positions, bounds: boundsOf(positions, sheet, size) };

// -- recursive placers -----

function placeMindmap(id: string, cx: number, cy: number, side: -1 | 0 | 1): void {
  const n = sheet.nodes[id];
  if (!n) return;
  const { w, h } = size(n);
  positions.set(id, isAuto(id) ? { x: cx - w / 2, y: cy - h / 2 } : { x: n.position.x, y: n.position.y });

  if (n.collapsed || n.childrenIds.length === 0) return;
  const kids = n.childrenIds.map((c) => sheet.nodes[c]).filter((c): c is MindNode => !!c);

  if (side === 0) {
    const manualKids = kids.filter((k) => !isAuto(k.id));
    const autoKids = kids.filter((k) => isAuto(k.id));

    const childSide = (kid: MindNode): -1 | 1 => {
      const existing = positions.get(kid.id);
      const p = existing ?? { x: kid.position.x, y: kid.position.y };
      return p.x + size(kid).w / 2 >= cx ? 1 : -1;
    };

    for (const kid of manualKids) {
      const { w: kw, h: kh } = size(kid);
      placeMindmap(kid.id, kid.position.x + kw / 2, kid.position.y + kh / 2, childSide(kid));
    }

    if (autoKids.length === 0) return;

    let rightGroup: MindNode[];
    let leftGroup: MindNode[];
    if (manualKids.length > 0) {
      rightGroup = autoKids.filter((k) => childSide(k) === 1);
      leftGroup = autoKids.filter((k) => childSide(k) === -1);
      if (rightGroup.length + leftGroup.length !== autoKids.length) {
        rightGroup = autoKids;
        leftGroup = [];
      }
    } else {
      let rightCount = autoKids.length;
      if (st.autoBalance && autoKids.length > 1) {
        const heights = autoKids.map((k) => subtreeHeight(k.id));
        const total = heights.reduce((a, b) => a + b, 0);
        let acc = 0;
        let bestDiff = Infinity;
        let best = 1;
        for (let i = 0; i < autoKids.length - 1; i++) {
          acc += heights[i];
          const diff = Math.abs(total - 2 * acc);
          if (diff < bestDiff) {
            bestDiff = diff;
            best = i + 1;
          }
        }
        rightCount = best;
      }
      rightGroup = autoKids.slice(0, rightCount);
      leftGroup = autoKids.slice(rightCount);
    }
    for (const group of [rightGroup, leftGroup]) {
      if (group.length === 0) continue;
      const sideSign: -1 | 1 = group === rightGroup ? 1 : -1;
      const { centers } = stacked(group, h);
      for (const kid of group) {

```

```

    placeMindmap(kid.id, cx + sideSign * (w / 2 + st.spacing + size(kid).w / 2), cy + centers.get(kid.id)!, sideSign);
  }
} else {
  // XMind behavior: for non-root branches, children are laid out in a
  // sequential top-to-bottom column centered on the parent's center.
  // Also, if the parent has a manual position, use its actual coordinates
  // so dragged parents correctly anchor their children.
  const parentNode = sheet.nodes[id];
  const parentSize = { w, h };
  const currentCx = !isAuto(id) && parentNode ? parentNode.position.x + parentSize.w / 2 : cx;
  const currentCy = !isAuto(id) && parentNode ? parentNode.position.y + parentSize.h / 2 : cy;

  const totalKidsH = kids.reduce((acc, kid) => acc + subtreeHeight(kid.id), 0) + st.branchSpacing * Math.max(0, kids.length - 1);
  let yCursor = currentCy - totalKidsH / 2;
  for (const kid of kids) {
    const kh = subtreeHeight(kid.id);
    const ke = size(kid);
    const kidCy = yCursor + kh / 2;
    placeMindmap(kid.id, currentCx + side * (w / 2 + st.spacing + ke.w / 2), kidCy, side);
    yCursor += kh + st.branchSpacing;
  }
}
}

function placeHierarchical(id: string, cx: number, cy: number, dir: "right" | "down"): void {
  const n = sheet.nodes[id];
  if (!n) return;
  const { w, h } = size(n);
  positions.set(id, isAuto(id) ? { x: cx - w / 2, y: cy - h / 2 } : { x: n.position.x, y: n.position.y });

  if (n.collapsed || n.childrenIds.length === 0) return;
  const kids = n.childrenIds.map((c) => sheet.nodes[c]).filter((c): c is MindNode => !!c);

  if (dir === "right") {
    const { centers } = stacked(kids, h);
    for (const kid of kids) {
      placeHierarchical(kid.id, cx + w / 2 + st.spacing + size(kid).w / 2, cy + centers.get(kid.id)!, dir);
    }
  } else {
    const widths = kids.map((kid) => subtreeWidth(kid.id));
    const totalW = widths.reduce((acc, v) => acc + v, 0) + st.branchSpacing * Math.max(0, kids.length - 1);
    let cursor = cx - totalW / 2;
    for (let i = 0; i < kids.length; i++) {
      const kid = kids[i];
      const sw = widths[i];
      placeHierarchical(kid.id, cursor + sw / 2, cy + h / 2 + st.spacing + size(kid).h / 2, dir);
      cursor += sw + st.branchSpacing;
    }
  }
}

/** Resolve vertical collisions without moving manually positioned anchors. */
function resolveIntersections(rootId: string): void {
  const branchBounds = (id: string): { minY: number; maxY: number } => {
    let minY = Infinity;
    let maxY = -Infinity;
    const visit = (nodeId: string): void => {
      const node = sheet.nodes[nodeId];
      if (!node) return;
      const p = positions.get(nodeId) ?? { x: node.position.x, y: node.position.y };
      const h = size(node).h;
      minY = Math.min(minY, p.y);
      maxY = Math.max(maxY, p.y + h);
      if (!node.collapsed) for (const childId of node.childrenIds) visit(childId);
    };
    visit(id);
    return { minY, maxY };
  };

  const shiftAutoSubtree = (id: string, dy: number): void => {
    const node = sheet.nodes[id];
    if (!node) return;
    if (isAuto(id)) {
      const p = positions.get(id) ?? { x: node.position.x, y: node.position.y };
      positions.set(id, { x: p.x, y: p.y + dy });
    }
    if (!node.collapsed) for (const childId of node.childrenIds) shiftAutoSubtree(childId, dy);
  };

  const resolveNode = (parentId: string): void => {
    const parent = sheet.nodes[parentId];
    if (!parent || parent.collapsed) return;
    for (const childId of parent.childrenIds) resolveNode(childId);
    const children = parent.childrenIds.map((id) => sheet.nodes[id]).filter((n): n is MindNode => !!n && positions.has(n.id));
    if (children.length < 2) return;

```

```

const parentPosition = positions.get(parentId) ?? { x: parent.position.x, y: parent.position.y };
const parentCenterX = parentPosition.x + size(parent).w / 2;
const parentCenterY = parentPosition.y + size(parent).h / 2;
const groups: MindNode[][] = st.structureType === "mindmap" && parentId === rootId
  ? [
    children.filter((child) => {
      const p = positions.get(child.id)!;
      return p.x + size(child).w / 2 < parentCenterX;
    }),
    children.filter((child) => {
      const p = positions.get(child.id)!;
      return p.x + size(child).w / 2 >= parentCenterX;
    }),
  ]
  : [children];

for (const group of groups) {
  for (let pass = 0; pass < group.length * 3; pass++) {
    group.sort((a, b) => branchBounds(a.id).minY - branchBounds(b.id).minY);
    let changed = false;
    for (let i = 1; i < group.length; i++) {
      const previous = group[i - 1];
      const current = group[i];
      const previousBounds = branchBounds(previous.id);
      const currentBounds = branchBounds(current.id);
      const overlap = previousBounds.maxY + gap - currentBounds.minY;
      if (overlap <= 0) continue;
      const currentCenter = (currentBounds.minY + currentBounds.maxY) / 2;
      if (isAuto(current.id)) {
        shiftAutoSubtree(current.id, currentCenter < parentCenterY ? -overlap : overlap);
        changed = true;
      } else if (isAuto(previous.id)) {
        const previousCenter = (previousBounds.minY + previousBounds.maxY) / 2;
        shiftAutoSubtree(previous.id, previousCenter < parentCenterY ? -overlap : overlap);
        changed = true;
      }
    }
    if (!changed) break;
  }
}
};
resolveNode(rootId);
}
}

function boundsOf(
  positions: Map<string, { x: number; y: number }>,
  sheet: Sheet,
  size: (n: MindNode) => { w: number; h: number }
): { minX: number; minY: number; maxX: number; maxY: number } {
  let minX = Infinity, minY = Infinity, maxX = -Infinity, maxY = -Infinity;
  for (const [id, p] of positions.entries()) {
    const n = sheet.nodes[id];
    if (!n) continue;
    const { w, h } = size(n);
    minX = Math.min(minX, p.x);
    minY = Math.min(minY, p.y);
    maxX = Math.max(maxX, p.x + w);
    maxY = Math.max(maxY, p.y + h);
  }
  if (!isFinite(minX)) return { minX: 0, minY: 0, maxX: 0, maxY: 0 };
  return { minX, minY, maxX, maxY };
}

/** Apply a layout result onto the sheet (auto nodes only, or all if force). */
export function applyLayout(sheet: Sheet, force: boolean, measurer?: TextMeasurer, clearManual = false): void {
  const result = layoutSheet(sheet, force, measurer);
  for (const [id, p] of result.positions) {
    const n = sheet.nodes[id];
    if (!n) continue;
    if (clearManual || force || !n.position.manual) {
      n.position = {
        x: p.x,
        y: p.y,
        manual: clearManual ? false : n.position.manual,
        ...(clearManual && n.position.offsetX !== undefined ? { offsetX: n.position.offsetX } : {}),
        ...(clearManual && n.position.offsetY !== undefined ? { offsetY: n.position.offsetY } : {}),
      };
    }
  }
}
}

```

### 30. src/render/theme.ts

```
/**
 * Design tokens for the canvas renderer.
 * The same palette drives the CSS chrome (see styles.css) so the light
 * theme stays coherent everywhere. (Dark theme was intentionally removed.)
 */
export type ThemeName = "light";

export interface RenderTheme {
  name: ThemeName;
  background: string;
  grid: string;
  text: string;
  textMuted: string;
  selection: string;
  selectionFill: string;
  dropIndicator: string;
  rootFill: string;
  rootText: string;
  branch: string[]; // branch palette, rotated by main-branch index
  branchSoft: string[]; // lighter fills for descendants of each main branch
  connector: string;
  collapsedBadge: string;
  collapsedBadgeText: string;
  nodeBorder: string;
  shadow: string;
}

export const THEMES: Record<ThemeName, RenderTheme> = {
  light: {
    name: "light",
    background: "ffffff",
    grid: "#f5f7fa",
    text: "#111827",
    textMuted: "#6b7280",
    selection: "#4f46e5",
    selectionFill: "rgba(79, 70, 229, 0.10)",
    dropIndicator: "#4f46e5",
    rootFill: "ffffff",
    rootText: "#111827",
    branch: ["#ff646b", "#ff9a66", "#4eb5e8", "#55c9bd", "#a7d9bb", "#d979e5", "#70b9e8", "#f0bd62"],
    branchSoft: ["#ffdfef", "#ffe8dc", "#dff4ff", "#dff8f5", "#e5f6ec", "#f6e1f9", "#deeffb", "#fff0d3"],
    connector: "#9aa3b2",
    collapsedBadge: "#e5e7eb",
    collapsedBadgeText: "#374151",
    nodeBorder: "rgba(0,0,0,0.06)",
    shadow: "rgba(28,35,51,0.14)",
  },
};
// Dark theme intentionally removed – the app is light-only.
```

### 31. src/ui/Outliner.tsx

```
import { useState } from "react";
import { useSyncExternalStore } from "react";
import { useStore } from "../editor/context";
import type { MindNode } from "../core/types";
import { makeOp, type Op } from "../core/ops";
import { plainToRuns } from "../core/text";

export function Outliner(): JSX.Element {
  const store = useStore();
  const root = store.doc.rootNode;

  return (
    <div className="outliner">
      <div className="outliner-header">
        <span>Outline</span>
        <button className="btn small" onClick={() => store.toast("Import from Markdown lands in Phase 4")}>
          Import
        </button>
      </div>
      <div className="outliner-body">
        <OutlinerRow node={root} depth={0} />
      </div>
    </div>
  );
}

function OutlinerRow({ node, depth }: { node: MindNode; depth: number }): JSX.Element {
  const store = useStore();
```

```

const state = useSyncExternalStore(store.subscribe, store.getSnapshot);
const [text, setText] = useState(node.title);
const selected = state.selection.includes(node.id);
const hasChildren = node.childrenIds.length > 0;

return (
  <div className="outliner-node">
    <div
      className={`outliner-row${selected ? " selected" : ""}`}
      style={{ paddingLeft: 10 + depth * 18 }}
      onClick={() => store.select(node.id, { center: true })}
      onDoubleClick={() => store.startEdit(node.id)}
    >
      <button
        className={`outliner-caret${node.collapsed ? " collapsed" : ""}`}
        disabled={!hasChildren}
        onClick={(e) => { e.stopPropagation(); if (hasChildren) store.toggleCollapsed(node.id); }}
      >
        {hasChildren ? "▼" : "▶"}
      </button>
      {node.task ? (
        <button
          className="outliner-check"
          onClick={(e) => { e.stopPropagation(); store.toggleTaskComplete(node.id); }}
        >
          {node.task.status === "completed" ? "☑" : "☐"}
        </button>
      ) : (
        <span className="outliner-check placeholder" />
      )}
      <input
        className="outliner-title"
        value={text}
        onChange={(e) => setText(e.target.value)}
        onBlur={() => { if (text !== node.title) store.execOps([makeOp<Op & { type: "setTitle" }>("setTitle", { id: node.id, title: text, prev:
node.title, titleRuns: plainToRuns(text), prevRuns: node.titleRuns })]); }}
        onKeyDown={(e) => {
          e.stopPropagation();
          if (e.key === "Enter") (e.target as HTMLInputElement).blur();
        }}
        onClick={(e) => e.stopPropagation()}
      />
      {node.task && node.task.priority && node.task.priority !== "none" && <span className={`prio prio-${node.task.priority}`} title={`Priority:
${node.task.priority}` } />}
    </div>
    {!node.collapsed &&
      node.childrenIds.map((id) => {
        const child = store.doc.node(id);
        return child ? <OutlinerRow key={id} node={child} depth={depth + 1} /> : null;
      })
    }
  </div>
);
}

```

## 32. src-tauri/src/main.rs

```

// Prevents an extra console window on Windows in release builds.
#![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]

fn main() {
  r_node::run()
}

```

## 33. tests/exportBridge.test.ts

```

import "fake-indexeddb/auto";
import { describe, expect, it } from "vitest";
import { unzipSync, strFromU8 } from "fflate";
import { IndexedDbAssetStore, sha256Hex, type AssetMeta } from "../src/persist/assets";
import { buildRnodeZip, estimateRnodeZip, importRnodeZip, type LevelGenerator } from "../src/editor/exportBridge";
import type { MindNode, RnodeDocument, Sheet } from "../src/core/types";

let dbCounter = 0;
function uniqueDb(): string {
  return `assets-test-${++dbCounter}-${Math.random().toString(36).slice(2)}`;
}

/**
 * Incompressible payload (true random, not a periodic pattern): deflate must
 * not shrink it, or the size estimate would overstate the produced file.

```

```

*/
function pngishBytes(_seed: number, size: number): Uint8Array<ArrayBuffer> {
  const out = new Uint8Array(size);
  crypto.getRandomValues(out);
  return out;
}

/** Node test seam: node has no canvas, so levels are the source as-is. */
const fakeGenerate: LevelGenerator = async (blob: Blob, _mime: string) => ({
  original: { blob, w: 64, h: 48 },
  large: { blob, w: 64, h: 48 },
  small: { blob, w: 64, h: 48 },
});

function makeDoc(imageId?: string): RnodeDocument {
  const nodes: Record<string, MindNode> = {
    root: {
      id: "root",
      type: "main",
      parentId: null,
      childrenIds: imageId ? ["child"] : [],
      title: "Root",
      titleRuns: [{ text: "Root" }],
      position: { x: 0, y: 0, manual: false },
      style: {},
      collapsed: false,
      labels: [],
      markers: [],
      notes: "",
      task: null,
      metadata: { createdAt: "t", updatedAt: "t" },
    },
  };
  const sheet: Sheet = {
    sheetId: "s1",
    title: "Map 1",
    structure: {
      structureType: "mindmap",
      orientation: "horizontal",
      spacing: 180,
      branchSpacing: 14,
      padding: 18,
      compactMode: false,
      autoBalance: true,
      freePositioningBranches: false,
      allowManualPositioning: true,
      connectorStyle: "curved",
    },
    rootNodeId: "root",
    nodes,
    relationships: [],
    boundaries: [],
    summaries: [],
    callouts: [],
    labels: [],
    zones: [],
    attachments: imageId
      ? [{ id: imageId, mime: "image/png", w: 64, h: 48, bytes: 4096, name: "img.png" }]
      : [],
    comments: [],
    presentation: {},
  };
  return {
    schemaVersion: "1",
    documentId: "doc-1",
    title: "Test map",
    createdAt: "t",
    updatedAt: "t",
    archived: false,
    pinned: false,
    settings: { theme: "light", showOutliner: false, showInspector: true },
    themeId: "r-node-light",
    sheets: [sheet],
  };
}

/** A doc whose root node references `imageId` (with the card). */
function makeDocWithImage(imageId: string): RnodeDocument {
  const doc = makeDoc(imageId);
  const root = doc.sheets[0].nodes.root;
  root.childrenIds = ["child"];
  root.style.image = imageId;
  doc.sheets[0].nodes.child = {
    id: "child",
    type: "subtopic",
    parentId: "root",
    childrenIds: [],
  };
}

```

```

    title: "Child",
    titleRuns: [{ text: "Child" }],
    position: { x: 0, y: 0, manual: false },
    style: {},
    collapsed: false,
    labels: [],
    markers: [],
    notes: "",
    task: null,
    metadata: { createdAt: "t", updatedAt: "t" },
  };
  return doc;
}

async function putTestAsset(store: IndexedDbAssetStore): Promise<string> {
  // Distinct sizes per level: complete export carries the 8KB original,
  // compact the 4KB display level – the ratio is what the mode tests assert.
  const original = pngishBytes(42, 8192);
  const large = pngishBytes(43, 4096);
  const small = pngishBytes(44, 1024);
  const meta: Omit<AssetMeta, "id"> = { mime: "image/png", w: 64, h: 48, bytes: original.length, name: "img.png" };
  return store.put(
    {
      original: { blob: new Blob([original], { type: "image/png" }), w: 64, h: 48 },
      large: { blob: new Blob([large], { type: "image/png" }), w: 32, h: 24 },
      small: { blob: new Blob([small], { type: "image/png" }), w: 16, h: 12 },
    },
    meta
  );
}

describe(".rnode.zip export", () => {
  it("contains document.json, manifest.json and one entry per referenced asset", async () => {
    const store = new IndexedDbAssetStore(uniqueDb());
    const id = await putTestAsset(store);
    const doc = makeDocWithImage(id);

    const bytes = await buildRnodeZip(doc, doc.sheets[0], store, "complete");
    const files = unzipSync(bytes);

    expect(Object.keys(files)).toContain("document.json");
    expect(Object.keys(files)).toContain("manifest.json");
    const assetNames = Object.keys(files).filter((n) => n.startsWith("assets/"));
    expect(assetNames).toEqual(`assets/${id}.png`);

    // The document inside is byte-identical to today's .rnode.json.
    expect(JSON.parse(strFromU8(files["document.json"]))).toEqual(doc);
    expect(JSON.parse(strFromU8(files["manifest.json"]))).toEqual({ mode: "complete" });
  });

  it("skips assets the nodes do not reference", async () => {
    const store = new IndexedDbAssetStore(uniqueDb());
    await putTestAsset(store); // stored, but no node references it
    const doc = makeDoc(); // no image at all

    const files = unzipSync(await buildRnodeZip(doc, doc.sheets[0], store, "complete"));
    expect(Object.keys(files).filter((n) => n.startsWith("assets/"))).toEqual([]);
  });

  it("estimates the size within 10% of the produced file", async () => {
    const store = new IndexedDbAssetStore(uniqueDb());
    const id = await putTestAsset(store);
    const doc = makeDocWithImage(id);

    const estimate = await estimateRnodeZip(doc, doc.sheets[0], store, "complete");
    const produced = (await buildRnodeZip(doc, doc.sheets[0], store, "complete")).length;

    expect(Math.abs(estimate - produced) / produced).toBeLessThan(0.1);
  });

  it("compact mode stores only the display level, not the original", async () => {
    const store = new IndexedDbAssetStore(uniqueDb());
    const id = await putTestAsset(store);
    const doc = makeDocWithImage(id);
    const sheet = doc.sheets[0];

    const originalBlob = await store.get(id, "original");
    const completeBytes = await buildRnodeZip(doc, sheet, store, "complete");
    const compactBytes = await buildRnodeZip(doc, sheet, store, "compact");

    // Same entries in both, but compact carries the 4KB display level, not
    // the 8KB original.
    const cFiles = unzipSync(completeBytes);
    const pFiles = unzipSync(compactBytes);
    expect(Object.keys(cFiles)).toEqual(Object.keys(pFiles));
    expect(pFiles[`assets/${id}.png`].length).toBeLessThan(cFiles[`assets/${id}.png`].length);
    expect(JSON.parse(strFromU8(pFiles["manifest.json"]))).toEqual({ mode: "compact" });
  });
}

```

```

    expect(originalBlob).not.toBeNull();
  });
});

describe(".rnode.zip import", () => {
  it("round-trips into a fresh store: export, wipe, import restores the asset under the same id", async () => {
    const source = new IndexedDbAssetStore(uniqueDb());
    const id = await putTestAsset(source);
    const doc = makeDocWithImage(id);
    const zip = await buildRnodeZip(doc, doc.sheets[0], source, "complete");

    // A clean profile: a brand-new store with nothing in it.
    const fresh = new IndexedDbAssetStore(uniqueDb());
    expect(await fresh.list()).toEqual([]);

    const imported = await importRnodeZip(zip, fresh, fakeGenerate);
    expect(imported).toEqual(doc);
    expect(await fresh.list()).toEqual([id]);
    expect(await fresh.get(id, "large")).not.toBeNull();
    expect(await fresh.get(id, "small")).not.toBeNull();
    const meta = await fresh.meta(id);
    expect(meta?.mime).toBe("image/png");
  });

  it("reimporting the same file does not duplicate anything", async () => {
    const source = new IndexedDbAssetStore(uniqueDb());
    const id = await putTestAsset(source);
    const doc = makeDocWithImage(id);
    const zip = await buildRnodeZip(doc, doc.sheets[0], source, "complete");

    const fresh = new IndexedDbAssetStore(uniqueDb());
    await importRnodeZip(zip, fresh, fakeGenerate);
    await importRnodeZip(zip, fresh, fakeGenerate);

    expect(await fresh.list()).toEqual([id]);
  });

  it("compact import regenerates the display levels under the document id", async () => {
    const source = new IndexedDbAssetStore(uniqueDb());
    const id = await putTestAsset(source);
    const doc = makeDocWithImage(id);
    const zip = await buildRnodeZip(doc, doc.sheets[0], source, "compact");

    const fresh = new IndexedDbAssetStore(uniqueDb());
    const imported = await importRnodeZip(zip, fresh, fakeGenerate);

    // The document keeps its original id (content address of the original,
    // which compact did not carry) – and the store serves it under that id.
    expect(imported).not.toBeNull();
    expect(await fresh.list()).toEqual([id]);
    expect(await fresh.get(id, "large")).not.toBeNull();
    expect(await fresh.get(id, "small")).not.toBeNull();
  });

  it("compact import marks the asset cards as originallost", async () => {
    const source = new IndexedDbAssetStore(uniqueDb());
    const id = await putTestAsset(source);
    const doc = makeDocWithImage(id);
    const zip = await buildRnodeZip(doc, doc.sheets[0], source, "compact");

    const fresh = new IndexedDbAssetStore(uniqueDb());
    const imported = (await importRnodeZip(zip, fresh, fakeGenerate))!;
    expect(imported.sheets[0].attachments.find((a) => a.id === id)?.originallost).toBe(true);
  });

  it("complete import leaves the cards unmarked", async () => {
    const source = new IndexedDbAssetStore(uniqueDb());
    const id = await putTestAsset(source);
    const doc = makeDocWithImage(id);
    const zip = await buildRnodeZip(doc, doc.sheets[0], source, "complete");

    const fresh = new IndexedDbAssetStore(uniqueDb());
    const imported = (await importRnodeZip(zip, fresh, fakeGenerate))!;
    expect(imported.sheets[0].attachments.find((a) => a.id === id)?.originallost).toBeUndefined();
  });

  it("a complete export of a compact-imported doc flags degraded in the manifest", async () => {
    const source = new IndexedDbAssetStore(uniqueDb());
    const id = await putTestAsset(source);
    const doc = makeDocWithImage(id);
    const compactZip = await buildRnodeZip(doc, doc.sheets[0], source, "compact");

    const fresh = new IndexedDbAssetStore(uniqueDb());
    const imported = (await importRnodeZip(compactZip, fresh, fakeGenerate))!;

    // Store serves the display level under the same id, and the complete
    // export of the degraded document says so in its manifest.
  });

```

```

const completeZip = await buildRNodeZip(imported, imported.sheets[0], fresh, "complete");
const manifest = JSON.parse(strFromU8(unzipSync(completeZip)["manifest.json"]));
expect(manifest).toEqual({ mode: "complete", degraded: true });

// A fresh complete export of the same doc (no degradation) stays clean.
const cleanZip = await buildRNodeZip(doc, doc.sheets[0], source, "complete");
expect(JSON.parse(strFromU8(unzipSync(cleanZip)["manifest.json"]))).toEqual({ mode: "complete" });
});

it("returns null for a corrupt container", async () => {
  const store = new IndexedDbAssetStore(uniqueDb());
  expect(await importRNodeZip(new Uint8Array([1, 2, 3]), store, fakeGenerate)).toBeNull();
  expect(await importRNodeZip(new Uint8Array(), store, fakeGenerate)).toBeNull();
});

it("stores the payload under the SHA-256 of the original bytes", async () => {
  // The document's references survive the round-trip by construction: the
  // id inside the zip equals the hash of the bytes that get stored.
  const bytes = pngishBytes(7, 2048);
  const id = await sha256Hex(bytes);
  expect(id).toMatch(/^[0-9a-f]{64}$/);

  const blob = new Blob([bytes], { type: "image/png" });
  const store = new IndexedDbAssetStore(uniqueDb());
  await store.putUnderId(
    id,
    { original: { blob, w: 64, h: 48 }, large: { blob, w: 64, h: 48 }, small: { blob, w: 64, h: 48 } },
    { id, mime: "image/png", w: 64, h: 48, bytes: bytes.length }
  );
  expect(await store.get(id, "original")).not.toBeNull();
});
});

```

## 34. tests/lexicalRuns.test.ts

```

import { describe, expect, it } from "vitest";
import { $createLineBreakNode, $createParagraphNode, $createTextNode, $getRoot, createEditor } from "lexical";
import { ListItemNode, ListNode } from "@lexical/list";
import { editorStateToRuns, setEditorRuns } from "../src/ui/lexicalRuns";
import type { TextRun } from "../src/core/types";

// FORMAT_* bits mirroring Lexical's TextNode format flags.
const F_BOLD = 1;
const F_ITALIC = 2;

async function makeEditorWith(texts: Array<{ text: string; format?: number; style?: string }>): Promise<ReturnType<typeof createEditor>> {
  const editor = createEditor();
  await editor.update(() => {
    const root = $getRoot();
    root.clear();
    const p = $createParagraphNode();
    for (const t of texts) {
      const n = $createTextNode(t.text);
      if (t.format) n.setFormat(t.format);
      if (t.style) n.setStyle(t.style);
      p.append(n);
    }
    root.append(p);
  });
  return editor;
}

describe("lexicalRuns bridge", () => {
  it("converts bold/italic/underline/color editor state into TextRuns", async () => {
    const editor = await makeEditorWith([
      { text: "Bold", format: F_BOLD },
      { text: " italic", format: F_ITALIC },
      { text: " color", style: "color: #ff0000" },
      { text: " plain" },
    ]);
    const runs = editorStateToRuns(editor.getEditorState());
    const find = (s: string) => runs.find((r) => r.text.includes(s));
    expect(find("Bold").bold).toBe(true);
    expect(find("italic").italic).toBe(true);
    expect(find("color").color).toBe("#ff0000");
    expect(find("plain").bold ?? false).toBe(false);
  });

  it("flattens line breaks and paragraphs into \\n-separated runs", async () => {
    const editor = createEditor();
    await editor.update(() => {
      const root = $getRoot();
      root.clear();
      const p = $createParagraphNode();

```

```

    p.append($createTextNode("line1"), $createLineBreakNode(), $createTextNode("line2"));
    root.append(p);
  });
  const runs = editorStateToRuns(editor.getEditorState());
  const plain = runs.map((r) => r.text).join("");
  expect(plain).toBe("line1\nline2");
});

it("seeds an editor from runs (bold + color) and reads them back unchanged", async () => {
  const editor = createEditor();
  setEditorRuns(editor, [
    { text: "Rich ", bold: true },
    { text: "text", color: "#d43a3a" },
  ]);
  // editor.update() schedules the commit on a microtask even though its
  // declared return type is void.
  await Promise.resolve();
  const runs = editorStateToRuns(editor.getEditorState());
  expect(runs).toEqual([
    { text: "Rich ", bold: true },
    { text: "text", color: "#d43a3a" },
  ]);
});

it("drops zero-length runs when normalizing", async () => {
  const editor = createEditor();
  setEditorRuns(editor, [{ text: "" }, { text: "x" }]);
  await Promise.resolve();
  const runs = editorStateToRuns(editor.getEditorState());
  expect(runs).toHaveLength(1);
  expect(runs[0].text).toBe("x");
});

describe("lexicalRuns round trip (runs → editor → runs)", () => {
  async function roundTrip(runs: TextRun[]): Promise<TextRun[]> {
    const editor = createEditor({ nodes: [ListNode, ListItemNode], onError: (e) => { throw e; } });
    setEditorRuns(editor, runs);
    await Promise.resolve();
    return editorStateToRuns(editor.getEditorState());
  }

  const LIST: TextRun[] = [
    { text: "one", listIndent: 1 },
    { text: "\n" },
    { text: "two", listIndent: 1 },
    { text: "\n" },
    { text: "nested", listIndent: 2 },
  ];

  it("keeps nested list depth (structural nesting and getIndent are one level, not two)", async () => {
    const out = await roundTrip(LIST);
    expect(out.filter((r) => r.listIndent).map((r) => r.listIndent)).toEqual([1, 1, 2]);
  });

  it("is idempotent: editing a title twice must not change it", async () => {
    // The canvas lays out whatever comes back from here, so any drift between
    // cycles shows up as the node silently growing/indenting on every edit.
    const once = await roundTrip(LIST);
    const twice = await roundTrip(once);
    expect(twice).toEqual(once);
    const thrice = await roundTrip(twice);
    expect(thrice).toEqual(once);
  });

  it("does not accumulate the newline that closes a list", async () => {
    const once = await roundTrip(LIST);
    const twice = await roundTrip(once);
    const tail = (rs: TextRun[]): string => rs.map((r) => r.text).join("").match(/\n*$/)[0];
    expect(tail(twice)).toBe(tail(once));
    expect(tail(once).length).toBeLessThanOrEqual(1);
  });

  it("keeps paragraphs separate across a round trip", async () => {
    const out = await roundTrip([
      { text: "first" },
      { text: "second", paraGap: true }
    ]);
    expect(out.map((r) => r.text)).toEqual(["first", "second"]);
    expect(out[1].paraGap).toBe(true);
  });
});

```

## 35. .gitignore

```

node_modules/
dist/

```

```
src-tauri/target/
src-tauri/gen/
*.log
.DS_Store

# Generated code snapshots for AI-assisted review. They go stale the moment
# anything is committed, and a stale snapshot read as documentation is worse
# than no snapshot: regenerate on demand instead of tracking them.
repomix-output.xml
docs/layout-bundle.html
```

## 36. docs/RICH\_TEXT\_EDITOR.md

```
# R-node – Editor rich text: come funziona

Questo documento spiega l'architettura ibrida scoped dell'editing di testo
di R-node: come il testo dei nodi viene modellato, misurato, disegnato sul
canvas e modificato tramite l'overlay Lexical, e come i due mondi (browser e
canvas) restano allineati.

> Stato: in sviluppo attivo. La parità geometrica fra editor e canvas è
> misurata, non asserita: vedi §7bis, oggi 16 casi su 16 allineati. Le sezioni
> *Limitazioni note* elencano i punti ancora aperti.

---

## 1. Panoramica: l'architettura ibrida

R-node disegna tutta la mappa su un solo <canvas> 2D (mai un DOM o SVG
per nodo). Il testo dei nodi è rich text, quindi servono due mondi:

| Aspetto | Dove vive |
|---|---|
| Modello dati | TextRun[] nello store (nativo, serializzabile) |
| Rendering statico | canvas 2D, con bitmap cache per nodo |
| Editing | un solo overlay HTML (Lexical) alla volta |
| Misura | wrapRunLines condivisa tra layout e renderer |
| Undo/redo | una sola history (Pattern Command nello store) |

Scelte architetturali esplicite (non ridiscutibili):
- Lexical non ha la propria history: è un puro generatore di draft.
- Mai più di un overlay montato: CanvasView lo chiama con key={editingId}.
- Il nodo in editing non viene disegnato dal canvas (niente ghosting).
- La rotella funziona anche durante l'editing: l'overlay deriva la sua
  posizione dalla camera dello store a ogni render, quindi pan/zoom resta
  incollato al nodo (il guard editing sul wheel è stato rimosso).

---

## 2. Il modello dati: TextRun[]

```ts
export interface TextRun {
  text: string;
  bold?: boolean;
  italic?: boolean;
  underline?: boolean;
  color?: string; // esadecimale; se assente eredita il colore del tema
  fontSize?: number; // heading: h1=26 ... h6=14 (in unità world)
  paraGap?: boolean; // inizio di un nuovo blocco → gap verticale extra
  listIndent?: number; // inizio di un item di lista (1 = primo livello)
}
```

Il titolo di un nodo non è una stringa ma una sequenza di run stilizzati.
Vale l'invariante:

```
node.title === runsToPlain(node.titleRuns)
```

cioè il titolo "piatto" resta sempre sincronizzato con i run: search,
outliner, export e i test continuano a leggere la stringa, mentre canvas ed
editor usano i run. I vecchi documenti senza titleRuns funzionano (fallback:
tutto il titolo come un singolo run plain).

Semantica di blocco (aggiunta per replicare la spaziatura di Draw.io/Word):
- fontSize → un run più grande (heading) alza l'altezza della riga;
- paraGap → il blocco successivo parte con un gap verticale;
- listIndent → il renderer disegna il pallino "•" e fa l'hanging indent
  sulle righe di continuazione.

Helper in src/core/text.ts: runsToPlain, plainToRuns, normalizeRuns
(unisce run adiacenti con formattazione identica), nodeRuns, runsEqual,
```

```

`trimRuns`, `isEmptyRuns`.

---

## 3. Il ciclo di vita di una modifica

### 3.1 Apertura (doppio click / F2 / type-to-edit)

1. `store.startEdit(id)`:
  - committa eventuale draft in corso (`commitDraftOnLeave`);
  - imposta `editingId` e la selezione;
  - salva `editOriginal` (titolo + runs pre-edit, per undo e per Escape);
  - **semina il draft** con i runs correnti del nodo (`applyDraftRuns`), così
    il layout continua a misurare il nodo alla dimensione giusta finché
    l'editor non riporta la prima modifica.
2. React monta `` (unico overlay) con `key={editingId}`: aprire
  l'editing su un altro nodo smonta il precedente.

### 3.2 Digitazione (draft live)

- Ad ogni modifica, il listener di Lexical (`DraftSyncPlugin`) converte lo
  stato dell'editor in `TextRun[]` e chiama `store.setEditingDraftRuns(...)`.
- Lo store applica i runs al nodo **in modo effimero** (nessuna op, nessuna
  history) e ricalcola il layout con debounce 30ms: la box del nodo cresce e
  il testo si ri-avvolge **mentre digiti**.

### 3.3 Commit (un'unica op)

- **Enter** (senza Shift) o click fuori → `commitEdit()`:
  - trim dei runs; se il titolo è vuoto e il nodo non ha figli, il nodo viene
    eliminato (altrimenti ripristina il titolo originale);
  - se il contenuto è cambiato → **una sola op `setTitle`** con
    `titleRuns` / `prevRuns` nella history dello store;
  - se non è cambiato → nessuna op, ripristino del titolo originale.
- **Escape** → `cancelEdit()`: scarta il draft e ripristina esattamente
  `editOriginal`.
- **Ctrl+S** mentre si digita → `commitDraftKeepEditing()`: committa il draft
  come op ma **senza chiudere** l'editor (salvi e continui).

Undo/redo passano esclusivamente dalla history dello store: Lexical non ha
HistoryPlugin attivo. `undo()` ripristina i runs pre-edit esatti.

### 3.4 Type/paste-to-edit

Digitare un carattere o incollare con un nodo selezionato avvia l'editing con
quel contenuto: `typeToEdit(text)` imposta `pendingInsert`, e l'editor lo
consuma al mount (`consumePendingInsert`). `appendPendingInsert` bufferizza i
keystroke persi tra l'avvio e il mount (per i dattilografi veloci).

---

## 4. L'overlay Lexical (`src/ui/RichEditor.tsx`)

Struttura:

...
.topic-rich-editor      ← posizionamento assoluto (left/top in px schermo)
└─ .topic-rich-inner    ← layout in WORLD units, transform: scale(zoom)
    └─ LexicalComposer
        ├── Toolbar      ← B / I / colore / barrato / clear (flottante sopra)
        ├── RichTextPlugin + ContentEditable (.topic-rich-editable)
        ├── ListPlugin
        ├── DraftSyncPlugin ← seed + listener editor → runs
        ├── PasteSanitizerPlugin
        └── KeysPlugin    ← Enter/Esc/Ctrl+S/Tab in fase di capture
    ...

**Perché il `transform: scale`? L'overlay viene ridimensionato in unità
world e scalato tutto insieme: font, padding, bordo e gap di blocco
coincidono col canvas pixel per pixel a qualsiasi zoom. (Scalare le singole
proprietà rompeva il WYSIWYG: i font-size degli heading erano px assoluti e i
padding px schermo, quindi il testo si gonfiava e avvolgeva altrove.)

- `fontSize` dell'inner = font del nodo (world, non scalato);
- `lineHeight: 1.25` = `LINE_HEIGHT_FACTOR`;
- il gap di blocco condiviso è passato come variabile CSS
  `--rnode-block-gap: calc(0.6 * 1.25em)` (costanti TS → CSS, unica fonte);
- la **toolbar flotta sopra la box** (`top: -30px` world): la box è misurata
  per il solo testo, se la toolbar stesse dentro ruberebbe altezza e il
  contenuto scorre/taglia;
- il padding dell'editable replica esattamente la geometria del canvas
  (vedi §7).

---

## 5. Il bridge Lexical ↔ runs (`src/ui/lexicalRuns.ts`)

```

Due direzioni:

```
- setEditorRuns(editor, runs) - semina l'editor dai runs del nodo:
  paragrafi → <p>, heading → <p> con span font-size: Xpx, liste →
  <ul><li> annidati; caret alla fine.
- editorStateToRuns(editorState) - ad ogni keystroke converte lo stato
  in runs:
  - bold/italic/underline → format bit del TextNode;
  - colore e font-size → inline style del TextNode (color: #hex,
    font-size: Xpx);
  - ogni figlio radice (paragrafo/list/heading) → gruppo; il primo run di un
    gruppo dopo il primo riceve paraGap;
  - ListItemNode → primo run con listIndent (il renderer disegna il
    pallino), item separati da run \n;
  - HeadingNode → run con fontSize assoluto (h1=26 ... h6=14);
  - \n interno al paragrafo → run \n (linea senza gap).
```

La struttura a blocchi sopravvive al round-trip: è questo che permette al canvas di riprodurre la spaziatura del contenuto incollato da Draw.io/Word.

---

## 6. Sanitizzazione del paste (`src/ui/pasteSanitizer.ts`)

Tre strati di difesa:

1. **sanitizeHtml(html)** - rimuove script/img/href/CSS di layout, ma **prima** legge `font-weight`/`font-style` delle span di Word/Google Docs e le converte in `<strong>`/`<em>` (altrimenti l'enfasi va persa silenziosamente); ricostruisce le liste Word (`MsoListParagraph` + `margin-left` + `mso-list`) in `<ul><li>` annidati; conserva h1-h6.
2. **htmlToRuns(html)** - HTML pulito → runs con `fontSize` (heading), `paraGap` (paragrafi), `listIndent` (item).
3. **runsToParagraphNodes(runs)** - runs → paragrafi/liste Lexical reali (l'utente può continuare a modificarli).

**Perché** il paste passa sempre dal percorso runs? `insertNodes` di Lexical converte un `HeadingNode` in paragraph quando inserito a selezione collassata (bug riprodotto nei test). Il percorso runs produce paragrafi con font-size inline: struttura identica, nessuna dipendenza dalle import rules di Lexical.

---

## 7. Misura condivisa e WYSIWYG (`src/layout/measure.ts`)

Layout e renderer usano **la stessa** `wrapRunLines`, con le stesse costanti:

| Costante                                            | Valore              | Significato                                                     |
|-----------------------------------------------------|---------------------|-----------------------------------------------------------------|
| --- --- ---                                         |                     |                                                                 |
| <code>LINE_HEIGHT_FACTOR</code>                     | 1.25                | line-height unitless, come nel CSS                              |
| <code>BLOCK_GAP_FACTOR</code>                       | 0.6                 | gap di blocco = $0.6 \times 1.25 \times$ <b>font</b> del blocco |
| <code>TEXT_INSET</code>                             | 6                   | insetto orizzontale dentro la box (entrambi i lati)             |
| <code>BULLET_WIDTH_EM</code>                        | 1.2                 | larghezza della colonna del pallino, in em                      |
| <code>FONT_STACK</code>                             | <code>--font</code> | unico stack tipografico per canvas e overlay                    |
| <code>MIN_TOPIC_W</code> / <code>MAX_TOPIC_W</code> | 84 / 280            | clamp della larghezza                                           |

Regole del modello (tutte verificate dall'harness, \$7bis):

- l'altezza di riga è la **line box** del CSS: `max(above) + max(below)` su ogni inline box della riga - lo **strut** del blocco compreso - dove ciascuno contribuisce `ascent + half-leading` e `descent + half-leading`. Non è «font-size più grande  $\times 1.25$ »: quella formula ignora lo strut (un run da 10px dentro un topic da 14px dava 12.5px contro i 17.5px del browser) e sbaglia la baseline quando sulla riga convivono più corpi;
- la **baseline** è calcolata qui e il renderer la usa così com'è;
- ogni **confine di blocco** aggiunge `BLOCK_GAP_FACTOR`  $\times$  `LINE_HEIGHT_FACTOR`  $\times$  `fontSize` del nodo. Il gap viene dallo **strut**, non dall'altezza della riga che segue: `margin-top: calc(.6 * 1.25em)` si risolve sull'`em` del blocco, e la dimensione di un heading vive su una span interna;
- un run con `paraGap` apre un blocco **anche** senza `\n` (l'editor segna il confine fra root children solo così);
- `listIndent` → il pallino sta in una colonna larga `BULLET_WIDTH_EM em` e il testo parte a `depth`  $\times$  `bulletW` su **tutte** le righe dell'item;
- lo spazio finale non conta nella larghezza della riga (il CSS lo fa debordare), un token più largo della colonna va a capo a metà parola;
- una riga vuota fra due paragrafi è una riga vera; quelle in coda no.

L'overlay replica le stesse regole in CSS:

- `line-height: 1.25` (ereditato, unitless → relativo al font di ogni riga);
- `p, ul, ol, li { margin: 0 }` (zero margini nativi del browser);
- `> :not(:first-child), li + li, li > ul { margin-top: var(--rnode-block-gap) }` - stesso gap del canvas, liste annidate comprese;
- `li { padding-left: var(--rnode-bullet-w); text-indent: calc(-1 * ...) }` con il pallino disegnato da `li::before`: il marker nativo del browser non è utilizzabile, la sua larghezza è decisa dallo UA e nessuna lunghezza CSS la eguaglia;
- padding dell'editable calcolato da `pad` e `TEXT_INSET`:
  - verticale: `pad + 2`, e il blocco è centrato da `justify-content: center`

```
(come il renderer centra la bitmap: con `flex: 1` restava in alto e
derivava su diamond/hexagon/circle e dopo ogni resize);
- orizzontale: `pad - 2` sx, `pad + TEXT_INSET - 2` dx (dentro il bordo da
2px) → la larghezza di avvolgimento coincide con `boxW - pad*2 - TEXT_INSET`.
```

---

## 7bis. L'harness di parità (`dev/parity.html`)

La parità non è un'opinione: `npm run dev` →  
`http://localhost:5173/dev/parity.html` monta un corpus di casi nel DOM vero  
dell'editable, ne estrae le line box reali carattere per carattere con  
`Range.getClientRects()` , e le confronta con l'output di `wrapRunLines`  
(punti di a-capo, baseline, avanzamento riga, `left`, altezza totale).  
Risultati anche su `window.\_\_parity`.

Due trappole di misura che l'harness deve evitare – entrambe producevano  
divergenze fantasma:

- il rect di un carattere è l'`inline box` (~19px a 14px), più alto della  
line box da 17.5px: la sovrapposizione verticale non separa le righe (si usa  
il ritorno a sinistra) e la sua altezza non è l'altezza di riga (si usa  
l'avanzamento);
- il `\*top\*` del glifo sta a distanze diverse dalla line box per ogni corpo, per  
cui il confronto è sulla `baseline`, ricavata dal rect e dall'ascent.

Stato: \*\*16 casi su 16 allineati\*\*, residuo massimo 0.5px (arrotondamento di  
`fontBoundingBox`).

---

## 8. Rendering del nodo sul canvas (`src/render/renderer.ts`)

Il renderer dipinge: sfondo/griglia → relazioni → connettori → nodi →  
indicatore di drop. Per ogni nodo (`drawNode`):

1. **Forma e bordo** (rounded/rect/capsule/circle/diamond/hexagon/underline/  
none) + fill + bordo + ombra.
2. **Anello di selezione** + maniglie di resize (solo bordo, con alone bianco  
per la visibilità su fill scuri).
3. **Testo rich** – solo se il nodo NON è in editing (no-ghosting).

### Bitmap cache per nodo

Il testo di ogni nodo viene renderizzato **una volta** in un offscreen canvas  
(`renderTextBitmap`) e blittato con `drawImage` durante pan/zoom. La chiave  
della cache include `JSON.stringify(titleRuns)` + stile + colore risolto +  
larghezza di wrap + bucket di risoluzione (potenza di 2 dello zoom): si  
invalida solo quando il contenuto cambia davvero, non ad ogni frame.

### Baseline come il browser

Il canvas **non** calcola più la baseline: la prende da `line.baseline`, che  
`wrapRunLines` costruisce con la regola del CSS (half-leading per ogni inline  
box, strut incluso – vedi §7). Prima il renderer applicava un solo  
half-leading per riga, e con corpi misti la posizione verticale dei glifi si  
scostava da quella dell'editor.

Sottolineato a ~0.1em sotto la baseline, barrato a ~0.28em sopra.

Il pallino di una lista viene disegnato in `line.bullet` alla sua `x` di  
colonna, non è più un segmento di testo: la sua larghezza deve essere una  
lunghezza che anche il CSS può esprimere.

---

## 9. Allineamento editor ↔ canvas: cosa è stato allineato

| Aspetto         | Valore condiviso                                                         |
|-----------------|--------------------------------------------------------------------------|
| --- ---         |                                                                          |
| Line box        | `max(ascent+half) + max(descent+half)` su ogni inline box, strut incluso |
| Gap di blocco   | `0.6 × 1.25 × font del blocco` (CSS var `--rnode-block-gap`)             |
| Margini nativi  | azzerati in CSS (p/ul/ol/li) come nel modello                            |
| Posizione glifi | `line.baseline`, calcolata una volta sola nella misura                   |
| Colonna pallino | `BULLET_WIDTH_EM em` (CSS var `--rnode-bullet-w`)                        |
| Wrap width      | `boxW - pad*2 - TEXT_INSET` (padding dell'editable calcolato)            |
| Centrata blocco | verticale, `justify-content: center` su entrambi i lati                  |
| Font            | unico `FONT_STACK` (= `--font`), più weight/italic/decorazioni del nodo  |
| Colori          | fill e colore testo risolti da `Renderer.nodeColors`                     |
| Scala           | overlay in world units + `transform: scale(zoom)`; toolbar <b>fuori</b>  |

---

## 10. Tasti e interazioni durante l'editing

- **Enter** → commit dell'edit (Enter in capture-phase, prima dei handler di  
Lexical).
- **Esc** → annulla (ripristina `editOriginal`).

- **Ctrl/Cmd+S** → `saveNow()` con commit del draft, editor aperto.
- **Tab** → inserisce un carattere tab letterale nel titolo.
- **B / I / colore / barrato / clear** → toolbar.
- Rotella → pan/zoom **sempre attivi**, anche durante l'editing (l'overlay segue la camera). Anche il drag-pan (tasto destro o medio) funziona mentre si edita: sopra l'overlay lo intercettano gli handler in capture phase sul wrapper (`onWrapPointerDown`), che catturano il puntatore e panano senza rubare il click sinistro all'editor (selezione testo intatta).

---

## ## 11. Bug noti risolti lungo la strada

- **React StrictMode** impediva la registrazione del listener di sincronia (effetto mount → cleanup → mount + guard `seeded`): seed e listener sono ora in **due effetti separati**.
- **insertNodes** appiattisce gli heading a selezione collassata: il paste passa sempre dal percorso runs.
- **Liste Word**: `- ` dentro `- ` → `HierarchyRequestError`; `replaceWith` di un discendente col proprio antenato → albero invalido. Risolti con contenuto del wrapper + `insertBefore`.
- **Toolbar** dentro la box rubava altezza → contenuto con scrollbar: ora flottante sopra.
- **Scalatura** per-proprietà dell'overlay (heading px assoluti, padding px schermo) gonfiava il contenuto: sostituita dal `transform: scale`.
- **cp** dentro `- `: un figlio `*block*` con `list-style-position: inside` manda il marker su una riga sua (item da 17.5px misurato 35px) – una riga intera di deriva per ogni elemento di lista. Ora gli item hanno figli inline.
- **Round-trip** non idempotente: `runs → editor → runs` cambiava il contenuto, quindi un titolo derivava a ogni modifica. Due cause: la profondità di lista contata due volte (nidificazione strutturale `getIndent()` sommati) e il `\n` di chiusura di una lista riemesso ogni volta, che si accumulava in una riga vuota disegnata dal canvas e mai mostrata dall'editor.
- **stripEmptyNestedLists** non rimuoveva nulla: iterava i figli della lista (tutti `ListItemNode`) cercando `$isListNode` – ogni `- ` si portava dietro un `<ul>` vuoto.
- **Marker** nativo del browser: la sua larghezza è decisa dallo UA e nessuna lunghezza CSS la eguaglia, quindi un item andato a capo non poteva allinearsi. Sostituito da una colonna di larghezza nota disegnata da `li::before`.

---

## ## 12. Limitazioni note e lavori in corso

- **Link nel testo**: non supportati (nessun campo `link` su `TextRun`) – i link di Draw.io vanno persi.
- **Liste numerate**: gli item vengono resi con pallini (`list-style: disc`); il tipo numerato non è ancora modellato.
- **Allineamento per paragrafo**: solo quello del nodo intero (`center/left`), non per singolo blocco. Gli **item** di lista sono sempre allineati a sinistra su entrambi i lati: centrare un hanging indent non è ben definito e i due renderer non potrebbero accordarsi.
- **Liste con contenuto misto**: un gruppo `paraGap` che contiene sia testo normale sia item di lista perde il testo che precede il primo item (`buildList` parte dal primo run con `listIndent`). Non raggiungibile dall'editor – `editorStateToRuns` mette `paraGap` su ogni root child, quindi la lista finisce sempre in un gruppo suo – ma runs costruiti a mano o migrati possono incapparci.
- **Toolbar**: mancano barrato, sottolineato, liste e heading; liste e heading si ottengono oggi solo incollando.
- **Note dei nodi**: ancora plain text (il rich text vale per i titoli).
- **Outliner/search/export**: mostrano il testo appiattito (`runsToPlain`).
- **Tabelle nel titolo**: non supportate.
- **RTL**: non implementato.

---

## ## 13. Mappa dei file

...

|                                                                                      |                                                      |
|--------------------------------------------------------------------------------------|------------------------------------------------------|
| src/core/types.ts                                                                    | TextRun, MindNode, Style, ...                        |
| src/core/text.ts                                                                     | helper runs: normalize, plain<->runs, nodeRuns...    |
| src/core/ops.ts                                                                      | op system (setTitle con titleRuns/prevRuns)          |
| src/editor/store.ts                                                                  | EditorStore: draft, commit, undo/redo, resize...     |
| src/ui/RichEditor.tsx                                                                | overlay Lexical (unico), toolbar, tasti, paste       |
| src/ui/lexicalRuns.ts                                                                | bridge editor <-> runs                               |
| src/ui/pasteSanitizer.ts                                                             | sanitizeHtml / htmlToRuns (Word, Draw.io, web)       |
| src/layout/measure.ts                                                                | wrapRunLines condiviso, line box CSS, costanti       |
| src/render/renderers.ts                                                              | drawNode, renderTextBitmap (cache), nodeColors       |
| src/ui/CanvasView.tsx                                                                | overlay positioning (segue la camera), resize handle |
| dev/parity.html · dev/parity.ts                                                      | harness di parità editor + canvas (\$7bis)           |
| tests/pasteSanitizer.test.ts · lexicalRuns.test.ts · measure.test.ts · store.test.ts |                                                      |

...

### 37. src/editor/exportBridge.ts

```
/**
 * Portable document with images – `rnode.zip` (ROADMAP T15).
 *
 * The container follows the same shape as `xmind` / `docx`:
 *
 * document.json    the document, byte-identical to today's rnode.json
 * manifest.json    { mode: "complete" | "compact" } – how the assets are stored
 * assets/<id>.<ext> one file per referenced asset (id = SHA-256 hex)
 *
 * Two export modes, because the weight differs by an order of magnitude:
 * - complete: the ORIGINAL bytes (faithful, can be hundreds of MB);
 * - compact: only the `large` (1024px) display level (tens of MB, visually
 *           identical on screen).
 *
 * The estimate is computed BEFORE generating (the UI shows it first), and is
 * within ~10% of the produced file because raster assets are incompressible.
 *
 * Import reconstructs the full three-level set from what the container
 * carries (via an injectable level generator – canvas in the browser, a stub
 * in node tests) and stores it under the id the document already references
 * (putUnderId: a compact export has no original, so the content address
 * cannot be re-derived). Ids are content hashes, so reimporting the same file
 * never duplicates anything.
 */
import type { RnodeDocument, Sheet } from "../core/types";
import {
  referencedAssetIds,
  type AssetBlob,
  type AssetLevel,
  type AssetMeta,
  type AssetStore,
} from "../persist/assets";
import { zip, unzip, deflateSync, strToU8, strFromU8 } from "fflate";

export type RnodeZipMode = "complete" | "compact";

// -----
// PNG export hook (pre-existing feature). CanvasView registers the renderer's
// exportPng as the handler on mount; Palette and TopBar trigger it. Kept here
// because this file is the app's export bridge – image and document alike.
// -----

let pngHandler: (() => void) | null = null;

export function setExportPngHandler(fn: (() => void) | null): void {
  pngHandler = fn;
}

export function runExportPng(): void {
  pngHandler?.();
}

/** Per-entry zip overhead (local header + central-directory entry + slack). */
const ZIP_ENTRY_OVERHEAD = 96;
/** Zip structure beyond the entries (end-of-central-directory, slack). */
const ZIP_BASE_OVERHEAD = 256;

/** Long side of the display levels the renderer ever requests. */
const LEVEL_LONG_SIDE: Record<Exclude<AssetLevel, "original">, number> = {
  large: 1024,
  small: 256,
};

const EXT_BY_MIME: Record<string, string> = {
  "image/png": "png",
  "image/jpeg": "jpg",
  "image/gif": "gif",
  "image/webp": "webp",
  "image/svg+xml": "svg",
};

const MIME_BY_EXT: Record<string, string> = {
  png: "image/png",
  jpg: "image/jpeg",
  jpeg: "image/jpeg",
  gif: "image/gif",
  webp: "image/webp",
  svg: "image/svg+xml",
};

function extForMime(mime: string): string {
  return EXT_BY_MIME[mime] ?? "bin";
}
```

```

function mimeForExt(ext: string): string {
    return MIME_BY_EXT[ext] ?? "application/octet-stream";
}

function zipAsync(files: Record<string, Uint8Array>): Promise<Uint8Array> {
    return new Promise((resolve, reject) => {
        zip(files, {}, (err, data) => (err ? reject(err) : resolve(data)));
    });
}

function unzipAsync(data: Uint8Array): Promise<Record<string, Uint8Array>> {
    return new Promise((resolve, reject) => {
        unzip(data, (err, files) => (err ? reject(err) : resolve(files)));
    });
}

/** The level an export mode stores per asset. */
function levelForMode(mode: RnodeZipMode): AssetLevel {
    return mode === "complete" ? "original" : "large";
}

/**
 * Estimated .rnode.zip size, BEFORE generating. The document is actually
 * deflated (it compresses well and would otherwise skew the total); the
 * asset levels are added raw because raster formats are incompressible and
 * deflate leaves them within a fraction of a percent. Entry overhead is
 * accounted per asset so the estimate holds for many small images too.
 */
export async function estimateRnodeZip(
    doc: RnodeDocument,
    sheet: Sheet,
    store: AssetStore,
    mode: RnodeZipMode
): Promise<number> {
    const docJson = deflateSync(strToU8(JSON.stringify(doc))).length;
    const manifest = deflateSync(strToU8(JSON.stringify({ mode }))).length;
    const ids = [...referencedAssetIds(sheet)];
    let assets = 0;
    const level = levelForMode(mode);
    for (const id of ids) {
        const blob = await store.get(id, level);
        if (blob) assets += blob.size;
    }
    return docJson + manifest + assets + ids.length * ZIP_ENTRY_OVERHEAD + ZIP_BASE_OVERHEAD;
}

/** Build the .rnode.zip bytes. Only assets the nodes reference are included. */
export async function buildRnodeZip(
    doc: RnodeDocument,
    sheet: Sheet,
    store: AssetStore,
    mode: RnodeZipMode
): Promise<Uint8Array> {
    const ids = [...referencedAssetIds(sheet)];
    // A referenced asset whose original is gone (compact import, I11): a
    // "complete" export of it is degraded – the zip carries the resized level.
    // The manifest says so, and the store warns the user before generating.
    const degraded = ids.some((id) => sheet.attachments.some((a) => a.id === id && a.originalLost));
    const files: Record<string, Uint8Array> = {
        "document.json": strToU8(JSON.stringify(doc)),
        "manifest.json": strToU8(
            JSON.stringify(degraded ? { mode, degraded: true } : { mode })
        ),
    };
    const level = levelForMode(mode);
    for (const id of ids) {
        const [meta, blob] = await Promise.all([store.meta(id), store.get(id, level)]);
        if (!meta || !blob) continue; // referenced but absent: the zip cannot carry it
        files[`assets/${id}.${extForMime(meta.mime)}`] = new Uint8Array(await blob.arrayBuffer());
    }
    return zipAsync(files);
}

/**
 * Reconstruct the full three-level set from a single available level.
 * Browser implementation (canvas): `original` is the source as-is, `large`
 * and `small` are downscaled copies. For a complete export the source IS the
 * original; for a compact export the source is the 1024px level, which then
 * doubles as `original` – visually identical, byte-wise best effort.
 */
export async function generateLevelsFromSource(
    blob: Blob,
    mime: string
): Promise<Record<AssetLevel, AssetBlob>> {
    const source = await createImageBitmap(blob);
    const srcW = source.width;
    const srcH = source.height;

```

```

const levels = {
  original: { blob, w: srcW, h: srcH },
} as Record<AssetLevel, AssetBlob>;
const outMime =
  mime === "image/png" || mime === "image/webp" || mime === "image/gif" ? "image/png" : "image/jpeg";
for (const [key, targetLong] of [
  ["large", LEVEL_LONG_SIDE.large],
  ["small", LEVEL_LONG_SIDE.small],
] as const) {
  const scale = targetLong / Math.max(srcW, srcH);
  if (scale >= 1) {
    // Already smaller than the target: the source IS the level.
    levels[key] = { blob, w: srcW, h: srcH };
    continue;
  }
  const w = Math.max(1, Math.round(srcW * scale));
  const h = Math.max(1, Math.round(srcH * scale));
  const canvas = new OffscreenCanvas(w, h);
  const ctx = canvas.getContext("2d");
  if (!ctx) throw new Error("offscreen 2d context unavailable");
  ctx.drawImage(source, 0, 0, w, h);
  levels[key] = { blob: await canvas.convertToBlob({ type: outMime, quality: 0.85 }), w, h };
}
source.close();
return levels;
}

/** Test seam: node has no canvas, so the browser generator is injected. */
export type LevelGenerator = (
  blob: Blob,
  mime: string
) => Promise<Record<AssetLevel, AssetBlob>>;

/**
 * Open a .rnode.zip: extract every referenced asset into the store and return
 * the document. Returns null when the container is not a valid R-node zip.
 * Reimporting the same file twice stores each id once (content addressing +
 * first-write-wins putUnderId).
 */
export async function importRnodeZip(
  data: Uint8Array,
  store: AssetStore,
  generate: LevelGenerator = generateLevelsFromSource
): Promise<RnodeDocument | null> {
  let files: Record<string, Uint8Array>;
  try {
    files = await unzipAsync(data);
  } catch {
    return null;
  }
  const docRaw = files["document.json"];
  if (!docRaw) return null;

  // A compact container carries display levels only: the assets it restores
  // have NO original, and the document must say so – otherwise a later
  // "complete" export would claim to ship originals it does not have (I11).
  let mode: RnodeZipMode = "complete";
  const manifestRaw = files["manifest.json"];
  if (manifestRaw) {
    try {
      const parsed = JSON.parse(strFromU8(manifestRaw)) as { mode?: RnodeZipMode };
      if (parsed.mode === "compact") mode = "compact";
    } catch {
      // no readable manifest: assume the only level present is the original
    }
  }

  let doc: RnodeDocument;
  try {
    doc = JSON.parse(strFromU8(docRaw)) as RnodeDocument;
  } catch {
    return null;
  }
  const sheet = doc.sheets[0];
  if (!sheet) return null;

  const referenced = referencedAssetIds(sheet);
  for (const [name, bytes] of Object.entries(files)) {
    if (!name.startsWith("assets/")) continue;
    const dot = name.lastIndexOf(".");
    if (dot < 0) continue;
    const id = name.slice("assets/".length, dot);
    if (!referenced.has(id)) continue; // skip junk that no node points to
    const mime = mimeForExt(name.slice(dot + 1));
    // slice() copies into a fresh ArrayBuffer: a view over the zip's buffer
    // would keep the whole archive alive as long as the blob does.
    const blob = new Blob([bytes.slice().buffer], { type: mime });

```

```

const levels = await generate(blob, mime);
const card = sheet.attachments.find((a) => a.id === id);
if (mode === "compact" && card) card.originalLost = true;
const meta: AssetMeta = card ?? {
  id,
  mime,
  w: levels.original.w,
  h: levels.original.h,
  bytes: blob.size,
};
await store.putUnderId(id, levels, meta);
}
return doc;
}

```

### 38. src/ui/lexicalRuns.ts

```

/**
 * Bridge between the Lexical editor state (single overlay) and the store's
 * TextRun[] draft/model. Two directions:
 * - setEditorRuns: seed the editor from runs (node title / type-to-edit);
 * - editorStateToRuns: every keystroke converts the editor back to runs so
 *   the store can lay out the topic live and commit one op at the end.
 *
 * Bold/italic/underline map to Lexical TextNode format bits; color and
 * per-run font size live in the TextNode inline style (color: #hex;
 * font-size: Xpx), which Lexical carries through splits and serialization.
 *
 * Block structure survives the round trip so pasted content (Draw.io/Word/
 * web) keeps its spatial layout on the canvas:
 * - paragraphs → runs flagged paraGap (extra vertical gap);
 * - ListItemNode → first run flagged listIndent (renderer draws the bullet
 *   glyph and hangs continuation lines);
 * - HeadingNode → runs with a larger fontSize.
 */
import {
  $createLineBreakNode,
  $createParagraphNode,
  $createTextNode,
  $getRoot,
  IS_BOLD,
  IS_ITALIC,
  IS_UNDERLINE,
  type EditorState,
  type ElementNode,
  type LexicalEditor,
  type LexicalNode,
  TextNode,
} from "lexical";
import { $isHeadingNode } from "@lexical/rich-text";
import { $createListItemNode, $createListNode, $isListItemNode, $isListNode, type ListNode } from "@lexical/list";
import type { TextRun } from "../core/types";
import { normalizeRuns } from "../core/text";

// Lexical TextNode format bits – imported from the library, NOT hardcoded:
// in newer Lexical the values moved (IS_STRIKETHROUGH is 4, IS_UNDERLINE is 8),
// so a local copy silently misread underline as strikethrough and seeded
// underline text as strikethrough in the editor.
const FORMAT_BOLD = IS_BOLD;
const FORMAT_ITALIC = IS_ITALIC;
const FORMAT_UNDERLINE = IS_UNDERLINE;

const COLOR_RE = /color:\s*(#[0-9a-fA-F]{3,8})\b/;
const FONT_SIZE_RE = /font-size:\s*(\d+(?:\.\d+)?)\s*px/;
const MAX_INDENT = 8;

/** Heading level → font size in px (absolute, independent of node font size). */
const HEADING_SIZES: Record<string, number> = { h1: 26, h2: 21, h3: 18, h4: 16, h5: 15, h6: 14 };

export function textNodeToRun(n: TextNode, ctx: { headingSize?: number }): TextRun {
  const format = n.getFormat();
  const run: TextRun = { text: n.getTextContent() };
  if (format & FORMAT_BOLD) run.bold = true;
  if (format & FORMAT_ITALIC) run.italic = true;
  if (format & FORMAT_UNDERLINE) run.underline = true;
  const m = n.getStyle().match(COLOR_RE);
  if (m) run.color = m[1];
  if (ctx.headingSize) run.fontSize = ctx.headingSize;
  else {
    const fm = n.getStyle().match(FONT_SIZE_RE);
    if (fm) run.fontSize = Math.round(parseFloat(fm[1]));
  }
  return run;
}

```

```

function pushRun(runs: TextRun[], r: TextRun): void {
  if (r.text.length === 0) return;
  const last = runs[runs.length - 1];
  if (
    last &&
    (last.bold ?? false) === (r.bold ?? false) &&
    (last.italic ?? false) === (r.italic ?? false) &&
    (last.underline ?? false) === (r.underline ?? false) &&
    (last.color ?? null) === (r.color ?? null) &&
    (last.fontSize ?? null) === (r.fontSize ?? null) &&
    (last.paraGap ?? false) === (r.paraGap ?? false) &&
    (last.listIndent ?? null) === (r.listIndent ?? null)
  ) {
    last.text += r.text;
  } else {
    runs.push({ ...r });
  }
}

interface CollectCtx {
  headingSize?: number;
  listDepth: number;
}

/** Flatten a Lexical node tree (paragraphs, line breaks, lists, headings) into runs. */
function collectRuns(node: LexicalNode, runs: TextRun[], ctx: CollectCtx): void {
  if (node instanceof TextNode) {
    pushRun(runs, textNodeToRun(node, ctx));
    return;
  }
  if (node.getType() === "linebreak") {
    pushRun(runs, { text: "\n" });
    return;
  }
  if ($isListNode(node)) {
    const inner: TextRun[] = [];
    for (const child of (node as ElementNode).getChildren()) {
      collectRuns(child, inner, { ...ctx, listDepth: ctx.listDepth + 1 });
    }
    // The newline after the LAST item is redundant – the list block ends there.
    // Kept, every runs → editor → runs cycle appended one more, and a title
    // that had been edited twice grew a blank line the canvas drew and the
    // editor never showed.
    for (const r of trimBlockNewline(inner)) pushRun(runs, r);
    return;
  }
  if ($isListItemNode(node)) {
    // Structural nesting (ul > li > ul) and Lexical's own indent property are
    // two spellings of the SAME level, not two levels to add up: buildList
    // rebuilds nesting from listIndent, so summing them made the depth grow on
    // every round trip. Take whichever spelling the editor used.
    const indent = Math.min(MAX_INDENT, Math.max(ctx.listDepth, 1 + (node.getIndent() || 0)));
    const inner: TextRun[] = [];
    for (const child of (node as ElementNode).getChildren()) {
      if ($isListNode(child)) {
        // nested list starts on its own line. Recurse with ctx UNCHANGED: the
        // $isListNode branch above already counts the level, and incrementing
        // here too made every nesting level count twice (a level-2 item came
        // back as listIndent 4 and the canvas indented it twice as far as the
        // editor did).
        if (inner.length > 0) pushRun(inner, { text: "\n" });
        collectRuns(child, inner, ctx);
      } else {
        collectRuns(child, inner, { ...ctx, headingSize: undefined });
      }
    }
    // the item's first run carries the bullet indent marker
    if (inner.length > 0 && inner[0].listIndent === undefined) inner[0].listIndent = indent;
    for (const r of inner) pushRun(runs, r);
    pushRun(runs, { text: "\n" });
    return;
  }
  if ($isHeadingNode(node)) {
    const tag = node.getTag();
    const level = tag.startsWith("h") ? tag : `h${tag}`;
    for (const child of (node as ElementNode).getChildren()) {
      collectRuns(child, runs, { ...ctx, headingSize: HEADING_SIZES[level] ?? HEADING_SIZES.h2 });
    }
    return;
  }
  for (const child of (node as ElementNode).getChildren()) collectRuns(child, runs, ctx);
}

/** The whole editor state → runs (one topic title). */
export function editorStateToRuns(editorState: EditorState): TextRun[] {
  const runs: TextRun[] = [];

```

```

editorState.read() => {
  const root = $getRoot();
  const children = root.getChildren();
  // Collect each root child (paragraph / list / heading) separately so a
  // block boundary becomes a paragraph gap instead of a plain newline.
  const groups: TextRun[][] = [];
  for (const child of children) {
    const g: TextRun[] = [];
    collectRuns(child, g, { listDepth: 0 });
    groups.push(g);
  }
  for (let i = 0; i < groups.length; i++) {
    const g = groups[i];
    if (i > 0 && g.length > 0 && g[0].text !== "\n") g[0].paraGap = true;
    for (const r of g) pushRun(runs, r);
  }
});
return normalizeRuns(runs);
}

// -----
// Reverse: runs → Lexical nodes (seeding the editor / inserting pasted runs)
// -----

function makeTextNode(text: string, run: TextRun): TextNode {
  const node = $createTextNode(text);
  let format = 0;
  if (run.bold) format |= FORMAT_BOLD;
  if (run.italic) format |= FORMAT_ITALIC;
  if (run.underline) format |= FORMAT_UNDERLINE;
  if (format) node.setFormat(format);
  const styles: string[] = [];
  if (run.color) styles.push(`color: ${run.color}`);
  if (run.fontSize) styles.push(`font-size: ${run.fontSize}px`);
  if (styles.length > 0) node.setStyle(styles.join("; "));
  return node;
}

/**
 * Drop the newline that merely CLOSES a block – the block boundary is already
 * the line break. Kept, it made Lexical emit a <br> plus its own "managed"
 * linebreak, i.e. a blank line the editor showed and the canvas did not draw
 * (wrapRunLines discards a trailing empty paragraph): a full extra line of
 * drift on every list item and every pasted block.
 */
function trimBlockNewline(runs: TextRun[]): TextRun[] {
  const out = runs.map((r) => ({ ...r }));
  while (out.length > 0) {
    const last = out[out.length - 1];
    if (last.text === "") {
      out.pop();
      continue;
    }
    if (last.text.endsWith("\n")) {
      last.text = last.text.slice(0, -1);
      if (last.text === "") out.pop();
    }
    break;
  }
  return out;
}

/** Inline children for a block: \n → LineBreakNode, styles per run. */
function inlineChildren(runs: TextRun[]): LexicalNode[] {
  const out: LexicalNode[] = [];
  for (const run of runs) {
    const parts = run.text.split("\n");
    parts.forEach((seg, i) => {
      if (i > 0) out.push($createLineBreakNode());
      if (seg.length === 0) return;
      out.push(makeTextNode(seg, run));
    });
  }
  return out;
}

/** One paragraph of runs: \n → LineBreakNode, styles per run. */
function buildParagraph(runs: TextRun[]): LexicalNode {
  const p = $createParagraphNode();
  for (const child of inlineChildren(trimBlockNewline(runs))) p.append(child);
  return p;
}

/** Rebuild (nested) bullet lists from a group of listIndent runs. */
function buildList(group: TextRun[]): ListNode {
  const root = $createListNode("bullet");

```

```

// Split the group into items. A run with listIndent starts an item; a "\n"
// run is an item boundary only when followed by another listIndent run.
const items: { indent: number; runs: TextRun[] }[] = [];
let cur: { indent: number; runs: TextRun[] } | null = null;
for (let i = 0; i < group.length; i++) {
  const run = group[i];
  if (run.listIndent !== undefined) {
    if (cur) items.push(cur);
    cur = { indent: run.listIndent, runs: [run] };
    continue;
  }
  if (!cur) continue;
  if (run.text === "\n" || run.text === "\n\n") {
    let j = i + 1;
    while (j < group.length && (group[j].text === "\n" || group[j].text.trim() === "")) j++;
    if (j < group.length && group[j].listIndent !== undefined) {
      items.push(cur);
      cur = null;
      continue;
    }
  }
  cur.runs.push(run);
}
if (cur) items.push(cur);

const stack: { list: ListNode; indent: number }[] = [{ list: root, indent: 0 }];
for (const item of items) {
  while (stack.length > 1 && stack[stack.length - 1].indent >= item.indent) stack.pop();
  const li = $createListItemNode();
  // strip listIndent/paraGap so the item content renders as plain text
  const clean = item.runs.map((r) => ({ ...r, listIndent: undefined, paraGap: undefined }));
  // INLINE children, never a nested <p>: a block child inside <li> pushes the
  // bullet onto a line of its own (measured: a 17.5px item became 35px), so
  // every list item cost the canvas a full extra line of drift.
  for (const child of inlineChildren(trimBlockNewline(clean))) li.append(child);
  const childList = $createListNode("bullet");
  li.append(childList);
  stack[stack.length - 1].list.append(li);
  stack.push({ list: childList, indent: item.indent });
}
stripEmptyNestedLists(root);
return root;
}

/**
 * Drop the placeholder sub-lists buildList appends to every item. They hang off
 * the ITEMS, not off the list, so the old version – which only looked at the
 * list's own children (all ListItemNodes) – never removed a single one and
 * every <li> shipped a dead empty <ul> the caret could fall into.
 */
function stripEmptyNestedLists(list: ListNode): void {
  for (const item of (list as ElementNode).getChildren()) {
    if (!$isListItemNode(item)) continue;
    for (const child of (item as ElementNode).getChildren()) {
      if (!$isListNode(child)) continue;
      if ((child as ElementNode).getChildren().length === 0) child.remove();
      else stripEmptyNestedLists(child as ListNode);
    }
  }
}

/** Runs → top-level nodes: paragraphs separated at paraGap, lists as lists. */
export function runsToParagraphNodes(runs: TextRun[]): LexicalNode[] {
  const groups: TextRun[][] = [];
  for (const run of normalizeRuns(runs)) {
    if (run.paraGap) groups.push([]);
    if (groups.length === 0) groups.push([]);
    groups[groups.length - 1].push({ ...run, paraGap: false });
  }
  if (groups.length === 0) groups.push([]);

  const out: LexicalNode[] = [];
  for (const group of groups) {
    if (group.some((r) => r.listIndent !== undefined)) out.push(buildList(group));
    else out.push(buildParagraph(group));
  }
  return out;
}

/** Seed the editor with runs: paragraphs/lists, caret at the end. */
export function setEditorRuns(editor: LexicalEditor, runs: TextRun[]): void {
  editor.update(() => {
    const root = $getRoot();
    root.clear();
    const nodes = runsToParagraphNodes(runs);
    for (const n of nodes) root.append(n);
    const last = nodes[nodes.length - 1];
  });
}

```

```

    if (last && "selectEnd" in last) (last as ElementNode).selectEnd();
  });
}

```

### 39. src/ui/Sidebar.tsx

```

import { useEffect, useRef, useState } from "react";
import { useSyncExternalStore } from "react";
import { useStore } from "../editor/context";
import type { RnodeDocument } from "../core/types";

export function Sidebar(): JSX.Element {
  const store = useStore();
  const state = useSyncExternalStore(store.subscribe, store.getSnapshot);
  const [query, setQuery] = useState("");
  const searchRef = useRef<HTMLInputElement>(null);
  const [renaming, setRenaming] = useState<string | null>(null);

  useEffect(() => {
    const onFocus = (): void => searchRef.current?.focus();
    window.addEventListener("r-node:focus-search", onFocus);
    return () => window.removeEventListener("r-node:focus-search", onFocus);
  }, []);

  const docs = state.docs.filter((d) => !d.archived && (query === "" || d.title.toLowerCase().includes(query.toLowerCase())));
  const archived = state.docs.filter((d) => d.archived);

  return (
    <aside className="sidebar">
      <div className="brand">
        <span className="brand-dot" />
        <span className="brand-name">R-node</span>
      </div>

      <div className="sidebar-search">
        <input
          ref={searchRef}
          placeholder="Search documents... (Ctrl+F)"
          value={query}
          onChange={(e) => setQuery(e.target.value)}
        />
      </div>

      <div className="sidebar-actions">
        <button className="btn primary" onClick={() => store.newDocument()}>
          + New document
        </button>
        <button className="btn" onClick={() => void store.loadFile()} title="Open a .rnode.json file">
          Open file...
        </button>
        <button className="btn" onClick={() => store.switchToDoc(store.duplicateSample())} title="Create a doc from the roadmap template">
          From template
        </button>
      </div>

      <div className="sidebar-list">
        <div className="sidebar-label">Documents</div>
        {docs.map((d) => (
          <DocRow key={d.documentId} doc={d} active={d.documentId === state.activeDocId} renaming={renaming === d.documentId} onRename={(t) => {
            store.renameDocument(d.documentId, t); setRenaming(null); }} onStartRename={() => setRenaming(d.documentId)} />
        ))}
        {docs.length === 0 && <div className="sidebar-empty">No documents match</div>}
      </div>

      {archived.length > 0 && (
        <div className="sidebar-list">
          <div className="sidebar-label">Archived</div>
          {archived.map((d) => (
            <DocRow key={d.documentId} doc={d} active={d.documentId === state.activeDocId} renaming={false} archived onRename={() => undefined}
            onStartRename={() => undefined} />
          ))}
        </div>
      )}

      <div className="sidebar-footer">
        <span>Enter: sibling · Tab: child · Shift+Tab: promote · Space: collapse</span>
      </div>
    </aside>
  );
}

function DocRow({ doc, active, renaming, onRename, onStartRename, archived }: { doc: RnodeDocument; active: boolean; renaming: boolean; onRename: (t: string) => void; onStartRename: () => void; archived?: boolean }): JSX.Element {
  const store = useStore();

```

```

const [text, setText] = useState(doc.title);

if (renaming) {
  return (
    <input
      className="doc-rename"
      value={text}
      autoFocus
      onChange={(e) => setText(e.target.value)}
      onBlur={() => onRename(text)}
      onKeyDown={(e) => {
        if (e.key === "Enter") onRename(text);
        if (e.key === "Escape") onRename(doc.title);
      }}
    />
  );
}

return (
  <div className={`doc-row${active ? " active" : ""}`} onClick={() => store.switchToDoc(doc.documentId)} onDoubleClick={onStartRename}>
    <span className="doc-title">{doc.title}</span>
    <span className="doc-row-actions">
      <button
        title="Duplicate"
        onClick={(e) => { e.stopPropagation(); store.duplicateDocument(doc.documentId); }}
      >
        📄
      </button>
      {archived ? (
        <button
          title="Restore"
          onClick={(e) => { e.stopPropagation(); store.toggleArchive(doc.documentId); }}
        >
          ↶
        </button>
      ) : (
        <button
          title="Archive"
          onClick={(e) => { e.stopPropagation(); store.toggleArchive(doc.documentId); }}
        >
          📁
        </button>
      )}
      <button
        title="Delete permanently"
        onClick={(e) => { e.stopPropagation(); store.deleteDocument(doc.documentId); }}
      >
        🗑️
      </button>
    </span>
  </div>
  );
}

```

## 40. src/App.tsx

```

import { useEffect } from "react";
import { useSyncExternalStore } from "react";
import { useStore } from "../editor/context";
import { handleShortcut } from "../editor/shortcuts";
import { viewSize } from "../editor/view";
import { Sidebar } from "../ui/Sidebar";
import { TopBar } from "../ui/TopBar";
import { CanvasView } from "../ui/CanvasView";
import { Outliner } from "../ui/Outliner";
import { Inspector } from "../ui/Inspector";
import { Palette } from "../ui/Palette";

export function App(): JSX.Element {
  const store = useStore();
  const state = useSyncExternalStore(store.subscribe, store.getSnapshot);

  useEffect(() => {
    document.documentElement.dataset.theme = state.theme;
  }, [state.theme]);

  useEffect(() => {
    const onKey = (e: KeyboardEvent): void => {
      const target = e.target as HTMLElement | null;
      const tag = target?.tagName ?? "";
      const isField = tag === "INPUT" || tag === "TEXTAREA" || tag === "SELECT" || !!target?.isContentEditable;
      if (isField) {
        // Save/Open must work even from a text field (Inspector title, doc
        // title, search...): blur it first so its onBlur commit fires, then run

```

```

    // the shortcut. Otherwise Ctrl+S falls through to the browser's
    // "Save page" dialog and nothing is saved.
    const mod = e.ctrlKey || e.metaKey;
    const key = e.key.toLowerCase();
    if (mod && (key === "s" || key === "o")) {
      e.preventDefault();
      (target as HTMLElement).blur();
      if (handleShortcut(store, e, viewSize.w, viewSize.h)) e.preventDefault();
    }
    return;
  }
  if (handleShortcut(store, e, viewSize.w, viewSize.h)) e.preventDefault();
};
window.addEventListener("keydown", onKey);
return () => window.removeEventListener("keydown", onKey);
}, [store]);

const zen = state.zen;

return (
  <div className="app">
    {!zen && <Sidebar />}

    <div className="main">
      {!zen && <TopBar />}

      <div className="workspace">
        <div className="canvas-area">
          <CanvasView />
          {!zen && state.showOutliner && <Outliner />}
        </div>
        {!zen && state.showInspector && <Inspector />}
      </div>

      {!zen && <StatusBar />}
    </div>

    <Palette />

    {state.message && (
      <div className="toast" onClick={() => (store as unknown as { toast: (m: string) => void }).toast("")}>
        {state.message}
      </div>
    )}
  </div>
);
}

function StatusBar(): JSX.Element {
  const store = useStore();
  const state = useSyncExternalStore(store.subscribe, store.getSnapshot);
  return (
    <footer className="statusbar">
      <span>{Math.round(state.camera.scale * 100)}%</span>
      <span className="sep">·</span>
      <span>{state.selection.length > 0 ? ` ${state.selection.length} selected` : ` ${state.doc.visibleNodeCount} topics`}</span>
      <span className="sep">·</span>
      <span className={state.sync}>{state.sync === "saved" ? "Saved" : "Unsaved changes"}</span>
      <span className="spacer" />
      {state.relFrom && <span className="status-hint">Link mode: click a target</span>}
      <span className="sep">·</span>
      <span>Enter/Tab create · drag to move · right-drag to pan · Ctrl+scroll to zoom</span>
    </footer>
  );
}

```

## 41. src-tauri/capabilities/default.json

```

{
  "$schema": "../gen/schemas/desktop-schema.json",
  "identifier": "default",
  "description": "Default capability for the main window",
  "windows": ["main"],
  "permissions": [
    "core:default",
    "dialog:default",
    {
      "identifier": "http:default",
      "allow": [{ "url": "https://**" }, { "url": "http://**" }]
    }
  ]
}

```

## 42. tests/tauriAdapter.test.ts

```
import { afterEach, describe, expect, it, vi } from "vitest";
import { TauriAssetStore, type AssetMeta } from "../src/persist/assets";
import { TauriStorageAdapter } from "../src/persist/storage";
import { EditorStore } from "../src/editor/store";
import type { RnodeDocument } from "../src/core/types";

type InvokeArgs = Record<string, unknown>;

/** Install a fake window.__TAURI__.core.invoke; records every call. */
function installTauri(impl: (cmd: string, args: InvokeArgs) => Promise<unknown>): { cmd: string; args: InvokeArgs }[] {
  const calls: { cmd: string; args: InvokeArgs }[] = [];
  const invoke = vi.fn(async (cmd: string, args: InvokeArgs) => {
    calls.push({ cmd, args });
    return impl(cmd, args);
  });
  (globalThis as unknown as { window?: unknown }).window = { __TAURI__: { core: { invoke } } };
  return calls;
}

afterEach(() => {
  delete (globalThis as unknown as { window?: unknown }).window;
  vi.restoreAllMocks();
});

const ID = "a".repeat(64);
const meta: AssetMeta = { id: ID, mime: "image/png", w: 10, h: 10, bytes: 3, name: "x.png" };
const metaBytes = [...new TextEncoder().encode(JSON.stringify(meta))];

/** Raw bytes as Tauri actually delivers them: an ArrayBuffer. */
function raw(bytes: number[]): ArrayBuffer {
  return Uint8Array.from(bytes).buffer;
}

describe("TauriAssetStore", () => {
  it("passes the current path to every command", async () => {
    const calls = installTauri(async (cmd, args) => {
      if (cmd === "default_document_path") return "C:/default";
      if (cmd === "get_asset" && args.level === "meta") return metaBytes;
      if (cmd === "get_asset") return raw([1, 2, 3]);
      return null;
    });
    const store = new TauriAssetStore();
    await store.get(ID, "large");
    await store.meta(ID);
    await store.list();
    const paths = calls.filter((c) => c.cmd !== "default_document_path").map((c) => c.args.path);
    expect(paths.every((p) => p === "C:/default")).toBe(true);
  });

  it("get returns a Blob from raw IPC bytes (ArrayBuffer)", async () => {
    const calls = installTauri(async (cmd, args) => {
      if (cmd === "default_document_path") return "C:/default";
      if (cmd === "get_asset" && args.level === "meta") return metaBytes;
      if (cmd === "get_asset") return raw([1, 2, 3]);
      return null;
    });
    const store = new TauriAssetStore();
    const blob = await store.get(ID, "large");
    expect(blob).not.toBeNull();
    expect(new Uint8Array(await blob!.arrayBuffer())).toEqual(Uint8Array.from([1, 2, 3]));
  });

  it("get of a missing asset returns null (empty body)", async () => {
    const calls = installTauri(async (cmd) => (cmd === "default_document_path" ? "C:/default" : new ArrayBuffer(0)));
    const store = new TauriAssetStore();
    expect(await store.get(ID, "large")).toBeNull();
  });

  it("setRoot switches the SAME instance to a new path (the Renderer trap)", async () => {
    const calls = installTauri(async () => null);
    const store = new TauriAssetStore();

    store.setRoot("C:/mapA.rnode");
    await store.get(ID, "large");
    store.setRoot("C:/mapB.rnode");
    await store.get(ID, "large");

    const reads = calls.filter((c) => c.cmd === "get_asset").map((c) => c.args.path);
    expect(reads).toEqual(["C:/mapA.rnode", "C:/mapB.rnode"]);
  });

  it("adoptFile copies every referenced asset to the new file BEFORE switching", async () => {
    const OLD = "C:/old";
    const NEW = "C:/new.rnode";
  });
});
```

```

const calls = installTauri(async (cmd, args) => {
  if (cmd === "default_document_path") return OLD;
  if (cmd === "get_asset" && args.level === "meta") return metaBytes;
  if (cmd === "get_asset" && args.level === "original") return raw([1, 2, 3]);
  if (cmd === "get_asset" && args.level === "large") return raw([4, 5]);
  if (cmd === "get_asset" && args.level === "small") return raw([6]);
  return null;
});
const store = new TauriAssetStore();
await store.adoptFile(NEW, [ID]);

// Reads happened against the OLD path...
const reads = calls.filter((c) => c.cmd === "get_asset").map((c) => c.args.path);
expect(reads.length).toBeGreaterThan(0);
expect(reads.every((p) => p === OLD)).toBe(true);

// ...and the writes went to the NEW one: all four levels per asset.
const puts = calls.filter((c) => c.cmd === "put_asset");
expect(puts).toHaveLength(4);
expect(puts.every((p) => p.args.path === NEW)).toBe(true);
expect(puts.map(p => p.args.level).sort()).toEqual(["large", "meta", "original", "small"]);

// After the switch, reads come from the new file.
calls.length = 0;
await store.get(ID, "large");
expect(calls.find((c) => c.cmd === "get_asset")?.args.path).toBe(NEW);
});

it("adoptFile with no referenced assets just switches the path", async () => {
  const calls = installTauri(async (cmd) => (cmd === "default_document_path" ? "C:/default" : null));
  const store = new TauriAssetStore();
  await store.adoptFile("C:/fresh.rnode", []);
  expect(calls.filter((c) => c.cmd === "put_asset")).toHaveLength(0);
  calls.length = 0;
  await store.list();
  expect(calls.find((c) => c.cmd === "list_assets")?.args.path).toBe("C:/fresh.rnode");
});
});

describe("TauriStorageAdapter", () => {
  const doc = {
    documentId: "d1",
    title: "M",
    sheets: [{ rootNodeId: "r", nodes: {} }],
  } as unknown as RnodeDocument;

  it("starts without a root and reports it after setRoot", () => {
    installTauri(async () => null);
    const a = new TauriStorageAdapter();
    expect(a.hasRoot).toBe(false);
    a.setRoot("C:/map.rnode");
    expect(a.hasRoot).toBe(true);
  });

  it("save writes the document row of the current file", async () => {
    const calls = installTauri(async () => null);
    const a = new TauriStorageAdapter();
    a.setRoot("C:/map.rnode");
    await a.save([doc]);
    const write = calls.find((c) => c.cmd === "write_document");
    expect(write?.args.path).toBe("C:/map.rnode");
    expect(JSON.parse(write?.args.data as string)).toEqual(doc);
  });

  it("save without a path throws instead of guessing", async () => {
    installTauri(async () => null);
    const a = new TauriStorageAdapter();
    await expect(a.save([doc])).rejects.toThrow(/no document file/);
  });

  it("load returns the document of the current file", async () => {
    installTauri(async (cmd) => (cmd === "read_document" ? JSON.stringify(doc) : null));
    const a = new TauriStorageAdapter();
    a.setRoot("C:/map.rnode");
    expect(await a.load()).toEqual([doc]);
  });

  it("load returns [] before a path is chosen, without any command", async () => {
    const calls = installTauri(async () => null);
    const a = new TauriStorageAdapter();
    expect(await a.load()).toEqual([]);
    expect(calls).toHaveLength(0);
  });

  it("readDocumentAt reads a file without switching the path", async () => {
    installTauri(async (cmd) => (cmd === "read_document" ? JSON.stringify(doc) : null));
    const a = new TauriStorageAdapter();
  });

```

```

    expect(await a.readDocumentAt("C:/other.rnode")).toEqual(doc);
    expect(a.hasRoot).toBe(false);
  });
});

describe("desktop save flow", () => {
  it("first save without a file is cancelled when the picker is dismissed", async () => {
    installTauri(async (cmd) => (cmd === "pick_document_file" ? null : null));
    const store = new EditorStore(new TauriStorageAdapter());
    await store.init();
    await store.saveNow();
    expect(store.getSnapshot().sync).toBe("dirty");
  });

  it("first save adopts the scratch assets into the chosen file and writes the document", async () => {
    const calls = installTauri(async (cmd) => {
      if (cmd === "pick_document_file") return "C:/MiaMappa.rnode";
      return null;
    });
    const store = new EditorStore(new TauriStorageAdapter());
    await store.init();
    await store.saveNow();
    expect(store.getSnapshot().sync).toBe("saved");
    // The empty sample map has no referenced assets: adoptFile writes nothing.
    expect(calls.filter((c) => c.cmd === "put_asset")).toHaveLength(0);
    const write = calls.find((c) => c.cmd === "write_document");
    expect(write?.args.path).toBe("C:/MiaMappa.rnode");
    // The picker for the save mode carries the mode argument and the document
    // title as the suggested file name – the filesystem dialog already knows it.
    const pick = calls.find((c) => c.cmd === "pick_document_file");
    expect(pick?.args.mode).toBe("save");
    expect(pick?.args.suggestedName).toBe("R-node_Roadmap");
  });

  it("renaming the document renames the REAL file on the next save", async () => {
    const calls = installTauri(async (cmd) => {
      if (cmd === "pick_document_file") return "C:/Docs/Old.rnode";
      if (cmd === "document_file_exists") return false;
      return null;
    });
    const store = new EditorStore(new TauriStorageAdapter());
    await store.init();
    await store.saveNow(); // first save -> C:/Docs/Old.rnode
    store.renameDocument(store.getSnapshot().activeDocId, "Vacation");
    await store.saveNow();

    const writes = calls.filter((c) => c.cmd === "write_document");
    // The second save wrote to the NEW name, not the old file.
    expect(writes[writes.length - 1].args.path).toBe("C:/Docs/Vacation.rnode");
    // The old file was removed only after the new one was written.
    const removals = calls.filter((c) => c.cmd === "remove_document");
    expect(removals).toHaveLength(1);
    expect(removals[0].args.path).toBe("C:/Docs/Old.rnode");
    const writeIdx = writes.findIndex((w) => w.args.path === "C:/Docs/Vacation.rnode");
    const removeIdx = calls.findIndex((c) => c.cmd === "remove_document");
    expect(writeIdx).toBeGreaterThan(-1);
    expect(removeIdx).toBeGreaterThan(writeIdx); // remove comes after the write
    // The rename did not re-ask where to save.
    expect(calls.filter((c) => c.cmd === "pick_document_file")).toHaveLength(1);
  });

  it("opening a file whose title differs from its name does NOT rename it on save", async () => {
    const doc = {
      documentId: "d1",
      title: "Internal Title",
      sheets: [{ rootNodeId: "r", nodes: { r: { id: "r", title: "r", childrenIds: [] } } }],
    };
    const calls = installTauri(async (cmd) => {
      if (cmd === "pick_document_file") return "C:/Docs/FileOnDisk.rnode";
      if (cmd === "read_document") return JSON.stringify(doc);
      if (cmd === "document_file_exists") return false;
      return null;
    });
    const store = new EditorStore(new TauriStorageAdapter());
    await store.init();
    await store.loadFile(); // open FileOnDisk.rnode (internal title: Internal Title)
    expect(store.getSnapshot().docs.length).toBe(2);
    await store.saveNow();

    const writes = calls.filter((c) => c.cmd === "write_document");
    expect(writes[writes.length - 1].args.path).toBe("C:/Docs/FileOnDisk.rnode");
    expect(calls.filter((c) => c.cmd === "remove_document")).toHaveLength(0);
  });

  it("a later save overwrites the same file without asking again", async () => {
    const calls = installTauri(async (cmd) => {
      if (cmd === "pick_document_file") return "C:/MiaMappa.rnode";

```

```

    return null;
  });
  const store = new EditorStore(new TauriStorageAdapter());
  await store.init();
  await store.saveNow();
  await store.saveNow();
  const picks = calls.filter((c) => c.cmd === "pick_document_file");
  expect(picks).toHaveLength(1); // the picker ran only for the first save
  expect(calls.filter((c) => c.cmd === "write_document")).toHaveLength(2);
  });
});

```

## 43. index.html

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no" />
    <meta name="description" content="R-node – visual thinking workspace: mind maps, trees, timelines, matrices." />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <link rel="icon" type="image/png" sizes="32x32" href="/favicon-32.png" />
    <link rel="icon" type="image/png" sizes="16x16" href="/favicon-16.png" />
    <link rel="apple-touch-icon" href="/apple-touch-icon.png" />
    <meta name="theme-color" content="#11141c" />
    <title>R-node</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>

```

## 44. src/core/ops.ts

```

/**
 * Operation system.
 *
 * Every meaningful edit is expressed as an Operation. Operations are:
 * - self-contained (carry the data needed to apply AND to reverse);
 * - idempotent / replayable (same starting state -> same result);
 * - tagged with opId/actorId/timestamp so the same log can later feed the
 *   collaboration layer (CRDT/OT) unchanged.
 *
 * applyOp mutates the sheet in place. inverseOf returns the operation(s)
 * that exactly reverse it, which is what the History stack uses for undo.
 */
import type { Group, MindNode, Position, Relationship, Sheet, StructureConfig, Style, Summary, TaskInfo, TextRun } from "../types";
import { nowIso } from "../doc";

// -----
// Op definitions
// -----

export type Op =
  | { opId: string; actorId: string; ts: string; type: "createNode"; id: string; nodeType: MindNode["type"]; parentId: string | null; index: number;
    title: string; titleRuns?: TextRun[]; style?: Style; task?: TaskInfo | null; position?: { x: number; y: number; manual: boolean } }
  | { opId: string; actorId: string; ts: string; type: "restoreNode"; id: string; parentId: string | null; index: number; subtree: MindNode[];
    removedRelationships: Relationship[] }
  | { opId: string; actorId: string; ts: string; type: "deleteNode"; id: string; parentId: string | null; index: number; subtree: MindNode[];
    removedRelationships: Relationship[] }
  | { opId: string; actorId: string; ts: string; type: "setTitle"; id: string; title: string; prev: string; titleRuns?: TextRun[]; prevRuns?: TextRun[] }
  | { opId: string; actorId: string; ts: string; type: "setStyle"; id: string; style: Style; prev: Style }
  | { opId: string; actorId: string; ts: string; type: "setNodeImage"; nodeId: string; imageId: string | null; prevImageId: string | null }
  | { opId: string; actorId: string; ts: string; type: "setPosition"; id: string; x: number; y: number; manual: boolean; offsetX?: number; offsetY?:
    number; prev: Position }
  | { opId: string; actorId: string; ts: string; type: "setCollapsed"; id: string; collapsed: boolean; prev: boolean }
  | { opId: string; actorId: string; ts: string; type: "moveNode"; id: string; fromParentId: string | null; fromIndex: number; toParentId: string |
    null; toIndex: number }
  | { opId: string; actorId: string; ts: string; type: "sortSiblings"; parentId: string; order: string[]; prevOrder: string[] }
  | { opId: string; actorId: string; ts: string; type: "setTask"; id: string; task: TaskInfo | null; prev: TaskInfo | null }
  | { opId: string; actorId: string; ts: string; type: "setNotes"; id: string; notes: string; prev: string }
  | { opId: string; actorId: string; ts: string; type: "setSheetTitle"; title: string; prev: string }
  | { opId: string; actorId: string; ts: string; type: "setStructure"; config: StructureConfig; prev: StructureConfig }
  | { opId: string; actorId: string; ts: string; type: "createRelationship"; relationship: Relationship }
  | { opId: string; actorId: string; ts: string; type: "deleteRelationship"; id: string; relationship: Relationship }
  | { opId: string; actorId: string; ts: string; type: "setRelationship"; id: string; relationship: Relationship; prev: Relationship }
  | { opId: string; actorId: string; ts: string; type: "createGroup"; group: Group }
  | { opId: string; actorId: string; ts: string; type: "deleteGroup"; id: string; group: Group }

```

```

| { opId: string; actorId: string; ts: string; type: "setGroup"; id: string; group: Group; prev: Group }
| { opId: string; actorId: string; ts: string; type: "createSummary"; summary: Summary }
| { opId: string; actorId: string; ts: string; type: "deleteSummary"; id: string; summary: Summary }
| { opId: string; actorId: string; ts: string; type: "setSummary"; id: string; summary: Summary; prev: Summary };

export interface OpMeta {
  actorId?: string;
}

let seqCounter = 0;

/** Build an op with the standard envelope filled in. */
export function makeOp<T extends { type: Op["type"] }>(
  type: T["type"],
  payload: Omit<T, "type" | "opId" | "actorId" | "ts">,
  meta?: OpMeta
): Op {
  return { opId: `op_${Date.now().toString(36)}_${(seqCounter++).toString(36)}`, actorId: meta?.actorId ?? "local", ts: nowIso(), type, ...payload } as
  unknown as Op;
}

export function cloneNode(n: MindNode): MindNode {
  return {
    ...n,
    childrenIds: [...n.childrenIds],
    style: { ...n.style },
    labels: [...n.labels],
    markers: [...n.markers],
    task: n.task ? { ...n.task } : null,
    metadata: { ...n.metadata },
    position: { ...n.position },
    titleRuns: n.titleRuns ? n.titleRuns.map((r) => ({ ...r })) : undefined,
  };
}

// -----
// applyOp
// -----

export function applyOp(sheet: Sheet, op: Op): void {
  const nodes = sheet.nodes;
  switch (op.type) {
    case "createNode": {
      const parent = op.parentId ? nodes[op.parentId] : undefined;
      const node: MindNode = {
        id: op.id,
        type: op.nodeType,
        parentId: op.parentId,
        childrenIds: [],
        title: op.title,
        titleRuns: op.titleRuns ? op.titleRuns.map((r) => ({ ...r })) : undefined,
        position: op.position ?? { x: 0, y: 0, manual: false },
        style: op.style ?? {},
        collapsed: false,
        labels: [],
        markers: [],
        notes: "",
        task: op.task ?? null,
        metadata: { createdAt: nowIso(), updatedAt: nowIso() },
      };
      nodes[op.id] = node;
      if (parent) parent.childrenIds.splice(clampIndex(op.index, parent.childrenIds.length), 0, op.id);
      break;
    }
    case "restoreNode": {
      for (const n of op.subtree) nodes[n.id] = cloneNode(n);
      if (op.parentId) {
        const parent = nodes[op.parentId];
        if (parent) parent.childrenIds.splice(clampIndex(op.index, parent.childrenIds.length), 0, op.id);
      }
      for (const rel of op.removedRelationships) sheet.relationships.push(rel);
      break;
    }
    case "deleteNode": {
      for (const n of op.subtree) delete nodes[n.id];
      if (op.parentId) {
        const parent = nodes[op.parentId];
        if (parent) {
          const idx = parent.childrenIds.indexOf(op.id);
          if (idx >= 0) parent.childrenIds.splice(idx, 1);
        }
      }
      sheet.relationships = sheet.relationships.filter((r) => !op.removedRelationships.some((rr) => rr.id === r.id));
      break;
    }
    case "setTitle": {
      const node = nodes[op.id];

```

```

    node.title = op.title;
    if (op.titleRuns) node.titleRuns = op.titleRuns.map((r) => ({ ...r }));
    else delete node.titleRuns;
    break;
}
case "setStyle":
    nodes[op.id].style = { ...op.style };
    break;
case "setNodeImage": {
    const node = nodes[op.nodeId];
    if (!node) break;
    // The op carries ONLY the id - never image bytes (ADR-001 §12).
    const style = { ...node.style };
    if (op.imageId) style.image = op.imageId;
    else delete style.image;
    node.style = style;
    break;
}
case "setPosition":
    nodes[op.id].position = { x: op.x, y: op.y, manual: op.manual, offsetX: op.offsetX, offsetY: op.offsetY };
    break;
case "setCollapsed":
    nodes[op.id].collapsed = op.collapsed;
    break;
case "moveNode": {
    // fromIndex/toIndex are FINAL indices: toIndex is the position the node
    // will occupy in the destination array after removal. Commands compute
    // them by simulating removal first, so applyOp needs no adjustment.
    const fromParent = op.fromParentId ? nodes[op.fromParentId] : undefined;
    const toParent = op.toParentId ? nodes[op.toParentId] : undefined;
    if (!nodes[op.id]) break;
    if (fromParent) {
        const idx = fromParent.childrenIds.indexOf(op.id);
        if (idx < 0) break;
        fromParent.childrenIds.splice(idx, 1);
    } else if (nodes[op.id].parentId !== null) {
        break;
    }
    if (toParent) toParent.childrenIds.splice(clampIndex(op.toIndex, toParent.childrenIds.length), 0, op.id);
    nodes[op.id].parentId = op.toParentId;
    break;
}
case "sortSiblings": {
    const parent = nodes[op.parentId];
    if (!parent) break;
    parent.childrenIds = [...op.order];
    break;
}
case "setTask":
    nodes[op.id].task = op.task ? { ...op.task } : null;
    break;
case "setNotes":
    nodes[op.id].notes = op.notes;
    break;
case "setSheetTitle":
    sheet.title = op.title;
    break;
case "setStructure":
    sheet.structure = { ...op.config };
    break;
case "createRelationship":
    sheet.relationships.push({ ...op.relationship });
    break;
case "deleteRelationship":
    sheet.relationships = sheet.relationships.filter((r) => r.id !== op.id);
    break;
case "setRelationship": {
    const idx = sheet.relationships.findIndex((r) => r.id === op.id);
    if (idx >= 0) sheet.relationships[idx] = { ...op.relationship };
    break;
}
case "createGroup":
    sheet.boundaries.push({ ...op.group, memberIds: [...op.group.memberIds] });
    break;
case "deleteGroup":
    sheet.boundaries = sheet.boundaries.filter((g) => g.id !== op.id);
    break;
case "setGroup": {
    const idx = sheet.boundaries.findIndex((g) => g.id === op.id);
    if (idx >= 0) sheet.boundaries[idx] = { ...op.group, memberIds: [...op.group.memberIds] };
    break;
}
case "createSummary":
    sheet.summaries.push({ ...op.summary, memberIds: [...op.summary.memberIds] });
    break;
case "deleteSummary":
    sheet.summaries = sheet.summaries.filter((s) => s.id !== op.id);

```

```

    break;
  case "setSummary": {
    const idx = sheet.summaries.findIndex((s) => s.id === op.id);
    if (idx >= 0) sheet.summaries[idx] = { ...op.summary, memberIds: [...op.summary.memberIds] };
    break;
  }
}
}

function clampIndex(i: number, len: number): number {
  return Math.max(0, Math.min(i, len));
}

// -----
// inverseOf – self-contained reversals used by undo
// -----

export function inverseOf(op: Op, meta?: OpMeta): Op[] {
  const m = { actorId: meta?.actorId ?? "local" };
  switch (op.type) {
    case "createNode":
      return [makeOp<Op & { type: "deleteNode" }>("deleteNode", { id: op.id, parentId: op.parentId, index: op.index, subtree: [], removedRelationships: [], m })];
    case "deleteNode":
      return [makeOp<Op & { type: "restoreNode" }>("restoreNode", { id: op.id, parentId: op.parentId, index: op.index, subtree: op.subtree, removedRelationships: op.removedRelationships }, m)];
    case "restoreNode":
      return [makeOp<Op & { type: "deleteNode" }>("deleteNode", { id: op.id, parentId: op.parentId, index: op.index, subtree: op.subtree, removedRelationships: op.removedRelationships }, m)];
    case "setTitle":
      return [makeOp<Op & { type: "setTitle" }>("setTitle", { id: op.id, title: op.prev, prev: op.title, titleRuns: op.prevRuns, prevRuns: op.titleRuns }, m)];
    case "setStyle":
      return [makeOp<Op & { type: "setStyle" }>("setStyle", { id: op.id, style: op.prev, prev: op.style }, m)];
    case "setNodeImage":
      return [makeOp<Op & { type: "setNodeImage" }>("setNodeImage", { nodeId: op.nodeId, imageId: op.prevImageId, prevImageId: op.imageId }, m)];
    case "setPosition":
      return [makeOp<Op & { type: "setPosition" }>("setPosition", {
        id: op.id,
        x: op.prev.x,
        y: op.prev.y,
        manual: op.prev.manual,
        offsetX: op.prev.offsetX,
        offsetY: op.prev.offsetY,
        prev: { x: op.x, y: op.y, manual: op.manual, offsetX: op.offsetX, offsetY: op.offsetY },
      }, m)];
    case "setCollapsed":
      return [makeOp<Op & { type: "setCollapsed" }>("setCollapsed", { id: op.id, collapsed: op.prev, prev: op.collapsed }, m)];
    case "moveNode":
      return [makeOp<Op & { type: "moveNode" }>("moveNode", { id: op.id, fromParentId: op.toParentId, fromIndex: op.toIndex, toParentId: op.fromParentId, toIndex: op.fromIndex }, m)];
    case "sortSiblings":
      return [makeOp<Op & { type: "sortSiblings" }>("sortSiblings", { parentId: op.parentId, order: op.prevOrder, prevOrder: op.order }, m)];
    case "setTask":
      return [makeOp<Op & { type: "setTask" }>("setTask", { id: op.id, task: op.prev, prev: op.task }, m)];
    case "setNotes":
      return [makeOp<Op & { type: "setNotes" }>("setNotes", { id: op.id, notes: op.prev, prev: op.notes }, m)];
    case "setSheetTitle":
      return [makeOp<Op & { type: "setSheetTitle" }>("setSheetTitle", { title: op.prev, prev: op.title }, m)];
    case "setStructure":
      return [makeOp<Op & { type: "setStructure" }>("setStructure", { config: op.prev, prev: op.config }, m)];
    case "createRelationship":
      return [makeOp<Op & { type: "deleteRelationship" }>("deleteRelationship", { id: op.relationship.id, relationship: op.relationship }, m)];
    case "deleteRelationship":
      return [makeOp<Op & { type: "createRelationship" }>("createRelationship", { relationship: op.relationship }, m)];
    case "setRelationship":
      return [makeOp<Op & { type: "setRelationship" }>("setRelationship", { id: op.id, relationship: op.prev, prev: op.relationship }, m)];
    case "createGroup":
      return [makeOp<Op & { type: "deleteGroup" }>("deleteGroup", { id: op.group.id, group: op.group }, m)];
    case "deleteGroup":
      return [makeOp<Op & { type: "createGroup" }>("createGroup", { group: op.group }, m)];
    case "setGroup":
      return [makeOp<Op & { type: "setGroup" }>("setGroup", { id: op.id, group: op.prev, prev: op.group }, m)];
    case "createSummary":
      return [makeOp<Op & { type: "deleteSummary" }>("deleteSummary", { id: op.summary.id, summary: op.summary }, m)];
    case "deleteSummary":
      return [makeOp<Op & { type: "createSummary" }>("createSummary", { summary: op.summary }, m)];
    case "setSummary":
      return [makeOp<Op & { type: "setSummary" }>("setSummary", { id: op.id, summary: op.prev, prev: op.summary }, m)];
  }
}

/**
 * Apply an op and return its inverse ops (for the history stack).
 * createNode's inverse needs the actual node after creation, so it is
 * captured here with sheet access; every other op is self-contained.
 */

```

```

export function applyWithInverse(sheet: Sheet, op: Op): Op[] {
  applyOp(sheet, op);
  if (op.type === "createNode") {
    const node = sheet.nodes[op.id];
    const deleteOp = makeOp<Op & { type: "deleteNode" }>("deleteNode", {
      id: op.id,
      parentId: op.parentId,
      index: op.index,
      subtree: node ? [cloneNode(node)] : [],
      removedRelationships: [],
    }, { actorId: op.actorId });
    return [deleteOp];
  }
  return inverseOf(op, { actorId: op.actorId });
}

```

## 45. src/dev/trace.ts

```

/**
 * Session tracer – a repro capture, NOT a monitoring system.
 *
 * The problem it solves: a person testing the app can see the effect of a bug
 * ("the arrows disappear when I zoom out") but never the cause, because the
 * decisive information is invisible from outside – whether the renderer drew
 * the arrows at all, whether an input was deliberately ignored, whether the
 * layout ran. Describing the symptom in prose forces whoever fixes it to
 * re-derive all of that by reading code.
 *
 * So this records the DECISIONS, not just the actions. Every guard that drops
 * an input logs why it dropped it (example shape):
 *
 * [INPUT] wheel ctrl=true → ignored (reason)
 *
 * That single line answers "why did this gesture do nothing", which no amount
 * of prose from the user ever could. Applied gestures are traced too, so a
 * dead input is distinguishable from a handled one.
 *
 * Usage: press Ctrl+Shift+D at the moment the bug happens; a JSON bundle of
 * the last few hundred events downloads. Paste it to whoever (or whatever) is
 * fixing it. Also on `window.__rnodeTrace` for console use.
 *
 * Disabled in production builds: every entry point returns immediately, so
 * the cost is a boolean test.
 */

const CAPACITY = 500;

export type TraceEvent =
  | { kind: "input"; t: number; n: number; what: string; outcome: "applied" | "ignored"; reason?: string; detail?: Detail }
  | { kind: "op"; t: number; n: number; types: string; count: number; ms: number }
  | { kind: "layout"; t: number; n: number; nodes: number; ms: number }
  | { kind: "render"; t: number; n: number; frames: number; scale: number; nodes: number; visible: number; rels: number; relsDrawn: number; links:
number; linksDrawn: number; textHits: number; textMisses: number; imgVisible: number; imgCached: number; imgBytes: number; imgInflight: number; maxMs:
number }
  | { kind: "edit"; t: number; n: number; what: string; nodeId?: string; detail?: Detail }
  | { kind: "invariant"; t: number; n: number; message: string }
  | { kind: "error"; t: number; n: number; message: string; stack?: string };

type Detail = Record<string, string | number | boolean | null | undefined>;

function devEnabled(): boolean {
  try {
    return Boolean((import.meta as unknown as { env?: { DEV?: boolean } }).env?.DEV);
  } catch {
    return false;
  }
}

let enabled = devEnabled();
const buffer: TraceEvent[] = [];
const t0 = typeof performance !== "undefined" ? performance.now() : 0;

function now(): number {
  return Math.round(((typeof performance !== "undefined" ? performance.now() : 0) - t0) * 10) / 10;
}

/**
 * Append, coalescing repeats. Without this a wheel gesture (dozens of events)
 * or a render loop would flush the whole buffer in a second and the capture
 * would hold nothing but the last instant – exactly the part you already know.
 */
function push(ev: TraceEvent, mergeKey: string): void {
  if (!enabled) return;
  const last = buffer[buffer.length - 1] as (TraceEvent & { _k?: string }) | undefined;

```

```

    if (last && last._k === mergeKey) {
      last.n += 1;
      last.t = ev.t;
      return;
    }
    (ev as TraceEvent & { _k?: string })._k = mergeKey;
    buffer.push(ev);
    if (buffer.length > CAPACITY) buffer.shift();
  }

  // -----
  // Render coalescing: 60 fps would drown everything else, but the per-frame
  // counters are the whole point for "it isn't painted" bugs. So frames are
  // aggregated and flushed at most every 250 ms, or immediately when the shape
  // of the frame changes (zoom bucket, visible count, relationships drawn).
  // -----

interface RenderSample {
  scale: number;
  nodes: number;
  visible: number;
  rels: number;
  relsDrawn: number;
  links: number;
  linksDrawn: number;
  textHits: number;
  textMisses: number;
  imgVisible: number;
  imgCached: number;
  imgBytes: number;
  imgInflight: number;
}

let pending: RenderSample | null = null;
let pendingFrames = 0;
let pendingMaxMs = 0;
let lastFlush = -Infinity;

function shapeOf(s: RenderSample): string {
  return `${s.scale.toFixed(2)}|${s.visible}|${s.rels}|${s.relsDrawn}|${s.linksDrawn}`;
}

function flushRender(): void {
  if (!pending) return;
  push(
    {
      kind: "render",
      t: now(),
      n: 1,
      frames: pendingFrames,
      scale: Math.round(pending.scale * 1000) / 1000,
      nodes: pending.nodes,
      visible: pending.visible,
      rels: pending.rels,
      relsDrawn: pending.relsDrawn,
      links: pending.links,
      linksDrawn: pending.linksDrawn,
      textHits: pending.textHits,
      textMisses: pending.textMisses,
      imgVisible: pending.imgVisible,
      imgCached: pending.imgCached,
      imgBytes: pending.imgBytes,
      imgInflight: pending.imgInflight,
      maxMs: Math.round(pendingMaxMs * 100) / 100,
    },
    `render:${shapeOf(pending)}`
  );
  pending = null;
  pendingFrames = 0;
  pendingMaxMs = 0;
}

// -----
// Public API
// -----

export const trace = {
  get enabled(): boolean {
    return enabled;
  },

  enable(on = true): void {
    enabled = on;
  },

  /** An input that was acted on. */
  applied(what: string, detail?: Detail): void {

```

```

    if (!enabled) return;
    push({ kind: "input", t: now(), n: 1, what, outcome: "applied", detail }, `input:${what}:applied`);
  },

  /**
   * An input that was deliberately NOT acted on, and why. This is the most
   * valuable call in the file: "nothing happened" is indistinguishable from
   * "it was dropped on purpose" from the outside.
   */
  ignored(what: string, reason: string, detail?: Detail): void {
    if (!enabled) return;
    push({ kind: "input", t: now(), n: 1, what, outcome: "ignored", reason, detail }, `input:${what}:ignored:${reason}`);
  },

  op(types: string, count: number, ms: number): void {
    if (!enabled) return;
    push({ kind: "op", t: now(), n: 1, types, count, ms: Math.round(ms * 100) / 100 }, `op:${types}`);
  },

  layout(nodes: number, ms: number): void {
    if (!enabled) return;
    push({ kind: "layout", t: now(), n: 1, nodes, ms: Math.round(ms * 100) / 100 }, `layout:${nodes}`);
  },

  edit(what: string, nodeId?: string, detail?: Detail): void {
    if (!enabled) return;
    push({ kind: "edit", t: now(), n: 1, what, nodeId, detail }, `edit:${what}:${nodeId ?? ""}`);
  },

  invariant(message: string): void {
    if (!enabled) return;
    push({ kind: "invariant", t: now(), n: 1, message }, `invariant:${message}`);
  },

  error(message: string, stack?: string): void {
    if (!enabled) return;
    push({ kind: "error", t: now(), n: 1, message, stack }, `error:${message}`);
  },

  /** One painted frame. Aggregated – see the note above. */
  render(sample: RenderSample, ms: number): void {
    if (!enabled) return;
    const changed = pending !== null && shapeOf(pending) !== shapeOf(sample);
    if (changed) flushRender();
    pending = sample;
    pendingFrames += 1;
    pendingMaxMs = Math.max(pendingMaxMs, ms);
    const t = now();
    if (changed || t - lastFlush > 250) {
      lastFlush = t;
      flushRender();
    }
  },

  /** Everything needed to reproduce, as a plain object. */
  capture(note?: string, extra?: Detail): unknown {
    flushRender();
    const counts: Record<string, number> = {};
    for (const ev of buffer) counts[ev.kind] = (counts[ev.kind] ?? 0) + ev.n;
    return {
      README:
        "R-node session trace. `events` is chronological, `t` is ms since page load and `n` is how many identical " +
        "events were coalesced into that entry. Read the `input` entries with outcome: `ignored` first – they say " +
        "which gesture was deliberately dropped and why. `render` entries are aggregated over `frames` frames: if " +
        "`relsDrawn` equals `rels` while the user reports missing arrows, the bug is in painting, not in culling.",
      note: note ?? null,
      at: new Date().toISOString(),
      counts,
      env:
        typeof window === "undefined"
          ? null
          : {
              url: window.location.href,
              userAgent: navigator.userAgent,
              dpr: window.devicePixelRatio,
              viewport: { w: window.innerWidth, h: window.innerHeight },
            },
      ...extra,
      events: buffer.map(({ _k, ...ev }: TraceEvent & { _k?: string }) => ev),
    };
  },

  download(note?: string, extra?: Detail): void {
    if (typeof document === "undefined") return;
    const blob = new Blob([JSON.stringify(this.capture(note, extra), null, 2)], { type: "application/json" });
    const url = URL.createObjectURL(blob);
    const a = document.createElement("a");

```

```

    a.href = url;
    a.download = `rnode-trace-${new Date().toISOString().replace(/[:.]/g, "-").json`;
    a.click();
    setTimeout(() => URL.revokeObjectURL(url), 1000);
  },

  clear(): void {
    buffer.length = 0;
    pending = null;
    pendingFrames = 0;
  },
};

/**
 * Install the capture hotkey and the console handle. Uncaught errors are
 * folded into the same stream so a crash arrives with the twenty events that
 * led to it instead of on its own.
 */
export function installTrace(): () => void {
  if (!enabled || typeof window === "undefined") return () => {};
  (window as unknown as { __rnodeTrace: typeof trace }).__rnodeTrace = trace;

  const onKey = (e: KeyboardEvent): void => {
    if (e.ctrlKey && e.shiftKey && e.key.toLowerCase() === "d") {
      e.preventDefault();
      const note = window.prompt("What did you expect to happen? (optional)") ?? undefined;
      trace.download(note);
      // Reset after capturing, never before: the bug has already happened by
      // the time anyone reaches for this, so clearing first would discard the
      // very evidence being asked for. Each capture is scoped to what came
      // after the previous one.
      trace.clear();
    }
  };
  const onError = (e: ErrorEvent): void => trace.error(e.message, e.error?.stack);
  const onRejection = (e: PromiseRejectionEvent): void => trace.error(String(e.reason), (e.reason as Error)?.stack);

  window.addEventListener("keydown", onKey, true);
  window.addEventListener("error", onError);
  window.addEventListener("unhandledrejection", onRejection);
  // eslint-disable-next-line no-console
  console.info("[trace] recording - Ctrl+Shift+D to capture, window.__rnodeTrace in the console");

  return () => {
    window.removeEventListener("keydown", onKey, true);
    window.removeEventListener("error", onError);
    window.removeEventListener("unhandledrejection", onRejection);
  };
}

```

## 46. src/main.tsx

```

import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { App } from "./App";
import { EditorStore } from "./editor/store";
import { StoreContext } from "./editor/context";
import { LocalStorageAdapter, TauriStorageAdapter } from "./persist/storage";
import "./styles.css";

// Same runtime switch as the asset factory (T19): the Tauri webview carries
// window.__TAURI__, a plain browser tab does not. On desktop the document is
// a single .rnode file, so localStorage never touches the document.
const adapter =
  typeof window !== "undefined" && window.__TAURI__ ? new TauriStorageAdapter() : new LocalStorageAdapter();

const store = new EditorStore(adapter);

// Dev-only debug handle for performance spikes / automated testing.
(window as unknown as Record<string, unknown>).__rnode = { store };

void store.init().then(() => {
  createRoot(document.getElementById("root")!).render(
    <StrictMode>
      <StoreContext.Provider value={store}>
        <App />
      </StoreContext.Provider>
    </StrictMode>
  );
});

```

## 47. tests/assets.test.ts

```
import "fake-indexeddb/auto";
import { afterEach, describe, expect, it, vi } from "vitest";
import {
  IndexedDbAssetStore,
  collectOrphans,
  getAssetStore,
  referencedAssetIds,
  sha256Hex,
  type AssetBlob,
  type AssetLevel,
  type AssetMeta,
} from "../src/persist/assets";
import { DEFAULT_STRUCTURE, type MindNode, type Sheet } from "../src/core/types";

afterEach(() => {
  vi.restoreAllMocks();
});

function blob(text: string, mime = "image/png"): Blob {
  return new Blob([text], { type: mime });
}

/** The three levels for one asset. Dimensions are arbitrary but consistent. */
function makeLevels(originalText = "ORIGINAL-IMAGE-BYTES"): Record<AssetLevel, AssetBlob> {
  return {
    original: { blob: blob(originalText), w: 1200, h: 800 },
    large: { blob: blob("LARGE-1024"), w: 1024, h: 683 },
    small: { blob: blob("SMALL-256"), w: 256, h: 171 },
  };
}

function meta(overrides: Partial<Omit<AssetMeta, "id">> = {}): Omit<AssetMeta, "id"> {
  return { mime: "image/png", w: 1200, h: 800, bytes: 21, ...overrides };
}

let dbCounter = 0;
/** Each test gets its own in-memory database: no teardown races. */
function uniqueDb(): string {
  dbCounter += 1;
  return `rnode-assets-test-${dbCounter}`;
}

function nodeWithImage(id: string, imageId: string): MindNode {
  return {
    id,
    type: "subtopic",
    parentId: "root",
    childrenIds: [],
    title: "Node",
    titleRuns: [{ text: "Node" }],
    position: { x: 0, y: 0, manual: false },
    style: { image: imageId },
    collapsed: false,
    labels: [],
    markers: [],
    notes: "",
    task: null,
    metadata: { createdAt: "2026-01-01T00:00:00.000Z", updatedAt: "2026-01-01T00:00:00.000Z" },
  };
}

function attachmentCard(id: string, name: string): Sheet["attachments"][number] {
  return { id, mime: "image/png", w: 1200, h: 800, bytes: 9, name };
}

function makeSheet(overrides: Partial<Sheet> = {}): Sheet {
  return {
    sheetId: "s1",
    title: "Test",
    structure: DEFAULT_STRUCTURE,
    rootNodeId: "root",
    nodes: {},
    relationships: [],
    boundaries: [],
    summaries: [],
    callouts: [],
    labels: [],
    zones: [],
    attachments: [],
    comments: [],
    presentation: {},
    ...overrides,
  };
}
```

```

describe("IndexedDbAssetStore", () => {
  it("addresses by content: the id is the SHA-256 of the original bytes", async () => {
    const store = new IndexedDbAssetStore(uniqueDb());
    const id = await store.put(makeLevels("DUPLICATE-CHECK"), meta());
    expect(id).toBe(await sha256Hex(new TextEncoder().encode("DUPLICATE-CHECK")));
  });

  it("put twice with the same content returns the same id and does not duplicate", async () => {
    const store = new IndexedDbAssetStore(uniqueDb());
    const id1 = await store.put(makeLevels(), meta({ name: "first.png" }));
    const id2 = await store.put(makeLevels(), meta({ name: "second.png" }));

    expect(id2).toBe(id1);
    // One record in the store, not two.
    expect(await store.list()).toEqual([id1]);
    // The existing data was not touched: the first write's name survives.
    expect((await store.meta(id1))?.name).toBe("first.png");
  });

  it("returns each of the three levels with their own bytes", async () => {
    const store = new IndexedDbAssetStore(uniqueDb());
    const id = await store.put(makeLevels(), meta());

    const original = await store.get(id, "original");
    const large = await store.get(id, "large");
    const small = await store.get(id, "small");

    expect(await original?.text()).toBe("ORIGINAL-IMAGE-BYTES");
    expect(await large?.text()).toBe("LARGE-1024");
    expect(await small?.text()).toBe("SMALL-256");
  });

  it("returns null (not an exception) for an unknown id", async () => {
    const store = new IndexedDbAssetStore(uniqueDb());
    await store.put(makeLevels(), meta());

    expect(await store.get("deadbeef", "original")).toBeNull();
    expect(await store.get("deadbeef", "small")).toBeNull();
    expect(await store.meta("deadbeef")).toBeNull();
  });

  it("putUnderId stores levels under a caller-supplied id and is idempotent", async () => {
    const store = new IndexedDbAssetStore(uniqueDb());
    // The .rnode.zip importer case: no original to re-derive the address from.
    const foreignId = "a".repeat(64);
    const full: AssetMeta = { id: foreignId, mime: "image/png", w: 1200, h: 800, bytes: 21, name: "first.png" };
    await store.putUnderId(foreignId, makeLevels(), full);
    expect(await store.get(foreignId, "large")).not.toBeNull();
    expect((await store.meta(foreignId))?.id).toBe(foreignId);

    // First write wins: re-importing the same zip must not overwrite.
    await store.putUnderId(foreignId, makeLevels("OTHER"), { ...full, name: "second.png" });
    expect(await store.get(foreignId, "original")).not.toBeNull();
    expect((await store.meta(foreignId))?.name).toBe("first.png");
    expect((await store.list()).length).toBe(1);
  });

  it("meta exposes the original's dimensions and weight", async () => {
    const store = new IndexedDbAssetStore(uniqueDb());
    const id = await store.put(
      makeLevels(),
      meta({ mime: "image/jpeg", name: "photo.jpg" })
    );

    expect(await store.meta(id)).toEqual({
      id,
      mime: "image/jpeg",
      w: 1200,
      h: 800,
      bytes: 21,
      name: "photo.jpg",
    });
  });

  it("delete removes the asset and list stops reporting it", async () => {
    const store = new IndexedDbAssetStore(uniqueDb());
    const keepId = await store.put(makeLevels("KEEP"), meta());
    const dropId = await store.put(makeLevels("DROP"), meta());

    await store.delete(dropId);

    expect(await store.list()).toEqual([keepId]);
    expect(await store.get(dropId, "original")).toBeNull();
    expect(await store.meta(dropId)).toBeNull();
    // The sibling asset is untouched.
    expect(await store.get(keepId, "original")).not.toBeNull();
  });
});

```

```

});

it("F3 – concurrent puts of the same content are atomic: one record, first wins", async () => {
  const store = new IndexedDbAssetStore(uniqueDb());

  // Release both digest calls at the same instant so the two puts race
  // through their existence checks instead of serializing. Without the
  // single-transaction put, both checks pass and the second write wins.
  let release: () => void = () => {};
  const barrier = new Promise<void>((r) => {
    release = r;
  });
  let pending = 0;
  const realDigest = crypto.subtle.digest.bind(crypto.subtle);
  vi.spyOn(crypto.subtle, "digest").mockImplementation(
    async (...args: Parameters<SubtleCrypto["digest"]>) => {
      const out = await realDigest(...args);
      pending += 1;
      if (pending === 2) release();
      await barrier;
      return out;
    },
  );

  const [id1, id2] = await Promise.all([
    store.put(makeLevels(), meta({ name: "first.png" })),
    store.put(makeLevels(), meta({ name: "second.png" })),
  ]);

  expect(id2).toBe(id1);
  // One record, not two; the first writer's metadata survives.
  expect(await store.list()).toEqual([id1]);
  expect((await store.meta(id1)).name).toBe("first.png");
});

describe("referencedAssetIds", () => {
  it("roots are the nodes only: an unattached attachment card is not a root", () => {
    const sheet = makeSheet({
      attachments: [attachmentCard("card-only", "card.png")],
      nodes: { n1: nodeWithImage("n1", "via-node") },
    });

    expect(referencedAssetIds(sheet)).toEqual(new Set(["via-node"]));
  });
});

describe("collectOrphans", () => {
  it("F1 – reports orphan cards and orphan blobs, and deletes nothing", async () => {
    const store = new IndexedDbAssetStore(uniqueDb());
    const viaNode = await store.put(makeLevels("VIA-NODE"), meta());
    const cardOnly = await store.put(makeLevels("CARD-ONLY"), meta());
    const blobOnly = await store.put(makeLevels("BLOB-ONLY"), meta());

    const sheet = makeSheet({
      // A card for B with no node behind it: garbage itself, not a root.
      attachments: [attachmentCard(cardOnly, "card.png")],
      // A node that references A.
      nodes: { n1: nodeWithImage("n1", viaNode) },
    });

    const report = await collectOrphans(sheet, store);

    // The orphaned card is reported as a card and as a blob (nothing in the
    // store references it either); C is a blob-only orphan.
    expect(report.cards).toEqual([cardOnly]);
    expect(report.blobs).toContain(cardOnly);
    expect(report.blobs).toContain(blobOnly);
    expect(report.blobs).not.toContain(viaNode);

    // Nothing was deleted: all three assets are still listed and readable.
    expect((await store.list()).sort()).toEqual([viaNode, cardOnly, blobOnly].sort());
    expect(await store.get(viaNode, "original")).not.toBeNull();
    expect(await store.get(cardOnly, "original")).not.toBeNull();
    expect(await store.get(blobOnly, "original")).not.toBeNull();
  });

  it("returns empty cards and blobs when every stored id is referenced", async () => {
    const store = new IndexedDbAssetStore(uniqueDb());
    const id = await store.put(makeLevels(), meta());

    const sheet = makeSheet({ nodes: { n1: nodeWithImage("n1", id) } });
    expect(await collectOrphans(sheet, store)).toEqual({ cards: [], blobs: [] });
  });
});

describe("getAssetStore", () => {

```

```

it("returns the same singleton instance every call", () => {
  const a = getAssetStore();
  const b = getAssetStore();
  expect(a).toBe(b);
});

it("picks IndexedDB when no Tauri global is present (the node test env)", () => {
  expect(getAssetStore()).toBeInstanceOf(IndexedDbAssetStore);
});
});

```

## 48. tests/ops.test.ts

```

import { describe, expect, it } from "vitest";
import { DocumentModel, uid } from "../src/core/doc";
import { applyOp, applyWithInverse, makeOp, type Op } from "../src/core/ops";
import { History } from "../src/core/history";
import type { Group, MindNode, Relationship, RnodeDocument, Summary } from "../src/core/types";

function freshDoc(): { doc: RnodeDocument; model: DocumentModel; root: MindNode } {
  const model = new DocumentModel(DocumentModel.blank("Test"));
  return { doc: model.doc, model, root: model.rootNode };
}

function child(model: DocumentModel, parentId: string, title: string): string {
  const id = uid("n");
  const parent = model.node(parentId)!;
  applyOp(model.sheet, makeOp<Op & { type: "createNode" }>("createNode", { id, nodeType: "subtopic", parentId, index: parent.childrenIds.length, title }));
  return id;
}

function exec(model: DocumentModel, history: History, ops: Op[]): void {
  const inverses: Op[][] = [];
  for (const op of ops) inverses.push(applyWithInverse(model.sheet, op));
  history.push(ops, inverses);
}

function undo(model: DocumentModel, history: History): void {
  const ops = history.undo();
  if (ops) for (const op of ops) applyWithInverse(model.sheet, op);
}

function redo(model: DocumentModel, history: History): void {
  const ops = history.redo();
  if (ops) for (const op of ops) applyWithInverse(model.sheet, op);
}

describe("relationship / group / summary ops", () => {
  it("setRelationship applies, undoes and redoes", () => {
    const { model, root } = freshDoc();
    const history = new History();
    const a = child(model, root.id, "A");
    const b = child(model, root.id, "B");
    const rel: Relationship = { id: "rel1", fromId: a, toId: b, label: "links" };
    exec(model, history, [makeOp<Op & { type: "createRelationship" }>("createRelationship", { relationship: rel })]);
    const patch: Relationship = { id: "rel1", fromId: a, toId: b, label: "renamed", bidirectional: true };
    exec(model, history, [makeOp<Op & { type: "setRelationship" }>("setRelationship", { id: "rel1", relationship: patch, prev: rel })]);
    expect(model.sheet.relationships[0].label).toBe("renamed");
    expect(model.sheet.relationships[0].bidirectional).toBe(true);
    undo(model, history);
    expect(model.sheet.relationships[0].label).toBe("links");
    expect(model.sheet.relationships[0].bidirectional).toBeUndefined();
    redo(model, history);
    expect(model.sheet.relationships[0].label).toBe("renamed");
  });

  it("createGroup/deleteGroup/setGroup round-trip through undo", () => {
    const { model, root } = freshDoc();
    const history = new History();
    const a = child(model, root.id, "A");
    const b = child(model, root.id, "B");
    const g: Group = { id: "g1", memberIds: [a, b], label: "group" };
    exec(model, history, [makeOp<Op & { type: "createGroup" }>("createGroup", { group: g })]);
    expect(model.sheet.boundaries).toHaveLength(1);
    expect(model.sheet.boundaries[0].memberIds).toEqual([a, b]);
    exec(model, history, [makeOp<Op & { type: "setGroup" }>("setGroup", { id: "g1", group: { ...g, label: "x" }, prev: g })]);
    expect(model.sheet.boundaries[0].label).toBe("x");
    undo(model, history);
    expect(model.sheet.boundaries[0].label).toBe("group");
    undo(model, history);
    expect(model.sheet.boundaries).toHaveLength(0);
    redo(model, history);
    redo(model, history);
  });
});

```

```

    expect(model.sheet.boundaries).toHaveLength(1);
  });

  it("createSummary/deleteSummary round-trip through undo", () => {
    const { model, root } = freshDoc();
    const history = new History();
    const a = child(model, root.id, "A");
    const b = child(model, root.id, "B");
    const sum: Summary = { id: "s1", memberIds: [a, b], label: "Summary" };
    exec(model, history, [makeOp<Op & { type: "createSummary" }>("createSummary", { summary: sum })]);
    expect(model.sheet.summaries).toHaveLength(1);
    exec(model, history, [makeOp<Op & { type: "deleteSummary" }>("deleteSummary", { id: "s1", summary: sum })]);
    expect(model.sheet.summaries).toHaveLength(0);
    undo(model, history);
    expect(model.sheet.summaries).toHaveLength(1);
  });
});

describe("create + delete", () => {
  it("creates a child and links it to the parent", () => {
    const { model, root } = freshDoc();
    const id = child(model, root.id, "Hello");
    const node = model.node(id)!;
    expect(node.parentId).toBe(root.id);
    expect(root.childrenIds).toContain(id);
    expect(node.title).toBe("Hello");
  });

  it("undo restores the exact previous tree", () => {
    const { model, root } = freshDoc();
    const history = new History();
    const a = child(model, root.id, "A");
    const b = child(model, root.id, "B");
    exec(model, history, [makeOp<Op & { type: "deleteNode" }>("deleteNode", { id: a, parentId: root.id, index: root.childrenIds.indexOf(a), subtree:
[model.node(a)!], removedRelationships: [] })]);
    expect(root.childrenIds).toEqual([b]);
    const undo = history.undo()!;
    for (const op of undo) applyWithInverse(model.sheet, op);
    expect(root.childrenIds).toEqual([a, b]);
    expect(model.node(a)!.title).toBe("A");
    // redo
    const redo = history.redo()!;
    for (const op of redo) applyWithInverse(model.sheet, op);
    expect(root.childrenIds).toEqual([b]);
  });

  it("deleting a branch removes descendants and restores them on undo", () => {
    const { model, root } = freshDoc();
    const main = child(model, root.id, "Main");
    const sub1 = child(model, main, "Sub1");
    const sub2 = child(model, main, "Sub2");
    const history = new History();
    const subtree = model.subtreeIds(main).map((id) => model.node(id)!);
    exec(model, history, [makeOp<Op & { type: "deleteNode" }>("deleteNode", { id: main, parentId: root.id, index: 0, subtree, removedRelationships: []
})]);
    expect(model.node(main)).toBeUndefined();
    expect(model.node(sub1)).toBeUndefined();
    expect(model.node(sub2)).toBeUndefined();
    expect(root.childrenIds).toHaveLength(0);
    for (const op of history.undo()!) applyWithInverse(model.sheet, op);
    expect(model.node(main)!.childrenIds).toEqual([sub1, sub2]);
    expect(model.node(sub1)!.parentId).toBe(main);
    expect(root.childrenIds).toEqual([main]);
  });

  it("protects nothing structurally: central topic can still host edits", () => {
    const { model, root } = freshDoc();
    const id = child(model, root.id, "X");
    expect(model.depth(id)).toBe(1);
  });
});

describe("moveNode", () => {
  it("reparents a node and undoes cleanly", () => {
    const { model, root } = freshDoc();
    const a = child(model, root.id, "A");
    const b = child(model, root.id, "B");
    const history = new History();
    exec(model, history, [makeOp<Op & { type: "moveNode" }>("moveNode", { id: a, fromParentId: root.id, fromIndex: 0, toParentId: b, toIndex: 0 })]);
    expect(model.node(a)!.parentId).toBe(b);
    expect(root.childrenIds).toEqual([b]);
    expect(model.node(b)!.childrenIds).toEqual([a]);
    for (const op of history.undo()!) applyWithInverse(model.sheet, op);
    expect(model.node(a)!.parentId).toBe(root.id);
    expect(root.childrenIds).toEqual([a, b]);
  });
});

```

```

it("reorders siblings with final-index semantics", () => {
  const { model, root } = freshDoc();
  const a = child(model, root.id, "A");
  const b = child(model, root.id, "B");
  const c = child(model, root.id, "C");
  // move C before B: desired final index 1 (after removing C from [A,B,C])
  applyOp(model.sheet, makeOp<Op & { type: "moveNode" }>("moveNode", { id: c, fromParentId: root.id, fromIndex: 2, toParentId: root.id, toIndex: 1
})));
  expect(root.childrenIds).toEqual([a, c, b]);
});

it("promote = move under grandparent after parent", () => {
  const { model, root } = freshDoc();
  const main = child(model, root.id, "Main");
  const sub = child(model, main, "Sub");
  const idx = model.node(main)!.childrenIds.indexOf(sub);
  const grandIdx = root.childrenIds.indexOf(main);
  applyOp(model.sheet, makeOp<Op & { type: "moveNode" }>("moveNode", { id: sub, fromParentId: main, fromIndex: idx, toParentId: root.id, toIndex:
grandIdx + 1 })));
  expect(model.node(sub)!.parentId).toBe(root.id);
  expect(root.childrenIds).toEqual([main, sub]);
});

it("moves a floating topic into the tree and undoes back to floating", () => {
  const { model, root } = freshDoc();
  const floating = uid("n");
  applyOp(model.sheet, makeOp<Op & { type: "createNode" }>("createNode", {
    id: floating,
    nodeType: "floating",
    parentId: null,
    index: 0,
    title: "Free",
    position: { x: 100, y: 200, manual: true },
  })));
  const history = new History();
  exec(model, history, [makeOp<Op & { type: "moveNode" }>("moveNode", {
    id: floating,
    fromParentId: null,
    fromIndex: 0,
    toParentId: root.id,
    toIndex: 0,
  })]);

  expect(model.node(floating)!.parentId).toBe(root.id);
  for (const op of history.undo(!)) applyWithInverse(model.sheet, op);
  expect(model.node(floating)!.parentId).toBeNull();
});
});

describe("history batching", () => {
  it("a two-op batch undoes as a single step", () => {
    const { model, root } = freshDoc();
    const history = new History();
    const wrapId = uid("n");
    const kidId = uid("n");
    exec(model, history, [
      makeOp<Op & { type: "createNode" }>("createNode", { id: wrapId, nodeType: "main", parentId: root.id, index: 0, title: "Wrap" }),
      makeOp<Op & { type: "createNode" }>("createNode", { id: kidId, nodeType: "subtopic", parentId: wrapId, index: 0, title: "Kid" }),
    ]);
    expect(history.canUndo).toBe(true);
    expect(model.node(wrapId)!.childrenIds).toHaveLength(1);
    const undo = history.undo(!);
    for (const op of undo) applyWithInverse(model.sheet, op);
    expect(model.node(wrapId)).toBeUndefined();
    expect(model.node(kidId)).toBeUndefined();
    expect(history.canRedo).toBe(true);
    const redo = history.redo(!);
    for (const op of redo) applyWithInverse(model.sheet, op);
    expect(model.node(wrapId)).toBeDefined();
    expect(root.childrenIds).toContain(wrapId);
  });
});

it("preserves child order when duplicating a subtree", () => {
  const { model, root } = freshDoc();
  const parent = child(model, root.id, "Parent");
  child(model, parent, "First");
  child(model, parent, "Second");
  child(model, parent, "Third");
  const source = model.subtreeIds(parent).map((id) => model.node(id)!.filter(Boolean));
  const store = {
    remapOps(source: MindNode[], parentId: string, index: number, rootType: string) {
      const idMap = new Map<string, string>();
      const ops: Op[] = [];
      const srcRoot = source.find((n) => !source.some((o) => o.childrenIds.includes(n.id)));
      if (!srcRoot) return ops;
      const newRoot = uid("n");
      idMap.set(srcRoot.id, newRoot);
    }
  };

```

```

ops.push(
  makeOp<Op & { type: "createNode" }>("createNode", {
    id: newRoot,
    nodeType: rootType as any,
    parentId,
    index,
    title: srcRoot.title,
    style: srcRoot.style,
    task: srcRoot.task,
  })
);
const queue: MindNode[] = [srcRoot];
while (queue.length > 0) {
  const src = queue.shift();
  for (const cid of src.childrenIds) {
    const c = source.find((node) => node.id === cid);
    if (!c) continue;
    const newId = uid("n");
    idMap.set(cid, newId);
    ops.push(
      makeOp<Op & { type: "createNode" }>("createNode", {
        id: newId,
        nodeType: "subtopic",
        parentId: idMap.get(src.id)!,
        index: src.childrenIds.indexOf(cid),
        title: c.title,
        style: c.style,
        task: c.task,
      })
    );
    queue.push(c);
  }
}
return ops;
},
);
const ops = store.remapOps(source, root.id, root.childrenIds.length, "main");
for (const op of ops) applyOp(model.sheet, op);
const duplicated = root.childrenIds[root.childrenIds.length - 1];
const newParent = model.node(duplicated)!;
expect(newParent.childrenIds).toEqual([
  expect.any(String),
  expect.any(String),
  expect.any(String),
]);
expect(model.node(newParent.childrenIds[0])!.title).toBe("First");
expect(model.node(newParent.childrenIds[1])!.title).toBe("Second");
expect(model.node(newParent.childrenIds[2])!.title).toBe("Third");
});
});

describe("relationships", () => {
  it("create/delete relationship round-trips", () => {
    const { model, root } = freshDoc();
    const a = child(model, root.id, "A");
    const b = child(model, root.id, "B");
    const rel = { id: uid("rel"), fromId: a, toId: b };
    const history = new History();
    exec(model, history, [makeOp<Op & { type: "createRelationship" }>("createRelationship", { relationship: rel })]);
    expect(model.sheet.relationships).toHaveLength(1);
    for (const op of history.undo(!)) applyWithInverse(model.sheet, op);
    expect(model.sheet.relationships).toHaveLength(0);
    for (const op of history.redo(!)) applyWithInverse(model.sheet, op);
    expect(model.sheet.relationships).toHaveLength(1);
  });

  it("deleting a topic removes relationships touching it", () => {
    const { model, root } = freshDoc();
    const a = child(model, root.id, "A");
    const b = child(model, root.id, "B");
    model.sheet.relationships.push({ id: uid("rel"), fromId: a, toId: b });
    const history = new History();
    const removed = model.sheet.relationships.filter((r) => r.fromId === a || r.toId === a);
    exec(model, history, [makeOp<Op & { type: "deleteNode" }>("deleteNode", { id: a, parentId: root.id, index: 0, subtree: [model.node(a)!],
      removedRelationships: removed })]);
    expect(model.sheet.relationships).toHaveLength(0);
    for (const op of history.undo(!)) applyWithInverse(model.sheet, op);
    expect(model.sheet.relationships).toHaveLength(1);
  });
});

describe("styles & tasks", () => {
  it("setNodeImage applies, undoes and redoes – carrying only the id", () => {
    const { model, root } = freshDoc();
    const history = new History();
    expect(root.style.image).toBeUndefined();
  });
});

```

```

// attach
exec(model, history, [makeOp<Op & { type: "setNodeImage" }>{"setNodeImage", { nodeId: root.id, imageId: "sha-abc", prevImageId: null }}});
expect(model.node(root.id)!.style.image).toBe("sha-abc");
expect(JSON.stringify(model.node(root.id)!.style)).toContain("sha-abc"); // only the id, never bytes

undo(model, history);
expect(model.node(root.id)!.style.image).toBeUndefined();

redo(model, history);
expect(model.node(root.id)!.style.image).toBe("sha-abc");

// replace + remove round-trip
exec(model, history, [makeOp<Op & { type: "setNodeImage" }>{"setNodeImage", { nodeId: root.id, imageId: "sha-other", prevImageId: "sha-abc" }}});
expect(model.node(root.id)!.style.image).toBe("sha-other");
undo(model, history);
expect(model.node(root.id)!.style.image).toBe("sha-abc");
exec(model, history, [makeOp<Op & { type: "setNodeImage" }>{"setNodeImage", { nodeId: root.id, imageId: null, prevImageId: "sha-abc" }}});
expect(model.node(root.id)!.style.image).toBeUndefined();
undo(model, history);
expect(model.node(root.id)!.style.image).toBe("sha-abc");
});

it("setStyle undo restores previous style", () => {
  const { model, root } = freshDoc();
  const history = new History();
  const originalFill = root.style.fill;
  exec(model, history, [makeOp<Op & { type: "setStyle" }>{"setStyle", { id: root.id, style: { fill: "#ff0000" }, prev: root.style }}});
  expect(model.rootNode.style.fill).toBe("#ff0000");
  for (const op of history.undo(!) applyWithInverse(model.sheet, op);
  expect(model.rootNode.style.fill).toBe(originalFill);
});

it("task progress roll-up is derivable from descendants", () => {
  const { model, root } = freshDoc();
  const a = child(model, root.id, "A");
  const b = child(model, root.id, "B");
  model.node(a)!.task = { status: "completed", priority: "high", progress: 100 };
  model.node(b)!.task = { status: "in-progress", priority: "medium", progress: 50 };
  const kids = [a, b].map((id) => model.node(id)!.filter((n) => n.task);
  const avg = Math.round(kids.reduce((s, n) => s + (n.task?.progress ?? 0), 0) / kids.length);
  expect(avg).toBe(75);
});
});

describe("schema", () => {
  it("round-trips through JSON without losing structure", () => {
    const model = new DocumentModel(DocumentModel.sample());
    const json = JSON.stringify(model.doc);
    const restored = new DocumentModel(JSON.parse(json) as RnodeDocument);
    expect(restored.doc.sheets[0].rootNodeId).toBe(model.doc.sheets[0].rootNodeId);
    expect(Object.keys(restored.doc.sheets[0].nodes).length).toBe(Object.keys(model.doc.sheets[0].nodes).length);
    expect(restored.doc.schemaVersion).toBe("0.1.0");
  });
});
});

```

## 49. docs/ARCHITECTURE.md

### # R-node – Architecture

Status: **\*\*Phase 1 (core editor) done; node images and the desktop app shipped.\*\*** The web and desktop targets share one codebase, one document model and one op stream. AI services and the full security model remain explicitly deferred per the product owner; the seams for both exist but are not built.

> This file describes the **\*\*structure and the plan\*\***. Before changing code,  
 > read [AGENT\_GUIDE.md](AGENT\_GUIDE.md) – the invariants, the editor+canvas  
 > parity contract and the definition of done – and take work from  
 > [ROADMAP.md](ROADMAP.md), in order.

#### ## 1. Product requirements (v0.1 scope)

A visual thinking workspace: mind maps, trees, org charts, timelines, matrices, freeform canvases – keyboard-first, canvas-rendered, offline-first, multi-client (desktop Tauri, web, mobile later).

#### Acceptance for Phase 1:

- Create a map without reading docs.
- Enter/Tab/Shift+Tab create sibling/child/promote; typing starts immediately.
- Every edit is undoable/redoable (single gesture = single undo step).
- Maps stay readable as they grow (layout never overlaps).
- Canvas and Outliner stay synchronized.
- Saving is explicit (Ctrl+S) and nothing is silently lost: there is no autosave timer. `saveNow()` writes through the storage adapter AND the portable ``.rnode.json``.

- The interface is original (no Xmind assets or pixel layouts).

## ## 2. Information architecture

```
...
Workspace (local-first, adapter-backed)
├── Documents (list, archived flag, pinned)
│   └── Document (schemaVersion, title, settings, themeId)
│       └── Sheets (multiple later; Phase 1 = one active)
│           ├── structure config (type, orientation, spacing...)
│           ├── nodes (Record<id, MindNode>) – tree
│           ├── relationships (independent of hierarchy)
│           └── (boundaries, summaries, callouts, zones... Phase 3)
├── Templates (sample roadmap template today)
└── Assets (Phase 4+)
...
```

## ## 3. Component tree

```
...
App
├── Sidebar      – documents, search, create/duplicate/archive/delete
├── TopBar       – title, undo/redo, save status, zoom, export, palette, theme, zen
├── Workspace
│   ├── CanvasArea
│   │   ├── CanvasView – <canvas> renderer + pointer/wheel handling
│   │   ├── RichEditor  – the ONE Lexical overlay for the topic being edited
│   │   └── Outliner    – synchronized indented list (bottom sheet)
│   └── Inspector      – node style/task/notes + sheet structure controls
├── StatusBar        – zoom %, selection, sync, hints
└── Palette           – command palette (Ctrl+K)
...
```

Engine modules (framework-free, all testable):

```
...
core/      types (schema) · doc (model+walks) · ops (op system) · history (undo/redo) · tree
layout/    mindmap.ts (measure, place, bounds)
render/    viewport.ts (camera) · renderer.ts (canvas paint, hit-test, PNG export) · theme.ts
editor/     store.ts (orchestration) · shortcuts.ts · context.ts · view.ts · exportBridge.ts
persist/    storage.ts (web: localStorage · desktop: one .rnode SQLite file) · assets.ts (AssetStore: IndexedDB on web, SQLite on desktop)
...
```

## ## 4. Document schema

`src/core/types.ts` is the single source of truth (versioned `SCHEMA\_VERSION`). Design rules: IDs everywhere (never indices), relationships independent of the tree, nodes keyed by id, `position.manual` marks user-placed nodes the layout engine must preserve. Example node carries `{ id, type, parentId, childrenIds, title, position, style, collapsed, labels, markers, notes, task, metadata }`.

## ## 5. API design (editor <-> engine)

The editor never mutates the document directly. Every change is an `**Op**`: `{ opId, actorId, ts, type, ...payload }`. Ops are self-contained, idempotent, and carry the data needed to reverse themselves. The store applies ops via `applyWithInverse(sheet, op)` which returns the inverse ops; `History` keeps batches so multi-op gestures undo in one step. The same op stream will feed collaboration (OT/CRDT) unchanged – each op already has `actorId/ts`.

## ## 6. Synchronization strategy

Deferred to Phase 5, but prepared for:

- Ops as the unit of sync (idempotent + replayable + tagged).
- Local-first: adapter writes are the source of truth; a sync layer replays ops against the local document after reconnect.
- Conflict resolution policy TBD (CRDT per-node vs. OT with server ordering)
  - documented as an open decision below.

## ## 7. Layout-engine strategy

Pure functions in `src/layout/`:

- `measure.ts` – the single source of truth for topic extents. Pluggable `TextMeasurer`: the renderer injects a canvas-backed one (real `measureText` with caching); pure code and tests use a deterministic heuristic. `measureTopic` computes width/height from wrapped text, padding, shape allowances (circle is square, diamond/hexagon grow the box) and explicit overrides. Layout and paint use the SAME measurer and the SAME word-wrap, so they never disagree.
- `mindmap.ts` – `subtreeHeight` (block height; collapsed subtrees count as leaves) and `layoutSheet` (mindmap / logic / tree / org / timeline / freeform).
- `applyLayout` – writes positions back, never touching `manual` nodes unless forced. Layout is derived data, excluded from undo history.

### ### Layout behavior (observable model)

The engine reproduces the observable behavior of classic mind-mapping tools (behavioral description; no proprietary code):

1. **\*\*Every topic has an extent\*\*** – `width = textWidth + padding\*2 + shape`, `height = lines\*lineHeight + padding\*2 + shape`; a two-line title makes the topic taller.
2. **\*\*Subtree block\*\*** – a topic's block = its own extent + its descendants, plus sibling spacing; the block grows when any descendant grows.
3. **\*\*Straddle distribution\*\*** – children are split into an above group and a below group (balanced by total block height, leaning below like Xmind), each separated from the parent box by a guaranteed gap: `blockH = aboveTotal + gap + parentH + gap + belowTotal`. No child can ever overlap its parent, and growing a topic's own title pushes its neighbors apart.
4. **\*\*Upward propagation\*\*** – any deep change recomputes the whole map (full relayout on a 30 ms debounce): the branch redistributes, the root re-splits children left/right by subtree height (size-aware balance, not raw count), and the overall map area changes. Stable positions for untouched branches are a Phase 3 refinement (incremental layout).

Renderer: single Canvas2D with viewport culling; WebGPU/WebGL2 is the upgrade path only if ADR-001 §12 is reopened (renderer is isolated behind `Renderer` with hit-test + export).

## ## 8. Import/export strategy

Today: JSON (native `.rnode.json`), Markdown, PNG (canvas export) and `.rnode.zip` (document + images: complete = originals, compact = display levels only). Import: `.rnode.json` / `.rmind.json` / `.rnode.zip`, Markdown outline, clipboard paste of JSON subtrees, and sanitized rich text from Word / Google Docs / Draw.io. Phase 4+: SVG, PDF, DOCX, PPTX, OPML, FreeMind, MindManager, OCR, background workers.

## ## 9. Security model (deferred, seams exist)

- Deferred by owner: auth, RBAC, encryption, XSS hardening (rich text comes with sanitization), audit logs.
- In place today: all data local, no network calls, no telemetry, XSS risk minimized by canvas rendering (HTML only for text editing).
- AI privacy rules (consent before upload, snapshots before transformation) are documented requirements for Phase 6.

## ## 10. Phased plan

- **\*\*Phase 1 (done):\*\*** document model, ops + undo/redo, one sheet, central/main/subtopic/floating, canvas renderer, keyboard + mouse, zoom/pan, drag-drop with drop indicator, manual save, outliner, inspector, palette, tests.
- **\*\*Images & desktop (done):\*\*** content-addressed asset store, node images (drop/paste, resize, byte-budgeted decode), portable `.rnode.zip`, Tauri bring-up, single-file `.rnode` (SQLite) desktop documents.
- **\*\*Phase 2 (in progress):\*\*** themes/styles, markers, labels, rich notes, boundaries, callouts – relationships UI, summaries, groups and attachments have already landed.
- **\*\*Phase 3:\*\*** logic/tree/org/timeline/fishbone/matrix/treetable/mixed.
- **\*\*Phase 4:\*\*** Gantt, search+, templates, presentation, Zen polish.
- **\*\*Phase 5:\*\*** accounts, sharing, comments, real-time ops sync, version history.
- **\*\*Phase 6:\*\*** AI provider abstraction + the 9 AI features.
- **\*\*Phase 7:\*\*** ecosystem: full import/export, plugins, mobile clients.

Each phase ends with: working code, tests, docs, accessibility + performance check, known limitations.

## ## 11. Risk register

| Risk                                    | Mitigation                                                                                                 |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------|
| Canvas2D perf at 10k nodes              | Culling + layout caching; WebGL2/WebGPU path isolated                                                      |
| Rust toolchain missing on a new machine | Core is framework-free TS: tests and the web build need no Rust; desktop needs `rustup` (per-user install) |
| Ops replay drift after schema changes   | Versioned schema + migration tests; ops carry explicit data                                                |
| Manual-position fights with auto layout | `manual` flag honored; force-layout is explicit                                                            |
| Undo of complex gestures                | Batch history entries + inverse-op capture                                                                 |

## ## 12. Assumptions & open decisions

- One active sheet per document in Phase 1 (schema supports many).
- Web persists to localStorage; desktop persists to one `.rnode` file (SQLite) holding document and images in a single transaction. IndexedDB (web assets) and SQLite (desktop) are adapters behind the same interfaces, not rewrites.
- Sync conflict model (CRDT vs OT) – open, will be chosen in Phase 5.
- Layout animations – deferred to Phase 3 (stable positions already).
- Light theme only (`ThemeName = "light"`). The renderer already takes the theme as data, so adding a dark one is a palette, not a refactor.
- Fonts: system stack (no bundling).

## ## Tech-stack mapping (per owner)

| Layer           | Now                                         | Later                                    |
|-----------------|---------------------------------------------|------------------------------------------|
| Desktop shell   | Tauri 2 (`src-tauri`), compiled and working | –                                        |
| Document engine | `src/core` (TS)                             | Rust crate mirroring the same schema/ops |

```
| Persistence | web: localStorage · desktop: one `rnode` SQLite file (document + assets) | IndexedDB (web) |
| Renderer | Canvas2D | WebGPU / WebGL2 – only if ADR-001 §12 is reopened |
| UI | React + TS + Vite | unchanged |
| Layout/export perf | TS, debounced | Rust background tasks / Web Workers |
```

## 50. src/layout/measure.ts

```
/**
 * Topic measurement – the single source of truth for extents.
 *
 * Observable layout behavior (see docs/ARCHITECTURE.md, "Layout behavior"):
 * every topic has an extent computed from text width/height, padding, shape
 * and border; a two-line title makes the topic taller; that size change
 * ripples through the subtree and the whole branch.
 *
 * Titles are rich text: a sequence of TextRuns where each run may carry its
 * own bold/italic/color. Wrapping and extent computation walk the runs so a
 * bold run really is wider, and the box always contains the styled text.
 *
 * Layout and rendering AGREE on sizes – both use `measureTopic` with the
 * same `TextMeasurer`:
 * - the renderer injects a canvas-backed measurer (real `measureText`);
 * - pure layout code and tests default to a deterministic heuristic.
 */
import type { MindNode, Sheet, Style, TextRun } from "../core/types";
import { nodeRuns } from "../core/text";

// -----
// Text measurer
// -----

export interface TextMetricsInput {
  fontSize: number;
  fontFamily?: string;
  fontWeight?: number;
  italic?: boolean;
}

export interface TextMeasurer {
  measure(text: string, style: TextMetricsInput): { width: number };
  /**
   * Font ascent/descent around the baseline, in px. Needed to build line boxes
   * the way CSS does (see `wrapRunLines`); when absent the caller falls back to
   * a 0.8/0.2 split, which reproduces the old fontSize × LINE_HEIGHT_FACTOR
   * behavior exactly.
   */
  metrics?(style: TextMetricsInput): { ascent: number; descent: number };
}

/** Deterministic fallback (no DOM): ~0.55 × fontSize per char. */
export const HEURISTIC_MEASURER: TextMeasurer = {
  measure: (t, s) => ({ width: t.length * (s.fontSize * 0.55) }),
  metrics: (s) => ({ ascent: s.fontSize * 0.8, descent: s.fontSize * 0.2 }),
};

let sharedCanvasMeasurer: TextMeasurer | null = null;

/** Canvas-backed measurer (real text metrics). Falls back to heuristic in Node. */
export function createCanvasTextMeasurer(): TextMeasurer {
  if (typeof document === "undefined") return HEURISTIC_MEASURER;
  if (sharedCanvasMeasurer) return sharedCanvasMeasurer;
  const canvas = document.createElement("canvas");
  const ctx = canvas.getContext("2d");
  if (!ctx) return HEURISTIC_MEASURER;
  const cache = new Map<string, number>();
  const metricsCache = new Map<string, { ascent: number; descent: number }>();
  const fontOf = (s: TextMetricsInput): string =>
    `${s.italic ? "italic " : ""}${s.fontWeight ?? 400} ${s.fontSize}px ${s.fontFamily ?? FONT_STACK}`;
  sharedCanvasMeasurer = {
    measure(text, s) {
      const key = `${s.fontWeight ?? 400}|${s.italic ? 1 : 0}|${s.fontSize}|${s.fontFamily ?? "system-ui"}|${text}`;
      const hit = cache.get(key);
      if (hit !== undefined) return { width: hit };
      ctx.font = fontOf(s);
      const width = ctx.measureText(text).width;
      if (cache.size > 20_000) cache.clear();
      cache.set(key, width);
      return { width };
    },
    metrics(s) {
      const font = fontOf(s);
      const hit = metricsCache.get(font);
      if (hit) return hit;
      ctx.font = font;
    }
  };
  return sharedCanvasMeasurer;
}
```

```

    const m = ctx.measureText("M");
    const ascent = m.fontBoundingBoxAscent > 0 ? m.fontBoundingBoxAscent : s.fontSize * 0.8;
    const descent = m.fontBoundingBoxDescent > 0 ? m.fontBoundingBoxDescent : s.fontSize * 0.2;
    const out = { ascent, descent };
    metricsCache.set(font, out);
    return out;
  },
};
return sharedCanvasMeasurer;
}

// -----
// Wrapping (shared by layout and renderer so line counts always agree)
// -----

export const TEXT_INSET = 6; // horizontal inset inside the topic box (both sides)
export const LINE_HEIGHT_FACTOR = 1.25;
/**
 * The one font stack for topic text. Must stay identical to `--font` in
 * styles.css: the canvas measured/drew with "system-ui, -apple-system,
 * sans-serif" while the overlay inherited `--font`, so on any machine where
 * those resolve differently the editor and the node used different faces –
 * different metrics, different wrap, for no visible reason.
 */
export const FONT_STACK = 'system-ui, -apple-system, "Segoe UI", Roboto, sans-serif';
/**
 * Block gap shared by BOTH renderers (single source of truth): every block
 * boundary (paragraph end, list item end) advances by this fraction of the
 * line height. The Lexical overlay mirrors it via --rnode-block-gap
 * (calc(BLOCK_GAP_FACTOR * LINE_HEIGHT_FACTOR em)) so the editor and the
 * canvas keep the same vertical rhythm.
 */
export const BLOCK_GAP_FACTOR = 0.6;
export const MIN_TOPIC_W = 84;
export const MAX_TOPIC_W = 280;
/** Default display width of a node image in world units (Xmind-style cap). */
export const MAX_IMAGE_W = 240;
/** Vertical gap between a node's image and its text, in world units. */
export const IMAGE_GAP = 6;

/**
 * Width of the bullet column, in em of the list item's font size. Shared with
 * the overlay as `--rnode-bullet-w`: the canvas indents the text by exactly
 * this and the CSS hangs the item by exactly this, so a wrapped list item
 * lines up identically on both sides.
 *
 * Measuring the glyph string ("• ") instead – what this used to do – could
 * never match the browser, because with `list-style` the marker width is
 * UA-defined and no CSS length can be derived from it.
 */
export const BULLET_WIDTH_EM = 1.2;

const BULLET_CHARS = ["•", "◦", "▪"];

/** Bullet marker for a list item at `depth` (1 = top level). */
export function bulletChar(depth: number): string {
  return BULLET_CHARS[Math.min(BULLET_CHARS.length - 1, Math.max(0, depth - 1))];
}

/** One wrapped line of rich text: segments reference their source run. */
export interface TextRunLine {
  segments: { text: string; run: TextRun }[];
  /** Visible width – trailing whitespace excluded, as CSS does when centering. */
  width: number;
  /** Line height in px – the CSS line box over every inline box on the line. */
  height?: number;
  /** Distance from the line's top to its baseline (CSS half-leading applied). */
  baseline?: number;
  /** x where the TEXT starts (list items: depth × bullet width, every line). */
  indent?: number;
  /** Extra vertical gap before this line (paragraph boundary). */
  gapBefore?: boolean;
  /**
   * The gap in px. Derived from the block's STRUT, never from this line's
   * height: the overlay applies `margin-top: calc(0.6 * 1.25em)` on the block
   * element, whose em is the topic's font size – a heading's size lives on an
   * inner span and does not enlarge its own block gap. Sizing the gap from the
   * line height made a heading after a paragraph sit 9px too low.
   */
  gapPx?: number;
  /** First line of a list item: which marker to draw and where. */
  bullet?: { char: string; x: number; run: TextRun };
}

function runMetrics(run: TextRun, style: Style): TextMetricsInput {
  return {
    fontSize: run.fontSize ?? style.fontSize ?? 14,
  }

```

```

    fontFamily: style.fontFamily,
    fontWeight: run.bold ? 700 : style.fontWeight,
    italic: (run.italic ?? false) || (style.italic ?? false),
  });
}

/**
 * Wrap rich text across runs at `maxW`, breaking only on whitespace.
 * A word may span runs (bold "he" + plain "llo" stays one token) so emphasis
 * never splits mid-word. Trailing whitespace is collapsed at line breaks.
 *
 * Block semantics:
 * - a run with `paraGap` marks a paragraph boundary: its paragraph gets an
 *   extra vertical gap (gapBefore on the first line);
 * - a run with `listIndent` starts a bullet item: the paragraph's first line
 *   carries the bullet glyph, continuation lines hang-indent under it;
 * - each line's height is the tallest run on that line (per-run fontSize).
 */
export function wrapRunLines(runs: TextRun[], maxW: number, measurer: TextMeasurer, style: Style): TextRunLine[] {
  const widthOf = (text: string, run: TextRun): number => measurer.measure(text, runMetrics(run, style)).width;
  /**
   * The CSS strut: every line box is at least as tall as the block's own font
   * size, even when every inline box on it is smaller. Without this a run of
   * 10px text inside a 14px topic produced a 12.5px line on the canvas and a
   * 17.5px line in the editor.
   */
  const strut = style.fontSize ?? 14;
  /** `margin-top: calc(BLOCK_GAP_FACTOR * LINE_HEIGHT_FACTOR em)` on the block. */
  const blockGapPx = strut * LINE_HEIGHT_FACTOR * BLOCK_GAP_FACTOR;

  /**
   * One inline box's extent above and below the baseline, CSS-style: the
   * leading (line-height - content height) is split in half around the font's
   * content area. A line box is then max(above) + max(below) over EVERY inline
   * box on the line, the strut included - not `tallest font-size × 1.25`,
   * which put a 26px heading 1px off from the browser on every line.
   */
  const boxOf = (m: TextMetricsInput): { above: number; below: number } => {
    const met = measurer.metrics ? measurer.metrics(m) : { ascent: m.fontSize * 0.8, descent: m.fontSize * 0.2 };
    const half = (m.fontSize * LINE_HEIGHT_FACTOR - (met.ascent + met.descent)) / 2;
    return { above: met.ascent + half, below: met.descent + half };
  };
  const strutBox = boxOf({ fontSize: strut, fontFamily: style.fontFamily, fontWeight: style.fontWeight, italic: style.italic });

  // 1) Split the run stream into paragraphs (each paragraph is a segment list
  // that may span multiple runs - emphasis survives a break). A paragraph
  // ends at a literal \n, and ALSO at a run that opens a new block.
  interface Seg {
    text: string;
    run: TextRun;
    paraGap: boolean;
  }
  const paragraphs: Seg[][] = [[]];
  const openParagraph = (): void => {
    if (paragraphs[paragraphs.length - 1].length > 0) paragraphs.push([]);
  };
  for (const run of runs) {
    const parts = run.text.split("\n");
    for (let i = 0; i < parts.length; i++) {
      if (i > 0) paragraphs.push([]);
      // A paraGap/listIndent run starts a new block even WITHOUT a newline:
      // editorStateToRuns marks a root-child boundary with paraGap alone, so
      // without this two typed paragraphs ran together on one canvas line
      // while the editor showed them as two separate <p>.
      else if (run.paraGap || run.listIndent !== undefined) openParagraph();
      if (parts[i].length > 0) {
        // only the first segment of the run carries the paragraph-gap flag
        paragraphs[paragraphs.length - 1].push({ text: parts[i], run, paraGap: i === 0 && !!run.paraGap });
      }
    }
  }
}

/** Longest prefix of `text` that fits `budget` (at least one character). */
const fitPrefix = (text: string, run: TextRun, budget: number): number => {
  let lo = 1;
  let hi = text.length;
  let best = 1;
  while (lo <= hi) {
    const mid = (lo + hi) >> 1;
    if (widthOf(text.slice(0, mid), run) <= budget) {
      best = mid;
      lo = mid + 1;
    } else {
      hi = mid - 1;
    }
  }
  return best;
}

```

```

};

const out: TextRunLine[] = [];
let emittedBlocks = 0;
// Blank paragraphs are only real blank lines when content follows them; the
// trailing ones are just the newlines that close the last block. Tolerating
// any number of them keeps documents written by older builds (which could
// accumulate "\n\n") rendering the same as the editor shows them.
let lastContentIdx = -1;
for (let i = 0; i < paragraphs.length; i++) if (paragraphs[i].length > 0) lastContentIdx = i;
for (let pi = 0; pi < paragraphs.length; pi++) {
  const para = paragraphs[pi];
  // Bullet metadata for this paragraph (first listIndent run wins).
  let bulletRun: TextRun | null = null;
  for (const seg of para) if (seg.run.listIndent) { bulletRun = seg.run; break; }
  const depth = bulletRun?.listIndent ?? 0;
  // Text column of a list item: depth × bullet width on EVERY line (the
  // marker sits in the column to its left). Mirrors the CSS padding-left +
  // negative text-indent on .topic-rich-editable li.
  const bulletW = BULLET_WIDTH_EM * (bulletRun?.fontSize ?? strut);
  const textIndent = depth * bulletW;
  const budget = Math.max(1, maxW - textIndent);
  // Block boundary → fixed gap. A paragraph flagged paraGap OR a list item
  // (bulletRun) starts a new block; the very first block of the topic has
  // no leading gap. The overlay applies the same rule with --rnode-block-gap.
  const gapBefore = (para.length === 0 ? false : para[0].paraGap || !!bulletRun) && emittedBlocks > 0;

  // 2) Tokenize the paragraph into whitespace-separated fragments (whitespace
  // kept). A word may span runs (bold "he" + plain "llo" stays one token)
  // so emphasis never splits mid-word.
  const tokens: { text: string; run: TextRun }[] = [];
  for (const seg of para) {
    for (const part of seg.text.split(/(\s+)/)) {
      if (part.length > 0) tokens.push({ text: part, run: seg.run });
    }
  }

  const startOutLen = out.length;
  let line: { text: string; run: TextRun }[] = [];
  let lineWidth = 0; // includes trailing whitespace
  let visibleWidth = 0; // excludes it — this is what CSS centers
  let firstLine = true; // the marker lives on the first line only
  let lineAbove = strutBox.above;
  let lineBelow = strutBox.below;
  const grow = (run: TextRun): void => {
    const bx = boxOf(runMetrics(run, style));
    lineAbove = Math.max(lineAbove, bx.above);
    lineBelow = Math.max(lineBelow, bx.below);
  };
  const push = (): void => {
    if (line.length === 0) return;
    // Trailing whitespace hangs past the line end in CSS: it must not count
    // towards the width, or every centered wrapped line sits half a space
    // off and the measured box is a couple of px too wide.
    while (line.length > 0 && /\s+$/ .test(line[line.length - 1].text)) line.pop();
    out.push({
      segments: line,
      width: visibleWidth,
      height: lineAbove + lineBelow,
      baseline: lineAbove,
      indent: textIndent,
      gapBefore: gapBefore && firstLine,
      gapPx: gapBefore && firstLine ? blockGapPx : undefined,
      bullet: bulletRun && firstLine ? { char: bulletChar(depth), x: Math.max(0, textIndent - bulletW), run: bulletRun } : undefined,
    });
    line = [];
    lineWidth = 0;
    visibleWidth = 0;
    lineAbove = strutBox.above;
    lineBelow = strutBox.below;
    firstLine = false;
  };

  for (const tok of tokens) {
    if (/^\s+$/ .test(tok.text)) {
      if (line.length > 0) {
        line.push(tok);
        lineWidth += widthOf(tok.text, tok.run);
      }
      continue; // leading/collapsed whitespace skipped
    }
    let rest = tok.text;
    // A token longer than the whole column must break mid-word, exactly as
    // `overflow-wrap: break-word` does in the overlay. Without this the
    // canvas drew past its bitmap (clipping the tail) while the editor let
    // it overflow the box.
    for (;;) {

```

```

const w = widthOf(rest, tok.run);
if (lineWidth + w <= budget || (line.length === 0 && w <= budget)) {
  line.push({ text: rest, run: tok.run });
  lineWidth += w;
  visibleWidth = lineWidth;
  grow(tok.run);
  break;
}
if (line.length > 0) {
  push();
  continue; // retry the whole token on a fresh line
}
const k = fitPrefix(rest, tok.run, budget);
if (k >= rest.length) {
  line.push({ text: rest, run: tok.run });
  lineWidth += w;
  visibleWidth = lineWidth;
  grow(tok.run);
  break;
}
const head = rest.slice(0, k);
line.push({ text: head, run: tok.run });
lineWidth += widthOf(head, tok.run);
visibleWidth = lineWidth;
grow(tok.run);
push();
rest = rest.slice(k);
}
}
push();
if (out.length > startOutLen) {
  emittedBlocks++;
} else if (pi < lastContentIdx) {
  // An empty paragraph is a real, visible blank line in the editor (the
  // canvas used to swallow it). A trailing empty paragraph is just the
  // newline that closes the last block, so it is not emitted.
  out.push({ segments: [], width: 0, height: strutBox.above + strutBox.below, baseline: strutBox.above, indent: textIndent, gapBefore, gapPx:
gapBefore ? blockGapPx : undefined });
  emittedBlocks++;
}
}
if (out.length === 0) out.push({ segments: [], width: 0, height: strutBox.above + strutBox.below, baseline: strutBox.above });
return out;
}

/**
 * String-based wrapping (legacy consumers). Runs collapse to plain text and
 * measure with the base style.
 */
export function wrapLines(text: string, maxW: number, measurer: TextMeasurer, style: Style): string[] {
  return wrapRunLines([ { text } ], maxW, measurer, style).map((l) => l.segments.map((s) => s.text).join("").trimEnd());
}

// -----
// Extent
// -----

export interface Extent {
  w: number;
  h: number;
}

/**
 * Resolve an attachment id to the ORIGINAL image's pixel size. Built from the
 * sheet's metadata cards in one place (invariant I9): every caller that
 * measures a node – layout, renderer, overlay – uses the same resolver, so
 * the three never disagree on a node's extent.
 */
export function imageResolver(sheet: Sheet): (id: string) => { w: number; h: number } | null {
  const byId = new Map<string, { w: number; h: number }>();
  for (const att of sheet.attachments) byId.set(att.id, { w: att.w, h: att.h });
  return (id: string) => byId.get(id) ?? null;
}

/**
 * Observable model:
 * width(topic) = width(text, wrapped) + paddingLeft + paddingRight + shapeAllowance
 * height(topic) = height(lines) + paddingTop + paddingBottom + shapeAllowance
 * Explicit style.width/height overrides win; an image (when resolvable) adds
 * its box above the text, separated by IMAGE_GAP.
 */
export function measureTopic(
  n: MindNode,
  measurer: TextMeasurer = HEURISTIC_MEASURER,
  resolveImage?: (id: string) => { w: number; h: number } | null,
): Extent {
  const style = n.style;

```

```

if (style.width && style.height) return { w: style.width, h: style.height };

const fontSize = style.fontSize ?? 14;
const pad = style.padding ?? 10;
// An explicit width fixes the box and re-wraps the text at it (Xmind-style
// resize); the height always follows the wrapped content.
const maxW = style.width ? Math.max(MIN_TOPIC_W, style.width) : MAX_TOPIC_W;
const textW = Math.max(24, maxW - pad * 2 - TEXT_INSET);
const runs = nodeRuns(n.title, n.titleRuns);
const lines = wrapRunLines(runs, textW, measurer, style);
// the list indent is part of the line's extent, not free space
const maxLineW = lines.reduce((acc, l) => Math.max(acc, (l.indent ?? 0) + l.width), 0);

// Image above the text: width from style.imageWidth (or the original's,
// capped at MAX_IMAGE_W), height keeps the aspect ratio. Unresolvable ids
// (no card in sheet.attachments) measure as if the node had no image.
let imgW = 0;
let imgH = 0;
if (style.image && resolveImage) {
  const att = resolveImage(style.image);
  if (att && att.w > 0) {
    imgW = style.imageWidth ?? Math.min(att.w, MAX_IMAGE_W);
    imgH = (imgW * att.h) / att.w;
  }
}
// An empty title still yields one strut line; "has text" means real content.
const hasText = lines.some((l) => l.segments.length > 0);
const textH = lines.reduce((acc, l) => acc + (l.height ?? fontSize * LINE_HEIGHT_FACTOR) + (l.gapPx ?? 0), 0);

let w = style.width ? Math.max(MIN_TOPIC_W, style.width) : Math.min(MAX_TOPIC_W, Math.max(MIN_TOPIC_W, Math.ceil(maxLineW) + pad * 2 + TEXT_INSET));
if (imgW > 0) w = Math.max(w, imgW + pad * 2);
// With an image the box is imgH + (gap + text only if real text) + padding;
// without one, the previous formula is untouched (the strut line included).
let h = style.height ?? Math.max(28, imgH > 0 ? imgH + (hasText ? IMAGE_GAP + textH : 0) + pad * 2 + 4 : textH + pad * 2 + 4);

const shape = style.shape ?? "rounded";
if (shape === "circle") {
  const d = Math.max(w, h);
  return { w: d, h: d };
}
if (shape === "diamond") {
  w += fontSize;
  h += fontSize;
} else if (shape === "hexagon") {
  w += 14;
  h += 10;
}
return { w, h };
}

export function measureNode(n: MindNode, measurer?: TextMeasurer, resolveImage?: (id: string) => { w: number; h: number } | null): Extent {
  return measureTopic(n, measurer, resolveImage);
}

```

## 51. src/persist/storage.ts

```

/**
 * Persistence layer.
 *
 * Adapter interface so the editor never cares where documents live:
 * - localStorage: default for the web MVP (autosave, crash-safe enough);
 * - IndexedDB: later web upgrade;
 * - SQLite via the Rust document engine: desktop (Tauri commands).
 */
import type { RnodeDocument } from "../core/types";

export interface StorageAdapter {
  readonly label: string;
  load(): Promise<RnodeDocument[]>;
  save(docs: RnodeDocument[]): Promise<void>;
}

// Storage key for the R-node brand. The app was previously named "R-mind":
// the old key is read once and migrated so existing documents are never lost.
const KEY = "r-node.docs.v1";
const LEGACY_KEY = "r-mind.docs.v1";

/** Rename the built-in sample document when migrating from the old app name. */
function migrateSample(doc: RnodeDocument): RnodeDocument {
  if (doc.title === "R-mind - Roadmap") doc.title = "R-node - Roadmap";
  const sheet = doc.sheets[0];
  const root = sheet.nodes[sheet.rootNodeId];
  if (root && root.title === "R-mind") root.title = "R-node";
  return doc;
}

```

```

}

export class LocalStorageAdapter implements StorageAdapter {
  readonly label = "localStorage";

  async load(): Promise<RnodeDocument[]> {
    try {
      const raw = localStorage.getItem(KEY);
      if (raw) {
        const parsed = JSON.parse(raw);
        if (!Array.isArray(parsed)) return [];
        return parsed.filter((d) => d && typeof d === "object" && Array.isArray(d.sheets));
      }
      // No data under the new key yet – migrate from the legacy "r-mind" key.
      const legacy = localStorage.getItem(LEGACY_KEY);
      if (!legacy) return [];
      const parsed = JSON.parse(legacy);
      if (!Array.isArray(parsed)) return [];
      const docs = parsed.filter((d) => d && typeof d === "object" && Array.isArray(d.sheets));
      docs.forEach(migrateSample);
      if (docs.length > 0) {
        try {
          localStorage.setItem(KEY, JSON.stringify(docs));
        } catch (e) {
          console.error("storage migration write failed", e);
        }
      }
      return docs;
    } catch {
      return [];
    }
  }

  async save(docs: RnodeDocument[]): Promise<void> {
    try {
      localStorage.setItem(KEY, JSON.stringify(docs));
    } catch (e) {
      console.error("localStorage save failed", e);
      throw e;
    }
  }
}

/**
 * Desktop: the document is ONE `.rnode` file (a SQLite database, T20). The
 * document JSON lives in its `document` table; the images are served by
 * TauriAssetStore from the same file. The path is mutable state set by
 * open/save-as – the same pattern as the asset store, so a file switch never
 * re-creates anything. No localStorage on desktop: the document survives
 * even if the webview profile is wiped.
 */
export class TauriStorageAdapter implements StorageAdapter {
  readonly label = "document file (rust)";
  private root: string | null = null;

  /** The document file this adapter serves. Null = nothing chosen yet. */
  setRoot(path: string | null): void {
    this.root = path;
  }

  get hasRoot(): boolean {
    return this.root !== null;
  }

  /** The current document path, or null before the first open/save-as. */
  get currentPath(): string | null {
    return this.root;
  }

  private invoke(cmd: string, args?: Record<string, unknown>): Promise<unknown> {
    if (typeof window === "undefined" || !window.__TAURI__) {
      throw new Error("TauriStorageAdapter used outside the Tauri webview");
    }
    return window.__TAURI__.core.invoke(cmd, args);
  }

  /** Read the document from `path` WITHOUT switching this adapter's path. */
  async readDocumentAt(path: string): Promise<RnodeDocument | null> {
    try {
      const text = (await this.invoke("read_document", { path })) as string | null;
      if (!text) return null;
      const doc = JSON.parse(text) as RnodeDocument;
      return Array.isArray(doc.sheets) ? doc : null;
    } catch {
      return null;
    }
  }
}

```

```

async load(): Promise<RnodeDocument[]> {
  if (!this.root) return [];
  const doc = await this.readDocumentAt(this.root);
  return doc ? [doc] : [];
}

async save(docs: RnodeDocument[]): Promise<void> {
  const doc = docs[0];
  if (!doc) return;
  if (!this.root) throw new Error("TauriStorageAdapter: no document file chosen");
  await this.invoke("write_document", { path: this.root, data: JSON.stringify(doc) });
}
}

```

## 52. src/ui/Palette.tsx

```

import { useEffect, useMemo, useRef, useState } from "react";
import { useSyncExternalStore } from "react";
import { useStore } from "../editor/context";
import { runExportPng } from "../editor/exportBridge";
import { viewSize } from "../editor/view";

interface PaletteItem {
  label: string;
  hint?: string;
  run: () => void;
}

export function Palette(): JSX.Element | null {
  const store = useStore();
  const state = useSyncExternalStore(store.subscribe, store.getSnapshot);
  const [query, setQuery] = useState("");
  const [index, setIndex] = useState(0);
  const inputRef = useRef<HTMLInputElement>(null);

  useEffect(() => {
    if (state.showPalette) {
      setQuery("");
      setIndex(0);
      setTimeout(() => inputRef.current?.focus(), 0);
    }
  }, [state.showPalette]);

  const items = useMemo<PaletteItem[]>(() => {
    const base: PaletteItem[] = [
      { label: "New document", hint: "", run: () => store.newDocument() },
      { label: "Save document", hint: "Ctrl+S", run: () => void store.saveNow() },
      { label: "Open file...", hint: "Ctrl+O", run: () => void store.loadFile() },
      { label: "Export as JSON", hint: "Ctrl+E", run: () => store.exportJson() },
      { label: "Export with images (.rnode.zip)", hint: "", run: () => void store.exportRnodeZip("complete") },
      { label: "Export with images, compact (.rnode.zip)", hint: "", run: () => void store.exportRnodeZip("compact") },
      { label: "Export as Markdown", hint: "", run: () => store.exportMarkdown() },
      { label: "Export as PNG", hint: "", run: () => runExportPng() },
      { label: "Toggle outline panel", hint: "", run: () => store.toggleOutliner() },
      { label: "Toggle inspector", hint: "", run: () => store.toggleInspector() },
      { label: "Zen / focus mode", hint: "Ctrl+Shift+F", run: () => store.toggleZen() },
      { label: "Fit map to view", hint: "Ctrl+1", run: () => store.fitView(viewSize.w, viewSize.h) },
      { label: "Undo", hint: "Ctrl+Z", run: () => store.undo() },
      { label: "Redo", hint: "Ctrl+Shift+Z", run: () => store.redo() },
      { label: "Create child topic", hint: "Tab", run: () => store.createChild() },
      { label: "Create sibling topic", hint: "Enter", run: () => store.createSibling() },
      { label: "Promote topic", hint: "Shift+Tab", run: () => store.promote() },
      { label: "Search documents...", hint: "Ctrl+F", run: () => window.dispatchEvent(new CustomEvent("r-node:focus-search")) },
      ...state.docs.filter((d) => !d.archived).map<PaletteItem>((d) => ({ label: `Open: ${d.title}`, hint: "document", run: () =>
store.switchToDoc(d.documentId) })),
    ];
    const q = query.trim().toLowerCase();
    return q ? base.filter((i) => i.label.toLowerCase().includes(q) || (i.hint ?? "").toLowerCase().includes(q)) : base;
  }, [query, state.docs, store]);

  if (!state.showPalette) return null;
  const clamped = Math.min(index, Math.max(items.length - 1, 0));

  return (
    <div
      className="palette-overlay"
      onPointerDown={(e) => {
        if (e.target === e.currentTarget) store.togglePalette();
      }}
    >
      <div className="palette">
        <input
          ref={inputRef}

```

```

placeholder="Type a command or document name..."
value={query}
onChange={(e) => { setQuery(e.target.value); setIndex(0); }}
onKeyDown={(e) => {
  e.stopPropagation();
  if (e.key === "Escape") store.togglePalette();
  else if (e.key === "ArrowDown") setIndex((i) => Math.min(i + 1, items.length - 1));
  else if (e.key === "ArrowUp") setIndex((i) => Math.max(i - 1, 0));
  else if (e.key === "Enter" && items[clamped]) {
    items[clamped].run();
    store.togglePalette();
  }
}}
/>
<div className="palette-list">
  {items.map((item, i) => (
    <button
      key={item.label + i}
      className={`palette-item${i === clamped ? " selected" : ""}`}
      onMouseEnter={() => setIndex(i)}
      onClick={() => { item.run(); store.togglePalette(); }}
    >
      <span>{item.label}</span>
      {item.hint && <span className="palette-hint">{item.hint}</span>}
    </button>
  ))}
  {items.length === 0 && <div className="palette-empty">No matches</div>}
</div>
<div className="palette-footer">↑↓ navigate · Enter run · Esc close</div>
</div>
</div>
);
}

```

## 53. src-tauri/Cargo.toml

```

[package]
name = "r-node"
version = "0.1.0"
description = "R-node desktop shell (Tauri 2)"
edition = "2021"

[build-dependencies]
tauri-build = { version = "2", features = [] }

[dependencies]
tauri = { version = "2", features = [] }
serde = { version = "1", features = ["derive"] }
serde_json = "1"
rusqlite = { version = "0.32", features = ["bundled"] }
tauri-plugin-dialog = "2"
tauri-plugin-http = "2"

[profile.release]
opt-level = 2

```

## 54. src-tauri/tauri.conf.json

```

{
  "$schema": "https://schema.tauri.app/config/2",
  "productName": "R-node",
  "version": "0.1.0",
  "identifier": "com.rnode.app",
  "build": {
    "beforeDevCommand": "npm run dev",
    "devUrl": "http://localhost:5173",
    "beforeBuildCommand": "npm run build",
    "frontendDist": "../dist"
  },
  "app": {
    "windows": [
      {
        "title": "R-node",
        "width": 1280,
        "height": 820,
        "minWidth": 900,
        "minHeight": 600,
        "dragDropEnabled": false
      }
    ]
  },
}

```

```

    "withGlobalTauri": true,
    "security": {
      "csp": null
    }
  },
  "bundle": {
    "active": true,
    "targets": "all",
    "icon": [
      "icons/32x32.png",
      "icons/128x128.png",
      "icons/128x128@2x.png",
      "icons/icon.icns",
      "icons/icon.ico"
    ]
  }
}

```

## 55. tests/measure.test.ts

```

import { describe, expect, it } from "vitest";
import { BULLET_WIDTH_EM, IMAGE_GAP, imageResolver, MAX_IMAGE_W, measureNode, MIN_TOPIC_W, wrapRunLines, HEURISTIC_MEASURER } from
"../src/layout/measure";
import { DocumentModel } from "../src/core/doc";
import { DEFAULT_STRUCTURE, type MindNode, type Sheet, type TextRun } from "../src/core/types";

function lineHeights(runs: TextRun[], maxW = 200): number[] {
  return wrapRunLines(runs, maxW, HEURISTIC_MEASURER, { fontSize: 14 }).map((l) => l.height ?? 14 * 1.25);
}

function node(overrides: Partial<MindNode>): MindNode {
  const model = new DocumentModel(DocumentModel.blank("m"));
  const root = model.rootNode;
  return { ...root, ...overrides, style: { fontSize: 14, ...(overrides.style ?? {}) } };
}

describe("topic extent (observable model)", () => {
  it("short single-line title → minimum width, one line of height", () => {
    const e = measureNode(node({ title: "Hi" }));
    expect(e.w).toBe(MIN_TOPIC_W);
    expect(e.h).toBeCloseTo(14 * 1.25 + 10 * 2 + 4, 5); // line + padding + frame
  });

  it("a long title wraps and the topic grows taller", () => {
    const short = measureNode(node({ title: "Short" }));
    const long = measureNode(node({ title: "word word word word word word word word word word word word word word word" }));
    expect(long.h).toBeGreaterThan(short.h);
    // 13 words ≈ 64 chars ≈ 8 wrapped lines at heuristic 7.7px/char → clearly > 1 line
    expect(long.h).toBeGreaterThan(short.h + 14);
  });

  it("explicit newlines force line breaks", () => {
    const e = measureNode(node({ title: "Line one\nLine two" }));
    expect(e.h).toBeCloseTo(2 * 14 * 1.25 + 10 * 2 + 4, 5);
  });

  it("larger padding grows the vertical extent (and wraps text harder)", () => {
    const a = measureNode(node({ title: "Padding padding padding padding", style: { padding: 10 } }));
    const b = measureNode(node({ title: "Padding padding padding padding", style: { padding: 24 } }));
    // less text room → more lines → taller box (narrower is expected and fine)
    expect(b.h).toBeGreaterThan(a.h);
    expect(b.w * b.h).toBeGreaterThan(a.w * a.h);
  });

  it("circle is square; diamond and hexagon grow the box", () => {
    const c = measureNode(node({ title: "O", style: { shape: "circle", fontSize: 14 } }));
    expect(c.w).toBe(c.h);

    const base = measureNode(node({ title: "T", style: { shape: "rounded" } }));
    const d = measureNode(node({ title: "T", style: { shape: "diamond" } }));
    expect(d.w).toBe(base.w + 14);
    expect(d.h).toBe(base.h + 14);

    const hx = measureNode(node({ title: "T", style: { shape: "hexagon" } }));
    expect(hx.w).toBe(base.w + 14);
    expect(hx.h).toBe(base.h + 10);
  });

  it("explicit width/height override wins", () => {
    const e = measureNode(node({ title: "A very long title that would wrap", style: { width: 400, height: 90 } }));
    expect(e).toEqual({ w: 400, h: 90 });
  });

  it("a fixed width re-wraps the text and the height follows (Xmind resize)", () => {

```

```

const title = "word word word word word word word word word";
const narrow = measureNode(node({ title, style: { width: 120 } }));
const wide = measureNode(node({ title, style: { width: 400 } }));
expect(narrow.w).toBe(120);
expect(wide.w).toBe(400);
// fewer chars fit per line at 120px → more lines → taller box
expect(narrow.h).toBeGreaterThan(wide.h);
// clearing the width returns to auto-fit (capped at MAX_TOPIC_W)
const auto = measureNode(node({ title }));
expect(auto.w).toBeLessThanOrEqual(280);
});

it("explicit widths below the minimum are clamped up", () => {
  const e = measureNode(node({ title: "T", style: { width: 40 } }));
  expect(e.w).toBe(MIN_TOPIC_W);
});

it("font size scales the extent", () => {
  const s = measureNode(node({ title: "Scale", style: { fontSize: 12 } }));
  const l = measureNode(node({ title: "Scale", style: { fontSize: 28 } }));
  expect(l.w).toBeGreaterThan(s.w);
  expect(l.h).toBeGreaterThan(s.h);
});

it("measurer is pluggable (deterministic in tests)", () => {
  const title = "iiiiiiiiiiiiiiiiiii"; // long enough to escape the min-width floor
  const wide = measureNode(node({ title }, { measure: (t, s) => ({ width: t.length * s.fontSize * 1.0 }) }));
  const narrow = measureNode(node({ title }, { measure: (t, s) => ({ width: t.length * s.fontSize * 0.2 }) }));
  expect(wide.w).toBeGreaterThan(narrow.w);
});

it("a paraGap run adds extra vertical height (paragraph spacing)", () => {
  // same two lines of text: with and without the paragraph gap
  const plain = measureNode(node({ title: "Line one\nLine two" }));
  const spaced = measureNode(node({ title: "Line one\nLine two", titleRuns: [{ text: "Line one" }, { text: "\n" }, { text: "Line two", paraGap: true } ] }));
  expect(spaced.h).toBeGreaterThan(plain.h);
});

it("a heading-size run makes its line taller", () => {
  const sizes = lineHeights([{ text: "Body" }, { text: "Heading", fontSize: 26 }]);
  // both runs share one line; the tallest run sets the line height
  expect(sizes[0]).toBeCloseTo(26 * 1.25, 5);
});

it("list items carry a bullet on the first line and indent the text column on every line", () => {
  const lines = wrapRunLines(
    [{ text: "A fairly long list item that will wrap onto multiple lines because the budget is small", listIndent: 1 }],
    80,
    HEURISTIC_MEASURER,
    { fontSize: 14 }
  );
  expect(lines.length).toBeGreaterThan(1);
  // The marker is metadata, not a text segment: it lives in its own column
  // of known width (BULLET_WIDTH_EM) so the overlay's CSS can reproduce it.
  const bulletW = BULLET_WIDTH_EM * 14;
  expect(lines[0].bullet?.char).toBe("•");
  expect(lines[0].bullet?.x).toBeCloseTo(0, 5);
  // the text column is indented on EVERY line – first line included, which
  // is what makes a wrapped item hang under its own text in both renderers
  for (const line of lines) {
    expect(line.indent).toBeCloseTo(bulletW, 5);
  }
  // and only the first line gets the marker
  for (const line of lines.slice(1)) expect(line.bullet).toBeUndefined();
});

it("nested list items sit one bullet column deeper", () => {
  const lines = wrapRunLines([{ text: "deep", listIndent: 2 }], 200, HEURISTIC_MEASURER, { fontSize: 14 });
  const bulletW = BULLET_WIDTH_EM * 14;
  expect(lines[0].indent).toBeCloseTo(bulletW * 2, 5);
  expect(lines[0].bullet?.x).toBeCloseTo(bulletW, 5);
  expect(lines[0].bullet?.char).toBe("•");
});

it("a paraGap run starts a new block even without a newline", () => {
  // editorStateToRuns marks a root-child boundary with paraGap alone; the
  // canvas used to run the two paragraphs together on a single line.
  const lines = wrapRunLines(
    [{ text: "first" }, { text: "second", paraGap: true }],
    400,
    HEURISTIC_MEASURER,
    { fontSize: 14 }
  );
  expect(lines).toHaveLength(2);
  expect(lines[1].gapPx).toBeCloseTo(14 * 1.25 * 0.6, 5);
});

```

```

it("the block gap comes from the strut, not from the following line's height", () => {
  // margin-top: calc(0.6 * 1.25em) resolves against the BLOCK's font size,
  // and a heading's size lives on an inner span – so a heading block's gap
  // is the same as any other block's.
  const lines = wrapRunLines(
    [{ text: "body" }, { text: "Heading", fontSize: 26, paraGap: true }],
    400,
    HEURISTIC_MEASURER,
    { fontSize: 14 }
  );
  expect(lines[1].gapPx).toBeCloseTo(14 * 1.25 * 0.6, 5);
});

it("a blank line between paragraphs survives, a trailing one does not", () => {
  const withBlank = wrapRunLines([
    { text: "a" }, { text: "\n" }, { text: "\n" }, { text: "b" }
  ], 400, HEURISTIC_MEASURER, { fontSize: 14 });
  expect(withBlank).toHaveLength(3);
  expect(withBlank[1].segments).toHaveLength(0);
  // the newline that merely closes the last block is not a blank line
  const trailing = wrapRunLines([
    { text: "a" }, { text: "\n" }
  ], 400, HEURISTIC_MEASURER, { fontSize: 14 });
  expect(trailing).toHaveLength(1);
});

it("a token wider than the column breaks mid-word instead of overflowing", () => {
  const lines = wrapRunLines(
    [{ text: "https://example.com/a/very/long/path/that/cannot/fit" }],
    80,
    HEURISTIC_MEASURER,
    { fontSize: 14 }
  );
  expect(lines.length).toBeGreaterThan(1);
  // nothing may stick out of the column: the canvas bitmap is exactly maxW
  // wide, so an unbroken token used to be silently clipped
  for (const line of lines) expect(line.width).toBeLessThanOrEqual(80 + 0.001);
});

it("trailing whitespace does not count towards the line width", () => {
  // CSS hangs it past the line end; counting it pushed centered lines half a
  // space off and made the measured box wider than the text.
  const [line] = wrapRunLines([
    { text: "word " }
  ], 400, HEURISTIC_MEASURER, { fontSize: 14 });
  expect(line.width).toBeCloseTo(HEURISTIC_MEASURER.measure("word", { fontSize: 14 }).width, 5);
});

describe("topic extent with an image (T12-1)", () => {
  const resolve = () => ({ w: 200, h: 100 });

  it("image 200x100 + one text line: imgH + IMAGE_GAP + lineH + pad*2 + 4", () => {
    const e = measureNode(node({ title: "Hi", style: { image: "img1" } }), HEURISTIC_MEASURER, resolve);
    expect(e.w).toBe(Math.max(MIN_TOPIC_W, 200 + 10 * 2)); // image wider than text
    expect(e.h).toBeCloseTo(100 + IMAGE_GAP + 14 * 1.25 + 10 * 2 + 4, 5);
  });

  it("image-only node: no IMAGE_GAP, no empty text line", () => {
    const e = measureNode(node({ title: "", style: { image: "img1" } }), HEURISTIC_MEASURER, resolve);
    expect(e.h).toBeCloseTo(100 + 10 * 2 + 4, 5);
  });

  it("explicit imageWidth changes width and height proportionally", () => {
    const e = measureNode(
      node({ title: "", style: { image: "img1", imageWidth: 120 } }),
      HEURISTIC_MEASURER,
      resolve
    );
    expect(e.w).toBe(120 + 10 * 2);
    expect(e.h).toBeCloseTo(120 * (100 / 200) + 10 * 2 + 4, 5); // imgH = 60
  });

  it("an image larger than MAX_IMAGE_W is capped at it (proportions kept)", () => {
    const e = measureNode(
      node({ title: "", style: { image: "big" } }),
      HEURISTIC_MEASURER,
      () => ({ w: 2000, h: 1000 })
    );
    expect(e.w).toBe(MAX_IMAGE_W + 10 * 2);
    expect(e.h).toBeCloseTo((MAX_IMAGE_W * 1000) / 2000 + 10 * 2 + 4, 5);
  });

  it("without a resolver (or unresolvable id) the node measures as text-only", () => {
    const withImg = node({ title: "Hi", style: { image: "img1" } });
    const plain = node({ title: "Hi" });
    expect(measureNode(withImg)).toEqual(measureNode(plain));
    expect(measureNode(withImg, HEURISTIC_MEASURER, () => null)).toEqual(measureNode(plain));
  });

  it("imageResolver reads the sheet's attachment cards", () => {
    const sheet: Sheet = {

```

```

    sheetId: "s1",
    title: "T",
    structure: DEFAULT_STRUCTURE,
    rootNodeId: "r",
    nodes: {},
    relationships: [],
    boundaries: [],
    summaries: [],
    callouts: [],
    labels: [],
    zones: [],
    attachments: [{ id: "a1", mime: "image/png", w: 200, h: 100, bytes: 10 }],
    comments: [],
    presentation: {},
  });
  const resolveImg = imageResolver(sheet);
  expect(resolveImg("a1")).toEqual({ w: 200, h: 100 });
  expect(resolveImg("missing")).toBeNull();
});
});

```

## 56. docs/AGENT\_GUIDE.md

# R-node – guida per chi modifica il codice (umano o AI)

Questo file è il contratto. Se una modifica lo viola, è sbagliata anche se compila, anche se i test passano, anche se "sembra più pulita".

**\*\*Leggi §1, §2 e §6 prima di toccare qualunque file.\*\*** Se il task riguarda testo, layout o editor, leggi anche §3 e §4 – quelle sezioni contengono errori già commessi e già pagati: ripeterli è puro spreco.

---

### ## 1. Come si lavora qui

1. **\*\*Un task alla volta\*\***, preso da [ROADMAP.md](ROADMAP.md) nell'ordine in cui è scritto. Non anticipare task successivi, non accorpare.
2. **\*\*Non riprogettare.\*\*** Se il task dice "aggiungi una funzione pura X", non trasformarlo in un refactoring dell'architettura. Le scelte architetturali di §2 non sono in discussione.
3. **\*\*Non toccare file fuori dalla lista `File` del task.\*\*** Se credi che serva, fermati e scrivilo nel messaggio finale invece di farlo.
4. **\*\*Definizione di "finito" ( §6):** tutti e tre i comandi verdi. Se uno fallisce, il task NON è finito – non commentare il test, non alzare la soglia, non aggiungere `skip`.
5. **\*\*Se non riesci, dillo.\*\*** Un task lasciato a metà con una spiegazione onesta vale più di un task "chiuso" disattivando un controllo. Non inventare risultati di test che non hai eseguito.

### ### Comandi

```

```bash
npm test
```

```

```

```bash
npm run typecheck
```

```

```

```bash
npm run dev
```

```

Il dev server serve l'app su `http://localhost:5173/` e l'harness di parità su `http://localhost:5173/dev/parity.html`.

---

### ## 2. Invarianti non negoziabili

Ognuna ha una ragione. La ragione serve a impedire che qualcuno la "ottimizzi" via senza capirla.

| #   | Invariante                                                                  | Perché                                                             |                                                                                |
|-----|-----------------------------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------|
| --- | ---                                                                         | ---                                                                |                                                                                |
| I1  | Un solo ` <canvas>`.</canvas>                                               | Mai un nodo DOM o SVG per topic.                                   | Obiettivo dichiarato: 1000+ nodi fluidi. Un div per nodo lo rende impossibile. |
| I2  | Al massimo <b>**un**</b> overlay di editing montato, con `key={editingId}`. | Due overlay = due sorgenti di verità sul testo.                    |                                                                                |
| I3  | Il nodo in editing <b>**non**</b> viene disegnato dal canvas.               | Altrimenti si vede il testo doppio (ghosting).                     |                                                                                |
| I4  | Lexical <b>**non**</b> ha `HistoryPlugin`.                                  | L'undo/redo vive solo nella history dello store.                   | Due stack di undo si desincronizzano e l'utente perde lavoro.                  |
| I5  | `node.title === runsToPlain(node.titleRuns)` sempre.                        | Search, outliner, export e i test leggono la stringa piatta.       |                                                                                |
| I6  | Il layout è <b>**dato derivato**</b> : mai nella history, mai in un op.     | Un undo deve ripristinare il contenuto, non le coordinate.         |                                                                                |
| I7  | Ogni `Op` porta i dati per invertirsi.                                      | Nessun op legge lo stato globale per calcolare il proprio inverso. | Serve per l'undo e servirà per la                                              |

```
sync (gli op sono già `actorId`/`ts`). |
| I8 | Il round-trip `runs → editor → runs` è **idempotente**. | Non lo era: un titolo derivava a ogni modifica. Test a 3 cicli in
`tests/lexicalRuns.test.ts`. |
| I9 | Le costanti condivise hanno **una** definizione, in `src/layout/measure.ts`. | Vedi §3. Duplicarle è il modo classico in cui i due renderer
divergono. |
| I10 | La parità editor+canvas è **misurata**, non dichiarata. | Vedi §3 e §4. |
| I11 | L'id di un asset è lo **SHA-256 dell'originale**, tranne per gli asset ripristinati da un container `.rnode.zip` compact (`putUnderId`): quelli
portano `AttachmentInfo.originalLost` e un export `complete` deve dichiararlo. | Il compact non porta gli originali: l'id non può essere ri-derivato. Se
nulla lo segna, un export `complete` dichiara originali che non ha. |
```

### Costanti condivise (I9)

Vivono in `src/layout/measure.ts` e **\*\*devono\*\*** essere consumate da entrambi i lati, mai riscritte a mano:

```
| Costante | Consumata da |
|---|---|
| `LINE_HEIGHT_FACTOR` (1.25) | misura + `line-height` dell'overlay |
| `BLOCK_GAP_FACTOR` (0.6) | misura + CSS var `--rnode-block-gap` |
| `BULLET_WIDTH_EM` (1.2) | misura + CSS var `--rnode-bullet-w` |
| `FONT_STACK` | misura, renderer, `RichEditor` – e deve restare **uguale a `--font`** in `styles.css` |
| `TEXT_INSET` (6) | misura + padding calcolato dell'editable |
```

Se cambi una di queste, entrambi i lati devono cambiare insieme e l'harness deve restare a 0 divergenze.

---

## 3. Il contratto di parità (la parte più facile da rompere)

Il canvas **\*\*imita\*\*** il layout testuale del browser. Ogni imitazione deriva. Queste sono le regole esatte che i due lati implementano; sono state ricavate misurando, non leggendo le specifiche CSS.

- \*\*Altezza di riga\*\*** = line box CSS: `max(above) + max(below)` su ogni inline box della riga, **\*\*strut del blocco incluso\*\***, dove ciascun box contribuisce `ascent + half-leading` e `descent + half-leading`, con  $\text{half-leading} = (\text{fontSize} \times 1.25 - (\text{ascent} + \text{descent})) / 2$ .  
**\*\*Non\*\*** è «font-size più grande  $\times 1.25$ ».
- \*\*Baseline\*\***: calcolata in `wrapRunLines` e messa in `line.baseline`. Il renderer la usa e basta – non ricalcolarla lì.
- \*\*Gap di blocco\*\*** = `BLOCK\_GAP\_FACTOR \times LINE\_HEIGHT\_FACTOR \times \text{fontSize}` del nodo. Viene dallo **\*\*strut\*\***, non dall'altezza della riga: in CSS `margin-top: calc(.6 * 1.25em)` si risolve sull'`em` del blocco, e la dimensione di un heading vive su una `` interna.
- \*\*Confine di blocco\*\***: un run con `paraGap` apre un blocco **\*\*anche senza\*\*** `\n`. `editorStateToRuns` segna il confine fra root children solo così.
- \*\*Liste\*\***: il pallino sta in una colonna larga `BULLET\_WIDTH\_EM` em; il testo parte a `depth \times \text{bulletW}` su **\*\*tutte\*\*** le righe dell'item. Gli item di lista sono **\*\*sempre\*\*** allineati a sinistra su entrambi i lati.
- \*\*Spazi finali\*\***: non contano nella larghezza della riga (il CSS li fa sporgere fuori dal box).
- \*\*Token più largo della colonna\*\***: va a capo a metà parola, come `overflow-wrap: break-word`.
- \*\*Riga vuota\*\*** fra due blocchi = riga vera. Le righe vuote **\*\*in coda\*\*** no: un `\n` finale chiude un blocco, non è una riga.

### Regola operativa

Se tocchi **\*\*uno qualsiasi\*\*** di questi file:

---

```
src/layout/measure.ts
src/render/renderer.ts      (parte testo)
src/ui/lexicalRuns.ts
src/styles.css              (regole .topic-rich-*)
src/ui/RichEditor.tsx
---
```

devi aprire `http://localhost:5173/dev/parity.html` e verificare **\*\*0 divergenze\*\***. Il risultato è anche su `window.__parity`.

---

## 4. Trappole già pagate

Non re-derivarle. Costano ore ciascuna.

**\*\*Misura e DOM\*\***

- \*\*jsdom non fa layout\*\*** `getBoundingClientRect()` restituisce zeri. Qualunque test di parità deve girare in un browser vero.
- Il rect di un carattere è l'**\*\*inline box\*\*** (~19px a 14px), più alto della line box (17.5px). Quindi: la sovrapposizione verticale **\*\*non\*\*** separa le righe (usa il ritorno a sinistra) e la sua altezza **\*\*non\*\*** è l'altezza di riga (usa l'avanzamento fra righe).
- Il **\*\*top\*\*** del glifo sta a distanza diversa dalla line box per ogni corpo: confronta le **\*\*baseline\*\***, non i top.

```

**Lexical**
- Un `<p>` dentro un `<li>` con `list-style-position: inside` manda il marker su una riga sua: un item da 17.5px ne misura 35. Gli item devono avere figli inline.
- Il marker nativo del browser (list-style) ha larghezza decisa dallo UA: nessuna lunghezza CSS la eguaglia, quindi un item andato a capo non può allinearsi. Si disegna con li::before a larghezza nota.
- $insertNodes converte un HeadingNode in paragraph a selezione collassata: il paste passa sempre dal percorso runs, mai dall'import DOM di Lexical.
- TextNode.setStyle() sostituisce l'intera stringa di stile: usarlo per il colore cancella il font-size. Usa $patchStyleText da @lexical/selection.
- Nidificazione strutturale (ul > li > ul) e ListItemNode.getIndent() sono la stessa informazione, non due livelli da sommare.
- Il \n che chiude una lista non va riemesso: si accumula a ogni round-trip.

```

```

**React**
- StrictMode fa mount → cleanup → mount. Il seeding dell'editor e la registrazione del listener stanno in due effetti separati: un solo effetto con guard`seeded` perde il listener.

```

```

**Store**
- Il draft di editing è effimero: nessun op, nessuna history, fino al commit.
- Guidare lo store da console mentre un overlay è montato non funziona come test: il listener dell'overlay riscrive il draft con il proprio contenuto.

```

---

## 4bis. Segnalare un bug: il tracer

Non descrivere i sintomi a parole. In dev l'app registra le proprie **decisioni** in un ring buffer (`src/dev/trace.ts`): premi il bottone **●** nella toolbar (o `Ctrl+Shift+D`) nell'istante in cui vedi il problema e scarichi un JSON con gli ultimi ~500 eventi. Quello si allega, non la descrizione. Entrambi catturano **e poi** azzerano – mai prima: quando ci arrivi il baco è già successo.

Cosa contiene, e perché conta:

```

| Evento | A cosa risponde |
|---|---|
| `input` ... outcome: "ignored", reason` | «perché non è successo niente?» – distingue un baco da una guardia deliberata |
| `render` ... visible/nodes, relsDrawn/rels, linksDrawn/links` | «l'ho disegnato ma non si vede» oppure «non l'ho proprio disegnato»: sono due bug diversi |
| `render` ... textHits/textMisses` | regressioni della cache del testo |
| `op`, `layout` con durata | quale sottosistema è rallentato |
| `invariant`, `error` | arrivano con i venti eventi che li hanno preceduti |

```

### Leggere una cattura

Il JSON ha un campo `README` che si spiega da solo, poi `counts` e `events` in ordine cronologico. `t`` sono i millisecondi dal caricamento della pagina, `n`` è quanti eventi identici consecutivi sono stati fusi in quella voce.

Ordine di lettura consigliato:

- input** con `outcome: "ignored"` – se il sintomo è «non succede niente», la risposta è quasi sempre qui, con il motivo già scritto.
- error** e **invariant** – arrivano con la loro cronologia davanti.
- render** – è la voce che separa due bug che sembrano identici:
  - `linksDrawn < links` → il renderer **non** ha disegnato: bug di culling;
  - `linksDrawn === links` ma l'utente non vede nulla → ha disegnato e non si vede: bug di pittura (colore, spessore sub-pixel, ordine di disegno).
  - Lo stesso vale per `relsDrawn/rels` e `visible/nodes`.
- op** e **layout** con le durate, se il sintomo è lentezza.

Esempio – un gesto scartato da una guardia (forma del log):

```

...
INPUT wheel:pan applied x1
INPUT wheel:zoom applied x1
INPUT wheel ignored(reason) x1    ← la risposta
...

```

Nota: la rotella NON è più bloccata durante l'editing – il `guard`editing`` sul wheel è stato rimosso (l'overlay deriva la sua posizione dalla camera dello store a ogni render, quindi pan/zoom resta incollato al nodo).

### Regola per chi scrive codice

**Ogni return anticipato in un handler di input deve dire perché.**

```

...ts
if (someGuard) return trace.ignored("pointerdown", "some-guard");
...

```

Un `return` muto rende un comportamento voluto indistinguibile da un difetto,

e nessuna segnalazione a parole può colmare quella differenza. Stessa regola per il renderer: se salti qualcosa, conta quanto ne hai saltato.

Il tracer è disattivato nelle build di produzione: ogni chiamata esce su un booleano. Istruzioni per catturare: [README](../README.md#reporting-a-bug).

---

## ## 5. Cosa NON fare (già fatto o inesistente)

Verificato nel codice. Se una lista di miglioramenti propone questi, salta.

| Proposta ricorrente | Realtà |  
 |---|---|  
 | «Aggiungere una cache al text measuring» | **Esiste già**: ``createCanvasTextMeasurer`` ha una ``Map`` con chiave ``weight`|italic`|size`|family`|text`` (cap 20k) più una ``metricsCache`` per `ascend/descent` – ``src/layout/measure.ts``. |  
 | «Rendere atomiche le transazioni nella history» | **Già atomiche**: ``execOps`` chiama ``history.push(ops, inverses)`` una volta per batch e ``push`` crea **una** ``HistoryEntry`` – ``src/core/history.ts``. Un gesto multi-op occupa già uno slot di undo. |  
 | «Risolvere la race fra ``layoutTimer`` e ``saveTimer``» | **``saveTimer`` non esiste.** Il salvataggio è manuale (Ctrl+S). Esiste solo ``layoutTimer`` (30ms). |  
 |  
 | «Ottimizzare ``mapBounds``» | Reale ma inutile: **un solo chiamante**, ``fitToScreen``. Non è un hot path. |  
 | «Il canvas somma le larghezze dei token e accumula errore» | **Ipotesi testata e smentita.** Caso ``real-topic-22px`` nell'harness: 9/9 righe, altezza identica. Le righe di troppo venivano dal round-trip non idempotente. |

---

## ## 6. Definizione di "finito"

Un task è chiuso quando **tutti** questi sono veri:

- ``npm run typecheck`` → 0 errori.
- ``npm test`` → tutti verdi. Nessun test nuovo ``skip``, nessuna soglia alzata per farlo passare.
- Se hai toccato i file di §3: harness a **0 divergenze**.
- Hai aggiunto almeno un test che **fallisce** senza la tua modifica. Se non sai scriverlo, probabilmente non hai capito il task.
- Non hai modificato file fuori dalla lista ``File`` del task.

### ### Commit

Un commit per task. Messaggio in inglese, imperativo, che spiega **perché** – non cosa (il diff dice già cosa):

---

Add runtime invariant checker for sheet topology

applyOp mutates the tree in place, so a wrong parentId silently corrupts the in-memory state and only surfaces much later as an unexplained crash. validateSheet turns that into an immediate, named failure.

---

## ## 7. Mappa rapida dei file

---

|                          |                                                       |
|--------------------------|-------------------------------------------------------|
| src/core/types.ts        | TextRun, MindNode, Style, Sheet – schema, unica fonte |
| src/core/text.ts         | helper runs: normalizeRuns, runsToPlain, nodeRuns...  |
| src/core/ops.ts          | op system: applyWithInverse                           |
| src/core/history.ts      | undo/redo, un HistoryEntry per batch                  |
| src/core/doc.ts          | DocumentModel: accesso ai nodi, walks                 |
| src/editor/store.ts      | EditorStore: execOps, draft, commit, camera, save     |
| src/layout/measure.ts    | wrapRunLines, line box CSS, costanti condivise        |
| src/layout/mindmap.ts    | layoutSheet, applyLayout                              |
| src/render/renderers.ts  | drawNode, renderTextBitmap, nodeColors, hit-test      |
| src/ui/RichEditor.tsx    | overlay Lexical (unico), toolbar, tasti, paste        |
| src/ui/lexicalRuns.ts    | bridge editor ↔ runs                                  |
| src/ui/pasteSanitizer.ts | sanitizeHtml / htmlToRuns (Word, Draw.io, web)        |
| src/ui/CanvasView.tsx    | posizionamento overlay, pan, resize handle            |
| dev/parity.ts            | harness di parità (§3)                                |

---

Documenti: [ARCHITECTURE.md](ARCHITECTURE.md) (struttura e fasi) ·  
 [RICH\_TEXT\_EDITOR.md](RICH\_TEXT\_EDITOR.md) (editor rich text in dettaglio) ·  
 [ROADMAP.md](ROADMAP.md) (cosa fare, in ordine).

## 57. src/editor/shortcuts.ts

```
/**
 * Keyboard shortcuts.
 *
 * Central, customizable mapping (a profile object so users can rebind later).
 * Returns true when a key combo was consumed. While a topic is being edited
 * inline, the editor overlay owns the keyboard and shortcuts are skipped.
```

```

*/
import type { EditorStore } from "./store";

export interface ShortcutProfile {
  [key: string]: string; // canonical combo -> action id
}

export const DEFAULT_SHORTCUTS: Record<string, string> = {
  Enter: "create-sibling",
  Tab: "create-child",
  "Shift+Tab": "promote",
  ArrowUp: "nav-up",
  ArrowDown: "nav-down",
  ArrowLeft: "nav-left",
  ArrowRight: "nav-right",
  F2: "edit",
  Space: "toggle-collapse",
  Delete: "delete",
  Backspace: "delete",
  "Mod+z": "undo",
  "Mod+Shift+z": "redo",
  "Mod+y": "redo",
  "Mod+c": "copy",
  "Mod+Shift+c": "copy-outline",
  "Mod+x": "cut",
  "Mod+v": "paste",
  "Mod+k": "palette",
  "Mod+f": "search",
  "Mod+s": "save",
  "Mod+o": "open",
  "Mod+e": "export-json",
  "Mod+d": "duplicate",
  "Mod+Enter": "task-complete",
  "Mod+=": "zoom-in",
  "Mod+-": "zoom-out",
  "Mod+0": "zoom-reset",
  "Mod+p": "present",
  "Mod+Shift+f": "zen",
  "Mod+l": "fit-view",
};

export function handleShortcut(store: EditorStore, e: KeyboardEvent, vw: number, vh: number): boolean {
  if (store.getSnapshot().editingId) {
    // While the type-to-edit editor is still mounting, buffer printable
    // characters into the pending insert so fast typing never loses a key.
    // Once the textarea is mounted (pending consumed) keys reach it directly.
    if (!e.ctrlKey && !e.metaKey && !e.altKey && e.key.length === 1 && !e.key.match(/\s/)) {
      return store.appendPendingInsert(e.key);
    }
    return false;
  }

  if (e.key === "Escape") {
    const s = store.getSnapshot();
    if (s.showPalette) {
      store.togglePalette();
    } else if (s.relFrom) {
      store.clearRelFrom();
    } else if (s.selection.length > 0 || s.imageSel) {
      store.clearSelection();
    }
    return true;
  }

  const mod = e.metaKey || e.ctrlKey;
  const parts: string[] = [];
  if (mod) parts.push("Mod");
  if (e.shiftKey && ![ "Tab", "ArrowUp", "ArrowDown", "ArrowLeft", "ArrowRight" ].includes(e.key)) parts.push("Shift");
  if (e.altKey) parts.push("Alt");
  parts.push(e.key === " " ? "Space" : e.key);
  const combo = parts.join("+");
  const key = e.key === " " ? "Space" : e.key;

  const action = DEFAULT_SHORTCUTS[combo] ?? (mod ? undefined : DEFAULT_SHORTCUTS[key]);
  if (!action) {
    // XMind-style type-to-edit: any printable character with a topic selected
    // starts editing it with that character. Modifier keys, Space (collapse)
    // and special keys never reach this branch.
    if (!mod && !e.altKey && e.key.length === 1 && !e.key.match(/\s/)) {
      store.typeToEdit(e.key);
      return true;
    }
    return false;
  }

  // Tab is special: never let it move focus out of the canvas.
  if (e.key === "Tab" || e.key === "Enter") e.preventDefault();

```

```

switch (action) {
  case "create-sibling":
    store.createSibling();
    return true;
  case "create-child":
    store.createChild();
    return true;
  case "promote":
    store.promote();
    return true;
  case "nav-up":
    store.navigate("up");
    return true;
  case "nav-down":
    store.navigate("down");
    return true;
  case "nav-left":
    store.navigate("left");
    return true;
  case "nav-right":
    store.navigate("right");
    return true;
  case "edit": {
    const n = store.selectionNode;
    if (n) store.startEdit(n.id);
    return true;
  }
  case "toggle-collapse": {
    const n = store.selectionNode;
    if (n) store.toggleCollapsed(n.id);
    return true;
  }
  case "delete": {
    e.preventDefault();
    const s = store.getSnapshot();
    // An image selected inside its node: Backspace/Delete removes ONLY the
    // image reference – never the node.
    if (s.imageSel) store.deleteSelectedImage();
    else if (s.relSel) store.deleteSelectedRelationship();
    else if (s.groupSel) store.deleteGroup(s.groupSel);
    else if (s.summarySel) store.deleteSummary(s.summarySel);
    else store.deleteSelection();
    return true;
  }
  case "copy-outline":
    void store.copySelectionOutline();
    return true;
  case "undo":
    e.preventDefault();
    store.undo();
    return true;
  case "redo":
    e.preventDefault();
    store.redo();
    return true;
  case "copy":
    void store.copySelection();
    return true;
  case "cut":
    void store.cutSelection();
    return true;
  case "paste":
    e.preventDefault();
    void store.paste();
    return true;
  case "palette":
    e.preventDefault();
    store.togglePalette();
    return true;
  case "search":
    e.preventDefault();
    window.dispatchEvent(new CustomEvent("r-node:focus-search"));
    return true;
  case "save":
    e.preventDefault();
    void store.saveNow();
    return true;
  case "open":
    e.preventDefault();
    void store.loadFile();
    return true;
  case "export-json":
    e.preventDefault();
    store.exportJson();
    return true;
  case "duplicate":

```

```

    store.duplicateTopic();
    return true;
  case "task-complete": {
    const n = store.selectionNode;
    if (n) store.toggleTaskComplete(n.id);
    return true;
  }
  case "zoom-in":
    store.zoomStep(1.2, vw, vh);
    return true;
  case "zoom-out":
    store.zoomStep(1 / 1.2, vw, vh);
    return true;
  case "zoom-reset":
    store.fitView(vw, vh);
    return true;
  case "present":
    store.toast("Presentation mode lands in Phase 4");
    return true;
  case "zen":
    store.toggleZen();
    return true;
  case "fit-view":
    store.fitView(vw, vh);
    return true;
  default:
    return false;
}
}

```

## 58. src/ui/Inspector.tsx

```

import { useEffect, useState } from "react";
import { useStore } from "../editor/context";
import type { Group, Relationship, Summary, TaskStatus, Priority, TopicShape } from "../core/types";
import { makeOp, type Op } from "../core/ops";
import { plainToRuns } from "../core/text";
import { MAX_IMAGE_W } from "../layout/measure";

const SHAPES: TopicShape[] = ["rounded", "rect", "capsule", "circle", "diamond", "hexagon", "underline", "none"];
const STATUSES: TaskStatus[] = ["not-started", "in-progress", "blocked", "completed", "cancelled"];
const PRIORITIES: Priority[] = ["none", "low", "medium", "high", "urgent"];
const STRUCTURES = ["mindmap", "logic", "tree", "org", "timeline", "fishbone", "matrix", "treetable", "freeform"] as const;

export function Inspector(): JSX.Element {
  const store = useStore();
  const node = store.selectionNode;
  const [title, setTitle] = useState(node?.title ?? "");
  const [notes, setNotes] = useState(node?.notes ?? "");

  // Sync when the selection changes OR the same node's title/notes change
  // from elsewhere (canvas editing, undo/redo, Tab defaults). While the user
  // types here, node.title is untouched, so this does not fight the input.
  useEffect(() => {
    setTitle(node?.title ?? "");
    setNotes(node?.notes ?? "");
  }, [node?.id, node?.title, node?.notes]); // eslint-disable-line react-hooks/exhaustive-deps

  const snap = store.getSnapshot();
  const rel = snap.relSel ? store.sheet.relationships.find((r) => r.id === snap.relSel) : undefined;
  const grp = snap.groupSel ? store.sheet.boundaries.find((g) => g.id === snap.groupSel) : undefined;
  const sum = snap.summarySel ? store.sheet.summaries.find((s) => s.id === snap.summarySel) : undefined;

  if (!node && (rel || grp || sum)) {
    return (
      <aside className="inspector">
        {rel && <RelationshipSection rel={rel} />}
        {grp && <GroupSection grp={grp} />}
        {sum && <SummarySection sum={sum} />}
        <SheetControls />
      </aside>
    );
  }

  if (!node) {
    return (
      <aside className="inspector">
        <div className="inspector-empty">
          <p>Select a topic to edit its style, task and notes.</p>
          <p className="muted">Double-click the canvas to create a floating topic.</p>
        </div>
        <SheetControls />
      </aside>
    );
  }
}

```

```

}

return (
  <aside className="inspector">
    <div className="inspector-section">
      <div className="inspector-label">Topic</div>
      <input
        className="inspector-title"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
        onBlur={() => { if (title !== node.title) store.execOps([makeOp<Op & { type: "setTitle" }>("setTitle", { id: node.id, title, prev: node.title,
titleRuns: plainToRuns(title), prevRuns: node.titleRuns }]]); }}
        onKeyDown={(e) => { if (e.key === "Enter") (e.target as HTMLInputElement).blur(); }}
      />
      <div className="inspector-actions">
        <button className="btn small" onClick={() => store.createChild()}>+ Child</button>
        <button className="btn small" onClick={() => store.createSibling()}>+ Sibling</button>
        <button className="btn small" onClick={() => store.promote()}>↑ Promote</button>
        <button className="btn small" onClick={() => store.demote()}>↓ Demote</button>
        <button className="btn small" onClick={() => store.duplicateTopic()}>📄 Duplicate</button>
        <button className="btn small danger" onClick={() => store.deleteNodes([node.id])}>Delete</button>
        <button className="btn small" onClick={() => store.beginRelationship(node.id)}>🔗 Link...</button>
        {snap.selection.length >= 2 && (
          <>
            <button className="btn small" onClick={() => store.createGroupFromSelection()}>📁 Group</button>
            <button className="btn small" onClick={() => store.createSummaryFromSelection()}>📄 Summary</button>
          </>
        )}
      </div>
      <button className="btn small" onClick={() => store.toggleCollapsed(node.id)}>
        {node.collapsed ? "Expand" : "Collapse"}
      </button>
    </div>
    {node.type === "main" && (
      <label className="field branch-free-position">
        <span>Free positioning branch</span>
        <input
          type="checkbox"
          checked={node.position.manual}
          onChange={(e) => store.setBranchFreePosition(node.id, e.target.checked)}
        />
      </label>
    )}
  </div>

  <div className="inspector-section">
    <div className="inspector-label">Style</div>
    <div className="field">
      <span>Fill</span>
      <input type="color" value={node.style.fill ?? "#4f46e5"} onChange={(e) => store.setNodeStyle(node.id, { fill: e.target.value })} />
    </div>
    <div className="field">
      <span>Text</span>
      <input type="color" value={node.style.textColor ?? "#ffffff"} onChange={(e) => store.setNodeStyle(node.id, { textColor: e.target.value })} />
    </div>
    <div className="field">
      <span>Font size</span>
      <input type="number" min={10} max={48} value={node.style.fontSize ?? 14} onChange={(e) => store.setNodeStyle(node.id, { fontSize:
Number(e.target.value) || 14 })} />
    </div>
    <div className="field">
      <span>Node width</span>
      <input
        type="range"
        min={90}
        max={640}
        step={5}
        value={node.style.width ?? 280}
        onChange={(e) => store.setNodeStyle(node.id, { width: Number(e.target.value) })}
        title="Fixed node width – the text re-wraps and the height follows"
      />
      <span className="muted">{node.style.width ? `${node.style.width}px` : "auto"}</span>
      <button className="btn small" title="Fit the box to the text again" onClick={() => store.setNodeStyle(node.id, { width: undefined })}>
        Auto
      </button>
    </div>
    {node.style.image && (() => {
      const att = store.sheet.attachments.find((a) => a.id === node.style.image);
      if (!att) return null;
      const natural = Math.min(att.w, MAX_IMAGE_W);
      const value = node.style.imageWidth ?? natural;
      const imgH = Math.round((value * att.h) / att.w);
      return (
        <div className="field">
          <span>Image size</span>
          <input
            type="range"
            min={48}

```

```

        max={640}
        step={5}
        value={value}
        onChange={(e) => store.setImageResizeDraft(node.id, Number(e.target.value))}
        onPointerUp={() => store.commitImageResize()}
        onKeyUp={() => store.commitImageResize()}
        onBlur={() => store.commitImageResize()}
        title="Display width – the height keeps the aspect ratio; one undo undoes the whole drag"
      />
      <span className="muted">
        {value}x{imgH}px
      </span>
      <button
        className="btn small"
        title="Back to the natural display size"
        onClick={() => store.resetImageWidth(node.id)}
      >
        Natural
      </button>
    </div>
  );
})();
<div className="field">
  <span>Bold</span>
  <input type="checkbox" checked={(node.style.fontWeight ?? 400) >= 600} onChange={(e) => store.setNodeStyle(node.id, { fontWeight:
e.target.checked ? 600 : 400 })} />
</div>
<div className="field">
  <span>Shape</span>
  <select value={node.style.shape ?? "rounded"} onChange={(e) => store.setNodeStyle(node.id, { shape: e.target.value as TopicShape })}>
    {SHAPES.map((s) => <option key={s} value={s}>{s}</option>)}
  </select>
</div>
<button className="btn small" onClick={() => store.setNodeStyle(node.id, { fill: undefined, textColor: undefined })}>
  Reset to branch color
</button>
</div>

<div className="inspector-section">
  <div className="inspector-label">Task</div>
  <div className="field">
    <span>Status</span>
    <select value={node.task?.status ?? "not-started"} onChange={(e) => store.setTask(node.id, { status: e.target.value as TaskStatus })}>
      {STATUSES.map((s) => <option key={s} value={s}>{s}</option>)}
    </select>
  </div>
  <div className="field">
    <span>Priority</span>
    <select value={node.task?.priority ?? "none"} onChange={(e) => store.setTask(node.id, { priority: e.target.value as Priority })}>
      {PRIORITIES.map((p) => <option key={p} value={p}>{p}</option>)}
    </select>
  </div>
  <div className="field">
    <span>Progress</span>
    <input type="range" min={0} max={100} step={5} value={node.task?.progress ?? 0} onChange={(e) => store.setTask(node.id, { progress:
Number(e.target.value), status: Number(e.target.value) >= 100 ? "completed" : (node.task?.status ?? "not-started") })} />
    <span className="muted">{node.task?.progress ?? 0}%</span>
  </div>
  <div className="field">
    <span>Assignee</span>
    <input value={node.task?.assignee ?? ""} placeholder="-" onChange={(e) => store.setTask(node.id, { assignee: e.target.value })} />
  </div>
</div>

<div className="inspector-section">
  <div className="inspector-label">Notes</div>
  <textarea
    className="notes-input"
    placeholder="Rich notes land in Phase 4; plain text for now..."
    value={notes}
    onChange={(e) => setNotes(e.target.value)}
    onBlur={() => { if (notes !== node.notes) store.execOps([makeOp<Op & { type: "setNotes" }>("setNotes", { id: node.id, notes, prev: node.notes
}]]); }}
  />
</div>

  <SheetControls />
</aside>
);
}

function SheetControls(): JSX.Element {
  const store = useStore();
  const sheet = store.sheet;
  const [title, setTitle] = useState(sheet.title);

  useEffect(() => setTitle(sheet.title), [sheet.title]);

```

```

return (
  <div className="inspector-section">
    <div className="inspector-label">Sheet</div>
    <input
      className="inspector-title"
      value={title}
      onChange={(e) => setTitle(e.target.value)}
      onBlur={() => { if (title !== sheet.title) store.setSheetTitle(title); }}
    />
    <div className="field">
      <span>Structure</span>
      <select value={sheet.structure.structureType} onChange={(e) => store.setStructure({ structureType: e.target.value as typeof STRUCTURES[number]
    })}>
        {STRUCTURES.map((s) => <option key={s} value={s}>{s}</option>)}
      </select>
    </div>
    <div className="field">
      <span>Orientation</span>
      <select value={sheet.structure.orientation} onChange={(e) => store.setStructure({ orientation: e.target.value as "horizontal" | "vertical" })}>
        <option value="horizontal">Horizontal</option>
        <option value="vertical">Vertical</option>
      </select>
    </div>
    <div className="field">
      <span>Level spacing</span>
      <input type="range" min={80} max={400} step={10} value={sheet.structure.spacing} onChange={(e) => store.setStructure({ spacing:
Number(e.target.value) })} />
      <span className="muted">{sheet.structure.spacing}</span>
    </div>
    <div className="field">
      <span>Branch spacing</span>
      <input type="range" min={4} max={80} step={2} value={sheet.structure.branchSpacing} onChange={(e) => store.setStructure({ branchSpacing:
Number(e.target.value) })} />
      <span className="muted">{sheet.structure.branchSpacing}</span>
    </div>
    <button className="btn small" onClick={() => store.autoLayoutAll()}>⌂ Auto layout</button>
    <div className="muted count">Visible topics: {store.doc.visibleNodeCount}</div>
  </div>
);
}

// -----
// Overlay sections: relationship, group, summary
// -----

function RelationshipSection({ rel }: { rel: Relationship }): JSX.Element {
  const store = useStore();
  const [label, setLabel] = useState(rel.label ?? "");
  useEffect(() => setLabel(rel.label ?? "", [rel.id, rel.label])); // eslint-disable-line react-hooks/exhaustive-deps
  const a = store.doc.node(rel.fromId);
  const b = store.doc.node(rel.toId);
  return (
    <div className="inspector-section">
      <div className="inspector-label">Relationship</div>
      <p className="muted">{a?.title ?? "?"} → {b?.title ?? "?"}</p>
      <div className="field">
        <span>Label</span>
        <input
          value={label}
          placeholder="(none)"
          onChange={(e) => setLabel(e.target.value)}
          onBlur={() => { if (label !== (rel.label ?? "")) store.setRelationship(rel.id, { label: label.trim() || undefined }); }}
          onKeyDown={(e) => { if (e.key === "Enter") (e.target as HTMLInputElement).blur(); }}
        />
      </div>
      <label className="field">
        <span>Bidirectional arrows</span>
        <input type="checkbox" checked={!!rel.bidirectional} onChange={(e) => store.setRelationship(rel.id, { bidirectional: e.target.checked })} />
      </label>
      <div className="field">
        <span>Line style</span>
        <select value={rel.lineStyle ?? "dashed"} onChange={(e) => store.setRelationship(rel.id, { lineStyle: e.target.value as
Relationship["lineStyle"] })}>
          <option value="solid">solid</option>
          <option value="dashed">dashed</option>
          <option value="dotted">dotted</option>
        </select>
      </div>
      <button className="btn small danger" onClick={() => store.deleteSelectedRelationship()}>Delete relationship</button>
    </div>
  );
}

function GroupSection({ grp }: { grp: Group }): JSX.Element {
  const store = useStore();

```

```

return (
  <div className="inspector-section">
    <div className="inspector-label">Group</div>
    <p className="muted">{grp.memberIds.length} topics enclosed</p>
    <button className="btn small danger" onClick={() => store.deleteGroup(grp.id)}>Delete group</button>
  </div>
);
}

function SummarySection({ sum }: { sum: Summary }): JSX.Element {
  const store = useStore();
  const [label, setLabel] = useState(sum.label ?? "");
  useEffect(() => setLabel(sum.label ?? ""), [sum.id, sum.label]); // eslint-disable-line react-hooks/exhaustive-deps
  return (
    <div className="inspector-section">
      <div className="inspector-label">Summary</div>
      <p className="muted">{sum.memberIds.length} topics spanned</p>
      <div className="field">
        <span>Label</span>
        <input
          value={label}
          placeholder="Summary"
          onChange={(e) => setLabel(e.target.value)}
          onBlur={() => { if (label !== (sum.label ?? "")) store.setSummary(sum.id, { label: label.trim() || undefined }); }}
          onKeyDown={(e) => { if (e.key === "Enter") (e.target as HTMLInputElement).blur(); }}
        />
      </div>
      <button className="btn small danger" onClick={() => store.deleteSummary(sum.id)}>Delete summary</button>
    </div>
  );
}

```

## 59. src/ui/TopBar.tsx

```

import { useEffect, useRef, useState } from "react";
import { useSyncExternalStore } from "react";
import { useStore } from "../editor/context";
import { runExportPng } from "../editor/exportBridge";
import { referencedAssetIds } from "../persist/assets";
import { viewSize } from "../editor/view";
import { trace } from "../dev/trace";

export function TopBar(): JSX.Element {
  const store = useStore();
  const state = useSyncExternalStore(store.subscribe, store.getSnapshot);
  const [title, setTitle] = useState(state.docTitle);
  const [exportOpen, setExportOpen] = useState(false);
  const exportRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    setTitle(state.docTitle);
  }, [state.docTitle]);

  useEffect(() => {
    const close = (e: MouseEvent): void => {
      if (exportRef.current && !exportRef.current.contains(e.target as Node)) setExportOpen(false);
    };
    window.addEventListener("mousedown", close);
    return () => window.removeEventListener("mousedown", close);
  }, []);

  return (
    <header className="topbar">
      <input
        className="doc-title-input"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
        onBlur={() => store.renameDocument(state.activeDocId, title || "Untitled map")}
        onKeyDown={(e) => {
          if (e.key === "Enter") (e.target as HTMLInputElement).blur();
        }}
      />

      <div className="topbar-group">
        <button className="btn icon" title="Undo (Ctrl+Z)" disabled={!state.canUndo} onClick={() => store.undo()}>
          ~
        </button>
        <button className="btn icon" title="Redo (Ctrl+Shift+Z)" disabled={!state.canRedo} onClick={() => store.redo()}>
          ~
        </button>
        <button className="btn primary" title="Save (Ctrl+S)" onClick={() => void store.saveNow()}>
          Save
        </button>
        <button className="btn" title="Open a .rnode.json file (Ctrl+O)" onClick={() => void store.loadFile()}>

```



## 60. package.json

```
{
  "name": "r-node",
  "private": true,
  "version": "0.1.0",
  "description": "R-node – a cross-platform visual thinking and mind-mapping workspace",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc --noEmit && vite build",
    "preview": "vite preview",
    "test": "vitest run",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "@lexical/html": "^0.49.0",
    "@lexical/list": "^0.49.0",
    "@lexical/react": "^0.49.0",
    "@lexical/rich-text": "^0.49.0",
    "@lexical/utils": "^0.49.0",
    "@tauri-apps/plugin-http": "^2.5.9",
    "fflate": "^0.8.3",
    "lexical": "^0.49.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@types/react": "^18.3.12",
    "@types/react-dom": "^18.3.1",
    "@vitejs/plugin-react": "^4.3.4",
    "fake-indexeddb": "^6.2.5",
    "jsdom": "^29.1.1",
    "typescript": "^5.6.3",
    "vite": "^6.0.7",
    "vitest": "^2.1.8"
  }
}
```

## 61. docs/ADR-001-immagini-e-piattaforma.md

```
# ADR-001 – Immagini nei nodi: storage, memoria di decodifica, piattaforma

**Stato:** DECISO – A1 + B1. Vedi §12.
**Data:** 2026-08-12
**Deciso da:** il proprietario del progetto
**Branch di lavoro:** `node-images`

> Le sezioni §5-§10 restano com'erano: documentano il ragionamento, non la
> conclusione. Chi rilegge per contestare la decisione parta da §12, che
> contiene anche *la condizione che la ribalterebbe*.

---

## 0. Come usare questo documento

Questo file è scritto per essere dato in pasto a un modello che non conosce
il progetto. Contiene quindi tutti i fatti necessari, e distingue
esplicitamente ciò che è stato misurato da ciò che è stimato.

**Cosa si chiede al lettore.** Non un riassunto delle opzioni: quelle sono già
enumerate in §5. Si chiede di rispondere alle domande aperte di §9, di
attaccare il ragionamento di §7 se è debole, e di segnalare opzioni mancanti.

**Cosa NON serve. In §10 c'è un elenco di proposte già valutate e scartate,
con il motivo. Riproporle non aiuta. In particolare: questa codebase ha già
ricevuto analisi automatiche che proponevano cose già implementate (una
cache del text measuring che esiste da mesi, transazioni atomiche nella history
che sono atomiche dal primo commit). Il codice va letto prima di proporre.

**Convenzione sulle affermazioni:

| Marca | Significato |
| --- | --- |
| [M] | misurato in questa codebase, con il numero riportato |
| [V] | verificato leggendo il codice |
| [S] | stima, con il ragionamento esplicitato |
| [?] | incerto, da verificare |

---

## 1. Il prodotto, in due righe
```

R-node è uno spazio di lavoro per mappe mentali: nodi collegati ad albero, disegnati su **un solo <canvas>** per reggere migliaia di elementi. Il testo dentro ogni nodo è **rich text** (grassetto, corsivo, colori, heading, liste).

Ora si vuole aggiungere le **immagini nei nodi**, in stile XMind: si trascina un file su un nodo, appare sopra il testo, si ridimensiona con uno slider.

Requisito dichiarato dal proprietario: **varie centinaia di immagini per mappa**, con gli **originali conservati a piena risoluzione**, e la possibilità di **mandare una mappa come file singolo**.

---

## ## 2. Lo stato attuale, in fatti

**Stack** [V]: TypeScript, React 18, Vite 6, Lexical 0.49, vitest.  
Nessuna dipendenza grafica: il rendering è Canvas2D scritto a mano.

**Struttura** [V] – il motore è framework-free, solo `ui/` usa React:

---

```
src/core/      schema, modello documento, sistema di operazioni, history
src/layout/    misura del testo e posizionamento (funzioni pure)
src/render/    renderer Canvas2D, viewport, temi, hit-test, export PNG
src/editor/    store (orchestrazione), scorciatoie
src/persist/   adapter di storage (localStorage oggi)
src/ui/        React: canvas, sidebar, topbar, outliner, inspector, palette
src-tauri/     shell desktop Tauri 2 – presente ma MAI COMPILATA
dev/           harness di parità testo (solo sviluppo)
---
```

**Test** [M]: 101 test in 8 suite, typecheck pulito.

**Performance** [M], dalla suite `tests/perf.test.ts` a 10.000 nodi:  
operazioni ~0.005 ms/op, layout ~150 ms, writeback ~135 ms.

**Rendering del testo** [V]: ogni nodo ha il titolo come `TextRun[]` (sequenza di run stilizzati). Il canvas lo disegna con una **cache bitmap per nodo**; durante pan e zoom si fa solo `drawImage`.

**Editing** [V]: quando si edita **un nodo**, si monta **un solo** overlay HTML (Lexical) sopra di esso, e il canvas smette di disegnare quel nodo. Mai più di un overlay.

**Parità editor+canvas** [M]: il canvas riproduce a mano il layout testuale del browser, e la coincidenza è **misurata**, non asserita – un harness confronta le line box reali del DOM con l'output della misura del canvas su 16 casi. Stato attuale: **16 su 16 allineati**, scarto massimo 0.5px. Ci è voluta una sessione di lavoro dedicata e ha prodotto la correzione di ~14 divergenze.

**Persistenza** [V]: `LocalStorageAdapter` scrive `JSON.stringify(docs)` in `localStorage`. Il salvataggio è **manuale** (Ctrl+S), non c'è autosave.  
`src-tauri/src/lib.rs` espone già `list_documents`, `load_document`, `save_document`, `delete_document` su SQLite – **mai compilati**, `rustc` e `cargo` non sono installati sulla macchina [M].

**Seam già previsti** [V]:

- `Style.image?: string` // attachment id esiste nello schema dal primo commit
- `Sheet.attachments: unknown[]` esiste, non tipizzato
- il renderer è **isolato** dietro una classe con hit-test ed export, e `docs/ARCHITECTURE.md` dichiara **WebGPU (preferred) / WebGL2 fallback** come percorso di aggrornamento previsto
- `StorageAdapter` è un'interfaccia con più implementazioni previste

---

## ## 3. Il problema, con i numeri

Sono due problemi distinti che è facile confondere.

### ### 3.1 Storage – dove stanno i byte

`localStorage` regge **~5MB** per origine e una mappa da 10k nodi ne occupa già ~3.6 [M, dal README del progetto]. Il base64 gonfia i byte del **33%**.

Centinaia di immagini, anche solo a 500KB l'una, fanno **centinaia di MB**: non è che va stretto, non ci sta di **due ordini di grandezza**. In più un documento da decine di MB blocca il thread principale a ogni `JSON.parse`.

Conclusione già presa: **le immagini escono dal documento**, in un archivio separato indirizzato per contenuto (SHA-256 → deduplicazione gratuita).

### ### 3.2 Memoria di decodifica – quanti pixel stanno in RAM

Questo è il problema **non risolto** e il motivo di questo documento.

Un'immagine compressa è piccola; decodificata è RGBA grezzo:  
`larghezza × altezza × 4 byte`.

```
| Bitmap | Memoria decodificata |  
|---|---|  
| 256×192 | 0,19 MB |  
| 512×384 | 0,75 MB |  
| 1024×768 | **3,0 MB** |  
| 2048×1536 | 12 MB |  
| 4000×3000 (12 megapixel) | **46 MB** |
```

Quindi: **\*\*dieci originali da 12MP decodificati sono ~460MB\*\***. E 100 bitmap da 1024px sono 300MB. Con "varie centinaia di immagini" si va fuori memoria banalmente.

Aggravante specifica di questa architettura: essendo tutto **\*\*un solo canvas\*\***, non esiste il lazy-loading gratuito che daresti per scontato con `` nel DOM. Il renderer deve gestire esplicitamente cosa decodificare, quando, e cosa liberare.

**\*\*Trappola tecnica\*\*** [V, comportamento noto della piattaforma]: la memoria di un `ImageBitmap` **\*\*non sta nello heap JavaScript\*\***. Non appare negli heap snapshot e non viene liberata dalla pressione del GC in modo prevedibile. Serve `bitmap.close()` esplicito. Con `new Image()` quel controllo non esiste.

### ### 3.3 Rischio di perdita dati

[V, comportamento della piattaforma] I browser **\*\*possono cancellare IndexedDB\*\***. Ma il rischio **\*\*non è uniforme\*\***: dipende da come l'app viene distribuita, e va valutato per canale invece che in astratto.

```
| Canale | Rischio | Perché |  
|---|---|---|  
| Chromium, `persist()` concesso o installata come PWA | **basso** | l'origine diventa persistente e non viene sfrattata automaticamente sotto pressione di spazio |  
| Chromium, sito normale senza `persist()` | **medio** | storage "best-effort", sfrattabile quando lo spazio scarseggia |  
| Safari, sito non installato | **alto** | l'ITP cancella lo storage scrivibile da script dopo ~7 giorni senza interazione, e `persist()` non protegge |  
| Firefox | **medio** | `persist()` richiede un permesso esplicito all'utente |  
| Desktop (Tauri) | **nullo** | file veri, nessuna politica di eviction |
```

`navigator.storage.persist()` resta una **\*richiesta\***, non una garanzia: su Chromium viene concessa in base a euristiche (installazione, engagement), non su domanda.

Conseguenza per la decisione: **\*\*questa casella non si può riempire senza sapere il canale di distribuzione\*\***. Se R-node vive come app installata o desktop, l'argomento perde peso; se deve funzionare come sito aperto in Safari, diventa il rischio principale – centinaia di originali insostituibili cancellati dopo una settimana di inattività.

Nota su un numero ricorrente: la quota **"~60% dello spazio disco"** è **\*\*specifica di Chrome\*\***. Firefox usa criteri diversi e Safari si attesta intorno a ~1GB prima di chiedere conferma. Ogni conclusione del tipo "sul web la quota basta" vale su Chromium e va riverificata altrove.

---

### ## 4. Vincoli

**\*\*Duri\*\*** (non negoziabili senza cambiare il prodotto):

1. **\*\*Un solo canvas.\*\*** Niente elemento DOM per nodo: è la ragione per cui la mappa regge migliaia di nodi.
2. **\*\*Il rich text nei nodi è il differenziatore del prodotto.\*\*** Ci è stato investito il grosso del lavoro finora, inclusa la parità misurata.
3. Undo/redo su ogni modifica, sistema di operazioni con inverso.
4. Local-first: nessuna dipendenza da un server.

**\*\*Morbidi\*\*** (preferenze, cedibili con una motivazione):

5. Il web resta un target – non necessariamente il principale.
6. Il core resta framework-free e testabile senza DOM.
7. Un solo linguaggio nel motore, per non moltiplicare la superficie di test.

**\*\*Del contesto, che pesano sulla decisione:\*\***

8. Sviluppo assistito da AI, con modelli di capacità diversa. Le scelte devono essere **\*\*verificabili\*\***: l'harness di parità esiste per questo.
9. Nessun team: costo di manutenzione a lungo termine su una persona.

---

### ## 5. Le opzioni

### Famiglia A – restare nel browser

#### A1. Status quo: Canvas2D + IndexedDB + varianti pre-scalate

L'archivio asset in IndexedDB conserva due varianti per immagine: l'originale e una `display` a 1024px. Il canvas decodifica solo la `display`, con `createImageBitmap`, per i soli nodi visibili, in una cache LRU a budget di byte, liberando con `close`.

- **Risolve**: la quota (IndexedDB regge centinaia di MB), la memoria (con il budget), la deduplicazione (hash).
- **Non risolve**: il rischio di eviction di §3.3; ogni cambio di zoom richiede una ri-decodifica al nuovo bucket di risoluzione.
- **Costo** [S]: basso. Nessun cambio di stack, il codice esistente resta.
- **Rischio**: il budget di memoria va calibrato a mano; sbagliarlo si vede solo sotto carico reale.

#### #### A2. A1 + renderer WebGPU

Il renderer passa da Canvas2D a WebGPU. Le immagini diventano texture con **mipmap**: si caricano una volta e la GPU sceglie il livello per lo zoom. Possibilità di **texture compresse** in VRAM (BC7): ~1MB invece di 3 per una 1024×768.

- **Risolve**: elimina i bucket di risoluzione e le ri-decodifiche; ~3× più immagini nella stessa memoria; anche il disegno dei nodi diventa più veloce.
- **Non risolve**: eviction di IndexedDB.
- **Costo** [S]: **alto**. Va riscritto tutto il disegno – forme, connettori, testo. Il testo su GPU richiede un **atlante di glifi** o il riuso delle bitmap attuali come texture. Settimane, non giorni.
- **Nota favorevole** [V]: il renderer è già isolato dietro una classe con hit-test ed export, e l'architettura dichiara WebGPU come percorso previsto. L'isolamento è già stato pagato.
- **Rischio**: WebGPU su Safari è arrivato tardi; serve un fallback WebGL2 o Canvas2D, quindi **due renderer da mantenere e da testare**.

#### #### A3. A1 + decodifica in Web Worker con OffscreenCanvas

Sposta decodifica e ridimensionamento fuori dal thread principale.

- **Risolve**: lo scatto durante l'import e durante la panoramica.
- **Non risolve**: la memoria totale (le bitmap pesano uguale).
- **Costo** [S]: medio-basso, incrementale su A1.
- **Nota**: `createImageBitmap` già decodifica fuori dal main thread; il guadagno aggiuntivo è sul ridimensionamento e sull'hashing.

#### ### Famiglia B – ibrido: shell nativa + interfaccia in webview

Tutte usano **Tauri 2**, che su Windows incorpora WebView2 (Chromium). `src-tauri/` esiste già [V].

> **Punto cruciale, spesso frainteso**: **Tauri** **usa comunque una webview**. Il > canvas, `createImageBitmap` e la memoria delle bitmap decodificate sono > identici al browser. Andare "in locale" sposta **dove stanno i byte**, non > **quanti pixel stanno in RAM**. Tutto §3.2 resta valido invariato.

#### #### B1. Tauri + asset su filesystem/SQLite

L'archivio asset diventa file veri o BLOB in SQLite.

- **Risolve**: quota (praticamente illimitata), **rischio di eviction (§3.3)**, export a file singolo più semplice.
- **Non risolve**: memoria di decodifica.
- **Costo** [S]: basso-medio. `rustup` da installare, primo build lungo, i comandi Rust per gli asset da aggiungere (i 4 per i documenti esistono già).
- **Rischio**: due target da mantenere se il web resta vivo; catena di build più lunga; testing del layer Tauri più scomodo.

#### #### B2. B1 + pipeline immagini lato Rust

All'import, Rust legge il file, calcola l'hash e genera i livelli ridotti su thread veri, in parallelo. La webview riceve immagini già della dimensione giusta e non ridimensiona mai nulla.

- **Risolve**: costo di import e di ridimensionamento tolto dal thread dell'interfaccia; parallelismo reale.
- **Non risolve**: la memoria delle bitmap a schermo.
- **Costo** [S]: medio. Una crate immagini in Rust (`image`), più i comandi.
- **Nota**: si combina bene con A2 – Rust può produrre direttamente texture compresse pronte per la GPU.

#### #### B3. B1 + B2 + renderer WebGPU (A2)

La combinazione massima senza uscire dalla webview.

- **Costo** [S]: somma dei precedenti. Alto.
- **Copre** [S]: ~80% del vantaggio di una soluzione nativa, mantenendo Lexical e quindi tutto l'editing di testo.

#### ### Famiglia C – nativo, niente webview

#### C1. GUI Rust (egui / iced / Slint / GPUI)

#### C2. Flutter (motore Impeller/Skia proprio)

#### C3. Skia o wgpu diretti, con GUI custom

Tutte e tre condividono benefici e costi, quindi si valutano insieme.

- **\*\*Risolvo\*\***: controllo totale su texture, mipmap e sfratto; texture compresse; niente webview né suo consumo di memoria; avvio più rapido; binario più piccolo. Sul problema §3.2 sono **\*\*tecnicamente superiori\*\***.  
**#### Il costo del rich text in nativo – versione corretta**

> Una prima stesura di questo ADR diceva «bisogna riscrivere un editor di rich text da zero, sono anni-persona». **\*\*È stato contestato ed era troppo assoluto\*\***: nessuno scriverebbe da zero. Qui la versione rivista.

Il problema si divide in cinque strati, e le librerie esistenti ne coprono quantità molto diverse:

| Strato | Stato dell'arte |  
|---|---|  
| 1. Shaping (glifi, kerning, legature, scritture complesse) | **\*\*risolto\*\***: HarfBuzz, rustybuzz, swash |  
| 2. Line breaking Unicode (UAX #14) e bidi (UAX #9) | **\*\*risolto\*\***: crate dedicate, ICU |  
| 3. Layout con run di stile misto, baseline condivisa | **\*\*buona copertura\*\*** |  
| 4. Modello di editing: caret per grafemi, selezione, word boundaries | **\*\*parziale\*\*** |  
| 5. Integrazione IME di sistema (TSF, NTextInputClient, ibus/fcitx) | **\*\*il punto dolente\*\*** |

**\*\*Flutter\*\*** [?, da riverificare: l'ecosistema si muove in fretta]: copre tutti e cinque **\*\*di serie\*\***. `EditableText` con alberi di `TextSpan`, e integrazione IME su tutte le piattaforme, perché Flutter porta il proprio motore e l'IME è una sua competenza centrale. Sopra esistono componenti maturi – **\*\*super\_editor\*\***, **\*\*AppFlowy Editor\*\***, **\*\*flutter\_quill\*\***, **\*\*fleather\*\*** – e AppFlowy è un prodotto reale che lo dimostra.

**\*\*Rust\*\*** [?]: `cosmic-text` e `parley` coprono bene 1-3 e in parte il 4, con attributi per run. Lo strato 5 dipende dalla crate di windowing e la qualità varia. Terreno in movimento rapido: verificare le versioni prima di decidere.

**\*\*Qt\*\***: `QTextDocument` è un motore di rich text maturo con IME funzionante.

**\*\*Un vantaggio di C che le prime stesure non riportavano\*\*** In nativo si usa **\*\*lo stesso motore di layout\*\*** per l'editing e per il disegno statico. Quindi **\*\*il problema della parità non esiste\*\***: non c'è nessun canvas che debba imitare nessun browser. L'intera sessione di lavoro che ha portato la parità a 16/16 non sarebbe mai stata necessaria. Va detto, perché è un argomento concreto a favore di C.

**\*\*Cosa resta contro C, dopo la correzione\*\***

1. **\*\*E una riscrittura totale, non incrementale\*\***: core, ops, history, layout, 101 test, tutta l'interfaccia React. Non esiste un percorso a tappe.
2. Flutter significa Dart e, di fatto, abbandonare il web.
3. In Rust lo strato 5 resta il rischio vero – quello che si scopre tardi, quando un utente giapponese segnala che non riesce a scrivere.
4. **\*\*vincolo specifico di questo prodotto\*\***: l'editor vive **\*\*dentro un canvas con zoom e pan arbitrari\*\***. Non è un campo di testo in un form, è un box trasformato. Quanto bene sopravviva il posizionamento dell'IME – la finestra dei candidati deve comparire al caret, trasformato – sotto una scala arbitraria è **\*\*la domanda da porre per prima\*\***, e non ha risposta certa per nessuna delle opzioni. Vedi §9, domanda 7.

- **\*\*Verdetto rivisto\*\*** [S]: non «proibitivo perché il testo», ma **\*\*costoso perché è una riscrittura totale\*\***. Il testo è il rischio **\*\*minore\*\*** con Flutter, il **\*\*maggiore\*\*** con Rust. Il criterio di §8 resta valido ma va letto con questa correzione.

---

## ## 6. Confronto

Legenda:  risolve ·  neutro ·  non risolve o peggiora.

|               |  | Quota                |  | Eviction §3.3                                                                       |  | Memoria decodifica                                                                  |  | Fluidità import                                                                     |  | Costo                                                                               |  | Rischio manutenzione                                                                |  |                                                                                     |  |                      |  |                              |  |
|---------------|--|----------------------|--|-------------------------------------------------------------------------------------|--|-------------------------------------------------------------------------------------|--|-------------------------------------------------------------------------------------|--|-------------------------------------------------------------------------------------|--|-------------------------------------------------------------------------------------|--|-------------------------------------------------------------------------------------|--|----------------------|--|------------------------------|--|
|               |  | ---                  |  | ---                                                                                 |  | ---                                                                                 |  | ---                                                                                 |  | ---                                                                                 |  | ---                                                                                 |  |                                                                                     |  |                      |  |                              |  |
| <b>**A1**</b> |  | Canvas2D + IndexedDB |  |  |  | (Chrome)                                                                            |  | dipende dal canale                                                                  |  |  |  | (a mano)                                                                            |  |  |  | Basso                |  | Basso                        |  |
| <b>**A2**</b> |  | + WebGPU             |  |  |  | (Chrome)                                                                            |  | dipende dal canale                                                                  |  |  |  | (mipmap)                                                                            |  |  |  | Alto                 |  | <b>**Alto**</b> (2 renderer) |  |
| <b>**A3**</b> |  | + Worker             |  |  |  | (Chrome)                                                                            |  | dipende dal canale                                                                  |  |  |  | <b>**come A1**</b>                                                                  |  |  |  | Basso-medio          |  | Basso                        |  |
| <b>**B1**</b> |  | Tauri + fs           |  |  |  |  |  |  |  | eredita dalla webview                                                               |  |  |  | Basso-medio                                                                         |  | Medio (2 target)     |  |                              |  |
| <b>**B2**</b> |  | + pipeline Rust      |  |  |  |  |  |  |  | eredita dalla webview                                                               |  |  |  |  |  | Medio                |  | Medio                        |  |
| <b>**B3**</b> |  | + WebGPU             |  |  |  |  |  |  |  |  |  |  |  | Alto                                                                                |  | Alto                 |  |                              |  |
| <b>**C**</b>  |  | Nativo               |  |  |  |  |  |  |  |  |  |  |  | <b>**Altissimo**</b>                                                                |  | <b>**Altissimo**</b> |  |                              |  |
| <b>**D**</b>  |  | Composizione OS      |  |  |  |  |  |  |  |  |  |  |  | <b>**Altissimo**</b>                                                                |  | <b>**Altissimo**</b> |  | <b>**Ingestibile**</b>       |  |

Tre avvertenze su come si legge questa tabella, perché da sola induce in errore.

**\*\*Il Worker non riduce la memoria.\*\*** A3 sposta *\*dove\** avviene la decodifica, non *\*quanto\** pesa la bitmap: una 1024x768 occupa 3MB nel worker esattamente come nel thread principale. Su quell'asse A3 è **\*\*uguale\*\*** ad A1, non migliore.

**\*\*Tauri non peggiora né migliora la memoria: la eredita.\*\*** B1 e B2 valgono quanto vale la strategia lato web che ci gira dentro.

**\*\*L'opzione D fallisce sull'interazione, non sull'import.\*\*** Comporre una finestra nativa sopra una webview significa due loop di rendering da sincronizzare a ogni scroll e zoom, più z-order, click-through, DPI misti fra monitor e tearing del compositore. L'import in Rust andrebbe benissimo: è l'interazione a rompersi.

> 🚩 **\*\*Questa tabella nasconde la variabile che decide.\*\*** Su queste sei  
> colonne C e B3 vincono quasi ovunque, perché la colonna che escluderebbe C  
> non esiste: **\*\*quanto editing di testo bisogna riscrivere\*\***. Shaping, line  
> breaking Unicode, bidi, caret, selezione, IME. In webview lo dà il browser;  
> in nativo sono anni-persona, compressi qui dentro la parola "Altissimo".  
> Chi ottimizza la tabella sceglie il nativo. **\*\*Leggere §8 prima di**  
> **concludere.\*\***

---

## ## 7. Il ragionamento attuale (da attaccare)

- \*\*§3.1 e §3.2 sono problemi diversi e vanno risolti separatamente.\*\***  
Confonderli porta a credere che "andare in locale" risolva la memoria.  
Non la risolve: Tauri è una webview.
- \*\*La memoria di decodifica si risolve nel renderer, non nello storage.\*\***  
Le leve sono: decodificare alla dimensione che serve (``createImageBitmap`` con ``resizeWidth``), budget in **\*\*byte\*\*** e non in numero di bitmap, sfratto LRU con ``close()``, decodifica solo dei nodi visibili, coda limitata.
- \*\*Il sistema si auto-bilancia\*\*** [S]: zoom alto e numero di immagini visibili sono inversamente proporzionali. A zoom 0,4 si vedono 300 immagini ma basta il livello più piccolo (~57MB totali); a zoom 3 servono 1024px ma il culling ne lascia a schermo una decina (~24MB). Non esiste uno stato che richieda **\*\*molte\*\*** immagini **\*\*e\*\*** ad alta risoluzione insieme.
- \*\*Lo storage e il rischio di perdita dati si risolvono andando desktop.\*\***  
È l'argomento migliore per B1, più della quota.
- \*\*Il rich text vincola la famiglia C fuori dal tavolo\*\***, finché resta il differenziatore del prodotto.
- \*\*Quindi\*\***: A1 subito (funziona ovunque), B1 appena ``rustup`` è installato (l'interfaccia ``AssetStore`` rende il passaggio indolore), A2/B3 solo se la fluidità con centinaia di immagini si rivela insufficiente **\*\*dopo averla misurata\*\***.

---

## ## 8. Il criterio che decide tutto

> **\*\*Il rich text dentro i nodi è \*il\* prodotto, o è un accessorio?\***

- Se è il prodotto: **\*\*restare in webview\*\***. Andare nativo significa buttare il differenziatore e ricostruirlo peggio.
- Se R-node diventasse un canvas di diagrammi ad altissime prestazioni con etichette semplici, allora la famiglia C diventa razionale e ``wgpu`` o Slint sono candidati seri.

Allo stato attuale il progetto è nettamente nel primo caso.

---

## ## 9. Domande aperte – è qui che serve il secondo parere

- \*\*Il budget di memoria di §7.2 è calibrato bene?\*** Un tetto di 128MB di bitmap decodificate è ragionevole per un'app desktop-like nel 2026, o è troppo aggressivo/conservativo? Esistono numeri pubblicati su cui basarsi?
- \*\*L'auto-bilanciamento di §7.3 regge davvero?\*** Esiste un pattern d'uso plausibile che richieda molte immagini ad alta risoluzione insieme – per esempio una mappa "moodboard" con 200 immagini grandi tutte visibili a zoom 1? In quel caso l'argomento cade e serve A2.
- \*\*Il costo di A2 (WebGPU) è stato sovrastimato?\*** Il testo è già renderizzato in bitmap cache per nodo: riusarle come texture invece di costruire un atlante di glifi renderebbe il porting molto più economico di quanto stimato. Qualcuno ha esperienza di questo percorso specifico?
- \*\*Esiste una quarta famiglia che non ho considerato?\*** Per esempio: webview per l'editing di testo + finestra nativa per il canvas, composite a livello di OS. È una via reale o una complicazione senza ritorno?

5. **\*\*Sul rischio eviction (§3.3)\*\*:** quanto è concreto nella pratica per un'app che l'utente usa quotidianamente e che chiama `persist()`? Ci sono dati su quanto spesso Chrome effettivamente cancella origini persistenti?

6. **\*\*Su C\*\*:** esiste oggi una libreria di rich text editing nativa, matura e multiplatforma che renda la famiglia C meno proibitiva? – **\*\*parzialmente risposta\*\***, vedi §5/C: sì per Flutter (`super_editor`, `AppFlowy Editor`), parzialmente per Rust (`cosmic-text`, `parley`, con lo strato IME incerto). La domanda residua è **\*\*quanto sono maturi oggi\*\***, perché quell'ecosistema si muove in fretta e le informazioni invecchiano in mesi.

7. **\*\*La domanda che vale più di tutte, e che resta aperta\*\*:** in R-node l'editor di testo vive **\*\*dentro un canvas con zoom e pan arbitrari\*\***. Il caret, la selezione e soprattutto la **\*\*finestra dei candidati IME\*\*** devono posizionarsi correttamente sotto una trasformazione di scala qualsiasi. Nessuna delle librerie citate – né Flutter, né `cosmic-text`, né Qt – nasce per questo caso: nascono per campi di testo in un layout. Qualcuno ha esperienza diretta di editing di testo con IME dentro una vista trasformata? È il punto su cui la famiglia C può rompersi in modo non recuperabile, e non è visibile finché non lo provi con una tastiera giapponese.

---

**## 10. Già valutato e scartato – non riproporre**

| Proposta | Perché no |  
|---|---|  
| Immagini come data URL nel documento | `localStorage` regge ~5MB, il documento ne farebbe centinaia. Bloccherebbe anche il `JSON.parse` all'apertura. |  
| Un elemento `<img>` per nodo, sopra il canvas | Viola il vincolo duro n.1: l'architettura a canvas singolo esiste per reggere migliaia di nodi. |  
| Decodificare l'originale e scarlarlo a schermo | Un originale da 12MP occupa 46MB decodificato per disegnarne 240 unità world. È esattamente il problema. |  
| Aggiungere una cache al text measuring | **\*\*Già esiste\*\*** in `src/layout/measure.ts`, con chiave `weight\|italic\|size\|family\|text`. |  
| Rendere atomiche le transazioni della history | **\*\*Già atomiche\*\***: un `execOps` = una `HistoryEntry`. |  
| Risolvere la race fra `layoutTimer` e `saveTimer` | **\*\*saveTimer non esiste\*\*: il salvataggio è manuale. |  
| Il canvas somma le larghezze dei token e accumula errore di wrap | **\*\*Ipotesi testata e smentita\*\*** con un caso dedicato nell'harness: 9/9 righe, altezze identiche. |  
| SVG fra i formati immagine accettati | Documento eseguibile; l'hardening XSS è esplicitamente differito. |  
| Monaco Editor al posto di Lexical | **\*\*Non** è un'alternativa né una soluzione locale. **\*\*È** una libreria web (gira comunque in webview) ed è un editor di \*codice\*: il suo modello è testo semplice, la colorazione è decorazione della vista calcolata da un tokenizer, quindi non può rappresentare formattazione autoriale come `TextRun[]`. In più è orientato a righe e monospazio, e avendo un proprio motore di layout invaliderebbe i 16 casi di parità. Avrebbe senso solo per una vista "sorgente" del documento (JSON/outline), che è una feature diversa. |  
| Sostituire Lexical con `ProseMirror` / `TipTap` / `Slate` / `Quill` | Sono alternative **\*legittime\*** (a differenza di Monaco), ma cambiarle ora invaliderebbe la parità misurata e Lexical funziona. Da riaprire solo davanti a un problema concreto che Lexical non risolve, non per preferenza. |**

---

**## 11. Riferimenti nel repository**

| File | Contenuto |  
|---|---|  
| `docs/ARCHITECTURE.md` | struttura, schema, piano per fasi, decisioni aperte |  
| `docs/AGENT_GUIDE.md` | invarianti, contratto di parità, trappole già pagate |  
| `docs/ROADMAP.md` | T12a-T15 specificano la feature immagini sull'ipotesi A1+B1 |  
| `docs/RICH_TEXT_EDITOR.md` | come funziona il rich text, e perché la parità è difficile |  
| `dev/parity.ts` | l'harness che misura la coincidenza editor\*canvas |  
| `src/render/renderer.ts` | il renderer Canvas2D, isolato dietro una classe |  
| `src/layout/measure.ts` | misura condivisa e costanti |  
| `src-tauri/src/lib.rs` | i 4 comandi SQLite già scritti, mai compilati |

---

**## 12. Decisione**

**\*\*A1 + B1\*\*:** Canvas2D con decodifica a livelli e budget in byte, su shell Tauri con gli asset su filesystem. **\*\*Non\*\*** WebGPU, **\*\*non\*\*** nativo.

**### Il vincolo che ha deciso**

Il proprietario ha dichiarato un requisito e un compromesso accettato:

> Migliaia di nodi e centinaia di immagini ad alta risoluzione. Zoomando dentro  
> l'immagine può degradarsi e diventare pixelata; zoomando fuori deve restare  
> riconoscibile.

**\*\*Quel compromesso elimina l'argomento principale a favore di A2/B3 e di C.\*\***  
Mipmap in VRAM e texture compresse servono a mantenere la qualità a ogni livello di zoom – cioè esattamente ciò a cui si è rinunciato. Pagare settimane di riscrittura del renderer per una qualità dichiarata non necessaria è sovra-ingegnerizzazione.

**### Il calcolo che lo sostiene**

Memoria di decodifica = `larghezza × altezza × 4`. Con decodifica al livello corrispondente allo zoom corrente, e solo per i nodi visibili:

| Regime | Immagini visibili | Dimensione a schermo | Livello | RAM |

```
|---|---|---|---|---|
| Vista d'insieme | 300 | ~60px | 128px | 14 MB |
| Zoom medio | 60 | ~200px | 256px | 11 MB |
| Zoom ravvicinato | 12 | ~500px | 512px | 9 MB |
| Zoom massimo | 3 | ~1000px | 1024px | 9 MB |
```

Il totale resta fra i 9 e i 14 MB **\*\*in ogni regime\*\***, perché zoom e numero di immagini visibili sono inversamente proporzionali e il compromesso accettato mette un **\*\*tetto duro\*\*** al livello massimo. Con un budget di 128MB il margine è di dieci volte: il problema che aveva motivato questo documento non si presenta.

### ### Configurazione

- **\*\*Su disco\*\*** (nessuna quota, via Tauri): **\*\*originale intatto\*\*** + due livelli derivati all'import, **\*\*256px\*\*** e **\*\*1024px\*\***.
- **\*\*In memoria\*\***: `createImageBitmap(blob, { resizeWidth })`` dal livello più vicino alla dimensione a schermo, **\*\*solo per nodi visibili\*\***, cache LRU con tetto in byte e `bitmap.close()` allo sfratto.
- **\*\*Tetto duro a 1024px\*\***: la pixelatura accettata diventa una regola, non un caso limite.
- **\*\*Facoltativo\*\***: se una singola immagine occupa quasi tutto il viewport, decodificarla dall'originale – una sola alla volta.

### ### Perché B1 e non il solo browser

1. Nessuna quota e **\*\*nessun rischio di eviction\*\*** (§3.3): centinaia di originali insostituibili non possono dipendere da IndexedDB, che Safari cancella dopo ~7 giorni di inattività e Chrome può sfrattare.
2. **\*\*Si batte XMind su avvio e RAM senza toccare il rendering\*\***: XMind è Electron con Chromium impacchettato, Tauri usa la webview di sistema – binario di pochi MB contro centinaia, footprint tipico 40-80MB contro 150-300.
3. La pipeline immagini può migrare a Rust (B2) in modo incrementale se l'import risulta lento.

### ### Ordine di esecuzione

1. **\*\*T0\*\*** – bring-up di Tauri (bloccato su `rustup`, vedi ROADMAP)
2. **\*\*T1a\*\*** – `AssetStore` (l'interfaccia e l'implementazione IndexedDB non dipendono da Tauri: si possono scrivere subito)
3. **\*\*T12 → T13 → T14\*\*** – modello e misura, ingestione, ridimensionamento
4. **\*\*T15\*\*** – `rnode.zip` per la condivisione
5. **\*\*Misura contro XMind\*\*** con il tracer, e solo allora si rivaluta

### ### Verificato sul campo (2026-08-12)

Le cifre della tabella qui sopra erano **\*\*stime\*\***. Ora sono misurate: 344 nodi, 300 immagini da 1600x1200, con i contatori del tracer.

- Mappa normale al fit: 22 immagini visibili → **\*\*5,6 MB\*\***, 2,5 ms/frame.
- **\*\*Caso moodboard\*\*** (i 300 nodi impacchettati in griglia, cioè la condizione di riapertura qui sotto): picco **\*\*17,4 MB\*\*** con 306 bitmap in cache, e 1,6-1,9 ms/frame a regime.

Contro un budget di 128 MB il margine è **\*\*7x\*\***, e nel caso peggiore previsto. Media ~57 KB per bitmap. **\*\*La condizione di riapertura non scatta.\*\***

Con un'avvertenza: la misura è a `devicePixelRatio = 1`. Su retina i byte crescono di circa 4x (picco stimato ~70 MB, margine 1,8x): resta sotto il budget, ma va rimisurato. Dettaglio in `[archive/PIANO-IMMAGINI.md](archive/PIANO-IMMAGINI.md)` §5.

### ### Cosa ribalterebbe questa decisione

Una condizione sola, ed è verificabile: un caso d'uso **\*\*moodboard\*\*** – ~200 immagini grandi tutte visibili contemporaneamente a zoom 1, ognuna oltre i 400px a schermo. Lì i livelli non aiutano, perché servono davvero molte immagini ad alta risoluzione insieme, e l'auto-bilanciamento di §7.3 cade.

Se quel caso entra nei requisiti, **\*\*questa decisione va riaperta\*\*** e A2/B3 tornano sul tavolo.

## 62. src/persist/assets.ts

```
/**
 * Asset store – image blobs living OUTSIDE the document (ADR-001 §12, T12a).
 *
 * The document keeps only `AttachmentInfo` metadata cards; the bytes are
 * stored in IndexedDB, addressed by content (id = SHA-256 of the original
 * file). The same image attached to N nodes therefore occupies space exactly
 * once, and there are no name collisions. No base64 anywhere: IndexedDB
 * stores native Blobs.
 *
 * The three levels arrive already generated – producing them needs a canvas
```

```

* and is the job of T13. This module only preserves and serves them.
*/
import type { Sheet } from "../core/types";

/** "original" = intatto · "large" = 1024px · "small" = 256px, lato lungo */
export type AssetLevel = "original" | "large" | "small";

export interface AssetBlob {
  blob: Blob;
  w: number;
  h: number;
}

export interface AssetMeta {
  id: string; // SHA-256 of the original
  mime: string;
  w: number; // dimensions of the original
  h: number;
  bytes: number; // weight of the original
  name?: string;
}

export interface AssetStore {
  /** The levels arrive ready-made: generating them is T13's job. */
  put(levels: Record<AssetLevel, AssetBlob>, meta: Omit<AssetMeta, "id">): Promise<string>;
  /**
   * Store levels under a caller-supplied id. The `rnode.zip` importer needs
   * it: a compact export carries no original, so re-deriving the id from the
   * stored levels (what `put` does) would change it and every reference in
   * the imported document would break. Idempotent like `put`: an id that
   * already exists is left untouched.
   *
   * INVARIANT EXCEPTION (AGENT_GUIDE I11): by taking the id as an argument
   * this breaks `id === sha256(original)`. That is only reachable through the
   * `rnode.zip` importer, and only for assets whose original is lost (the
   * compact mode carries display levels only) – those assets carry the
   * `originalLost` flag on their AttachmentInfo card so an export can warn.
   */
  putUnderId(id: string, levels: Record<AssetLevel, AssetBlob>, meta: AssetMeta): Promise<void>;
  get(id: string, level: AssetLevel): Promise<Blob | null>;
  meta(id: string): Promise<AssetMeta | null>;
  delete(id: string): Promise<void>;
  list(): Promise<string[]>;
}

/** Hex-encoded SHA-256 of the given bytes – the content address of an asset. */
export async function sha256Hex(data: BufferSource): Promise<string> {
  const digest = await crypto.subtle.digest("SHA-256", data);
  return Array.from(new Uint8Array(digest), (b) => b.toString(16).padStart(2, "0")).join("");
}

/** A stored asset: the three levels plus the metadata card. */
interface AssetRecord {
  id: string;
  original: AssetBlob;
  large: AssetBlob;
  small: AssetBlob;
  meta: AssetMeta;
}

const DEFAULT_DB_NAME = "r-node-assets";
const DB_VERSION = 1;
const STORE = "assets";

function openDb(dbName: string): Promise<IDBDatabase> {
  return new Promise((resolve, reject) => {
    const req = indexedDB.open(dbName, DB_VERSION);
    req.onupgradeneeded = () => {
      if (!req.result.objectStoreNames.contains(STORE)) {
        req.result.createObjectStore(STORE, { keyPath: "id" });
      }
    };
    req.onsuccess = () => resolve(req.result);
    req.onerror = () => reject(req.error);
    // A version-change upgrade waits for other open connections. Never hang
    // forever: reject loudly instead of leaving the promise pending.
    req.onblocked = () => {
      reject(new Error(`openDb("${dbName}")`: upgrade blocked by an open connection`));
    };
  });
}

/** Runs `fn` on the object store and resolves when the transaction completes. */
function txDone(db: IDBDatabase, mode: IDBTransactionMode, fn: (s: IDBObjectStore) => void): Promise<void> {
  return new Promise((resolve, reject) => {
    const tx = db.transaction(STORE, mode);
    tx.oncomplete = () => resolve();
  });
}

```

```

    tx.onerror = () => reject(tx.error);
    tx.onabort = () => reject(tx.error);
    fn(tx.objectStore(STORE));
  });
}

/** Fetches a record by key through a short-lived read-only transaction. */
function txGet<T>(db: IDBDatabase, id: string): Promise<T | undefined> {
  return new Promise((resolve, reject) => {
    const tx = db.transaction(STORE, "readonly");
    const req = tx.objectStore(STORE).get(id);
    req.onsuccess = () => resolve(req.result as T | undefined);
    req.onerror = () => reject(req.error);
  });
}

/**
 * Default (and, for now, only) AssetStore implementation. IndexedDB stores
 * native Blobs – no base64 inflation – with quotas in the hundreds of MB up
 * to GB, which is why the images cannot live in the document (localStorage
 * caps around 5MB).
 */
export class IndexedDbAssetStore implements AssetStore {
  private dbPromise: Promise<IDBDatabase> | null = null;

  constructor(private readonly dbName: string = DEFAULT_DB_NAME) {}

  private db(): Promise<IDBDatabase> {
    this.dbPromise ??= openDb(this.dbName);
    return this.dbPromise;
  }

  async put(levels: Record<AssetLevel, AssetBlob>, meta: Omit<AssetMeta, "id">): Promise<string> {
    const id = await sha256Hex(await levels.original.blob.arrayBuffer());
    await this.putUnderId(id, levels, { ...meta, id });
    return id;
  }

  async putUnderId(id: string, levels: Record<AssetLevel, AssetBlob>, meta: AssetMeta): Promise<void> {
    const db = await this.db();
    const record: AssetRecord = { id, ...levels, meta };
    // Existence check and write in ONE readwrite transaction: two concurrent
    // puts of identical content cannot both pass the check. Content-addressed
    // first write wins – an existing id is returned as-is, never rewritten.
    // Deliberately not add()-and-catch: a failed request aborts the
    // transaction unless its error event is default-prevented.
    await new Promise<void>((resolve, reject) => {
      const tx = db.transaction(STORE, "readwrite");
      tx.oncomplete = () => resolve();
      tx.onerror = () => reject(tx.error);
      tx.onabort = () => reject(tx.error);
      const s = tx.objectStore(STORE);
      const getReq = s.get(id);
      getReq.onsuccess = () => {
        if (getReq.result === undefined) s.put(record);
      };
      getReq.onerror = () => reject(getReq.error);
    });
  }

  async get(id: string, level: AssetLevel): Promise<Blob | null> {
    const record = await txGet<AssetRecord>(await this.db(), id);
    return record?.[level]?.blob ?? null;
  }

  async meta(id: string): Promise<AssetMeta | null> {
    const record = await txGet<AssetRecord>(await this.db(), id);
    return record?.meta ?? null;
  }

  async delete(id: string): Promise<void> {
    const db = await this.db();
    await txDone(db, "readwrite", (s) => s.delete(id));
  }

  async list(): Promise<string[]> {
    const db = await this.db();
    return new Promise((resolve, reject) => {
      const tx = db.transaction(STORE, "readonly");
      const req = tx.objectStore(STORE).getAllKeys();
      req.onsuccess = () => resolve(req.result.map(String));
      req.onerror = () => reject(req.error);
    });
  }
}

/**

```

```

* Tauri v2 exposes its API as `window.__TAURI__` when `withGlobalTauri` is
* enabled in tauri.conf.json. That global is the detection switch for the
* desktop build: it exists only inside the WebView2 window, never in a plain
* browser tab – exactly what the factory needs.
*/
type TauriInvoke = (cmd: string, args?: Record<string, unknown>) => Promise<unknown>;

declare global {
  interface Window {
    __TAURI__?: {
      core: {
        invoke: TauriInvoke;
      };
    };
  }
}

/**
* Desktop implementation: the levels live INSIDE the current `.rnode` file
* (a SQLite database) – `/scratch.rnode` before the
   * first save, then whatever open/save-as chose. Null resets to default. */
  private path: string | null = null;
  private defaultPath: Promise<string> | null = null;

  private invoke(): TauriInvoke {
    const api = typeof window !== "undefined" ? window.__TAURI__ : undefined;
    if (!api) throw new Error("TauriAssetStore used outside the Tauri webview");
    return api.core.invoke;
  }

  /** Switch this store to a document file. Same instance, new path. */
  setRoot(path: string | null): void {
    this.path = path;
    this.defaultPath = null;
    this.metaCache.clear();
  }

  private async currentPath(): Promise<string> {
    if (this.path) return this.path;
    this.defaultPath ??= this.invoke()("default_document_path") as Promise<string>;
    return this.defaultPath;
  }

  async put(levels: Record<AssetLevel, AssetBlob>, meta: Omit<AssetMeta, "id">): Promise<string> {
    const id = await sha256Hex(await levels.original.blob.arrayBuffer());
    await this.putUnderId(id, levels, { ...meta, id });
    return id;
  }

  async putUnderId(id: string, levels: Record<AssetLevel, AssetBlob>, meta: AssetMeta): Promise<void> {
    const path = await this.currentPath();
    const invoke = this.invoke();
    for (const level of ["original", "large", "small"] as const) {
      const bytes = new Uint8Array(await levels[level].blob.arrayBuffer());
      await invoke("put_asset", { path, id, level, bytes });
    }
    await invoke("put_asset", {
      path,
      id,
      level: "meta",
      bytes: new TextEncoder().encode(JSON.stringify(meta)),
    });
    this.metaCache.set(id, meta);
  }

  async get(id: string, level: AssetLevel): Promise<Blob | null> {
    const path = await this.currentPath();
    const raw = await this.invoke()("get_asset", { path, id, level });
    const bytes = toBytes(raw);
    if (!bytes || bytes.byteLength === 0) return null; // empty body = absent
    const mime = this.metaCache.get(id)?.mime;
    return new Blob([bytes], mime ? { type: mime } : undefined);
  }

  async meta(id: string): Promise<AssetMeta | null> {
    const cached = this.metaCache.get(id);
    if (cached) return cached;

```

```

const path = await this.currentPath();
const raw = await this.invoke()("get_asset", { path, id, level: "meta" });
const bytes = toBytes(raw);
if (!bytes || bytes.byteLength === 0) return null;
const meta = JSON.parse(new TextDecoder().decode(bytes)) as AssetMeta;
this.metaCache.set(id, meta);
return meta;
}

async delete(id: string): Promise<void> {
  const path = await this.currentPath();
  await this.invoke()("delete_asset", { path, id });
  this.metaCache.delete(id);
}

async list(): Promise<string[]> {
  return (await this.invoke()("list_assets", { path: await this.currentPath() })) as string[];
}

/**
 * Point this store at a NEW document file, copying every referenced asset
 * (all three levels + meta) from the current path into it FIRST. Reading
 * happens before the switch: after it, get() would read the new, empty
 * file. Same per-asset iteration the zip exporter uses – a document saved
 * into a fresh file never points at ids that are not there.
 */
async adoptFile(newPath: string, ids: string[]): Promise<void> {
  // The reads below go through this.currentPath(), still the OLD file: the
  // switch happens only after every asset is copied. Only the BYTES matter
  // for the copy – w/h live in the meta JSON.
  const payloads: { id: string; meta: AssetMeta; blobs: Record<AssetLevel, Blob> }[] = [];
  for (const id of ids) {
    const [meta, original, large, small] = await Promise.all([
      this.meta(id),
      this.get(id, "original"),
      this.get(id, "large"),
      this.get(id, "small"),
    ]);
    if (!meta || !original || !large || !small) continue; // referenced but absent
    payloads.push({ id, meta, blobs: { original, large, small } });
  }
  const invoke = this.invoke();
  for (const { id, meta, blobs } of payloads) {
    for (const level of ["original", "large", "small"] as const) {
      const bytes = new Uint8Array(await blobs[level].arrayBuffer());
      await invoke("put_asset", { path: newPath, id, level, bytes });
    }
    await invoke("put_asset", {
      path: newPath,
      id,
      level: "meta",
      bytes: new TextEncoder().encode(JSON.stringify(meta)),
    });
  }
  this.path = newPath;
  this.defaultPath = null;
  this.metaCache.clear();
}

/**
 * Raw IPC bytes arrive as an ArrayBuffer (Tauri 2 custom-protocol path for
 * `tauri::ipc::Response`); the postMessage fallback and test mocks may hand
 * back a Uint8Array or a plain number[]. Normalize all of them to a
 * Uint8Array, or null when the value is not bytes at all.
 */
function toBytes(raw: unknown): Uint8Array<ArrayBuffer> | null {
  if (raw === null || raw === undefined) return null;
  if (raw instanceof ArrayBuffer) return new Uint8Array(raw);
  if (raw instanceof Uint8Array) {
    // Fresh buffer: a view over a SharedArrayBuffer or a foreign buffer must
    // not leak into Blob (BlobPart wants ArrayBuffer-backed views).
    return Uint8Array.from(raw);
  }
  if (Array.isArray(raw)) return Uint8Array.from(raw as number[]);
  return null;
}

/**
 * The one factory every caller must use. The three former `new
 * IndexedDbAssetStore()` sites (store.ts, renderer.ts, RichEditor.tsx) each
 * opened their OWN database: images written through one store were invisible
 * to the others. A singleton also means the desktop build and the web build
 * each pick their backend exactly once, at first use.
 */
let sharedStore: AssetStore | null = null;

```

```

export function getAssetStore(): AssetStore {
  if (!sharedStore) {
    const tauri = typeof window !== "undefined" && !window.__TAURI__;
    sharedStore = tauri ? new TauriAssetStore() : new IndexedDbAssetStore();
  }
  return sharedStore;
}

/**
 * The single source of truth for "which asset is in use". The roots are the
 * nodes alone: an `attachments` card whose node is gone is garbage too, so
 * it must never act as a root. Future referrers (callouts, boundaries) are
 * added here, in one place.
 */
export function referencedAssetIds(sheet: Sheet): Set<string> {
  const referenced = new Set<string>();
  for (const node of Object.values(sheet.nodes)) {
    if (node.style.image) referenced.add(node.style.image);
  }
  return referenced;
}

export interface OrphanReport {
  /** Cards in sheet.attachments that no node references. */
  cards: string[];
  /** Ids in the store that no node references. */
  blobs: string[];
}

/**
 * What is unreachable from the nodes. Returns an OrphanReport WITHOUT
 * deleting anything: garbage collection must be an explicit user action (a
 * command), never a side-effect of undo or save.
 */
export async function collectOrphans(sheet: Sheet, store: AssetStore): Promise<OrphanReport> {
  const referenced = referencedAssetIds(sheet);
  const cards = sheet.attachments.map((a) => a.id).filter((id) => !referenced.has(id));
  const ids = await store.list();
  const blobs = ids.filter((id) => !referenced.has(id));
  return { cards, blobs };
}

```

## 63. src-tauri/src/lib.rs

```

/// R-node desktop backend.
///
/// The document is a SINGLE file `MiaMappa.rnode`, which is an SQLite
/// database holding the document JSON and every image level (T20). The
/// frontend keeps using its StorageAdapter and AssetStore; the Tauri
/// adapters call these commands.
///
/// Deliberately NOT using `journal_mode=WAL`: WAL writes `-wal` and `-shm`
/// files next to the document, which defeats the single-file requirement.
/// The default rollback journal (DELETE) creates a `~journal` only during a
/// write transaction and removes it on commit – at rest there is exactly one
/// file.
///
/// Each command opens the `.rnode` file at the path the frontend chose; the
/// backend keeps no hidden state. Connections are short-lived, so there is
/// nothing to share between commands.

use rusqlite::Connection;
use std::fs;
use std::path::Path;
use tauri::ipc::Response;
use tauri::{AppHandle, Manager};
use tauri_plugin_dialog::DialogExt;

/// Schema of a `.rnode` file (one file = one document).
///
/// `meta` carries `app` and `schemaVersion` so the file is recognizable and
/// can be migrated in the future without guessing. The `document` row is the
/// JSON document; `assets` holds the image levels (the `meta` level is the
/// per-asset metadata card, same as the T19 `assets/<id>/meta` file).
const SCHEMA: &str = "
CREATE TABLE IF NOT EXISTS meta      (key TEXT PRIMARY KEY, value TEXT NOT NULL);
CREATE TABLE IF NOT EXISTS document (id INTEGER PRIMARY KEY CHECK (id = 1), json TEXT NOT NULL);
CREATE TABLE IF NOT EXISTS assets    (id TEXT NOT NULL,      -- SHA-256
                                         level TEXT NOT NULL,  -- original | large | small | meta
                                         bytes BLOB NOT NULL,
                                         PRIMARY KEY (id, level));
";

const APP_NAME: &str = "r-node";

```

```

const SCHEMA_VERSION: &str = "1";

/// Create the schema and write the recognizer keys. Safe to run on a fresh
/// file AND on an existing document (all statements are IF NOT EXISTS /
/// OR IGNORE).
fn ensure_schema(conn: &Connection) -> Result<(), String> {
    conn.execute_batch(SCHEMA).map_err(|e| e.to_string());
    conn.execute(
        "INSERT OR IGNORE INTO meta (key, value) VALUES ('app', ?1), ('schemaVersion', ?2)",
        rusqlite::params![APP_NAME, SCHEMA_VERSION],
    )
    .map_err(|e| e.to_string());
    Ok(())
}

/// Open a `.rnode` file, creating it (and its parent dir) if needed, and
/// make sure it carries the schema. Used by every WRITE command.
fn open_writable(path: &str) -> Result<Connection, String> {
    if let Some(parent) = Path::new(path).parent() {
        if !parent.as_os_str().is_empty() {
            fs::create_dir_all(parent).map_err(|e| format!("cannot create {parent:?}: {e}"))?;
        }
    }
    let conn = Connection::open(path).map_err(|e| format!("cannot open {path}: {e}"))?;
    ensure_schema(&conn)?;
    Ok(conn)
}

/// Open a `.rnode` file WITHOUT creating it: `SQLITE_OPEN_READ_WRITE` (no
/// CREATE) fails when the file is missing, and still lets SQLite roll back a
/// hot journal left by a crash. Reads never leave a file behind.
fn open_readable(path: &str) -> Result<Connection, String> {
    Connection::open_with_flags(path, rusqlite::OpenFlags::SQLITE_OPEN_READ_WRITE)
        .map_err(|e| format!("cannot open {path}: {e}"))
}

/// Write one asset level. `INSERT OR IGNORE`: an existing (id, level) is
/// left untouched, matching the AssetStore contract (ids are content
/// hashes, so same id => same bytes; re-importing never rewrites anything).
#[tauri::command]
fn put_asset(path: String, id: String, level: String, bytes: Vec<u8>) -> Result<(), String> {
    let conn = open_writable(&path)?;
    conn.execute(
        "INSERT OR IGNORE INTO assets (id, level, bytes) VALUES (?1, ?2, ?3)",
        rusqlite::params![id, level, bytes],
    )
    .map_err(|e| e.to_string());
    Ok(())
}

/// Read one asset level as raw bytes. Missing -> EMPTY vec: the frontend
/// treats an empty result as "absent" (the AssetStore contract: get of a
/// missing id -> null). Image bytes are never legitimately empty, so the
/// convention cannot collide with data.
fn asset_bytes(path: &str, id: &str, level: &str) -> Result<Vec<u8>, String> {
    let conn = open_readable(&path)?;
    conn.query_row(
        "SELECT bytes FROM assets WHERE id = ?1 AND level = ?2",
        rusqlite::params![id, level],
        |row| row.get(0),
    )
    .map(Some)
    .or_else(|e| match e {
        rusqlite::Error::QueryReturnedNoRows => Ok(None),
        other => Err(other),
    })
    .map(|b| b.unwrap_or_default())
    .map_err(|e| e.to_string())
}

/// Read one asset level as RAW bytes over IPC (no JSON-array-of-numbers
/// inflation: an original of tens of MB must not be serialized as fifty
/// million numbers). `tauri::ipc::Response` sends an ArrayBuffer to the
/// frontend (verified for Tauri 2.11.5).
#[tauri::command]
fn get_asset(path: String, id: String, level: String) -> Result<Response, String> {
    Ok(Response::new(asset_bytes(&path, &id, &level)?))
}

/// Remove every level of an asset (the whole `assets/<id>` group).
#[tauri::command]
fn delete_asset(path: String, id: String) -> Result<(), String> {
    let conn = open_readable(&path)?;
    conn.execute("DELETE FROM assets WHERE id = ?1", [id])
        .map_err(|e| e.to_string());
    Ok(())
}

```

```

#[tauri::command]
fn list_assets(path: String) -> Result<Vec<String>, String> {
    let conn = open_readable(&path)?;
    let mut stmt = conn
        .prepare("SELECT DISTINCT id FROM assets ORDER BY id")
        .map_err(|e| e.to_string())?;
    let rows = stmt
        .query_map([], |row| row.get(0))
        .map_err(|e| e.to_string())?;
    let mut out = Vec::new();
    for r in rows {
        out.push(r.map_err(|e| e.to_string())?);
    }
    Ok(out)
}

/// The default working document BEFORE the first save: `<app-data>/
/// scratch.rnode`. Attaching an image to a never-saved map must work, so the
/// file is created (with schema) here, and the first "Save as..." adopts its
/// content into the user-chosen file.
#[tauri::command]
fn default_document_path(app: AppHandle) -> Result<String, String> {
    let dir = app
        .path()
        .app_data_dir()
        .map_err(|e| format!("no app data dir: {e}"))?;
    fs::create_dir_all(&dir).map_err(|e| format!("cannot create data dir: {e}"))?;
    let path = dir.join("scratch.rnode");
    let conn = Connection::open(&path).map_err(|e| e.to_string())?;
    ensure_schema(&conn)?;
    Ok(path.to_string_lossy().into_owned())
}

/// Native file picker restricted to `.rnode`. `mode` is "open" or "save"; a
/// cancelled dialog returns None. On save the dialog is pre-filled with
/// `suggested_name` (the document title from the GUI, sanitized by the
/// frontend) so the name typed in the app becomes the real file's name – the
/// filesystem dialog already knows it. The extension is guaranteed even when
/// the user types a name without it.
///
/// Uses `tauri-plugin-dialog` (NOT raw rfd): a blocking rfd dialog inside a
/// sync command runs on the main thread, fighting the webview's message pump
/// and appearing without a parent – the app showed exactly that bug (dialog
/// not opening, then reopening on its own). The plugin runs the dialog off
/// the main thread, parented to the window, and this command is async so the
/// main thread stays free.
#[tauri::command]
async fn pick_document_file(
    app: AppHandle,
    mode: String,
    suggested_name: Option<String>,
) -> Result<Option<String>, String> {
    let parent = app.get_webview_window("main");
    let dialog = |title: &str, save: bool| {
        let mut builder = app.dialog().file().add_filter("R-node document", &["rnode"]);
        if let Some(w) = &parent {
            builder = builder.set_parent(w);
        }
        builder = builder.set_title(title);
        if save {
            let mut name = suggested_name.clone().unwrap_or_else(|| "MiaMappa".into());
            if !name.to_lowercase().ends_with(".rnode") {
                name.push_str(".rnode");
            }
            builder = builder.set_file_name(name);
        }
        builder
    };

    let picked = match mode.as_str() {
        "open" => dialog("Open an R-node document", false).blocking_pick_file(),
        "save" => dialog("Save the R-node document", true).blocking_save_file(),
        _ => return Err(format!("unknown pick mode: {mode}")),
    };

    let Some(file) = picked else {
        return Ok(None); // user cancelled
    };

    let mut path = match file {
        tauri_plugin_dialog::FilePath::Path(p) => p.to_string_lossy().into_owned(),
        tauri_plugin_dialog::FilePath::Url(_) => return Err("dialog returned a URL, expected a path".into()),
    };

    if mode == "save" && !path.to_lowercase().ends_with(".rnode") {
        path.push_str(".rnode");
    }

    Ok(Some(path))
}

```

```

/// Delete the `.rnode` file. Used by the rename-on-save flow, AFTER the new
/// file is fully written and the app switched to it – the old file is only
/// removed once the document provably lives at the new path.
#[tauri::command]
fn remove_document(path: String) -> Result<(), String> {
    match fs::remove_file(&path) {
        Ok(()) => Ok(()),
        Err(e) if e.kind() == std::io::ErrorKind::NotFound => Ok(()), // already gone
        Err(e) => Err(e.to_string()),
    }
}

/// Whether a file exists at `path`. The rename flow refuses to clobber an
/// existing document under the new name.
#[tauri::command]
fn document_file_exists(path: String) -> Result<bool, String> {
    Ok(Path::new(&path).is_file())
}

/// Read the document JSON from `path`, or None when the file is missing or
/// is not an R-node document. Never creates the file.
#[tauri::command]
fn read_document(path: String) -> Result<Option<String>, String> {
    let Ok(conn) = open_readable(&path) else {
        return Ok(None); // no file yet – nothing to read, not an error
    };
    let json: Option<String> = conn
        .query_row("SELECT json FROM document WHERE id = 1", [], |row| row.get(0))
        .map(Some)
        .or_else(|e| match e {
            rusqlite::Error::QueryReturnedNoRows => Ok(None),
            rusqlite::Error::SqliteFailure(f, _) if f.code == rusqlite::ErrorCode::NotADatabase => Ok(None),
            other => Err(other),
        })
        .map_err(|e| e.to_string());
    Ok(json)
}

/// Write the document JSON into `path` – creating the file if needed – in
/// ONE transaction, so a crash mid-save never leaves a half-written
/// document. The rollback journal disappears on commit (no WAL).
#[tauri::command]
fn write_document(path: String, data: String) -> Result<(), String> {
    let mut conn = open_writable(&path)?;
    let tx = conn.transaction().map_err(|e| e.to_string());
    tx.execute(
        "INSERT INTO document (id, json) VALUES (1, ?1)
        ON CONFLICT(id) DO UPDATE SET json = excluded.json",
        [&data],
    )
    .map_err(|e| e.to_string());
    tx.commit().map_err(|e| e.to_string());
    Ok(())
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::path::PathBuf;

    /// Fresh, unique, pre-deleted document path inside its OWN directory, so
    /// "what is left on disk" is observable without other temp files around.
    fn temp_doc(name: &str) -> PathBuf {
        let dir = std::env::temp_dir().join(format!("rnode-t20-{}-{}", name, std::process::id()));
        let _ = fs::remove_dir_all(&dir);
        fs::create_dir_all(&dir).unwrap();
        dir.join("MiaMappa.rnode")
    }

    fn dir_files(dir: &Path) -> Vec<String> {
        let mut out: Vec<String> = fs::read_dir(dir)
            .map(|e| {
                e.flatten()
                    .filter(|e| e.file_type().map(|t| t.is_file()).unwrap_or(false))
                    .map(|e| e.file_name().to_string_lossy().into_owned())
                    .collect()
            })
            .unwrap_or_default();
        out.sort();
        out
    }

    #[test]
    fn asset_round_trip() {
        let path = temp_doc("assets.rnode");
        let p = path.to_string_lossy().into_owned();

```

```

let id = "a".repeat(64);

put_asset(p.clone(), id.clone(), "original".into(), vec![1, 2, 3]).unwrap();
put_asset(p.clone(), id.clone(), "meta".into(), b{"m":1}.to_vec()).unwrap();

assert_eq!(asset_bytes(&p, &id, "original").unwrap(), vec![1, 2, 3]);
assert_eq!(asset_bytes(&p, &id, "meta").unwrap(), b{"m":1}.to_vec());
// A missing level returns EMPTY bytes, not an error (contract: null).
assert_eq!(asset_bytes(&p, &id, "large").unwrap(), Vec::<u8>::new());

assert_eq!(list_assets(p.clone()).unwrap(), vec![id.clone()]);
delete_asset(p.clone(), id.clone()).unwrap();
assert_eq!(list_assets(p.clone()).unwrap(), Vec::<String>::new());
assert_eq!(asset_bytes(&p, &id, "original").unwrap(), Vec::<u8>::new());
}

#[test]
fn first_write_wins_for_an_existing_id() {
let path = temp_doc("fww.rnode");
let p = path.to_string_lossy().into_owned();
let id = "b".repeat(64);

put_asset(p.clone(), id.clone(), "large".into(), vec![7, 7]).unwrap();
put_asset(p.clone(), id.clone(), "large".into(), vec![9, 9, 9]).unwrap();
assert_eq!(asset_bytes(&p, &id, "large").unwrap(), vec![7, 7]);
}

#[test]
fn put_asset_creates_a_fresh_file_and_leaves_only_it_at_rest() {
let path = temp_doc("fresh.rnode");
let p = path.to_string_lossy().into_owned();
let dir = path.parent().unwrap();

// put_asset before any save (the scratch flow): the file appears.
put_asset(p.clone(), "c".repeat(64), "small".into(), vec![5]).unwrap();
write_document(p.clone(), "{doc:true}".into()).unwrap();
assert_eq!(read_document(p.clone()).unwrap().as_deref(), Some("{doc:true}"));

// A completed save leaves EXACTLY one file: no -wal, -shm, -journal.
assert_eq!(dir_files(dir), vec![path.file_name().unwrap().to_string_lossy().into_owned()]);
}

#[test]
fn document_round_trip_and_overwrite() {
let path = temp_doc("doc.rnode");
let p = path.to_string_lossy().into_owned();
write_document(p.clone(), "{v\":1}".into()).unwrap();
write_document(p.clone(), "{v\":2}".into()).unwrap();
assert_eq!(read_document(p.clone()).unwrap().as_deref(), Some("{v\":2}"));

// The scratch recognizer keys were written too.
let conn = open_readable(&p).unwrap();
let app: String = conn
.query_row("SELECT value FROM meta WHERE key = 'app'", [], |r| r.get(0))
.unwrap();
assert_eq!(app, APP_NAME);
}

#[test]
fn read_of_a_missing_file_is_none_and_creates_nothing() {
let path = temp_doc("missing.rnode");
assert_eq!(path.exists(), false);
assert_eq!(read_document(path.to_string_lossy().into_owned()).unwrap(), None);
assert_eq!(path.exists(), false);
}

#[test]
fn read_of_a_non_database_file_is_none() {
let path = temp_doc("garbage.rnode");
fs::write(&path, b"this is not sqlite").unwrap();
assert_eq!(read_document(path.to_string_lossy().into_owned()).unwrap(), None);
}

#[test]
fn write_document_is_atomic_single_transaction() {
// A valid document survives a second, overwriting save (transaction
// commits cleanly and the journal is gone afterwards).
let path = temp_doc("tx.rnode");
let p = path.to_string_lossy().into_owned();
let dir = path.parent().unwrap();
write_document(p.clone(), "{x\":1}".into()).unwrap();
assert_eq!(read_document(p.clone()).unwrap().as_deref(), Some("{x\":1}"));
assert_eq!(dir_files(dir).len(), 1, "no journal/wal/shm left at rest");
}

#[cfg_attr(mobile, tauri::mobile_entry_point)]

```

```

pub fn run() {
    tauri::Builder::default()
        .plugin(tauri_plugin_dialog::init())
        // CORS-free HTTP for images dragged from a browser page (text/uri-list).
        .plugin(tauri_plugin_http::init())
        .invoke_handler(tauri::generate_handler![
            put_asset,
            get_asset,
            delete_asset,
            list_assets,
            default_document_path,
            pick_document_file,
            read_document,
            write_document,
            remove_document,
            document_file_exists
        ])
        .run(tauri::generate_context!())
        .expect("error while running r-node");
}

```

## 64. docs/ROADMAP.md

# R-node – roadmap operativa

Task in ordine di esecuzione. **\*\*Fai un task alla volta, nell'ordine scritto.\*\***  
Prima di iniziare leggi [AGENT\_GUIDE.md](AGENT\_GUIDE.md) §1, §2, §6.

Ogni task è chiuso solo quando la sua sezione **\*\*Fatto quando\*\*** è interamente vera. Se non ci riesci, fermati e spiega: non disattivare test, non alzare soglie, non allargare lo scope.

Legenda: **\*\*P0\*\*** = blocca la qualità del resto · **\*\*P1\*\*** = alto valore ·  
**\*\*P2\*\*** = funzionalità mancanti.

Stato dei task: **\*\*✅ FATTO\*\*** = chiuso · **\*\*⚠️ PARZIALE\*\*** = obiettivo centrato solo in parte (la sezione dice cosa resta) · senza marcatore = aperto.

---

## T1 – Invariant checker della topologia · P0

**\*\*Obiettivo.\*\*** Una funzione pura che verifica che l'albero dei nodi sia coerente, invocata dopo ogni batch di operazioni in dev/test.

**\*\*Perché.\*\*** ``execOps`` (``src/editor/store.ts``) applica gli op mutando l'albero in place, senza alcuna validazione. Un ``parentId`` sbagliato non dà errore: resta in memoria, viene salvato su disco, e riemerge molto dopo come crash inspiegabile lontano dalla causa. Questo task trasforma una corruzione silenziosa in un errore immediato e con un nome.

**\*\*File.\*\*** ``src/core/validate.ts`` (nuovo) · ``src/editor/store.ts`` ·  
``tests/validate.test.ts`` (nuovo)

**\*\*Passi.\*\***

1. Crea ``src/core/validate.ts`` con:
 

```

      ```ts
      export class InvariantViolation extends Error {}
      export function validateSheet(sheet: Sheet): void
      ...
      
```
2. ``validateSheet`` deve lanciare ``InvariantViolation`` se **\*\*una\*\*** di queste è falsa:
  - **\*\*Coerenza parent/children\*\***: se ``B`` compare in ``A.childrenIds``, allora ``sheet.nodes[B].parentId === A.id``. E viceversa: se ``B.parentId === A.id``, allora ``B.id`` compare in ``A.childrenIds`` esattamente una volta.
  - **\*\*Riferimenti risolvibili\*\***: ogni id in ``childrenIds``, ogni ``parentId`` non nullo, e ogni ``fromId`/`toId`` in ``relationships`` esistono in ``sheet.nodes``.
  - **\*\*Nessun ciclo\*\***: partendo da ``rootNodeId`` e scendendo per ``childrenIds``, nessun nodo viene visitato due volte.
  - **\*\*Root\*\***: ``sheet.nodes[sheet.rootNodeId]`` esiste e ha ``parentId === null``.
  - **\*\*Nessun orfano\*\***: ogni nodo è raggiungibile da ``rootNodeId``, tranne quelli con ``type === "floating"``.
3. Il messaggio deve nominare gli id coinvolti, es.  
``InvariantViolation: node 'n_123' has parentId 'n_456' but appears in childrenIds of 'n_789'``.
4. In ``store.ts``, alla fine di ``execOps``, ``undo`` e ``redo``, invoca ``validateSheet(this.sheet)`` **\*\*solo fuori produzione\*\***:
 

```

      ```ts
      if (import.meta.env?.DEV ?? true) validateSheet(this.sheet);
      ...
      
```

In produzione deve costare zero.

**\*\*Fatto quando.\*\***

- ``tests/validate.test.ts`` copre i 5 casi sopra: uno sheet valido passa, e cinque sheet corrotti a mano lanciano ciascuno il proprio errore.

```

- Almeno un test verifica che dopo una sequenza reale di op via `EditorStore`
  (crea → sposta → cancella → undo → redo) lo sheet resti valido.
- `npm test` e `npm run typecheck` verdi.

**Non fare.** Non "riparare" automaticamente lo sheet quando trovi
un'incoerenza: deve fallire, non correggere. Non invocarlo in produzione.

---

## T2 – Harness di parità nella suite di test · P0

**Obiettivo.** I casi di `dev/parity.ts` diventano un test automatico che
fallisce da solo, invece di un URL da aprire a mano.

**Perché.** Oggi la fedeltà visiva del testo sul canvas è protetta solo se
qualcuno si ricorda di aprire `/dev/parity.html`. Chiunque può rompere
`measure.ts` senza che nessuno se ne accorga.

> **Prima di iniziare questo task, fermati e chiedi conferma.** Aggiunge una
> dipendenza di sviluppo (browser di test) e un browser da installare in CI:
> è una decisione del proprietario del progetto, non tua.

**File.** `package.json` · `vite.config.ts` · `tests/parity.browser.test.ts`
(nuovo) · `dev/parity.ts` (solo estrazione, vedi sotto)

**Passi.**
1. `jsdom` non serve a niente qui: non implementa il layout,
  `getBoundingClientRect()` restituisce zeri. Serve un browser vero –
  `@vitest/browser` con provider Playwright, oppure Playwright diretto.
2. Estrai da `dev/parity.ts` in un modulo condiviso (es.
  `dev/parityCore.ts`) tutto tranne il rendering del report HTML: il corpus
  `CASES`, `runsToDom`, `browserLines`, `canvasLines`, `runCase`. La pagina
  `dev/parity.html` continua a usarlo per l'ispezione manuale.
3. Il test importa `CASES` e `runCase` e asserisce, **per ogni caso**:
  - `linesCanvas === linesBrowser`
  - `firstBreakMismatch === null`
  - `maxDTop`, `maxDAdv`, `maxDLeft`, `|dTotalH|` tutti `<= 0.5`
4. Il messaggio di fallimento deve nominare il caso e il delta, es.
  `parity[bullets-nested]: maxDTop 10.5 > 0.5`.

**Fatto quando.**
- Il nuovo test gira con `npm test` (o uno script dedicato documentato nel
  README) e passa su tutti i 16 casi.
- Rompendo di proposito una costante (es. `BLOCK_GAP_FACTOR` da 0.6 a 0.5) il
  test **fallisce**. Verificalo davvero, poi ripristina.

**Non fare.** Non abbassare `EPS` sopra 0.5 per far passare il test. Non
riscrivere `wrapRunLines` in questo task.

---

## T3 – Property-based testing su apply/inverse · P1 · dipende da T1

**Obiettivo.** Verificare matematicamente che applicare op e poi i loro inversi
riporti allo stato di partenza, su sequenze generate a caso.

**Perché.** `tests/ops.test.ts` copre scenari scritti a mano. L'undo/redo è un
sistema algebrico: gli edge case veri stanno nelle combinazioni che nessuno
pensa di scrivere.

**File.** `package.json` (dev dep `fast-check`) · `tests/ops.property.test.ts`
(nuovo)

**Passi.**
1. Arbitrary per uno `Sheet` valido piccolo (3-10 nodi): genera un albero, poi
  fallo passare da `validateSheet` (T1) – se non è valido, l'arbitrary è
  sbagliato.
2. Arbitrary per una sequenza di 1-10 `Op` **strutturalmente applicabili** a
  quello sheet (`createNode`, `moveNode`, `setTitle`, `setStyle`,
  `deleteNode`). Gli id devono riferirsi a nodi esistenti.
3. Proprietà da verificare:
  - **Round-trip**: applica gli op raccogliendo gli inversi con
    `applyWithInverse`, poi applica gli inversi in ordine inverso → lo sheet
    è deep-equal a quello iniziale.
  - **Validità continua**: dopo ogni singolo op, `validateSheet` non lancia.
4. Almeno 200 run per proprietà.

**Fatto quando.** Entrambe le proprietà passano. Se `fast-check` trova un
controesempio reale, **è un bug del codice**: correggilo e aggiungi quel caso
come test deterministico in `tests/ops.test.ts`.

---

## T4 – Tipizzare i campi non implementati di `Sheet` · P1

**Obiettivo.** Impedire che vengano scritti dati arbitrari nei campi ancora
non progettati.

```

```

**Perché.** `src/core/types.ts` dichiara `callouts`, `zones`, `comments` come
`unknown[]`. `unknown[]` accetta qualunque cosa in scrittura: è un invito a
inventare schemi incoerenti. (`boundaries`, `summaries` e `labels` hanno già
tipi veri: sono stati implementati da altre feature.)

**File.** `src/core/types.ts` · eventuali punti che li inizializzano.

**Passi.**
1. Cambia a `never[]` i tre campi rimasti: `callouts`, `zones`, `comments`.
2. Aggiungi sopra un commento: sono feature di Fase 2-3, si tipizzano quando si
progettano, e `never[]` costringe a una decisione esplicita.

**Fatto quando.** `npm run typecheck` verde (gli array vuoti `[]` restano
validi) e `npm test` verde.

**Non fare.** **Non inventare** `CalloutInfo`, `ZoneInfo` ecc. Progettare ora
schemi per feature non implementate è esattamente il debito che questo task
evita.

---

## T5 – Budget di performance assertivi · P1

> **Stato: ☒ FATTO. I budget sono assertivi in `tests/perf.test.ts` come
> soglie di sanità (applyOps < 10s, layout < 5s, writeback < 5s a 1k/5k/10k),
> non le soglie 3x datate per taglia previste qui: l'intento – far fallire le
> regressioni algoritmiche invece di stamparle – è coperto, con un meccanismo
> più semplice.

**Obiettivo.** Far **fallire** i test se una modifica degrada le performance,
invece di stamparlo in console.

**Perché.** `tests/perf.test.ts` già misura `applyOps`, `layout`, `writeback`,
`walks` e li stampa. Nessuno legge la console: una regressione passa
inosservata.

**File.** `tests/perf.test.ts`

**Passi.**
1. Esegui `npm test` e annota i tempi attuali a 1.000, 5.000 e 10.000 nodi.
2. Aggiungi asserzioni con soglia **generosa (3x la misura attuale): servono
a catturare regressioni algoritmiche (10x), non il rumore della macchina.
3. Commenta ogni soglia con il valore misurato e la data.

**Fatto quando.** I test passano; alzando artificialmente il costo di
`layoutSheet` (es. un ciclo inutile) falliscono. Verificalo, poi ripristina.

**Non fare.** Soglie strette: un test perf che fallisce a caso viene disattivato
entro una settimana e non protegge più nulla.

---

## T6 – Chiave della cache testo fuori dall'hot path · P1

**Obiettivo.** Smettere di serializzare i runs di ogni nodo visibile a ogni
frame.

**Perché.** `Renderer.textCacheKey` (`src/render/renderer.ts`) costruisce la
chiave con `JSON.stringify(n.titleRuns ?? n.title)`, e `drawText` la chiama per
**ogni nodo visibile a ogni frame**. La cache evita di ri-disegnare, non di
costruire la chiave: con titoli lunghi è il costo dominante del pan.

**File.** `src/render/renderer.ts` · eventualmente `src/core/types.ts`

**Passi.** Scegli **una** delle due strade:
- **A (consigliata): `WeakMap<MindNode, entry>` come primo livello di lookup.
Funziona solo se gli op sostituiscono l'oggetto nodo invece di mutarlo:
**verificalo prima** in `src/core/ops.ts`. Se i nodi sono mutati in place,
scarta A.
- **B**: un contatore `titleVersion` sul nodo, incrementato dagli op che
cambiano titolo/stile, usato nella chiave al posto del `JSON.stringify`.

**Fatto quando.** Nessun `JSON.stringify` nel percorso di disegno per frame; il
rendering resta visivamente identico (harness a 0 divergenze) e i test passano.

---

## T7 – Il colore del testo non deve cancellare il font-size · P1

**Obiettivo.** Applicare il colore alla selezione senza distruggere gli altri
stili inline e senza colorare testo non selezionato.

**Perché.** `applyColor` in `src/ui/RichEditor.tsx` fa
`node.setStyle("color: ...")`: `setStyle` **sostituisce** l'intera stringa, quindi
colorare un heading gli toglie il `font-size`. E itera su `selection.getNodes()`
colorando **interi** `TextNode` anche quando la selezione ne copre solo una

```

parte: selezionare mezza parola colora tutta la parola.

```
**File.** `src/ui/RichEditor.tsx` · `tests/lexicalRuns.test.ts`
```

**\*\*Passi.\*\*** Sostituisci ``applyColor`` e ``clearColor`` con ``$patchStyleText`` da ``@lexical/selection``: fa lo split dei nodi ai bordi della selezione e fonde le proprietà invece di sostituirle. ``clearColor`` passa ``{ color: null }``.

**\*\*Fatto quando.\*\*** Un test verifica che colorare un run con ``fontSize: 26`` mantiene il ``fontSize`` nei runs risultanti, e che colorare una sotto-parte produce runs distinti.

---

**## T8 – Toolbar: liste, heading, sottolineato, barrato · P2**

> **\*\*Stato:** 🚧 **PARZIALE.** Sottolineato è in toolbar (pulsante U) e il barrato sopravvive al round-trip nel modello, ma la toolbar non ha ancora liste né selettore heading, e il barrato non è applicabile dall'interfaccia. Resta da > fare la parte del task qui sotto che non è barrata.

**\*\*Obiettivo.\*\*** Rendere raggiungibili dall'interfaccia le formattazioni che il modello già supporta.

**\*\*Perché.\*\*** ``TextRun`` supporta ``underline``, ``fontSize`` (heading) e ``listIndent``, il canvas li disegna e la sanitizzazione li produce – ma la toolbar ha solo B / I / U / colore / clear. Heading e liste si ottengono **\*\*solo incollandoli\*\***.

```
**File.** `src/ui/RichEditor.tsx` · `src/styles.css`
```

**\*\*Passi.\*\***

1. Aggiungi: `~~sottolineato~~` (fatto) e barrato (``FORMAT_TEXT_COMMAND`` con ``"underline" / "strikethrough"``), lista puntata (``INSERT_UNORDERED_LIST_COMMAND`` da ``@lexical/list``), e un selettore heading H1-H3 che imposta ``font-size`` inline sui ``TextNode`` selezionati via ``$patchStyleText``.
2. Usa le taglie già definite in ``HEADING_SIZES``.

**\*\*Fatto quando.\*\*** Ogni comando produce i runs attesi (``editorStateToRuns``), il round-trip resta idempotente (I8), e l'harness resta a 0 divergenze.

**\*\*Attenzione.\*\*** Gli heading devono restare ``font-size`` inline su ``<span>``, non ``HeadingNode``: il canvas e il calcolo del gap dipendono da questo (§3.3 della guida).

---

**## T9 – `PastePolicy`: preservare la struttura, imporre l'estetica R-node · P2**

**\*\*Obiettivo.\*\*** Una politica di incollamento esplicita e in un solo posto.

**\*\*Perché.\*\*** Oggi la politica è implicita e in parte contraddittoria:

- ``HEADING_SIZES`` è **\*\*duplicata\*\*** in ``src/ui/lexicalRuns.ts`` e ``src/ui/pasteSanitizer.ts``, ed è in px **\*\*assoluti\*\***: su un nodo con ``fontSize: 20``, un ``<h6>`` incollato risulta **\*\*più piccolo\*\*** del corpo;
- i colori del documento sorgente passano intatti, quindi l'estetica del brand non è imposta;
- ``applyInline`` non legge mai ``style.fontSize`` delle ``<span>``: Word esprime i suoi stili "Titolo" così, quindi la gerarchia visiva di un documento Word si appiattisce senza che nessuno lo dica;
- non c'è alcun limite di volume: incollare 40 pagine in un titolo produce un nodo alto qualche migliaio di px.

```
**File.** `src/ui/pastePolicy.ts` (nuovo) · `src/ui/pasteSanitizer.ts` ·  
`src/ui/lexicalRuns.ts` · `tests/pasteSanitizer.test.ts`
```

**\*\*Passi.\*\***

1. Un solo modulo con la mappa heading come **\*\*scale relative\*\*** (``h1: 1.8 ... h6: 1.0``), risolte al paste contro il ``fontSize`` del nodo e clampate a ``[base, base × 2]``.
2. ``font-size`` delle span sorgente: leggilo e mappalo allo step di brand più vicino, invece di scartarlo in silenzio.
3. Colori: snap alla palette entro una soglia; scarta quelli troppo vicini al colore di testo di default. Prevedi un modificatore per il paste fedele.
4. Cap su blocchi e caratteri, con troncamento **\*\*annunciato\*\*** all'utente.
5. Elimina la costante duplicata.

**\*\*Fatto quando.\*\*** I test coprono: heading relativi al font del nodo, colore snappato, font-size Word bucketizzato, cap rispettato. Nessuna regressione in ``tests/pasteSanitizer.test.ts``.

---

**## T10 – Paste ricco su nodo selezionato · P2**

> **\*\*Stato:** ✅ **FATTO.** ``store.paste`` usa ``navigator.clipboard.read()`` e > instrada ``text/html`` in ``sanitizeHtml`` → ``htmlToRuns``, con fallback a testo > piatto dove l'API non è disponibile (``src/editor/store.ts``).

**\*\*Obiettivo.\*\*** Incollare da Word con un nodo selezionato (senza aprire l'editor) deve conservare la formattazione.

**\*\*Perché.\*\*** ``EditorStore.paste`` usa ``navigator.clipboard.readText()``, quindi il percorso ``typeToEdit` → `plainToRuns`` **\*\*perde tutta la formattazione\*\***. Il paste ricco funziona solo dentro l'editor già aperto.

**\*\*File.\*\*** ``src/editor/store.ts`` · ``src/ui/RichEditor.tsx``

**\*\*Passi.\*\*** Usa ``navigator.clipboard.read()`` per ottenere ``text/html``, instradalo in ``sanitizeHtml` → `htmlToRuns`` (con la policy di T9) e introduci un ``pendingInsertRuns`` accanto a ``pendingInsert``, consumato al mount dell'overlay. Mantieni il fallback a testo piatto dove l'API non è disponibile.

**\*\*Fatto quando.\*\*** Un test dello store verifica che con HTML negli appunti il nodo riceve runs formattati, e che senza HTML il comportamento attuale non cambia.

---

## T11 – Guardia sulla composizione IME · P2

**\*\*Obiettivo.\*\*** Non convertire lo stato dell'editor in runs mentre l'utente sta componendo (cinese, giapponese, coreano, accenti con dead key).

**\*\*Perché.\*\*** ``DraftSyncPlugin`` chiama ``editorStateToRuns`` a **\*\*ogni\*\*** update, composizione inclusa: layout thrash e composizione potenzialmente rotta.

**\*\*File.\*\*** ``src/ui/RichEditor.tsx``

**\*\*Passi.\*\*** Nel listener, salta l'aggiornamento del draft se ``editor.isComposing()``, e forza una sincronizzazione al termine della composizione.

**\*\*Fatto quando.\*\*** Il draft non viene aggiornato durante la composizione e contiene il testo finale corretto alla fine.

---

## Retrospectiva – immagini e desktop (T0, T12a-T20) ·  FATTO

Questa voce sostituisce i task T0 e T12a-T15 della roadmap e i passi T16-T20 del piano di esecuzione: l'intero programma immagini + desktop è implementato e spedito.

- **\*\*T0 – bring-up di Tauri.\*\*** La shell desktop compila e funziona: Rust e la CLI Tauri sono installati, ``cargo tauri dev`` avvia l'app e un documento sopravvive a chiusura e riapertura.
- **\*\*T12a/T12b – archivio degli asset.\*\*** Store indirizzato per contenuto (SHA-256) dietro l'interfaccia ``AssetStore``, con le correzioni di T12b: GC con radici solo nei nodi, ``put`` atomico in una transazione, ``onblocked`` gestito. Implementazione IndexedDB per il web, SQLite su desktop.
- **\*\*T12 – immagini nel modello.\*\*** ``AttachmentInfo`` + ``Style.imageWidth``, misura con ``imageResolver`` passato da layout, renderer e overlay, disegno con cache bitmap a budget di byte (``createImageBitmap`` a livello corrispondente allo zoom, tetto 1024px, sfratto LRU con ``close()``), spazio riservato nell'overlay, op ``setNodeImage`` con inverso.
- **\*\*T13 – ingestione.\*\*** Pipeline di import in **\*\*Web Worker\*\*** (originale intatto + livelli 1024/256 da una sola lettura), drop sul nodo e paste, allowlist dei formati, rifiuti tracciati.
- **\*\*T14 – ridimensionamento.\*\*** Slider nell'Inspector e maniglia sul canvas, proporzioni bloccate, un solo op per gesto.
- **\*\*T15 – documento portabile.\*\*** ``rnode.zip`` (completo = originali, compatto = soli livelli display) con stima di dimensione prima della generazione e import senza duplicati (id = hash del contenuto).
- **\*\*T16-T20 – desktop definitivo.\*\*** Asset su filesystem via comandi Rust (T16); misura della memoria superata: picco **\*\*17,4 MB\*\*** di bitmap contro un budget di 128 MB nel caso moodboard, 306 bitmap in cache (T17); i quattro difetti emersi dalla revisione chiusi, incluso ``originalLost`` per gli import compact (T18); documento come cartella ``document.json`` + assets (T19); e il formato attuale – **\*\*un solo file ``rnode`` (SQLite)\*\*** con documento e immagini in una transazione, senza file ``-wal`/`-shm`` (T20).

Decisioni e numeri: [ADR-001 §12](ADR-001-immagini-e-piattaforma.md).

Dettagli di esecuzione (archiviati): [archive/PIANO-IMMAGINI.md](archive/PIANO-IMMAGINI.md).

---

## Fuori roadmap (decisione del proprietario)

Non iniziare senza richiesta esplicita:

- **\*\*Misura testuale via DOM\*\*** (``LineLayout`` iniettabile con implementazione che misura nel DOM e batcha i reflow). È la soluzione strutturale definitiva alla parità, ma oggi l'harness è a 0 divergenze: costo alto, guadagno attualmente nullo.

```
- **Link nei `TextRun`**, liste numerate, allineamento per singolo blocco,
  note dei nodi in rich text, tabelle, RTL. Vedi §12 di
  [RICH_TEXT_EDITOR.md](RICH_TEXT_EDITOR.md).
- Qualunque voce elencata in [AGENT_GUIDE.md](AGENT_GUIDE.md) §5.
```

## 65. src/core/types.ts

```
/**
 * R-node — document schema (version 0.1).
 *
 * The schema is the single source of truth shared by every client (desktop,
 * web, mobile) and by the Rust document engine that will take over
 * persistence/layout on the desktop build. It is plain, serializable JSON.
 *
 * Design rules (from the product spec):
 * - IDs everywhere, never array indices, for references.
 * - Relationships are independent from parent/child hierarchy.
 * - Versioned schema with migrations.
 */

export const SCHEMA_VERSION = "0.1.0";

// -----
// Core enums
// -----

export type NodeType =
  | "central" // root of a mind-map sheet (one per sheet)
  | "main" // direct child of central topic
  | "subtopic" // any regular descendant
  | "floating" // free, unparented topic
  | "summary" // groups a range of topics
  | "callout"; // annotation box linked to a topic

export type StructureType =
  | "mindmap" // classic radial mind map
  | "logic" // top-to-bottom / left-to-right reasoning chart
  | "tree" // hierarchical tree
  | "org" // org chart
  | "timeline" // chronological
  | "fishbone" // Ishikawa
  | "matrix" // rows x columns
  | "treetable" // hierarchical rows + columns
  | "freeform"; // grid/free canvas

export type Orientation = "horizontal" | "vertical";

export type ConnectorStyle = "curved" | "straight" | "elbow";

export type TaskStatus = "not-started" | "in-progress" | "blocked" | "completed" | "cancelled";

export type Priority = "none" | "low" | "medium" | "high" | "urgent";

export type TopicShape =
  | "rounded"
  | "rect"
  | "capsule"
  | "circle"
  | "diamond"
  | "hexagon"
  | "cloud"
  | "underline"
  | "none";

// -----
// Style
// -----

export interface Style {
  fill?: string;
  stroke?: string;
  borderWidth?: number;
  borderStyle?: "solid" | "dashed" | "dotted";
  cornerRadius?: number;
  shape?: TopicShape;
  textColor?: string;
  fontSize?: number;
  fontWeight?: number;
  fontFamily?: string;
  italic?: boolean;
  underline?: boolean;
  strikethrough?: boolean;
  opacity?: number;
  shadow?: boolean;
}
```

```

icon?: string;
image?: string; // attachment id
/** Display width of the image in world units; height follows the aspect ratio. */
imageWidth?: number;
link?: string;
padding?: number;
align?: "left" | "center";
width?: number;
height?: number;
rotation?: number;
locked?: boolean;
hidden?: boolean;
}

export const DEFAULT_STYLE: Style = {
  fill: undefined, // → theme branch palette
  stroke: "transparent",
  borderWidth: 0,
  borderStyle: "solid",
  cornerRadius: 10,
  shape: "rounded",
  textColor: undefined, // → theme
  fontSize: 14,
  fontWeight: 400,
  italic: false,
  underline: false,
  strikethrough: false,
  opacity: 1,
  shadow: false,
  icon: undefined,
  padding: 10,
  align: "center",
};

// -----
// Rich text (topic titles)
// -----

/**
 * A styled segment of a topic title. The title is rendered as a sequence of
 * runs: plain text plus per-run emphasis and color. A missing color inherits
 * the theme/branch text color of the node.
 *
 * Block-level semantics (so pasted content keeps its spatial structure,
 * Draw.io-style): a run may open a paragraph gap, carry its own font size
 * (headings), or start a bullet-list item at a given nesting depth.
 */
export interface TextRun {
  text: string;
  bold?: boolean;
  italic?: boolean;
  underline?: boolean;
  /** CSS color; when absent the node/theme text color is used. */
  color?: string;
  /** Per-run font size in px (headings). Absent = node font size. */
  fontSize?: number;
  /** Extra vertical gap before this run's paragraph (past a block boundary). */
  paraGap?: boolean;
  /** >0 → this run starts a bullet-list item at this depth (1 = top level). */
  listIndent?: number;
}

// -----
// Task
// -----

export interface TaskInfo {
  status: TaskStatus;
  priority: Priority;
  progress: number; // 0..100, 0 = not tracked
  assignee?: string;
  startDate?: string; // ISO date
  dueDate?: string;
  durationDays?: number;
  description?: string;
}

// -----
// Node
// -----

export interface Position {
  x: number;
  y: number;
  manual: boolean; // true = user-placed (layout engine must preserve)
  /** Locks a direct child of the central topic while its descendants reflow. */
  branchFree?: boolean;
}

```

```

    /** Legacy optional displacement retained for document compatibility. */
    offsetX?: number;
    offsetY?: number;
}

export interface MindNode {
    id: string;
    type: NodeType;
    parentId: string | null;
    childrenIds: string[];
    /** Plain-text title – always kept in sync with titleRuns (single run). */
    title: string;
    /**
     * Styled segments of the title. When absent the title is a single plain
     * run; every title mutation (styled or not) updates both fields so all
     * existing consumers (search, export, outliner, tests) keep working.
     */
    titleRuns?: TextRun[];
    position: Position;
    style: Style;
    collapsed: boolean;
    labels: string[];
    markers: string[];
    notes: string;
    task: TaskInfo | null;
    metadata: { createdAt: string; updatedAt: string };
}

// -----
// Relationships (independent from hierarchy)
// -----

export interface Relationship {
    id: string;
    fromId: string;
    toId: string;
    label?: string;
    color?: string;
    lineStyle?: "solid" | "dashed" | "dotted";
    bidirectional?: boolean;
}

// -----
// Groups & summaries (drawn over the map, derived from member geometry)
// -----

export interface Group {
    id: string;
    /** Topic ids enclosed by the dashed boundary (usually siblings). */
    memberIds: string[];
    label?: string;
}

export interface Summary {
    id: string;
    /** Topic ids spanned by the brace (usually a contiguous sibling range). */
    memberIds: string[];
    label?: string;
}

// -----
// Structure config
// -----

export interface StructureConfig {
    structureType: StructureType;
    orientation: Orientation;
    spacing: number; // gap between levels
    branchSpacing: number; // gap between siblings
    padding: number;
    compactMode: boolean;
    autoBalance: boolean;
    /** Keep main topics fixed; only their internal subtopics are auto-laid out. */
    freePositioningBranches: boolean;
    allowManualPositioning: boolean;
    connectorStyle: ConnectorStyle;
}

export const DEFAULT_STRUCTURE: StructureConfig = {
    structureType: "mindmap",
    orientation: "horizontal",
    spacing: 180,
    branchSpacing: 14,
    padding: 18,
    compactMode: false,
    autoBalance: true,
    freePositioningBranches: false,

```

```

    allowManualPositioning: true,
    connectorStyle: "curved",
  });

// -----
// Attachments
// -----

/**
 * Metadata card for an image attached to the map, kept INSIDE the document.
 * The bytes never live here: they are stored in the AssetStore (IndexedDB),
 * addressed by content – `id` is the SHA-256 of the original file and the
 * key of the asset in the store. `Style.image` on a node references the same
 * id, so one asset can be shared by many nodes.
 */
export interface AttachmentInfo {
  id: string; // SHA-256 of the original bytes – the key in the AssetStore
  mime: string; // image/png | image/jpeg | image/gif | image/webp
  w: number; // intrinsic pixels of the ORIGINAL
  h: number;
  bytes: number; // weight of the original, shown to the user
  name?: string;
  alt?: string;
}

/**
 * The original bytes are gone: the asset came from a compact .rnode.zip
 * import, which carries only display levels (AGENT_GUIDE I11). A complete
 * export of this document exports the resized level and must say so.
 */
originalLost?: boolean;
}

// -----
// Sheet & Document
// -----

export interface Sheet {
  sheetId: string;
  title: string;
  structure: StructureConfig;
  rootNodeId: string;
  /** nodes keyed by id – the single source of truth for the tree */
  nodes: Record<string, MindNode>;
  relationships: Relationship[];
  boundaries: Group[];
  summaries: Summary[];
  callouts: unknown[];
  labels: string[];
  zones: unknown[];
  attachments: AttachmentInfo[];
  comments: unknown[];
  presentation: Record<string, unknown>;
}

export interface RnodeDocument {
  schemaVersion: string;
  documentId: string;
  title: string;
  createdAt: string;
  updatedAt: string;
  archived: boolean;
  pinned: boolean;
  settings: {
    theme: "light" | "dark";
    showOutliner: boolean;
    showInspector: boolean;
  };
  themeId: string;
  sheets: Sheet[];
}

```

## 66. src/ui/RichEditor.tsx

```

/**
 * RichEditor – the ONE HTML overlay for topic editing (Lexical).
 *
 * Architecture (hybrid scoped):
 * - The canvas renders every non-editing node as cached rich text;
 * - while editing, this single Lexical overlay owns the node (the renderer
 *   skips it, so no ghosting);
 * - at most one overlay exists at any time – starting a new edit unmounts
 *   the previous one (CanvasView keys it by editingId);
 * - Lexical's HistoryPlugin is intentionally absent: undo/redo lives ONLY in
 *   the store's op history. The editor is a pure draft generator that pushes
 *   TextRun[] into the store on every change (editingDraftRuns), and the

```

```

*   store applies the final setTitle op on commit.
* - Wheel pan/zoom keeps working while editing: CanvasView positions this
*   overlay from the store camera on every render, so the overlay stays
*   glued to the node even while the map moves under it.
*/
import { useEffect, useRef, useState, type CSSProperties } from "react";
import { LexicalComposer } from "@lexical/react/LexicalComposer";
import { ContentEditable } from "@lexical/react/LexicalContentEditable";
import { LexicalErrorBoundary } from "@lexical/react/LexicalErrorBoundary";
import { RichTextPlugin } from "@lexical/react/LexicalRichTextPlugin";
import { ListPlugin } from "@lexical/react/LexicalListPlugin";
import { useLexicalComposerContext } from "@lexical/react/LexicalComposerContext";
import { $getRoot, $getSelection, $isTextNode, COMMAND_PRIORITY_HIGH, FORMAT_TEXT_COMMAND, INSERT_TAB_COMMAND, PASTE_COMMAND, type LexicalEditor, type TextFormatType } from "lexical";
import { HeadingNode } from "@lexical/rich-text";
import { ListItemNode, ListNode } from "@lexical/list";
import { useStore } from "../editor/context";
import type { EditorStore } from "../editor/store";
import type { MindNode, TextRun } from "../core/types";
import { nodeRuns, plainToRuns } from "../core/text";
import { BLOCK_GAP_FACTOR, BULLET_WIDTH_EM, FONT_STACK, IMAGE_GAP, LINE_HEIGHT_FACTOR, MAX_IMAGE_W, TEXT_INSET } from "../layout/measure";
import { getAssetStore } from "../persist/assets";
import { editorStateToRuns, runsToParagraphNodes, setEditorRuns } from "../lexicalRuns";
import { htmlToRuns, sanitizeHtml } from "../pasteSanitizer";

const editorTheme = {
  textBold: "rnode-text-bold",
  textItalic: "rnode-text-italic",
  textUnderline: "rnode-text-underline",
  textStrikethrough: "rnode-text-strike",
};

export function RichEditor({
  node,
  style,
  scale,
  colors,
}): {
  node: MindNode;
  style: CSSProperties;
  scale: number;
  /** Fill + text color the canvas paints this node with (see Renderer.nodeColors). */
  colors?: { fill: string; text: string };
}: JSX.Element {
  const store = useStore();
  const config = {
    namespace: "rnode-topic",
    nodes: [ListNode, ListItemNode, HeadingNode],
    theme: editorTheme,
    onError: (err: Error) => console.error("rich editor error", err),
  };
  const ns = node.style;
  // The overlay is laid out in WORLD units and scaled as a whole with a CSS
  // transform: fonts, paddings, the border and the block gaps then match the
  // canvas pixel-for-pixel at every zoom. (Scaling each property separately
  // broke the WYSIWYG: heading font-size was absolute px and the padding was
  // screen px, so the editor text inflated and wrapped elsewhere.)
  const decoration = [ns.underline ? "underline" : null, ns.strikethrough ? "line-through" : null].filter(Boolean).join(" ");
  const inner: CSSProperties = {
    width: Math.max(1, ((style.width as number) ?? 60) / scale),
    height: Math.max(1, ((style.height as number) ?? 28) / scale),
    fontSize: ns.fontSize ?? 14,
    lineHeight: LINE_HEIGHT_FACTOR,
    textAlign: (ns.align === "left" ? "left" : "center") as CSSProperties["textAlign"],
    // Everything the canvas paints the node with, so double-clicking a topic
    // does not change how it looks: same face, same weight, same emphasis,
    // same fill, same resolved text color.
    fontFamily: ns.fontFamily ?? FONT_STACK,
    fontWeight: ns.fontWeight ?? 400,
    fontStyle: ns.italic ? "italic" : "normal",
    textDecoration: decoration.length > 0 ? decoration : undefined,
    background: colors?.fill,
    color: colors?.text,
    transform: `scale(${scale})`,
    transformOrigin: "0 0",
  };
  // Shared block gap: the canvas advances BLOCK_GAP_FACTOR × line-height at
  // every block boundary (paragraph end, list item end) – the overlay applies
  // the identical rule via this CSS variable (see .topic-rich-editable).
  (inner as Record<string, string | number>)[`--rnode-block-gap`] = `calc(${BLOCK_GAP_FACTOR} * ${LINE_HEIGHT_FACTOR}em)`;
  // Bullet column width, the same constant wrapRunLines indents list text by.
  (inner as Record<string, string | number>)[`--rnode-bullet-w`] = `${BULLET_WIDTH_EM}em`;
  // The editable overlays the node's box EXACTLY like the canvas draws the
  // text: same wrap width (boxW - pad*2 - TEXT_INSET), and the block is
  // centered vertically by .topic-rich-inner (justify-content: center) exactly
  // as the renderer centers the bitmap.
  const pad = node.style.padding ?? 10;

```

```

const editablePad: CSSProperties = {
  paddingTop: pad + 2,
  paddingBottom: pad + 2,
  paddingLeft: pad - 2,
  paddingRight: pad + TEXT_INSET - 2,
};

// A node image reserves its rect above the editable (T12-3): without it the
// box's text would sit where the image is and the box would jump at the
// double click. Geometry mirrors measureTopic/renderer: imgW from
// style.imageWidth (capped at MAX_IMAGE_W), imgH by aspect ratio, IMAGE_GAP
// below – all in world units (the inner is scaled as a whole).
const card = node.style.image ? store.sheet.attachments.find((a) => a.id === node.style.image) : undefined;
const hasImage = !!card && card.w > 0;
const imgW = hasImage ? node.style.imageWidth ?? Math.min(card!.w, MAX_IMAGE_W) : 0;
const imgH = hasImage ? (imgW * card!.h) / card!.w : 0;
const boxW = Math.max(1, ((style.width as number) ?? 60) / scale);

return (
  <div className="topic-rich-editor" style={{ left: style.left, top: style.top }}>
    <LexicalComposer initialConfig={config}>
      {/* Outside the scaled box on purpose: inside it the toolbar shrank
        with the zoom (unreadable at 40%, huge at 300%). */}
      <Toolbar />
      <div className="topic-rich-inner" style={inner}>
        {hasImage && <NodeImageBlock id={node.style.image!} imgW={imgW} imgH={imgH} left={(boxW - imgW) / 2} top={pad} />}
        <RichTextPlugin
          contentEditable={
            <ContentEditable
              className="topic-rich-editable"
              style={
                {
                  hasImage
                    ? {
                        ...editablePad,
                        // The text lives below the image, in the remaining
                        // region – same placement the canvas gives it.
                        position: "absolute",
                        top: imgH + IMAGE_GAP,
                        left: 0,
                        right: 0,
                        bottom: 0,
                      }
                    : editablePad
                }
              }
              ariaLabel="Edit topic"
              spellCheck={false}
              onBlur={() => store.commitEdit()}
            />
          }
          placeholder={null}
          ErrorBoundary={LexicalErrorBoundary}
        />
      </div>
      <ListPlugin />
      <DraftSyncPlugin store={store} node={node} />
      <PasteSanitizerPlugin />
      <KeysPlugin store={store} />
    </LexicalComposer>
  </div>
);
}

// -----
// Reserved image block (T12-3): non-editable, sized to the node's image rect,
// sitting above the editable exactly where the canvas draws the image.
// -----

function NodeImageBlock({ id, imgW, imgH, left, top }: { id: string; imgW: number; imgH: number; left: number; top: number }): JSX.Element {
  const [url, setUrl] = useState<string | null>(null);
  useEffect(() => {
    let cancelled = false;
    let created: string | null = null;
    void (async () => {
      // Resolved here, at use time: the store singleton must be picked in the
      // running environment, not at module scope (T18-D).
      const blob = await getAssetStore().get(id, "large");
      if (cancelled || !blob) return;
      created = URL.createObjectURL(blob);
      setUrl(created);
    })();
    return () => {
      cancelled = true;
      if (created) URL.revokeObjectURL(created);
    };
  }, [id]);
  return (
    <div

```

```

    contentEditable={false}
    style={{
      position: "absolute",
      top,
      left,
      width: imgW,
      height: imgH,
      display: "flex",
      alignItems: "center",
      justifyContent: "center",
      overflow: "hidden",
      borderRadius: 4,
    }}
  >
    {url && <img src={url} alt="" draggable={false} style={{ maxWidth: "100%", maxHeight: "100%", objectFit: "contain", display: "block" }} />}
  </div>
);
}

// -----
// Keys: capture-phase listener on the editor root so we run BEFORE Lexical's
// bubble-phase handlers (Enter must commit, not insert a newline).
// -----

function KeysPlugin({ store }: { store: EditorStore }): null {
  const [editor] = useLexicalComposerContext();
  useEffect(() => {
    const handler = (e: KeyboardEvent): void => {
      // Inline formatting: Ctrl/Cmd+B / I / U toggle bold / italic /
      // underline on the current selection (or the pending format for the
      // next keystroke). Handled here, in capture phase, so the browser's
      // own contentEditable defaults never get a chance.
      if ((e.ctrlKey || e.metaKey) && !e.altKey && !e.shiftKey) {
        const k = e.key.toLowerCase();
        const fmt: TextFormatType | null = k === "b" ? "bold" : k === "i" ? "italic" : k === "u" ? "underline" : null;
        if (fmt) {
          e.preventDefault();
          e.stopPropagation();
          editor.dispatchCommand(FORMAT_TEXT_COMMAND, fmt);
          return;
        }
      }
      if ((e.ctrlKey || e.metaKey) && e.key.toLowerCase() === "s") {
        // Save while editing: commit the draft (editor stays open) so the
        // browser "Save page" dialog never eats the keystroke.
        e.preventDefault();
        e.stopPropagation();
        void store.saveNow();
        return;
      }
      if (e.key === "Enter" && !e.shiftKey) {
        e.preventDefault();
        e.stopPropagation();
        store.commitEdit();
        return;
      }
      if (e.key === "Escape") {
        e.preventDefault();
        e.stopPropagation();
        store.cancelEdit();
        return;
      }
      if (e.key === "Tab") {
        // Keep focus inside the overlay; insert a literal tab char in the
        // title (matches the previous textarea behavior).
        e.preventDefault();
        e.stopPropagation();
        editor.dispatchCommand(INSERT_TAB_COMMAND, undefined);
        return;
      }
    };
    return editor.registerRootListener((root, prev) => {
      if (prev) prev.removeEventListener("keydown", handler, true);
      if (root) root.addEventListener("keydown", handler, true);
    });
  }, [editor, store]);
  return null;
}

// -----
// Draft sync: seed content once, then mirror every change to the store.
// -----

function DraftSyncPlugin({ store, node }: { store: EditorStore; node: MindNode }): null {
  const [editor] = useLexicalComposerContext();
  const seeded = useRef(false);

```

```

// Debug handle for driving the editor from the console / preview.
useEffect(() => {
  (window as unknown as { __rnodeEditor?: LexicalEditor }).__rnodeEditor = editor;
  return () => {
    const w = window as unknown as { __rnodeEditor?: LexicalEditor };
    if (w.__rnodeEditor === editor) delete w.__rnodeEditor;
  };
}, [editor]);

// Update listener – lives in its OWN effect so it survives React
// StrictMode's mount → cleanup → mount cycle (and any re-render): the
// `seeded` guard in the seeding effect below would otherwise skip the
// re-registration and the draft would never follow keystrokes.
useEffect(() => {
  return editor.registerUpdateListener(({ editorState }) => {
    store.setEditingDraftRuns(editorStateToRuns(editorState));
  });
}, [editor, store]);

// Seeding – guarded so it only ever runs once per overlay instance.
useEffect(() => {
  if (seeded.current) return;
  seeded.current = true;
  // Type/paste-to-edit hands us initial content that replaces the title.
  const pending = store.consumePendingInsert();
  const initial = pending !== null ? plainToRuns(pending) : nodeRuns(node.title, node.titleRuns);
  setEditorRuns(editor, initial);
  store.setEditingDraftRuns(initial);
  const focusTimer = setTimeout(() => editor.focus(), 0);
  return () => clearTimeout(focusTimer);
}, [editor, store, node]);

return null;
}

// -----
// Paste sanitization: Word / Google Docs / Draw.io / web.
// -----

function PasteSanitizerPlugin(): null {
  const [editor] = useLexicalComposerContext();

  useEffect(() => {
    return editor.registerCommand(
      PASTE_COMMAND,
      (event) => {
        // PasteCommandType payload differs across Lexical versions; read the
        // clipboard defensively.
        const payload = (event ?? {}) as unknown as { clipboardData?: DataTransfer | null };
        if (!payload.clipboardData) return false;
        const html = payload.clipboardData.getData("text/html");
        if (!html) return false; // plain-text paste → Lexical default
        event.preventDefault();
        const cleaned = sanitizeHtml(html);
        editor.update(() => {
          // ALWAYS route through our run pipeline (not Lexical's DOM import):
          // $insertNodes converts HeadingNode to paragraph at a collapsed
          // selection, while plain paragraphs/lists survive – and our runs
          // carry heading size as inline font-size, paragraph gaps and list
          // indent, so the canvas renders the exact pasted structure.
          const paragraphs = runsToParagraphNodes(htmlToRuns(cleaned));
          const sel = $getSelection();
          if (sel) sel.insertNodes(paragraphs);
          else $getRoot().append(...paragraphs);
        });
        return true;
      },
      COMMAND_PRIORITY_HIGH
    );
  }, [editor]);

  return null;
}

// -----
// Mini formatting toolbar (B / I / color) – stays inside the overlay.
// -----

function Toolbar(): JSX.Element {
  const [editor] = useLexicalComposerContext();
  return (
    <div className="topic-rich-toolbar" onMouseDown={(e) => e.preventDefault()} onPointerDown={(e) => e.stopPropagation()}>
      <button
        type="button"
        title="Bold (Ctrl+B)"
        onMouseDown={(e) => {
          e.preventDefault();

```

```

        editor.dispatchCommand(FORMAT_TEXT_COMMAND, "bold");
    })
    >
    <b>B</b>
</button>
<button
  type="button"
  title="Italic (Ctrl+I)"
  onMouseDown={(e) => {
    e.preventDefault();
    editor.dispatchCommand(FORMAT_TEXT_COMMAND, "italic");
  }}
>
  <i>I</i>
</button>
<button
  type="button"
  title="Underline (Ctrl+U)"
  onMouseDown={(e) => {
    e.preventDefault();
    editor.dispatchCommand(FORMAT_TEXT_COMMAND, "underline");
  }}
>
  <u>U</u>
</button>
<input
  type="color"
  title="Text color"
  defaultValue="#d43a3a"
  onMouseDown={(e) => e.preventDefault()}
  onChange={(e) => applyColor(editor, e.target.value)}
/>
<button
  type="button"
  title="Clear color"
  onMouseDown={(e) => {
    e.preventDefault();
    clearColor(editor);
  }}
>
  <span style={{ textDecoration: "line-through" }}>A</span>
</button>
</div>
);
}

function applyColor(editor: LexicalEditor, color: string): void {
  editor.update(() => {
    const sel = $getSelection();
    if (!sel) return;
    for (const n of sel.getNodes()) {
      if ($isTextNode(n)) n.setStyle(`color: ${color}`);
    }
  });
}

function clearColor(editor: LexicalEditor): void {
  editor.update(() => {
    const sel = $getSelection();
    if (!sel) return;
    for (const n of sel.getNodes()) {
      if ($isTextNode(n)) n.setStyle("");
    }
  });
}

// Re-export the run type so callers stay type-safe.
export type { TextRun };

```

## 67. README.md

```

# R-node

**A cross-platform visual thinking and mind-mapping workspace.**

R-node is a keyboard-first mind-mapping application designed for brainstorming,
knowledge organization, project planning and presentations. It is built
original from the ground up – brand, design system and code are our own, and
nothing is copied from other mind-mapping products.

- **Fast** – single-canvas rendering, no DOM or SVG element per topic; smooth
  at 10,000+ topics.
- **Keyboard-first** – Enter/Tab/Shift+Tab to create and structure topics,
  arrows to navigate, every action undoable.

```

- **Local-first** – documents save when you decide (Ctrl+S, no hidden autosave). On the web they live in localStorage; on desktop each document is **one .rnode file** (a SQLite database holding the document **and** its images).
- **Images in nodes** – drop or paste an image onto a topic, resize it with a slider or a canvas handle, and share the map as a single **.rnode.zip**. Originals are preserved at full resolution; the canvas only ever decodes pre-scaled levels inside a byte budget.
- **Rich text on the canvas** – bold, italic, colours, headings and lists live in the topic itself, drawn by the canvas and edited in an overlay that matches it pixel for pixel (see [Text](#text-rich-text-on-a-canvas)).
- **Extensible by design** – a pure, framework-free core (document model, operation system, layout engine) that web, desktop and mobile clients share.

---

## ## Status

**Phase 1** (core editor) is implemented and tested; node images and the desktop app have landed. **188** unit tests passing across 13 suites, typecheck clean, performance-verified at 10k topics, editor/canvas text parity measured at **0** divergences (16/16 cases).

```
| | |
|---|---|
| Document model | versioned JSON schema, tree with stable IDs, multi-sheet schema (first sheet active) |
| Operation system | every edit is an idempotent, replayable op with an inverse – undo/redo for free, collaboration-ready |
| Rendering | single <canvas>, viewport culling, shapes, curved connectors, relationships, marquee selection, PNG export |
| Layout | mind map with balanced branches, exact measured extents, no overlap by construction |
| Text | rich text per topic (TextRun[]): bold/italic/underline/strikethrough, colours, heading sizes, nested bullet lists – drawn by the canvas,
edited in a single Lexical overlay |
| Editing | create / sibling / child / promote, rename, delete, duplicate, copy/paste/cut, drag-and-drop reparenting, type-to-edit |
| Structure ops | collapse/expand, sort siblings, tasks, notes, styles, relationships (arrows), groups, summaries |
| Images | attach by drop/paste, three stored levels (original / 1024px / 256px), decode-at-paint-size with a byte-budgeted bitmap cache, resize slider
+ canvas handle, select / delete / reassign |
| Workspace | outliner, inspector, command palette, search, light theme, Zen mode, manual save |
| Import/export | JSON, Markdown, PNG; .rnode.zip with images (complete or compact); clipboard import sanitized from Word / Google Docs / Draw.io |
| Desktop | Tauri 2 shell; each document is one .rnode SQLite file (document + assets in a single transaction), native open/save dialogs, drag files
from the OS onto topics |
```

---

## ## Quick start

Requirements: [Node.js](https://nodejs.org) 18+ (npm).

```
```bash
npm install
npm run dev      # start dev server → http://localhost:5173
npm test         # run the test suite (vitest)
npm run typecheck # TypeScript check only
npm run build    # typecheck + production build → dist/
```
```

The dev server also serves the **text parity harness** at `http://localhost:5173/dev/parity.html`, which must report **0** diverging cases (see [Text](#text-rich-text-on-a-canvas)).

Open `http://localhost:5173` and start typing – select the central topic and press **Tab** to add the first branch. The sample roadmap document shows the main interactions.

## ### Keyboard reference

```
| Keys | Action |
|---|---|
| `Enter` | create sibling topic |
| `Tab` | create child topic |
| `Shift+Tab` | promote topic (move up a level) |
| `Arrow keys` | navigate between topics |
| `F2` / double-click | edit topic text |
| `Space` | collapse / expand branch |
| `Delete` / `Backspace` | delete branch (or the selected image / relationship / group) |
| `Ctrl/Cmd+Z` / `Ctrl/Cmd+Shift+Z` / `Ctrl/Cmd+Y` | undo / redo |
| `Ctrl/Cmd+D` | duplicate topic |
| `Ctrl/Cmd+C` / `X` / `V` | copy / cut / paste |
| `Ctrl/Cmd+Shift+C` | copy selection as an outline |
| `Ctrl/Cmd+K` | command palette |
| `Ctrl/Cmd+F` | search |
| `Ctrl/Cmd+S` | save now |
| `Ctrl/Cmd+O` | open a document |
| `Ctrl/Cmd+E` | export JSON |
| `Ctrl/Cmd+Enter` | toggle task complete |
| `Ctrl/Cmd+I` / `0` | fit map to view |
| `Ctrl/Cmd+=` / `-` | zoom in / out |
| `Ctrl/Cmd+Shift+F` | Zen (focus) mode |
| `Escape` | close editor / clear selection |
```

```
| Drag | move / reparent (drop indicator shows child vs sibling) |
| Scroll / `Ctrl+scroll` / middle-drag | pan / zoom / pan |
```

Shortcuts live in a single profile object (`src/editor/shortcuts.ts`) so they can be rebound without touching the editor logic.

---

## ## Architecture

Documentation map:

```
| Document | What it is for |
|---|---|
| [docs/ARCHITECTURE.md](docs/ARCHITECTURE.md) | full design document: PRD, information architecture, schema, API design, sync and layout strategy, security model, phased plan, risk register |
| [docs/AGENT_GUIDE.md](docs/AGENT_GUIDE.md) | **read before changing code** – invariants, the editor+canvas parity contract, traps already paid for, definition of done |
| [docs/ROADMAP.md](docs/ROADMAP.md) | the ordered task list, one task at a time |
| [docs/RICH_TEXT_EDITOR.md](docs/RICH_TEXT_EDITOR.md) | how rich text is modelled, measured, drawn and edited |
| [docs/ADR-001-immagini-e-piattaforma.md](docs/ADR-001-immagini-e-piattaforma.md) | decision record: where images live, the decode-memory budget, and the desktop platform choice (A1+B1) |
| [docs/ELEMENTI_VISUALI.md](docs/ELEMENTI_VISUALI.md) | reference sheet: every modifiable visual element (theme, node, sheet, text), every GUI element, logo/branding assets |
```

The code is split into clean, framework-free layers – only the UI uses React:

---

```
src/
├─ core/      document schema, model + tree walks, operation system, history
├─ layout/    measurement + placement engines (pure math, fully testable)
├─ render/    camera/viewport, canvas renderer, themes, hit-testing, PNG export
├─ editor/    store (orchestration), keyboard shortcuts, context, export bridge,
              image-import pipeline (Web Worker)
├─ persist/   storage adapters (localStorage on web; `.rnode` SQLite via Tauri)
              + content-addressed asset store (IndexedDB on web, SQLite on desktop)
├─ ui/        React shell: canvas, sidebar, topbar, outliner, inspector, palette
dev/          text parity harness + session tracer (development only, not shipped)
```

## ### Document model & operations

The editor never mutates the document directly. Every change is an **\*\*Op\*\***:

```
``ts
{ opId, actorId, ts, type, ...payload }
``
```

Ops are self-contained, idempotent and carry everything needed to reverse themselves. The store applies them through a history layer that batches multi-step gestures (e.g. create + rename) into a single undo step. Because every edit is already an op with an actor and timestamp, the same stream is the future synchronization unit (OT/CRDT) – no rewrite needed when collaboration lands.

## ### Layout engine

Pure functions in `src/layout/`:

- **\*\*Shared measurement\*\*** – one measurer (canvas-backed in the app, a deterministic heuristic in tests) drives both layout and paint, so a topic can never be sized differently by the two.
- **\*\*Subtree blocks\*\*** – every topic's extent is its own box plus its descendants; collapsed branches count as leaves.
- **\*\*Straddle distribution\*\*** – children split above/below the parent box, balanced by block height, with guaranteed gaps: no overlaps, ever.
- **\*\*Size-aware balance\*\*** – root branches split left/right by subtree height, not raw count.
- **\*\*Upward propagation\*\*** – any deep change recomputes the map (debounced) and rebalances up to the root.

Manual positions are preserved; layout is derived data and never pollutes undo history.

## ### Text: rich text on a canvas

A topic title is not a string but a sequence of styled runs (`TextRun[]`), so one topic can carry bold, italics, colours, heading sizes and nested bullet lists. Keeping that on a single canvas means two renderers must agree:

- **\*\*the canvas\*\*** draws every topic at rest, with a per-topic bitmap cache so pan and zoom only blit;
- **\*\*one Lexical overlay\*\*** – never more than one – owns the topic being edited, and the canvas skips it so nothing is drawn twice.

The two share the measurement (`wrapRunLines`), the constants (`LINE\_HEIGHT\_FACTOR`, `BLOCK\_GAP\_FACTOR`, `BULLET\_WIDTH\_EM`, `FONT\_STACK`),

`TEXT\_INSET`) and the resolved colours – the overlay wears the node's own fill and text colour, so double-clicking a topic does not change how it looks.

Because the canvas \*imitates\* the browser's text layout, that agreement is **measured, not assumed**: `dev/parity.html` renders a corpus of cases into the real editable DOM, extracts the browser's true line boxes character by character, and diffs them against the canvas measurement – break points, baselines, line advances, lefts and total height. Current state: 16 of 16 cases aligned, worst residual 0.5px.

Pasting from Word, Google Docs or Draw.io is sanitized into that model: structure and emphasis survive, scripts, layout CSS and foreign fonts do not.

Full detail in [docs/RICH\_TEXT\_EDITOR.md](docs/RICH\_TEXT\_EDITOR.md).

### ### Images in nodes

An image attaches to a topic (drop it on the node, or paste with a topic selected) and is drawn above the text inside the same box. On import the original is stored **intact** plus two derived levels (1024px and 256px on the long side); the canvas decodes only the level closest to what is actually painted, inside a byte-budgeted LRU cache, so a map with hundreds of images stays fluid. Resize with the Inspector slider or the canvas handle (proportions locked); select, delete or reassign an image without touching the node. The decision behind this – storage, decode-memory budget, and why the desktop is the primary home for images – is recorded in [docs/ADR-001-immagini-e-piattaforma.md](docs/ADR-001-immagini-e-piattaforma.md).

### ### Performance

Verified with a permanent perf suite (`tests/perf.test.ts`). At 10k topics ops stay linear (~0.005-0.008 ms/op); the numbers the suite prints on each run are the reference – it reports `applyOps`, `layout`, `writeback` and tree walks separately at 1k / 5k / 10k so a regression is attributable to one subsystem. The real constraint at scale is **web** storage (a 10k map is ~3.6 MB in localStorage); on desktop the document is a single `.rnode` SQLite file, and images live in a content-addressed store behind a byte-budgeted decode cache (measured at ~17 MB peak for 306 on-screen bitmaps, against a 128 MB budget – see ADR-001 §12).

---

### ## Desktop app (Tauri 2 + Rust)

R-node runs as a native desktop app. Each document is **one** `.rnode` file – a SQLite database that holds the document JSON **and** its images, committed in a single transaction, so "a document" is a single file on disk (no folder, no sidecars). Open and save use native dialogs, and you can drag image files from the OS onto topics. The shell is compiled and working on this machine.

Requirements: [Rust](https://rustup.rs) (stable) and the Tauri CLI (`cargo install tauri-cli --locked`), plus `npm install`. WebView2 is built into Windows 11.

```
``bash
cargo tauri dev      # dev build – starts Vite and opens the desktop window
cargo tauri build    # release build → installer in src-tauri/target/release/
``
```

The web target keeps working unchanged (`npm run dev`) – it persists to localStorage and shares all the same code; only the storage adapter differs. Documents without images export as `.rnode.json`; maps with images share as a single `.rnode.zip` (complete = originals, compact = display levels only).

---

### ## Testing

```
``bash
npm test
``
```

188 tests across 13 suites: document operations, undo/redo round-trips, reparenting and subtree moves, layout geometry (no-overlap, straddle, size-aware balance, upward propagation), text measurement (line boxes, block gaps, bullet columns, mid-word breaks), the Lexical **↻** runs bridge (including round-trip idempotence), clipboard sanitization, viewport math, the asset store and image import pipeline, the Tauri storage adapter, export (including `.rnode.zip` round-trips), and performance ceilings at 1k/5k/10k topics.

Text parity between the canvas and the editor is checked separately, in a real browser, at `http://localhost:5173/dev/parity.html` – jsdom cannot do it because it has no layout engine.

---

### ## Reporting a bug

Don't describe the symptom – capture it. While running `npm run dev` the app records its own decisions in a rolling buffer.

**\*\*When something looks wrong, press the ● button in the toolbar\*\*** (or `Ctrl+Shift+D`, which also asks what you expected). A `node-trace-*.json` file downloads. Attach that file. It carries the last few hundred events: which inputs were acted on, which were deliberately ignored *\*and why\**, what the renderer actually drew each frame, how long layout took, and any errors – with the events that preceded them.

Both capture **\*\*and then reset\*\***, in that order: by the time you reach for the button the bug has already happened, so clearing first would throw away the evidence. Because the buffer restarts, each capture covers only what happened since the previous one – hit it once to get a clean slate, reproduce, hit it again, and the second file contains the repro and nothing else.

Console entry points: `__nodeTrace.capture()` returns the same bundle as an object (handy for inspecting in place), `__nodeTrace.download()` skips the prompt, `__nodeTrace.clear()` resets, `__nodeTrace.enabled` tells you whether recording is on.

The tracer is **\*\*development-only\*\*** – it is compiled out of production builds, so a `npm run build` bundle records nothing.

Why this instead of a written report: the decisive facts are invisible from outside the app. "The connector lines disappear when I pan" cannot tell anyone whether the renderer skipped them or drew them where you could not see them – two different bugs. A capture answers that in one line. How to read one is in [docs/AGENT\_GUIDE.md](docs/AGENT\_GUIDE.md) §4bis.

---

## ## Roadmap

- **\*\*Phase 1 – Core editor (done):\*\*** document model, ops + undo/redo, canvas renderer, mind-map layout, keyboard + mouse editing, drag-and-drop, marquee, outliner, inspector, palette, manual save, rich text topics with measured editor/canvas parity, JSON/Markdown/PNG export.
- **\*\*Images & desktop (done):\*\*** content-addressed asset store, drop/paste import in a Web Worker, decode-at-paint-size with a byte budget, resize slider + handle, portable `node.zip`, Tauri bring-up, and single-file `node` (SQLite) desktop documents. (ROADMAP tasks T12a-T20.)
- **\*\*Phase 2 – Visual richness (in progress):\*\*** markers, labels, rich notes, boundaries, callouts.
- **\*\*Phase 3 – Structures:\*\*** logic chart, tree chart, org chart, timeline, fishbone/Ishikawa, matrix, tree table, mixed structures.
- **\*\*Phase 4 – Productivity:\*\*** Gantt, full-text search+, templates, presentation mode, Zen polish, incremental layout (stable positions for untouched branches).
- **\*\*Phase 5 – Collaboration:\*\*** accounts, sharing, comments, real-time op sync, version history, offline sync.
- **\*\*Phase 6 – AI:\*\*** provider abstraction, map-from-prompt/text, expand, summarize, restructure, task extraction, map chat – with previews, diffs and privacy controls.
- **\*\*Phase 7 – Ecosystem:\*\*** full import/export (SVG, PDF, DOCX, PPTX, OPML, FreeMind...), plugins, mobile clients.

Each phase ends with working code, tests, docs, accessibility + performance checks, and a list of known limitations.

## 68. src/styles.css

```
/* =====
R-node design system
===== */

:root {
  --accent: #4f46e5;
  --accent-soft: rgba(79, 70, 229, 0.14);
  --radius: 8px;
  --radius-sm: 6px;
  --font: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  --mono: "SF Mono", "Cascadia Code", Consolas, monospace;
}

:root[data-theme="light"] {
  --bg: #ffffff;
  --surface: #ffffff;
  --surface-2: #f8fafc;
  --border: #e5e7eb;
  --text: #1c2333;
  --text-muted: #6b7280;
  --accent: #4f46e5;
```

```

--accent-soft: rgba(79, 70, 229, 0.1);
--danger: #dc2626;
--shadow: 0 10px 30px rgba(28, 35, 51, 0.16);
}

* {
  box-sizing: border-box;
}

html,
body,
#root {
  height: 100%;
  margin: 0;
}

body {
  font-family: var(--font);
  color: var(--text);
  background: var(--bg);
  overflow: hidden;
  font-size: 14px;
}

button {
  font-family: inherit;
  cursor: pointer;
}

input,
select,
textarea {
  font-family: inherit;
  color: var(--text);
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius-sm);
  padding: 5px 8px;
  font-size: 13px;
}

input:focus,
select:focus,
textarea:focus {
  outline: 2px solid var(--accent);
  outline-offset: -1px;
}

/* ----- App shell ----- */

.app {
  display: grid;
  grid-template-columns: 264px 1fr;
  height: 100vh;
}

.main {
  display: flex;
  flex-direction: column;
  min-width: 0;
}

.workspace {
  flex: 1;
  display: grid;
  grid-template-columns: 1fr;
  min-height: 0;
}

.workspace:has(.inspector) {
  grid-template-columns: 1fr 292px;
}

.canvas-area {
  position: relative;
  min-width: 0;
  min-height: 0;
}

/* ----- Sidebar ----- */

.sidebar {
  background: var(--surface);
  border-right: 1px solid var(--border);
  display: flex;
  flex-direction: column;
  min-height: 0;
}

```

```

}

.brand {
  display: flex;
  align-items: center;
  gap: 9px;
  padding: 14px 16px;
  font-weight: 700;
  font-size: 16px;
  letter-spacing: -0.01em;
}

.brand-dot {
  width: 14px;
  height: 14px;
  border-radius: 50%;
  background: conic-gradient(from 0deg, var(--accent), #db2777, #16a34a, var(--accent));
}

.sidebar-search {
  padding: 0 12px 10px;
}

.sidebar-search input {
  width: 100%;
}

.sidebar-actions {
  display: flex;
  gap: 8px;
  padding: 0 12px 12px;
}

.sidebar-actions .btn {
  flex: 1;
}

.sidebar-list {
  flex: 1;
  overflow-y: auto;
  padding: 4px 8px;
}

.sidebar-label {
  font-size: 11px;
  text-transform: uppercase;
  letter-spacing: 0.08em;
  color: var(--text-muted);
  padding: 8px 8px 6px;
}

.doc-row {
  display: flex;
  align-items: center;
  gap: 6px;
  padding: 7px 8px;
  border-radius: var(--radius-sm);
  cursor: pointer;
  user-select: none;
}

.doc-row:hover {
  background: var(--surface-2);
}

.doc-row.active {
  background: var(--accent-soft);
}

.doc-title {
  flex: 1;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.doc-row-actions {
  display: none;
  gap: 2px;
}

.doc-row:hover .doc-row-actions {
  display: flex;
}

.doc-row-actions button {
  border: none;
}

```

```

background: none;
color: var(--text-muted);
font-size: 13px;
padding: 2px 4px;
border-radius: 4px;
}

.doc-row-actions button:hover {
background: var(--surface-2);
color: var(--text);
}

.doc-rename {
width: 100%;
margin: 2px 0;
}

.sidebar-empty {
padding: 12px;
color: var(--text-muted);
font-size: 13px;
}

.sidebar-footer {
padding: 10px 12px;
border-top: 1px solid var(--border);
color: var(--text-muted);
font-size: 11.5px;
line-height: 1.5;
}

/* ----- Buttons ----- */

.btn {
border: 1px solid var(--border);
background: var(--surface);
color: var(--text);
border-radius: var(--radius-sm);
padding: 5px 11px;
font-size: 13px;
transition: background 0.12s ease, border-color 0.12s ease;
}

.btn:hover:not(:disabled) {
background: var(--surface-2);
}

.btn.disabled {
opacity: 0.4;
cursor: default;
}

.btn.primary {
background: var(--accent);
border-color: var(--accent);
color: #fff;
}

.btn.icon {
width: 30px;
height: 30px;
padding: 0;
display: inline-flex;
align-items: center;
justify-content: center;
font-size: 15px;
}

.btn.icon.on {
background: var(--accent-soft);
color: var(--accent);
}

.btn.small {
padding: 3px 8px;
font-size: 12px;
}

.btn.small.danger {
color: var(--danger);
}

/* ----- Top bar ----- */

.topbar {
display: flex;
align-items: center;

```

```

    gap: 14px;
    padding: 7px 12px;
    background: var(--surface);
    border-bottom: 1px solid var(--border);
}

.doc-title-input {
    font-weight: 600;
    font-size: 15px;
    border: 1px solid transparent;
    background: transparent;
    width: 220px;
}

.doc-title-input:hover,
.doc-title-input:focus {
    border-color: var(--border);
    background: var(--surface);
}

.topbar-group {
    display: flex;
    align-items: center;
    gap: 4px;
}

.save-status {
    font-size: 12px;
    color: var(--text-muted);
    margin-left: 6px;
}

.save-status.dirty {
    color: #f59e0b;
}

.statusbar .dirty {
    color: #f59e0b;
}

.zoom-group .zoom-pct {
    min-width: 58px;
    text-align: center;
    font-variant-numeric: tabular-nums;
}

.menu-wrap {
    position: relative;
}

.menu {
    position: absolute;
    top: calc(100% + 6px);
    right: 0;
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: var(--radius);
    box-shadow: var(--shadow);
    min-width: 200px;
    padding: 6px;
    z-index: 40;
    display: flex;
    flex-direction: column;
}

.menu button {
    border: none;
    background: none;
    color: var(--text);
    text-align: left;
    padding: 8px 10px;
    border-radius: var(--radius-sm);
    font-size: 13px;
}

.menu button:hover {
    background: var(--surface-2);
}

.menu-sep {
    height: 1px;
    background: var(--border);
    margin: 5px 4px;
}

/* ----- Status bar ----- */

```

```

.statusbar {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 4px 12px;
  background: var(--surface);
  border-top: 1px solid var(--border);
  font-size: 12px;
  color: var(--text-muted);
}

.statusbar .spacer {
  flex: 1;
}

.statusbar .sep {
  opacity: 0.5;
}

.status-hint {
  color: var(--accent);
  font-weight: 600;
}

/* ----- Inspector ----- */

.inspector {
  border-left: 1px solid var(--border);
  background: var(--surface);
  overflow-y: auto;
  padding: 14px;
}

.inspector-section {
  padding-bottom: 14px;
  margin-bottom: 14px;
  border-bottom: 1px solid var(--border);
}

.inspector-section:last-child {
  border-bottom: none;
  margin-bottom: 0;
}

.inspector-label {
  font-size: 11px;
  text-transform: uppercase;
  letter-spacing: 0.08em;
  color: var(--text-muted);
  margin-bottom: 8px;
}

.inspector-title {
  width: 100%;
  font-weight: 600;
  margin-bottom: 8px;
}

.inspector-actions {
  display: flex;
  flex-wrap: wrap;
  gap: 5px;
}

.field {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 7px;
}

.field > span:first-child {
  width: 86px;
  font-size: 12.5px;
  color: var(--text-muted);
  flex-shrink: 0;
}

.field input[type="range"] {
  flex: 1;
}

.field input[type="number"] {
  width: 64px;
}

.field select {

```

```

    flex: 1;
}

.field input[type="color"] {
    width: 42px;
    height: 26px;
    padding: 1px;
}

.branch-free-position {
    margin-top: 10px;
    padding: 7px 8px;
    border-radius: var(--radius-sm);
    background: var(--accent-soft);
}

.branch-free-position > span:first-child {
    width: auto;
    flex: 1;
    color: var(--text);
    font-weight: 600;
}

.notes-input {
    width: 100%;
    min-height: 84px;
    resize: vertical;
}

.muted {
    color: var(--text-muted);
    font-size: 12px;
}

.count {
    margin-top: 8px;
}

.inspector-empty {
    color: var(--text-muted);
    padding: 4px 0 16px;
    line-height: 1.5;
}

/* ----- Canvas ----- */

.canvas-wrap {
    position: absolute;
    inset: 0;
    overflow: hidden;
}

.canvas {
    position: absolute;
    inset: 0;
    width: 100%;
    height: 100%;
    display: block;
    touch-action: none;
}

/* Ghost preview of an image dragged from outside (Explorer / browser):
   follows the cursor over the map until the drop lands. Pure preview –
   never intercepts the pointer, so the drop still reaches the canvas. */
.ext-drop-ghost {
    position: absolute;
    z-index: 30;
    pointer-events: none;
    width: 150px;
    height: 110px;
    object-fit: contain;
    border: 2px dashed var(--accent);
    border-radius: 8px;
    background: var(--surface);
    box-shadow: var(--shadow);
    opacity: 0.9;
}

/* Rich-text topic editor (Lexical overlay – exactly one at any time).
   The OUTER box only positions; the INNER box is laid out in world units and
   scaled with a transform, so fonts, paddings, border and gaps match the
   canvas pixel-for-pixel at every zoom. */
.topic-rich-editor {
    position: absolute;
    z-index: 20;
    overflow: visible;
}

```

```

.topic-rich-inner {
  position: relative;
  transform-origin: 0 0;
  border: 2px solid var(--accent);
  border-radius: 6px;
  /* background/color come from RichEditor: while editing, the overlay must
     wear the NODE's fill and resolved text color, not the app surface – the
     canvas draws it that way at rest. */
  background: var(--surface);
  color: var(--text);
  box-shadow: var(--shadow);
  display: flex;
  flex-direction: column;
  justify-content: center;
  min-width: 60px;
  min-height: 28px;
  overflow: visible;
}

/* Floating above the box (Xmind-style): the box height is measured for the
   TEXT only, so the toolbar must not consume any of it – otherwise the text
   area shrinks and the content scrolls/clips while the node view fits it.
   It is a sibling of .topic-rich-inner, NOT a child: inside the scaled box it
   inherited transform: scale(zoom) and became unreadable when zoomed out. */
.topic-rich-toolbar {
  position: absolute;
  top: -32px;
  left: 0;
  display: flex;
  align-items: center;
  gap: 2px;
  padding: 2px 4px;
  border: 1px solid var(--border);
  background: var(--surface);
  border-radius: 6px;
  box-shadow: var(--shadow);
  user-select: none;
  white-space: nowrap;
}

.topic-rich-toolbar button {
  background: none;
  border: none;
  border-radius: 3px;
  padding: 1px 6px;
  cursor: pointer;
  color: var(--text);
  font-size: 12px;
  line-height: 1.4;
}

.topic-rich-toolbar button:hover {
  background: var(--hover);
}

.topic-rich-toolbar input[type="color"] {
  width: 22px;
  height: 18px;
  padding: 0;
  border: 1px solid var(--border);
  border-radius: 3px;
  background: none;
  cursor: pointer;
}

.topic-rich-editable {
  /* Natural height, vertically centered by the flex parent – the canvas draws
     the text block centered in the node box too (startY = mid – totalH/2).
     With flex: 1 the block sat at the top instead, which drifted on every
     shape that inflates the box (diamond/hexagon/circle) and after a resize. */
  flex: 0 0 auto;
  outline: none;
  /* NEVER scroll: a scrollbar would steal width and re-wrap the text mid-typing,
     the one thing the canvas can never reproduce. The box already grows with
     the draft. */
  overflow: visible;
  /* matches the canvas, which now breaks a token that cannot fit its column */
  overflow-wrap: break-word;
  font-family: inherit;
}

.topic-rich-editable p {
  margin: 0;
}

/* WYSIWYG with the canvas: the same vertical rhythm the renderer uses.

```

```

    Every block boundary (paragraph end AND list item end) gets the shared
    --rnode-block-gap (BLOCK_GAP_FACTOR × line-height, set by RichEditor from
    the same constants the canvas uses), and lists are compact – no
    browser-default ul/ol margins or 40px indentation. */
.topic-rich-editable ul,
.topic-rich-editable ol {
    margin: 0;
    padding-left: 0;
    /* The UA marker cannot be used: its width is UA-defined, so no CSS length
       matches it and the canvas could never place a wrapped item's text at the
       same x. We draw the bullet ourselves in a column of known width. */
    list-style: none;
    text-indent: 0;
}

/* Bullet column: the marker occupies exactly --rnode-bullet-w
   (BULLET_WIDTH_EM em, the same constant wrapRunLines indents by), and the
   negative text-indent pulls the first line back into it so continuation
   lines hang under the text. Nesting comes from the PARENT item's
   padding-left, so nested <ul> must stay at padding-left: 0. */
.topic-rich-editable li {
    margin: 0;
    padding-left: var(--rnode-bullet-w, 1.2em);
    text-indent: calc(-1 * var(--rnode-bullet-w, 1.2em));
    /* centering a hanging indent is ill-defined – both sides pin lists to left */
    text-align: left;
}

.topic-rich-editable li::before {
    content: "•";
    display: inline-block;
    width: var(--rnode-bullet-w, 1.2em);
}

.topic-rich-editable li li::before {
    content: "◦";
}

.topic-rich-editable li li li::before {
    content: "•";
}

/* Every block boundary gets the gap: between top-level blocks, between
   sibling items, AND before a nested list – whose first item is :first-child
   of its own <ul> and so matched none of the other two rules, leaving a
   nested item 10.5px higher than the canvas drew it. */
.topic-rich-editable > :not(:first-child),
.topic-rich-editable li + li,
.topic-rich-editable li > ul,
.topic-rich-editable li > ol {
    margin-top: var(--rnode-block-gap, 0.75em);
}

.rnode-text-bold {
    font-weight: 700;
}

.rnode-text-italic {
    font-style: italic;
}

.rnode-text-underline {
    text-decoration: underline;
}

.rnode-text-strike {
    text-decoration: line-through;
}

.rel-pending {
    position: absolute;
    top: 12px;
    left: 50%;
    transform: translateX(-50%);
    z-index: 30;
    background: var(--accent);
    color: #fff;
    border-radius: 999px;
    padding: 6px 14px;
    font-size: 12.5px;
    box-shadow: var(--shadow);
}

/* Marquee (box selection) drawn as a DOM overlay – the only transient
   chrome that lives outside the canvas. Coordinates are canvas-wrap space. */
.marquee {
    position: absolute;

```

```

z-index: 15;
border: 1.5px solid var(--accent);
background: color-mix(in srgb, var(--accent) 12%, transparent);
pointer-events: none;
}

/* ----- Outliner ----- */

.outliner {
  position: absolute;
  left: 0;
  right: 0;
  bottom: 0;
  height: 38%;
  background: var(--surface);
  border-top: 1px solid var(--border);
  display: flex;
  flex-direction: column;
  z-index: 15;
  box-shadow: 0 -6px 24px rgba(0, 0, 0, 0.18);
}

.outliner-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 6px 12px;
  border-bottom: 1px solid var(--border);
  font-size: 12px;
  text-transform: uppercase;
  letter-spacing: 0.08em;
  color: var(--text-muted);
}

.outliner-body {
  flex: 1;
  overflow-y: auto;
  padding: 6px 0;
}

.outliner-row {
  display: flex;
  align-items: center;
  gap: 4px;
  padding: 2px 8px 2px 0;
  border-radius: var(--radius-sm);
  cursor: pointer;
}

.outliner-row:hover {
  background: var(--surface-2);
}

.outliner-row.selected {
  background: var(--accent-soft);
}

.outliner-caret {
  border: none;
  background: none;
  color: var(--text-muted);
  width: 18px;
  font-size: 11px;
  flex-shrink: 0;
}

.outliner-caret.disabled {
  opacity: 0.3;
}

.outliner-check {
  border: none;
  background: none;
  font-size: 14px;
  flex-shrink: 0;
  padding: 0;
  width: 18px;
}

.outliner-check.placeholder {
  opacity: 0;
}

.outliner-title {
  flex: 1;
  border: 1px solid transparent;
  background: transparent;

```

```

padding: 2px 4px;
font-size: 13px;
}

.outliner-title:hover,
.outliner-title:focus {
border-color: var(--border);
background: var(--surface);
}

.prio {
width: 8px;
height: 8px;
border-radius: 50%;
flex-shrink: 0;
margin-right: 4px;
}

.prio-low { background: #a3e635; }
.prio-medium { background: #fbbf24; }
.prio-high { background: #fb923c; }
.prio-urgent { background: #f87171; }

/* ----- Palette ----- */

.palette-overlay {
position: fixed;
inset: 0;
background: rgba(0, 0, 0, 0.35);
z-index: 100;
display: flex;
justify-content: center;
align-items: flex-start;
padding-top: 12vh;
}

.palette {
width: 520px;
max-width: 92vw;
background: var(--surface);
border: 1px solid var(--border);
border-radius: 12px;
box-shadow: var(--shadow);
overflow: hidden;
}

.palette input {
width: 100%;
border: none;
border-bottom: 1px solid var(--border);
border-radius: 0;
padding: 14px 16px;
font-size: 15px;
background: transparent;
}

.palette input:focus {
outline: none;
}

.palette-list {
max-height: 46vh;
overflow-y: auto;
padding: 6px;
}

.palette-item {
display: flex;
justify-content: space-between;
align-items: center;
width: 100%;
border: none;
background: none;
color: var(--text);
padding: 9px 12px;
border-radius: var(--radius-sm);
font-size: 13.5px;
text-align: left;
}

.palette-item.selected {
background: var(--accent-soft);
color: var(--accent);
}

.palette-hint {
font-size: 11px;

```

```

    color: var(--text-muted);
    font-family: var(--mono);
}

.palette-empty {
  padding: 16px;
  color: var(--text-muted);
  text-align: center;
}

.palette-footer {
  padding: 8px 14px;
  border-top: 1px solid var(--border);
  font-size: 11.5px;
  color: var(--text-muted);
}

/* ----- Toast ----- */

.toast {
  position: fixed;
  bottom: 44px;
  left: 50%;
  transform: translateX(-50%);
  background: var(--surface);
  border: 1px solid var(--border);
  color: var(--text);
  border-radius: 999px;
  padding: 8px 18px;
  font-size: 13px;
  box-shadow: var(--shadow);
  z-index: 200;
  cursor: pointer;
  animation: toast-in 0.16s ease;
}

@keyframes toast-in {
  from { opacity: 0; transform: translate(-50%, 8px); }
  to { opacity: 1; transform: translate(-50%, 0); }
}

```

## 69. tests/store.test.ts

```

import { describe, expect, it, vi } from "vitest";
import { EditorStore, docFileName, portableFileKey } from "../src/editor/store";
import { makeOp, type Op } from "../src/core/ops";
import type { StorageAdapter } from "../src/persist/storage";

const memoryAdapter: StorageAdapter = {
  label: "test",
  async load() { return []; },
  async save() { /* no-op */ },
};

describe("portableFileKey", () => {
  it("derives a distinct key per format for the same document", () => {
    const json = portableFileKey("doc-1", "json");
    const zip = portableFileKey("doc-1", "zip");
    expect(json).not.toBe(zip);
    expect(json).toBe("r-node.file-handle.doc-1:json");
    expect(zip).toBe("r-node.file-handle.doc-1:zip");
  });

  it("does not collide across documents", () => {
    expect(portableFileKey("doc-1", "json")).not.toBe(portableFileKey("doc-2", "json"));
  });
});

describe("docFileName", () => {
  it("makes a filesystem-safe name out of the GUI title", () => {
    expect(docFileName("Vacation Plan")).toBe("Vacation_Plan");
    expect(docFileName("R-node - Roadmap")).toBe("R-node_Roadmap");
    expect(docFileName("a:b*c\"d<e>f|g")).toBe("a_b_c_d_e_f_g");
    expect(docFileName("")).toBe("Untitled map");
  });
});

describe("new topic defaults", () => {
  it("gives every newly created topic a useful editable title", () => {
    const store = new EditorStore(memoryAdapter);
    const root = store.doc.node(store.sheet.rootNodeId!);
    const rootKidsBefore = root.childrenIds.length;

    // No selection → the child is created under the root as a main topic.

```

```

store.createChild();
expect(root.childrenIds.length).toBe(rootKidsBefore + 1);
const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1]);
expect(main.type).toBe("main");
expect(main.title).toMatch(/^Main Topic \d+$/);

// Tab keeps the source selected: with the main topic selected, the next
// child is a subtopic of it – created under the same parent, no nesting.
store.select(main.id);
store.createChild();
expect(main.childrenIds.length).toBe(1);
const child = store.doc.node(main.childrenIds[0]);
expect(child.type).toBe("subtopic");
expect(child.title).toBe("Subtopic 1");
});

it("keeps a manually pinned main topic pinned when reordered among root children", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId);
  const first = store.doc.node(root.childrenIds[0]);

  // select the root so the next createChild makes a sibling main topic
  store.select(store.sheet.rootNodeId);
  store.createChild();
  const second = root.childrenIds.find((id) => id !== first.id);
  expect(second).toBeTruthy();
  if (!second) return;

  store.doc.node(first.id).position = { x: 200, y: 100, manual: true };
  store.dropAt(first.id, second, "after", 250, 100);

  const moved = store.doc.node(first.id);
  expect(moved.position.manual).toBe(true);
  expect(moved.position.x).toBe(200);
  expect(moved.position.y).toBe(100);
});

it("Tab adds children under the same parent without selecting or editing", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId);
  const parent = store.doc.node(root.childrenIds[root.childrenIds.length - 1]);
  store.select(parent.id);

  // Tab 1: child created, selection stays on parent, editor NOT opened.
  store.createChild();
  expect(parent.childrenIds.length).toBe(1);
  expect(store.getSnapshot().selection).toEqual([parent.id]);
  expect(store.getSnapshot().editingId).toBeNull();

  // Tab 2: another child under the SAME parent (no nesting, no stealing).
  store.createChild();
  expect(parent.childrenIds.length).toBe(2);
  expect(store.getSnapshot().selection).toEqual([parent.id]);
  expect(store.sheet.nodes[parent.childrenIds[1]].parentId).toBe(parent.id);
});

describe("type-to-edit", () => {
  it("starts editing the selected topic with the typed character", () => {
    const store = new EditorStore(memoryAdapter);
    store.createChild();
    const root = store.doc.node(store.sheet.rootNodeId);
    const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1]);
    store.select(main.id);

    store.typeToEdit("H");
    const s = store.getSnapshot();
    expect(s.editingId).toBe(main.id);
    expect(s.selection).toEqual([main.id]);
    expect(s.pendingInsert).toBe("H");

    // TopicEditor consumes the pending insert once, then it is gone.
    expect(store.consumePendingInsert()).toBe("H");
    expect(store.consumePendingInsert()).toBeNull();
  });

  it("buffers extra keystrokes while the type-to-edit editor is mounting", () => {
    const store = new EditorStore(memoryAdapter);
    store.createChild();
    const root = store.doc.node(store.sheet.rootNodeId);
    const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1]);
    store.select(main.id);

    // First character starts editing; the next one arrives before the
    // textarea has consumed the pending insert – it must be buffered, not lost.

```

```

store.typeToEdit("f");
expect(store.appendPendingInsert("a")).toBe(true);
expect(store.appendPendingInsert("s")).toBe(true);
expect(store.consumePendingInsert()).toBe("fas");

// Once consumed (editor mounted), the textarea owns the keyboard: the
// buffer is closed and keys must NOT be appended.
expect(store.appendPendingInsert("t")).toBe(false);
});

it("commits the in-progress edit when the selection changes (click away)", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId!);
  const main = store.doc.node(root.childrenIds[0])!;
  store.select(main.id);
  store.typeToEdit("H");

  // TopicEditor keeps the store draft in sync with every keystroke.
  store.setEditingDraft("Hello typed text");

  // Clicking another node runs BEFORE the textarea blur: selection changes
  // must commit the draft instead of discarding it.
  store.select(root.id);
  expect(store.doc.node(main.id)!.title).toBe("Hello typed text");
  expect(store.getSnapshot().editingId).toBeNull();
});

it("commits the draft on commitEdit() and discards it on cancelEdit()", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId!);
  const main = store.doc.node(root.childrenIds[0])!;
  store.select(main.id);
  store.startEdit(main.id);
  store.setEditingDraft("Kept text");
  store.commitEdit();
  expect(store.doc.node(main.id)!.title).toBe("Kept text");

  store.startEdit(main.id);
  store.setEditingDraft("Discarded text");
  store.cancelEdit();
  expect(store.doc.node(main.id)!.title).toBe("Kept text");
  expect(store.getSnapshot().editingId).toBeNull();
});

it("Ctrl+S saves the in-progress draft without closing the editor", async () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId!);
  const main = store.doc.node(root.childrenIds[0])!;
  store.select(main.id);
  store.startEdit(main.id);
  store.setEditingDraft("Saved while typing");

  // first save downloads the .rnode.json – stub it (no DOM in node env)
  vi.spyOn(store as unknown as { download: () => void }, "download").mockImplementation(() => {});
  await store.saveNow();

  // The draft was committed to the document AND the editor stays open.
  expect(store.doc.node(main.id)!.title).toBe("Saved while typing");
  expect(store.getSnapshot().editingId).toBe(main.id);
  expect(store.getSnapshot().sync).toBe("saved");
});

it("commits a rich-text draft as a setTitle op with titleRuns", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId!);
  const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1])!;
  store.select(main.id);
  store.startEdit(main.id);

  // The Lexical overlay pushes runs on every keystroke (bold + color here).
  store.setEditingDraftRuns([
    { text: "Rich ", bold: true },
    { text: "text", color: "#ff0000" },
  ]);
  store.commitEdit();

  const node = store.doc.node(main.id)!;
  expect(node.title).toBe("Rich text");
  expect(node.titleRuns).toEqual([
    { text: "Rich ", bold: true },
    { text: "text", color: "#ff0000" },
  ]);
  expect(store.getSnapshot().editingId).toBeNull();
});

```

```

});

it("undo restores the exact pre-edit runs; redo reapplies them", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId!);
  const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1]!);
  // Give the node a rich baseline BEFORE editing.
  store.execOps([
    makeOp<Op & { type: "setTitle" }>("setTitle", {
      id: main.id,
      title: "Baseline",
      prev: main.title,
      titleRuns: [{ text: "Base", italic: true }],
      prevRuns: main.titleRuns,
    }),
  ]);
  const baselineRuns = store.doc.node(main.id)!.titleRuns;

  store.select(main.id);
  store.startEdit(main.id);
  store.setEditingDraftRuns([{ text: "Bold new title", bold: true }]);
  store.commitEdit();
  expect(store.doc.node(main.id)!.title).toBe("Bold new title");

  store.undo();
  // the original rich runs come back exactly
  expect(store.doc.node(main.id)!.title).toBe("Baseline");
  expect(store.doc.node(main.id)!.titleRuns).toEqual(baselineRuns);

  store.redo();
  expect(store.doc.node(main.id)!.title).toBe("Bold new title");
  expect(store.doc.node(main.id)!.titleRuns?.[0]?.bold).toBe(true);
});

it("cancelEdit discards rich runs and restores the original", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId!);
  const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1]!);
  store.select(main.id);
  store.startEdit(main.id);
  const before = store.doc.node(main.id)!.title;
  store.setEditingDraftRuns([{ text: "discarded", italic: true }]);
  store.cancelEdit();
  expect(store.doc.node(main.id)!.title).toBe(before);
  expect(store.getSnapshot().editingId).toBeNull();
});

it("pastes plain text into the selected topic (falls back from map paste)", async () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId!);
  const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1]!);
  store.select(main.id);

  vi.stubGlobal("navigator", { clipboard: { readText: async () => "plain pasted text" } });
  await store.paste();
  vi.unstubAllGlobals();

  const s = store.getSnapshot();
  expect(s.editingId).toBe(main.id);
  expect(s.pendingInsert).toBe("plain pasted text");
});

describe("save / load file", () => {
  it("round-trips a document through the .rnode.json format", () => {
    const store = new EditorStore(memoryAdapter);
    const original = store.doc.doc;

    const json = JSON.stringify(original);
    const id = store.importDocumentFromJson(json);

    expect(id).toBe(original.documentId);
    // the same documentId replaces the existing entry instead of duplicating
    expect(store.getSnapshot().docs.length).toBe(1);
    expect(store.sheet.nodes[store.sheet.rootNodeId].title).toBe(original.sheets[0].nodes[original.sheets[0].rootNodeId].title);
  });

  it("imports a second document as an additional entry and switches to it", () => {
    const store = new EditorStore(memoryAdapter);
    const second = JSON.parse(JSON.stringify(store.doc.doc));
    second.documentId = "d_other";
    second.title = "Imported map";
    const id = store.importDocumentFromJson(JSON.stringify(second));
  });

```

```

expect(id).toBe("d_other");
expect(store.getSnapshot().docs.length).toBe(2);
expect(store.getSnapshot().activeDocId).toBe("d_other");
expect(store.getSnapshot().docTitle).toBe("Imported map");
// loaded from disk → needs a save to persist in app storage
expect(store.getSnapshot().sync).toBe("dirty");
});

it("rejects malformed files", () => {
  const store = new EditorStore(memoryAdapter);
  expect(store.importDocumentFromJson("not json")).toBeNull();
  expect(store.importDocumentFromJson("{}")).toBeNull();
  expect(store.importDocumentFromJson(JSON.stringify({ foo: 1 }))).toBeNull();
  expect(store.importDocumentFromJson(JSON.stringify({ documentId: "d_x" }))).toBeNull();
  expect(store.importDocumentFromJson(JSON.stringify({ documentId: "d_x", sheets: [] }))).toBeNull();
  expect(store.importDocumentFromJson(JSON.stringify({ documentId: "d_x", sheets: [{ rootNodeId: "r" }] }))).toBeNull();
  expect(store.importDocumentFromJson(JSON.stringify({ documentId: "d_x", sheets: [{ rootNodeId: "r", nodes: {} }] }))).toBeNull();
});

it("writes the CURRENT content on every save (download fallback without the File System Access API)", async () => {
  const store = new EditorStore(memoryAdapter);
  const blobs: Blob[] = [];
  const download = vi
    .spyOn(store as unknown as { download: (blob: Blob) => void }, "download")
    .mockImplementation((blob: Blob) => { blobs.push(blob); });

  const root = store.doc.node(store.sheet.rootNodeId!);
  store.createChild();
  const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1]!);
  store.execOps([
    { opId: "t", actorId: "t", ts: new Date().toISOString(), type: "setTitle", id: main.id, title: "Versione corrente", prev: main.title } as never,
  ]);

  await store.saveNow();
  await store.saveNow(); // second save must not be stale either

  expect(store.getSnapshot().sync).toBe("saved");
  expect(download).toHaveBeenCalledTimes(2);
  const parsed = JSON.parse(await blobs[1].text());
  expect(parsed.sheets[0].nodes[main.id].title).toBe("Versione corrente");
});

it("saves a document loaded from storage with current content after reload", async () => {
  const stored: StorageAdapter = {
    label: "test",
    async load() {
      return [new EditorStore(memoryAdapter).doc.doc];
    },
    async save() { /* no-op */ },
  };
  const store = new EditorStore(stored);
  await store.init();
  const blobs: Blob[] = [];
  vi.spyOn(store as unknown as { download: (blob: Blob) => void }, "download")
    .mockImplementation((blob: Blob) => { blobs.push(blob); });

  await store.saveNow();
  expect(blobs).toHaveLength(1); // the file is written with current content
  expect(store.getSnapshot().sync).toBe("saved");
});

describe("image selection, delete and move", () => {
  const IMG_ID = "f".repeat(64);
  const card = { id: IMG_ID, mime: "image/png", w: 10, h: 10, bytes: 3 };

  function twoMainTopics(store: EditorStore): [string, string] {
    store.createChild();
    store.createChild();
    const root = store.doc.node(store.sheet.rootNodeId!);
    return [root.childrenIds[0], root.childrenIds[1]];
  }

  it("selects the image exclusively: clears node selection, and a node select clears it", () => {
    const store = new EditorStore(memoryAdapter);
    const [a] = twoMainTopics(store);
    store.attachImage(a, card);

    store.select(a);
    expect(store.getSnapshot().selection).toEqual([a]);

    store.selectImage(a);
    const s = store.getSnapshot();
    expect(s.imageSel).toBe(a);
    expect(s.selection).toEqual([]);

    // Selecting the node again drops the image selection.

```

```

const root = store.sheet.rootNodeId;
store.select(root);
expect(store.getSnapshot().imageSel).toBeNull();
});

it("deleteSelectedImage removes only the image (node survives) and undo restores it", () => {
  const store = new EditorStore(memoryAdapter);
  const [a] = twoMainTopics(store);
  store.attachImage(a, card);
  store.selectImage(a);

  store.deleteSelectedImage();
  const node = store.doc.node(a)!;
  expect(node.style.image).toBeUndefined();
  expect(node).toBeDefined(); // the node itself is NOT deleted
  expect(store.getSnapshot().imageSel).toBeNull();
  // The attachment card stays: GC is explicit (collectOrphans).
  expect(store.sheet.attachments.some((c) => c.id === IMG_ID)).toBe(true);

  store.undo();
  expect(store.doc.node(a)!.style.image).toBe(IMG_ID);
});

it("deleteSelectedImage on a node without an image is a no-op", () => {
  const store = new EditorStore(memoryAdapter);
  // The root has no image and the store still has zero ops in history.
  store.selectImage(store.sheet.rootNodeId);
  store.deleteSelectedImage();
  expect(store.getSnapshot().canUndo).toBe(false); // no op was created
});

it("assignImageToNode moves the image between nodes as ONE undoable batch", () => {
  const store = new EditorStore(memoryAdapter);
  const [a, b] = twoMainTopics(store);
  store.attachImage(a, card);

  store.assignImageToNode(a, b);
  expect(store.doc.node(a)!.style.image).toBeUndefined();
  expect(store.doc.node(b)!.style.image).toBe(IMG_ID);
  // The image on the target stays selected.
  expect(store.getSnapshot().imageSel).toBe(b);

  store.undo();
  expect(store.doc.node(a)!.style.image).toBe(IMG_ID);
  expect(store.doc.node(b)!.style.image).toBeUndefined();
  store.redo();
  expect(store.doc.node(a)!.style.image).toBeUndefined();
  expect(store.doc.node(b)!.style.image).toBe(IMG_ID);
});

it("assignImageToNode no-ops for the same node, an image-less source, or a missing target", () => {
  const store = new EditorStore(memoryAdapter);
  const rootId = store.sheet.rootNodeId;
  store.createChild();
  const main = store.doc.node(rootId)!.childrenIds[0];
  store.attachImage(main, card);
  expect(store.doc.node(main)!.style.image).toBe(IMG_ID);

  store.assignImageToNode(main, main); // same node
  store.assignImageToNode("missing", main); // missing source
  store.assignImageToNode(rootId, main); // image-less source
  expect(store.doc.node(main)!.style.image).toBe(IMG_ID);

  // None of the no-ops pushed history: one undo reverts the attach itself.
  store.undo();
  expect(store.doc.node(main)!.style.image).toBeUndefined();
  store.redo();
  expect(store.doc.node(main)!.style.image).toBe(IMG_ID);
});

describe("canvas resize drag", () => {
  it("commits a live width drag as ONE setStyle op with exact undo", () => {
    const store = new EditorStore(memoryAdapter);
    const root = store.doc.node(store.sheet.rootNodeId)!;
    const id = root.id;
    const before = { ...store.doc.node(id)!.style };

    store.beginResize(id);
    store.setResizeDraft(id, 200);
    store.setResizeDraft(id, 180);
    expect(store.doc.node(id)!.style.width).toBe(180);
    store.commitResize();

    // exactly one op in history (two live drafts collapsed into one commit)
    expect(store.getSnapshot().canUndo).toBe(true);
    const node = store.doc.node(id)!;

```

```

expect(node.style.width).toBe(180);

store.undo();
expect(store.doc.node(id)!.style.width).toBeUndefined();
expect(store.doc.node(id)!.style).toEqual(before);
});

it("clamps the width to the allowed range and no-ops on a click without drag", () => {
  const store = new EditorStore(memoryAdapter);
  const id = store.sheet.rootNodeId;
  store.beginResize(id);
  store.setResizeDraft(id, 5); // below MIN_TOPIC_W
  expect(store.doc.node(id)!.style.width).toBe(84);
  store.setResizeDraft(id, 5000); // above the 640 cap
  expect(store.doc.node(id)!.style.width).toBe(640);
  store.commitResize();
  expect(store.doc.node(id)!.style.width).toBe(640);

  // a pointer-down/up with no move must not create an op
  const before = { ...store.doc.node(id)!.style };
  store.beginResize(id);
  store.commitResize();
  expect(store.doc.node(id)!.style).toEqual(before);
  const historyOps = store.getSnapshot().canUndo;
  store.undo();
  store.undo();
  expect(store.getSnapshot().canUndo).toBe(false);
  expect(historyOps).toBe(true);
});

it("left-edge drag anchors the right edge: floating node keeps width + position, one undo restores both", () => {
  const store = new EditorStore(memoryAdapter);
  store.createFloatingAt(500, 200);
  store.cancelEdit(); // leave the auto-opened editor
  const f = Object.values(store.sheet.nodes).find((n) => n.type === "floating")!;
  store.select(f.id);
  store.beginResize(f.id);
  store.setResizeDraft(f.id, 220, { anchorRight: true, x: 480 }); // dragged 20 left
  store.commitResize();

  const node = store.doc.node(f.id)!;
  expect(node.style.width).toBe(220);
  expect(node.position.x).toBe(480);
  expect(node.position.manual).toBe(true);

  store.undo();
  const undone = store.doc.node(f.id)!;
  expect(undone.style.width).toBeUndefined();
  expect(undone.position.x).toBe(500);
});

it("left-edge drag on an auto-layout node keeps the width but returns the position to the layout slot", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId)!;
  const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1])!;
  const origX = main.position.x;
  store.beginResize(main.id);
  store.setResizeDraft(main.id, 260, { anchorRight: true, x: origX - 40 });
  store.commitResize();

  const node = store.doc.node(main.id)!;
  expect(node.style.width).toBe(260);
  // auto node: position restored (not manual) - width is what survives
  expect(node.position.x).toBe(origX);
  expect(node.position.manual).toBe(false);

  store.undo();
  expect(store.doc.node(main.id)!.style.width).toBeUndefined();
});

it("commits an image width drag as ONE setStyle op with exact undo", () => {
  const store = new EditorStore(memoryAdapter);
  const id = store.sheet.rootNodeId;
  const before = { ...store.doc.node(id)!.style };

  store.beginImageResize(id);
  store.setImageResizeDraft(id, 200);
  store.setImageResizeDraft(id, 180);
  expect(store.doc.node(id)!.style.imageWidth).toBe(180);
  store.commitImageResize();

  const node = store.doc.node(id)!;
  expect(node.style.imageWidth).toBe(180);

  store.undo();
  expect(store.doc.node(id)!.style).toEqual(before);
});

```

```

});

it("clamps the image width to 48px minimum and no-ops on a click without drag", () => {
  const store = new EditorStore(memoryAdapter);
  const id = store.sheet.rootNodeId;
  store.beginImageResize(id);
  store.setImageResizeDraft(id, 10); // below the 48 floor
  expect(store.doc.node(id)!.style.imageWidth).toBe(48);
  store.commitImageResize();
  expect(store.doc.node(id)!.style.imageWidth).toBe(48);

  // pointer-down/up with no move must not create an op
  const before = { ...store.doc.node(id)!.style };
  store.beginImageResize(id);
  store.commitImageResize();
  expect(store.doc.node(id)!.style).toEqual(before);
});

it("resetImageWidth removes the custom width in one undoable op", () => {
  const store = new EditorStore(memoryAdapter);
  const id = store.sheet.rootNodeId;
  store.beginImageResize(id);
  store.setImageResizeDraft(id, 220);
  store.commitImageResize();
  expect(store.doc.node(id)!.style.imageWidth).toBe(220);

  store.resetImageWidth(id);
  expect(store.doc.node(id)!.style.imageWidth).toBeUndefined();

  store.undo();
  expect(store.doc.node(id)!.style.imageWidth).toBe(220);
});

it("dragging a subtopic onto empty canvas does not pin an absolute position", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId)!;
  const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1])!;
  store.select(main.id);
  store.createChild();
  const sub = store.doc.node(main.childrenIds[0])!;
  const orig = { ...sub.position };

  store.dropAt(sub.id, null, "floating", 999, 888);

  const after = store.doc.node(sub.id)!;
  expect(after.position.manual).toBe(false);
  expect(after.position.x).toBe(orig.x);
  expect(after.position.y).toBe(orig.y);
});

it("dragging a main topic pins it and releases manual pins on its descendants so they follow", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId)!;
  const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1])!;
  store.select(main.id);
  store.createChild();
  const sub = store.doc.node(main.childrenIds[0])!;
  // a legacy/previous manual pin on the subtopic
  store.doc.node(sub.id)!.position = { x: 300, y: 200, manual: true };

  store.dropAt(main.id, null, "floating", 700, 350);

  const moved = store.doc.node(main.id)!;
  const child = store.doc.node(sub.id)!;
  expect(moved.position.manual).toBe(true);
  expect(moved.position.x).toBe(700);
  expect(child.position.manual).toBe(false);
  // one undo restores the whole batch (main back to auto, child pinned again)
  store.undo();
  expect(store.doc.node(main.id)!.position.manual).toBe(false);
  expect(store.doc.node(sub.id)!.position.manual).toBe(true);
  expect(store.doc.node(sub.id)!.position.x).toBe(300);
});

it("floating topics keep being freely movable to an absolute position", () => {
  const store = new EditorStore(memoryAdapter);
  store.createFloatingAt(500, 200);
  store.cancelEdit(); // leave the auto-opened editor
  const f = Object.values(store.sheet.nodes).find((n) => n.type === "floating")!;

  store.dropAt(f.id, null, "floating", 400, 300);

  const after = store.doc.node(f.id)!;
  expect(after.position.manual).toBe(true);
  expect(after.position.x).toBe(400);
});

```

```

    expect(after.position.y).toBe(300);
  });

describe("live node drag (fluid repositioning)", () => {
  it("a main topic follows the cursor and commits at the drop point, subtree settled immediately, one undo", () => {
    const store = new EditorStore(memoryAdapter);
    store.createChild();
    store.cancelEdit(); // leave the auto-opened editor
    const root = store.doc.node(store.sheet.rootNodeId!);
    const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1]!);
    // give the main a child
    store.select(main.id);
    store.createChild();
    store.cancelEdit();
    const child = store.doc.node(main.childrenIds[0]!);
    // Pin the main at a known spot and settle the child relative to it:
    // fresh nodes carry pre-layout positions, so the baseline must come
    // from a settled layout.
    store.doc.node(main.id)!.position = { x: 200, y: 100, manual: true };
    store.beginNodeDrag(main.id);
    store.setNodeDragDraft(main.id, 200, 100); // same spot, but settles the layout
    store.resetNodeDragDraft(main.id); // no-op: nothing changed, child stays settled
    const offBefore = { x: child.position.x - 200, y: child.position.y - 100 };

    store.beginNodeDrag(main.id);
    store.setNodeDragDraft(main.id, 700, 350);
    // live follow: the main is pinned at the draft spot and the subtree
    // keeps its offset (it slid with the pointer)
    expect(store.doc.node(main.id)!.position.x).toBe(700);
    expect(store.doc.node(main.id)!.position.y).toBe(350);
    const offDuring = { x: child.position.x - 700, y: child.position.y - 350 };
    expect(offDuring).toEqual(offBefore);

    store.commitNodeDrag(main.id, null, "floating", 700, 350);

    const after = store.doc.node(main.id!);
    expect(after.position).toEqual({ x: 700, y: 350, manual: true });
    // the subtree is settled IMMEDIATELY – no 30ms debounce window where
    // the child is still anchored to the main's old slot (release glitch)
    const offAfter = { x: child.position.x - 700, y: child.position.y - 350 };
    expect(offAfter).toEqual(offDuring);

    // one undo restores the exact pre-drag state
    store.undo();
    expect(store.doc.node(main.id)!.position).toEqual({ x: 200, y: 100, manual: true });
  });

  it("leaving the free-position zone mid-drag restores the pinned position", () => {
    const store = new EditorStore(memoryAdapter);
    store.createChild();
    store.cancelEdit(); // leave the auto-opened editor
    const root = store.doc.node(store.sheet.rootNodeId!);
    const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1]!);
    store.doc.node(main.id)!.position = { x: 123, y: 456, manual: true };

    store.beginNodeDrag(main.id);
    store.setNodeDragDraft(main.id, 700, 350);
    store.resetNodeDragDraft(main.id);
    expect(store.doc.node(main.id)!.position).toEqual({ x: 123, y: 456, manual: true });
  });

  it("a subtopic drag never pins an absolute position (deep nodes stay in the auto layout)", () => {
    const store = new EditorStore(memoryAdapter);
    store.createChild();
    store.cancelEdit();
    const root = store.doc.node(store.sheet.rootNodeId!);
    const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1]!);
    store.select(main.id);
    store.createChild();
    store.cancelEdit();
    const sub = store.doc.node(main.childrenIds[0]!);

    store.beginNodeDrag(sub.id);
    store.setNodeDragDraft(sub.id, 999, 888);
    store.commitNodeDrag(sub.id, null, "floating", 999, 888);

    const after = store.doc.node(sub.id!);
    expect(after.position.manual).toBe(false);
    expect(after.position.x).not.toBe(999);
    expect(after.position.y).not.toBe(888);
  });
});

it("creates a group only from at least two sibling topics, undoable", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId!);

```

```

const first = store.doc.node(root.childrenIds[0])!;
store.select(store.sheet.rootNodeId);
store.createChild();
const second = root.childrenIds.find((id) => id !== first.id!);

// single selection → rejected
store.select(second);
store.createGroupFromSelection();
expect(store.sheet.boundaries).toHaveLength(0);

store.selectMany([first.id, second]);
store.createGroupFromSelection();
expect(store.sheet.boundaries).toHaveLength(1);
expect(store.sheet.boundaries[0].memberIds).toEqual(expect.arrayContaining([first.id, second]));

store.undo();
expect(store.sheet.boundaries).toHaveLength(0);
});

it("groups ANY selected topics, not just siblings, undoable", () => {
  const store = new EditorStore(memoryAdapter);
  // two children of the root, then a grandchild of the first: not siblings
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId)!;
  const first = store.doc.node(root.childrenIds[0])!;
  store.select(first.id);
  store.createChild();
  const grandchild = store.doc.node(first.childrenIds[0])!;

  store.selectMany([first.id, grandchild.id]);
  store.createGroupFromSelection();
  expect(store.sheet.boundaries).toHaveLength(1);
  expect(store.sheet.boundaries[0].memberIds).toEqual(expect.arrayContaining([first.id, grandchild.id]));

  store.undo();
  expect(store.sheet.boundaries).toHaveLength(0);

  // but a summary still needs siblings
  store.selectMany([first.id, grandchild.id]);
  store.createSummaryFromSelection();
  expect(store.sheet.summaries).toHaveLength(0);
});

it("creates a summary from sibling selection, undoable", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId)!;
  const first = store.doc.node(root.childrenIds[0])!;
  store.select(store.sheet.rootNodeId);
  store.createChild();
  const second = root.childrenIds.find((id) => id !== first.id!);

  store.selectMany([first.id, second]);
  store.createSummaryFromSelection();
  expect(store.sheet.summaries).toHaveLength(1);
  expect(store.sheet.summaries[0].memberIds).toEqual(expect.arrayContaining([first.id, second]));

  store.undo();
  expect(store.sheet.summaries).toHaveLength(0);
});

it("deleting a main topic drops groups/summaries over its subtree", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId)!;
  const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1])!;
  store.select(main.id);
  store.createChild();
  store.createChild(); // same parent stays selected → two subtopics
  const subs = [...main.childrenIds];
  store.selectMany(subs);
  store.createGroupFromSelection();
  store.createSummaryFromSelection();
  expect(store.sheet.boundaries).toHaveLength(1);
  expect(store.sheet.summaries).toHaveLength(1);

  store.deleteNodes([main.id]);
  expect(store.sheet.boundaries).toHaveLength(0);
  expect(store.sheet.summaries).toHaveLength(0);
});

it("relationship label edits are undoable and deletable", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId)!;
  const first = store.doc.node(root.childrenIds[0])!;
  store.select(store.sheet.rootNodeId);

```

```

store.createChild();
const second = root.childrenIds.find((id) => id !== first.id!);
store.createRelationship(first.id, second);
const relId = store.sheet.relationships[0].id;

store.setRelationship(relId, { label: "causes" });
expect(store.sheet.relationships[0].label).toBe("causes");
store.undo();
expect(store.sheet.relationships[0].label).toBeUndefined();

store.selectRelationship(relId);
store.deleteSelectedRelationship();
expect(store.sheet.relationships).toHaveLength(0);
store.undo();
expect(store.sheet.relationships).toHaveLength(1);
});

it("copySelectionOutline writes an indented hierarchy to the clipboard", async () => {
  const writes: string[] = [];
  Object.defineProperty(globalThis, "navigator", {
    value: { clipboard: { writeText: async (t: string) => { writes.push(t); } } },
    configurable: true,
  });
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId!);
  const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1])!;
  store.doc.node(main.id)!.title = "A";
  store.select(main.id);
  store.createChild();
  const sub = store.doc.node(main.childrenIds[0])!;
  store.doc.node(sub.id)!.title = "A1";
  store.select(store.sheet.rootNodeId);
  store.createChild();
  const other = store.doc.node(root.childrenIds[root.childrenIds.length - 1])!;
  store.doc.node(other.id)!.title = "B";

  store.selectMany([main.id, other.id]);
  await store.copySelectionOutline();
  expect(writes).toHaveLength(1);
  expect(writes[0]).toBe("A\n  A1\nB");
});

it("attachImage registers the card once and undo removes the reference, keeping the card", () => {
  const store = new EditorStore(memoryAdapter);
  store.createChild();
  const root = store.doc.node(store.sheet.rootNodeId!);
  const main = store.doc.node(root.childrenIds[root.childrenIds.length - 1])!;
  const card = { id: "sha-img", mime: "image/png", w: 400, h: 300, bytes: 1234, name: "pic.png" };

  store.attachImage(main.id, card);
  expect(store.sheet.attachments).toHaveLength(1);
  expect(store.doc.node(main.id)!.style.image).toBe("sha-img");

  // Same image on a second node: still one card.
  store.attachImage(root.id, card);
  expect(store.sheet.attachments).toHaveLength(1);
  expect(store.doc.node(root.id)!.style.image).toBe("sha-img");

  // One undo per op removes the references, and the shared card stays –
  // removing it on undo could break another user of the same image;
  // collectOrphans is the GC.
  store.undo();
  expect(store.doc.node(root.id)!.style.image).toBeUndefined();
  store.undo();
  expect(store.doc.node(main.id)!.style.image).toBeUndefined();
  expect(store.sheet.attachments).toHaveLength(1);

  // No image bytes ever entered the document.
  expect(JSON.stringify(store.sheet.attachments)).not.toContain("\"image\":");
});
});

```

## 70. src/render/renderer.ts

```

/**
 * Canvas renderer – one <canvas>, no DOM/SVG nodes per topic.
 *
 * Paints relationships, connectors and nodes with viewport culling, then
 * provides hit testing over the same geometry. Text editing happens in an
 * HTML overlay (allowed by the architecture) – the renderer skips the title
 * of the node being edited so the overlay doesn't double-paint.
 */
import type { Group, MindNode, Sheet, StructureType, Orientation, Summary, TextRun } from "../core/types";

```

```

import { nodeRuns } from "../core/text";
import { createCanvasTextMeasurer, FONT_STACK, IMAGE_GAP, imageResolver, LINE_HEIGHT_FACTOR, MAX_IMAGE_W, measureNode, TEXT_INSET, wrapRunLines, type TextMeasurer } from "../layout/measure";
import { getAssetStore, type AssetLevel, type AssetStore } from "../persist/assets";
import { THEMES, type RenderTheme, type ThemeName } from "../theme";
import { trace } from "../dev/trace";
import type { Camera } from "../viewport";

export interface DropIndicator {
  mode: "child" | "before" | "after" | "floating" | "none";
  nodeId: string;
}

export interface RenderState {
  sheet: Sheet;
  camera: Camera;
  selection: Set<string>;
  editingId: string | null;
  hoverId: string | null;
  drop: DropIndicator | null;
  themeName: ThemeName;
  viewW: number;
  viewH: number;
  relSel?: string | null;
  groupSel?: string | null;
  summarySel?: string | null;
  /** Node whose image is selected (mutually exclusive with node selection). */
  imageSel?: string | null;
  /**
   * Ghost preview of an image being dragged over the map (internal
   * reassignment): the image bitmap follows the cursor, semi-transparent,
   * until the drop lands. x/y are world coords of the cursor.
   */
  ghostImage?: { imageId: string; x: number; y: number; nodeId: string } | null;
  /**
   * Marquee drag preview: ids of the topics inside the drag box – they wear
   * the selection ring BEFORE the release commits them (the box itself is a
   * DOM overlay; this is the live "what will be selected" feedback).
   */
  marqueeSel?: Set<string> | null;
  showHidden?: boolean; // export: include collapsed subtrees
}

interface Placed {
  node: MindNode;
  x: number; // top-left in world coords
  y: number;
  w: number;
  h: number;
  visible: boolean;
}

export class Renderer {
  private ctx: CanvasRenderingContext2D;
  private dpr = 1;
  /** Same measurer the layout engine uses – extents always agree. */
  private measurer: TextMeasurer = createCanvasTextMeasurer();
  /** Camera scale of the current frame (used to pick the text-cache resolution). */
  private curScale = 1;
  /**
   * Bitmap cache for node titles: the styled text of a node is rendered once
   * into an offscreen canvas and blitted with drawImage during pan/zoom – the
   * text layout is NOT recomputed every frame. The cache key covers the runs,
   * style, resolved color and a scale bucket, so it is invalidated only when
   * the title/content actually changes (or the zoom crosses a power-of-two
   * boundary, where the bitmap is re-rendered sharper).
   */
  private textCache = new Map<string, { canvas: HTMLCanvasElement; w: number; h: number }>();
  private textHits = 0;
  private textMisses = 0;

  /** Per-frame attachment resolver (invariant I9): set from state.sheet. */
  private resolveImage: ((id: string) => { w: number; h: number } | null) | null = null;
  /** Node ids with an image that were visible in the last painted frame. */
  private visibleImageNodes = new Set<string>();
  /**
   * Image bitmap cache (ADR-001 §12): decoded at the level closest to the
   * current zoom, LRU with a byte budget (w×h×4 per bitmap, 128MB cap).
   * Evicted bitmaps are closed – ImageBitmap memory is not JS heap and the
   * GC cannot free it predictably.
   */
  private imageCache = new Map<string, { bitmap: ImageBitmap; bytes: number }>();
  private imageBytes = 0;
  private imageFailed = new Set<string>();
  /** Decodes in flight, key → requesting nodeId (a finished one for a node
   * that scrolled off is closed immediately, never cached). */
  private inflight = new Map<string, string>();

```

```

private inflightCount = 0;
private readonly IMAGE_BUDGET = 128 * 1024 * 1024;
private readonly MAX_INFLIGHT = 5;
/** Decode-size buckets. The max is the width of the `large` level: decoding
 * wider than the stored source costs memory and adds no detail. */
private readonly IMAGE_BUCKET_MIN = 128;
private readonly IMAGE_BUCKET_MAX = 1024;
private assetStore: AssetStore;
private onRepaint: (() => void) | null = null;

constructor(
  canvas: HTMLCanvasElement,
  opts: { assetStore?: AssetStore; onRepaint?: () => void } = {}
) {
  const ctx = canvas.getContext("2d");
  if (!ctx) throw new Error("2d context unavailable");
  this.ctx = ctx;
  // Resolved at construction, not at module scope: getAssetStore() is a
  // singleton that sticks on first call, and module init may run before
  // Tauri injects window.__TAURI__ - the backend must be picked when the
  // renderer is actually created, in the running environment.
  this.assetStore = opts.assetStore ?? getAssetStore();
  this.onRepaint = opts.onRepaint ?? null;
}

resize(canvas: HTMLCanvasElement, cssW: number, cssH: number): void {
  this.dpr = Math.min(window.devicePixelRatio || 1, 2);
  canvas.width = Math.max(1, Math.round(cssW * this.dpr));
  canvas.height = Math.max(1, Math.round(cssH * this.dpr));
}

// -----
// Placement + culling
// -----

private placedNodes(state: RenderState): Placed[] {
  const { sheet, camera } = state;
  const vw = state.viewW / camera.scale;
  const vh = state.viewH / camera.scale;
  const cx = camera.x, cy = camera.y;
  const out: Placed[] = [];

  // Nodes with images must be measured with the sheet's attachment cards
  // (invariant I9): the layout, the renderer and the overlay all agree.
  this.resolveImage = imageResolver(state.sheet);
  const add = (n: MindNode): void => {
    const m = measureNode(n, this.mesurer, this.resolveImage ?? undefined);
    const x = n.position.x;
    const y = n.position.y;
    const visible =
      state.showHidden === true ||
      (x + m.w >= cx - vw / 2 - 40 && x <= cx + vw / 2 + 40 && y + m.h >= cy - vh / 2 - 40 && y <= cy + vh / 2 + 40);
    out.push({ node: n, x, y, w: m.w, h: m.h, visible });
  };

  // BFS from root, then floating nodes on top.
  const seen = new Set<string>();
  const queue = [sheet.rootNodeId];
  while (queue.length > 0) {
    const id = queue.shift()!;
    const n = sheet.nodes[id];
    if (!n || seen.has(id)) continue;
    seen.add(id);
    add(n);
    if (state.showHidden === true || !n.collapsed) queue.push(...n.childrenIds);
  }
  for (const n of Object.values(sheet.nodes)) {
    if (n.type === "floating" && !seen.has(n.id)) add(n);
  }
  return out;
}

// -----
// Main paint
// -----

render(state: RenderState): void {
  const ctx = this.ctx;
  const theme = THEMES[state.themeName];
  ctx.setTransform(this.dpr, 0, 0, this.dpr, 0, 0);
  ctx.clearRect(0, 0, state.viewW, state.viewH);
  ctx.fillStyle = theme.background;
  ctx.fillRect(0, 0, state.viewW, state.viewH);

  const s = state.camera.scale;
  this.curScale = s;
  const ox = state.viewW / 2 - state.camera.x * s;

```

```

const oy = state.viewH / 2 - state.camera.y * s;
ctx.setTransform(this.dpr * s, 0, 0, this.dpr * s, this.dpr * ox, this.dpr * oy);

this.drawGrid(theme, state);

const tStart = typeof performance !== "undefined" ? performance.now() : 0;
this.textHits = 0;
this.textMisses = 0;

const placed = this.placedNodes(state);
this.visibleImageNodes = new Set(
  placed.filter((p) => p.visible && !p.node.style.image).map((p) => p.node.id)
);
const byId = new Map(placed.map((p) => [p.node.id, p]));

// Viewport in world units, with the same 40px margin placedNodes uses.
const vw = state.viewW / s;
const vh = state.viewH / s;
const viewMinX = state.camera.x - vw / 2 - 40;
const viewMaxX = state.camera.x + vw / 2 + 40;
const viewMinY = state.camera.y - vh / 2 - 40;
const viewMaxY = state.camera.y + vh / 2 + 40;
/**
 * A line between two boxes is on screen whenever the box that CONTAINS
 * both of them is – not only when both endpoints are individually visible.
 * Requiring both is what made a connector vanish as soon as its parent
 * scrolled off, while the child was still in view.
 */
const linkVisible = (a: Placed, b: Placed): boolean =>
  state.showHidden === true ||
  (Math.max(a.x + a.w, b.x + b.w) >= viewMinX &&
   Math.min(a.x, b.x) <= viewMaxX &&
   Math.max(a.y + a.h, b.y + b.h) >= viewMinY &&
   Math.min(a.y, b.y) <= viewMaxY);

// 1) relationships
let rels = 0;
let relsDrawn = 0;
for (const rel of state.sheet.relationships) {
  rels++;
  const a = byId.get(rel.fromId);
  const b = byId.get(rel.toId);
  if (a && b && linkVisible(a, b)) {
    relsDrawn++;
    this.drawRelationship(theme, a, b, rel.color ?? theme.selection, rel.lineStyle ?? "dashed", rel.label, rel.bidirectional, state.relSel ===
rel.id);
  }
}

// 2) connectors
let links = 0;
let linksDrawn = 0;
for (const p of placed) {
  if (!p.node.parentId) continue;
  const parent = byId.get(p.node.parentId);
  if (!parent) continue;
  links++;
  if (!linkVisible(parent, p)) continue;
  linksDrawn++;
  this.drawConnector(
    parent,
    p,
    state.sheet.structure.structureType,
    state.sheet.structure.orientation,
    this.branchColor(theme, p.node, state.sheet)
  );
}

// 3) nodes
for (const p of placed) {
  if (!p.visible) continue;
  this.drawNode(theme, p, state);
}

// 4) groups & summaries (drawn over the nodes, geometry derived from members)
this.drawGroupsAndSummaries(theme, state, byId);

// 5) drop indicator
if (state.drop && state.drop.mode !== "none") {
  const target = byId.get(state.drop.nodeId);
  if (target) this.drawDropIndicator(theme, target, state.drop.mode);
}

// 6) ghost preview of an image being dragged (internal reassignment) –
// on top of everything, centered on the cursor, before the drop lands.
if (state.ghostImage) this.drawGhostImage(theme, state.ghostImage);

```

```

ctx.setTransform(this.dpr, 0, 0, this.dpr, 0, 0);

// Evict only at the end of the frame: a closed bitmap still referenced by
// the current paint would throw.
this.evictToBudget();

// Per-frame counters: they are what separates "not drawn" from "drawn but
// invisible" when someone reports something missing on screen.
trace.render(
  {
    scale: s,
    nodes: placed.length,
    visible: placed.reduce((a, p) => a + (p.visible ? 1 : 0), 0),
    rels,
    relsDrawn,
    links,
    linksDrawn,
    textHits: this.textHits,
    textMisses: this.textMisses,
    imgVisible: placed.filter(
      (p) => p.visible && !!p.node.style.image && !!this.resolveImage?.(p.node.style.image)
    ).length,
    imgCached: this.imageCache.size,
    imgBytes: this.imageBytes,
    imgInflight: this.inflightCount,
  },
  (typeof performance !== "undefined" ? performance.now() : 0) - tStart
);
}

// -----
// Pieces
// -----

private drawGrid(theme: RenderTheme, state: RenderState): void {
  const ctx = this.ctx;
  const s = state.camera.scale;
  const step = 40;
  ctx.strokeStyle = theme.grid;
  ctx.lineWidth = 1 / s;
  ctx.beginPath();
  const x0 = state.camera.x - state.viewW / 2 / s;
  const x1 = state.camera.x + state.viewW / 2 / s;
  const y0 = state.camera.y - state.viewH / 2 / s;
  const y1 = state.camera.y + state.viewH / 2 / s;
  const startX = Math.floor(x0 / step) * step;
  const startY = Math.floor(y0 / step) * step;
  for (let gx = startX; gx <= x1; gx += step) {
    ctx.moveTo(gx, y0);
    ctx.lineTo(gx, y1);
  }
  for (let gy = startY; gy <= y1; gy += step) {
    ctx.moveTo(x0, gy);
    ctx.lineTo(x1, gy);
  }
  ctx.stroke();
}

private drawConnector(
  parent: Placed,
  child: Placed,
  structure: StructureType,
  orientation: Orientation,
  color: string
): void {
  const ctx = this.ctx;
  void structure;
  const childLeft = child.x + child.w / 2 < parent.x + parent.w / 2;
  let sx: number, sy: number, ex: number, ey: number;
  // vertical connector when child is below parent and structure supports vertical layouts
  if (child.y > parent.y + parent.h - 1 && structure !== "mindmap" && !(structure === "logic" && orientation !== "vertical")) {
    // vertical tree: parent bottom -> child top
    sx = parent.x + parent.w / 2;
    sy = parent.y + parent.h;
    ex = child.x + child.w / 2;
    ey = child.y;
  } else {
    sx = childLeft ? parent.x : parent.x + parent.w;
    sy = parent.y + parent.h / 2;
    ex = childLeft ? child.x + child.w : child.x;
    ey = child.y + child.h / 2;
  }
  const dx = Math.abs(ex - sx);
  const cp1x = sx + (childLeft ? -dx * 0.45 : dx * 0.45);
  const cp2x = ex + (childLeft ? dx * 0.45 : -dx * 0.45);
  ctx.strokeStyle = color;
  ctx.lineWidth = 1.7;

```

```

    ctx.beginPath();
    ctx.moveTo(sx, sy);
    ctx.bezierCurveTo(cp1x, sy, cp2x, ey, ex, ey);
    ctx.stroke();
}

private nodeFill(theme: RenderTheme, p: Placed, state: RenderState): string {
    const n = p.node;
    if (n.style.fill) return n.style.fill;
    if (n.type === "central") return theme.rootFill;
    const color = this.branchColor(theme, n, state.sheet);
    return n.type === "subtopic" ? theme.branchSoft[this.branchIndex(n, state.sheet)] : color;
}

/**
 * The fill and text color a node is actually painted with. The editing
 * overlay wears these so the box does not change appearance the moment you
 * double-click it – resolving branch palettes here keeps the one source of
 * truth in the renderer.
 */
nodeColors(state: RenderState, id: string): { fill: string; text: string } | null {
    const n = state.sheet.nodes[id];
    if (!n) return null;
    const theme = THEMES[state.themeName];
    const fill =
        n.style.fill ??
        (n.type === "central"
            ? theme.rootFill
            : n.type === "subtopic"
            ? theme.branchSoft[this.branchIndex(n, state.sheet)]
            : this.branchColor(theme, n, state.sheet));
    return { fill, text: n.style.textColor ?? (n.type === "central" ? theme.rootText : theme.text) };
}

private branchColor(theme: RenderTheme, n: MindNode, sheet: Sheet): string {
    const branchRootId = this.branchRoot(n, sheet);
    const branchRoot = sheet.nodes[branchRootId];
    return branchRoot?.style.fill ?? theme.branch[this.branchIndex(n, sheet)];
}

private branchIndex(n: MindNode, sheet: Sheet): number {
    const branchRootId = this.branchRoot(n, sheet);
    const root = sheet.nodes[sheet.rootNodeId];
    const index = root?.childrenIds.indexOf(branchRootId) ?? -1;
    return (index >= 0 ? index : 0) % 8;
}

private branchRoot(n: MindNode, sheet: Sheet): string {
    let cur: string | null = n.id;
    let prev: string | null = null;
    while (cur && cur !== sheet.rootNodeId) {
        prev = cur;
        cur = sheet.nodes[cur]?.parentId ?? null;
    }
    return prev ?? n.id;
}

private drawNode(theme: RenderTheme, p: Placed, state: RenderState): void {
    const ctx = this.ctx;
    const n = p.node;
    const editing = state.editingId === n.id;
    const selected = state.selection.has(n.id);
    const fill = this.nodeFill(theme, p, state);
    const textColor = n.style.textColor ?? (n.type === "central" ? theme.rootText : theme.text);
    const opacity = n.style.opacity ?? 1;

    ctx.save();
    ctx.globalAlpha = opacity;
    if (n.style.rotation) ctx.rotate((n.style.rotation * Math.PI) / 180);

    // shadow
    if (n.style.shadow) {
        ctx.shadowColor = theme.shadow;
        ctx.shadowBlur = 12;
        ctx.shadowOffsetY = 4;
    }

    this.traceShape(ctx, p, n.style.shape ?? "rounded", n.style.cornerRadius ?? 10);
    ctx.fillStyle = fill;
    ctx.fill();
    ctx.shadowBlur = 0;
    ctx.shadowOffsetY = 0;
    if (n.style.borderWidth && n.style.stroke && n.style.stroke !== "transparent") {
        ctx.strokeStyle = n.style.stroke;
        ctx.lineWidth = n.style.borderWidth;
        ctx.setLineDash(n.style.borderStyle === "dashed" ? [6, 4] : n.style.borderStyle === "dotted" ? [2, 3] : []);
        ctx.stroke();
    }

```

```

    ctx.setLineDash([]);
} else if (theme.name === "light") {
    ctx.strokeStyle = theme.nodeBorder;
    ctx.lineWidth = 1;
    ctx.stroke();
}

// selection ring
if (selected) {
    ctx.strokeStyle = theme.selection;
    ctx.lineWidth = 2.5;
    ctx.setLineDash([]);
    const pad = 3;
    ctx.strokeRect(p.x - pad, p.y - pad, p.w + pad * 2, p.h + pad * 2);

    // Xmind-style resize handles on BOTH edges (hidden while editing – the
    // HTML overlay owns the node then). Dragging changes the width and the
    // text re-wraps; see hitTestResize + store.setResizeDraft. Outline only
    // (a filled square reads as a purple blob on the node); a white
    // under-stroke keeps them visible on dark/same-colored fills (e.g. the
    // indigo central topic) without adding a fill.
    const hs = 9;
    const hy = p.y + p.h / 2 - hs / 2;
    for (const hx of [p.x - hs / 2, p.x + p.w - hs / 2]) {
        this.roundRect(ctx, hx, hy, hs, hs, 2);
        ctx.strokeStyle = theme.background;
        ctx.lineWidth = 3.2;
        ctx.stroke();
        ctx.strokeStyle = theme.selection;
        ctx.lineWidth = 1.6;
        ctx.stroke();
    }
}

// Image resize handle (T14): bottom-right corner of the image, same
// outline style as the width handles. Same rect as imageWorldRect. Drawn
// for a selected node OR a selected IMAGE (the image is then the focus –
// resize must stay reachable without reselecting the node).
if (selected || state.imageSel === n.id) {
    const hs = 9;
    const ir = this.imageWorldRect(state, n.id);
    if (ir) {
        const hx = ir.x + ir.w - hs / 2;
        const hy = ir.y + ir.h - hs / 2;
        this.roundRect(ctx, hx, hy, hs, hs, 2);
        ctx.strokeStyle = theme.background;
        ctx.lineWidth = 3.2;
        ctx.stroke();
        ctx.strokeStyle = theme.selection;
        ctx.lineWidth = 1.6;
        ctx.stroke();
    }
}

// Marquee preview ring: the topic is inside the drag box but not yet
// committed – same ring as selection, no resize handles (it is not
// really selected until the release).
if (!selected && state.marqueeSel?.has(n.id)) {
    ctx.strokeStyle = theme.selection;
    ctx.lineWidth = 2.5;
    ctx.setLineDash([]);
    const pad = 3;
    ctx.strokeRect(p.x - pad, p.y - pad, p.w + pad * 2, p.h + pad * 2);
}

// Image selection ring: the image is selected (not the node) – outline
// around the image so Backspace/Delete knows what it will remove.
if (state.imageSel === n.id) {
    const ir = this.imageWorldRect(state, n.id);
    if (ir) {
        const pad = 3;
        ctx.strokeStyle = theme.selection;
        ctx.lineWidth = 2.5;
        ctx.setLineDash([]);
        ctx.strokeRect(ir.x - pad, ir.y - pad, ir.w + pad * 2, ir.h + pad * 2);
    }
}

if (state.hoverId === n.id && !selected) {
    ctx.strokeStyle = theme.selection;
    ctx.lineWidth = 1.5;
    ctx.globalAlpha = opacity * 0.55;
    ctx.strokeRect(p.x - 2, p.y - 2, p.w + 4, p.h + 4);
    ctx.globalAlpha = opacity;
}

// task ribbon
if (n.task && n.task.status === "completed") {
    ctx.fillStyle = "rgba(255,255,255,0.25)";
    ctx.fillRect(p.x, p.y, p.w, 3);
} else if (n.task && n.task.priority && n.task.priority !== "none") {

```

```

    ctx.fillStyle = n.task.priority === "high" || n.task.priority === "urgent" ? "#f87171" : n.task.priority === "medium" ? "#fbbf24" : "#a3e635";
    ctx.beginPath();
    ctx.arc(p.x + p.w - 8, p.y + p.h - 8, 3.5, 0, Math.PI * 2);
    ctx.fill();
}

// image between shape and text (ADR-001 §12); skipped while editing -
// the HTML overlay owns it, no double-render (same rule as the text).
const imgH = this.imageH(p);
if (!editing) {
    this.drawImage(p);
    this.drawText(theme, p, textColor, imgH);
}

// collapsed badge (mirror to the left when the branch is left-side)
if (n.collapsed && n.childrenIds.length > 0) {
    const count = this.hiddenCount(n, state.sheet);
    const centerX = p.x + p.w / 2;
    const isLeft = centerX < 0;
    const bx = isLeft ? p.x - 10 : p.x + p.w + 10;
    const by = p.y + p.h / 2;
    ctx.fillStyle = theme.collapsedBadge;
    ctx.beginPath();
    ctx.arc(bx, by, 11, 0, Math.PI * 2);
    ctx.fill();
    ctx.fillStyle = theme.collapsedBadgeText;
    ctx.font = `600 11px system-ui, sans-serif`;
    ctx.textAlign = "center";
    ctx.textBaseline = "middle";
    ctx.fillText(String(count), bx, by + 0.5);
}

ctx.restore();
}

private hiddenCount(n: MindNode, sheet: Sheet): number {
    let count = 0;
    const queue = [...n.childrenIds];
    while (queue.length) {
        const id = queue.shift();
        const c = sheet.nodes[id];
        if (!c) continue;
        count++;
        if (!c.collapsed) queue.push(...c.childrenIds);
    }
    return count;
}

private traceShape(ctx: CanvasRenderingContext2D, p: Placed, shape: string, radius: number): void {
    const { x, y, w, h } = p;
    ctx.beginPath();
    switch (shape) {
        case "rect":
            ctx.rect(x, y, w, h);
            break;
        case "capsule":
            this.roundRect(ctx, x, y, w, h, h / 2);
            break;
        case "circle": {
            const r = Math.max(w, h) / 2;
            ctx.arc(x + w / 2, y + h / 2, r, 0, Math.PI * 2);
            break;
        }
        case "diamond":
            ctx.moveTo(x + w / 2, y);
            ctx.lineTo(x + w, y + h / 2);
            ctx.lineTo(x + w / 2, y + h);
            ctx.lineTo(x, y + h / 2);
            ctx.closePath();
            break;
        case "hexagon":
            ctx.moveTo(x + w * 0.25, y);
            ctx.lineTo(x + w * 0.75, y);
            ctx.lineTo(x + w, y + h / 2);
            ctx.lineTo(x + w * 0.75, y + h);
            ctx.lineTo(x + w * 0.25, y + h);
            ctx.lineTo(x, y + h / 2);
            ctx.closePath();
            break;
        case "underline":
            this.roundRect(ctx, x, y, w, h, radius);
            break;
        case "none":
            // Text-only topics (the central topic in the XMind-inspired theme).
            break;
        default:
            this.roundRect(ctx, x, y, w, h, radius);
    }
}

```

```

    }
}

private roundRect(ctx: CanvasRenderingContext2D, x: number, y: number, w: number, h: number, r: number): void {
    const rr = Math.min(r, w / 2, h / 2);
    ctx.moveTo(x + rr, y);
    ctx.arcTo(x + w, y, x + w, y + h, rr);
    ctx.arcTo(x + w, y + h, x, y + h, rr);
    ctx.arcTo(x, y + h, x, y, rr);
    ctx.arcTo(x, y, x + w, y, rr);
    ctx.closePath();
}

/** Height of the node's image in world units (0 = none), matching measureTopic. */
private imageH(p: Placed): number {
    const n = p.node;
    if (!n.style.image || !this.resolveImage) return 0;
    const att = this.resolveImage(n.style.image);
    if (!att || att.w <= 0) return 0;
    const imgW = n.style.imageWidth ?? Math.min(att.w, MAX_IMAGE_W);
    return (imgW * att.h) / att.w;
}

private drawText(_theme: RenderTheme, p: Placed, color: string, imgH = 0): void {
    const n = p.node;
    const pad = n.style.padding ?? 10;
    const maxW = Math.max(20, p.w - pad * 2 - TEXT_INSET);
    // Resolution bucket: re-render the bitmap only when the zoom crosses a
    // power-of-two boundary; between boundaries pan/zoom just blits.
    const res = Math.max(1, Math.min(4, Math.ceil(this.curScale * this.dpr)));
    const key = this.textCacheKey(n, color, maxW, res);
    let entry = this.textCache.get(key);
    if (entry) this.textHits++;
    if (!entry) {
        this.textMisses++;
        entry = this.renderTextBitmap(n, color, maxW, res);
        this.textCache.set(key, entry);
        if (this.textCache.size > 5000) {
            const first = this.textCache.keys().next().value;
            if (first !== undefined) this.textCache.delete(first);
        }
    }
    const totalH = entry.h;
    // With an image above, the text occupies the space below it and centers
    // inside that area; otherwise it centers in the whole box (as before).
    const startY =
        imgH > 0
            ? p.y + pad + imgH + IMAGE_GAP + Math.max(0, (p.h - pad * 2 - imgH - IMAGE_GAP - totalH) / 2)
            : p.y + p.h / 2 - totalH / 2;
    const startX = p.x + pad;
    if (entry.w > 0 && entry.h > 0) this.ctx.drawImage(entry.canvas, startX, startY, entry.w, entry.h);
}

/**
 * Draw the node's image into its reserved rect (cached bitmap), or start a
 * decode. Sync by design: no await inside the paint path.
 */
private drawImage(p: Placed): void {
    const n = p.node;
    const imageId = n.style.image;
    if (!imageId || !this.resolveImage) return;
    const att = this.resolveImage(imageId);
    if (!att || att.w <= 0) return;
    const imgW = n.style.imageWidth ?? Math.min(att.w, MAX_IMAGE_W);
    const imgH = (imgW * att.h) / att.w;
    // Decode at the size THIS image is painted at, not at the size it happens
    // to be stored at. Driving the choice from the global zoom alone put every
    // bitmap on the 1024px level above zoom 0.5 on a retina screen – 3MB each,
    // so fifty visible image nodes blew the budget and the cache thrashed.
    const neededPx = imgW * this.curScale * this.dpr;
    // Quantised to powers of two: an exact size would mint a new cache key on
    // every micro-change of zoom and nothing would ever hit.
    const bucket = Math.max(this.IMAGE_BUCKET_MIN, Math.min(this.IMAGE_BUCKET_MAX, 2 ** Math.ceil(Math.log2(Math.max(1, neededPx)))));
    // Smallest stored level that can serve the bucket. The original is never
    // decoded: the hard 1024px cap of ADR-001 §12 is enforced here.
    const level: AssetLevel = bucket <= 256 ? "small" : "large";
    // The BUCKET keys the cache, not the level – with a variable decode size,
    // keying by level would reuse a 384px bitmap when 900 are needed and leave
    // that image blurred with nothing to invalidate it.
    const key = `${imageId}@${bucket}`;
    const entry = this.imageCache.get(key);
    if (entry) {
        // LRU refresh (Map insertion order = recency).
        this.imageCache.delete(key);
        this.imageCache.set(key, entry);
        const x = p.x + (p.w - imgW) / 2;
    }
}

```

```

    const y = p.y + (n.style.padding ?? 10);
    this.ctx.drawImage(entry.bitmap, x, y, imgW, imgH);
    return;
}
if (this.imageFailed.has(key)) return; // corrupt/unavailable: not per frame
if (this.inflight.has(key) || this.inflightCount >= this.MAX_INFLIGHT) return;
this.startDecode(key, imageId, n.id, level, bucket);
}

private async startDecode(key: string, assetId: string, nodeId: string, level: AssetLevel, bucket: number): Promise<void> {
    this.inflight.set(key, nodeId);
    this.inflightCount++;
    try {
        const blob = await this.assetStore.get(assetId, level);
        if (!blob) {
            this.imageFailed.add(key);
            return;
        }
        // resizeWidth downsamples DURING decode: the full-resolution surface is
        // never materialised. Height is omitted on purpose – the browser derives
        // it from the aspect ratio, and passing both risks distortion.
        const bitmap = await createImageBitmap(blob, { resizeWidth: bucket, resizeQuality: "high" });
        if (!this.visibleImageNodes.has(nodeId)) {
            // The requesting node scrolled off while decoding: close immediately,
            // never cache it.
            bitmap.close();
            return;
        }
        const bytes = bitmap.width * bitmap.height * 4;
        this.imageCache.set(key, { bitmap, bytes });
        this.imageBytes += bytes;
        this.evictToBudget();
        // A repaint lets the fresh bitmap appear without waiting for the next
        // user gesture (and starts the next pending decodes).
        this.onRepaint?();
    } catch {
        this.imageFailed.add(key);
    } finally {
        if (this.inflight.delete(key)) this.inflightCount--;
    }
}

/**
 * Ghost of the image being dragged to another node: the already-cached
 * bitmap, semi-transparent, centered on the cursor with a dashed outline.
 * Reuses the same bucket/level logic as drawImage so the preview is sharp
 * at the current zoom; if the bitmap isn't decoded yet, starts a decode
 * and draws nothing (a repaint brings it in when ready).
 */
private drawGhostImage(theme: RenderTheme, ghost: { imageId: string; x: number; y: number; nodeId: string }): void {
    const ctx = this.ctx;
    if (!this.resolveImage) return;
    const att = this.resolveImage(ghost.imageId);
    if (!att || att.w <= 0) return;
    const imgW = Math.min(att.w, MAX_IMAGE_W);
    const imgH = (imgW * att.h) / att.w;
    const neededPx = imgW * this.curScale * this.dpr;
    const bucket = Math.max(this.IMAGE_BUCKET_MIN, Math.min(this.IMAGE_BUCKET_MAX, 2 ** Math.ceil(Math.log2(Math.max(1, neededPx)))));
    const level: AssetLevel = bucket <= 256 ? "small" : "large";
    const key = `${ghost.imageId}@${bucket}`;
    const entry = this.imageCache.get(key);
    if (!entry) {
        if (!this.inflight.has(key) && this.inflightCount < this.MAX_INFLIGHT) {
            this.startDecode(key, ghost.imageId, ghost.nodeId, level, bucket);
        }
        return;
    }
    const x = ghost.x - imgW / 2;
    const y = ghost.y - imgH / 2;
    ctx.save();
    ctx.globalAlpha = 0.6;
    ctx.drawImage(entry.bitmap, x, y, imgW, imgH);
    ctx.strokeStyle = theme.selection;
    ctx.lineWidth = 2.5;
    ctx.setLineDash([6, 4]);
    ctx.strokeRect(x, y, imgW, imgH);
    ctx.restore();
}

/** LRU eviction under the byte budget. Only ever called between frames. */
private evictToBudget(): void {
    while (this.imageBytes > this.IMAGE_BUDGET && this.imageCache.size > 0) {
        const first = this.imageCache.entries().next().value;
        if (!first) break;
        const [k, v] = first as [string, { bitmap: ImageBitmap; bytes: number }];
        this.imageCache.delete(k);
        this.imageBytes -= v.bytes;
    }
}

```

```

        v.bitmap.close();
    }
}

private textCacheKey(n: MindNode, color: string, maxW: number, res: number): string {
    const st = n.style;
    const runs = JSON.stringify(n.titleRuns ?? n.title);
    return `${n.id}|${runs}|${st.fontSize ?? 14}|${st.fontFamily ?? ""}|${st.fontWeight ?? 400}|${st.italic ? 1 : 0}|${st.align ??
"center"}|${st.padding ?? 10}|${st.underline ? 1 : 0}|${st.strikethrough ? 1 : 0}|${color}|${maxW}|${res}`;
}

/** Render the styled title once into an offscreen canvas (world-unit sized, `res` pixels per unit). */
private renderTextBitmap(n: MindNode, color: string, maxW: number, res: number): { canvas: HTMLCanvasElement; w: number; h: number } {
    const size = n.style.fontSize ?? 14;
    const lines = wrapRunLines(nodeRuns(n.title, n.titleRuns), maxW, this.measurer, n.style);
    let totalH = 0;
    for (const line of lines) {
        const lh = line.height ?? size * LINE_HEIGHT_FACTOR;
        totalH += lh + (line.gapPx ?? 0);
    }
    const canvas = document.createElement("canvas");
    canvas.width = Math.max(1, Math.ceil(maxW * res));
    canvas.height = Math.max(1, Math.ceil(totalH * res));
    const bctx = canvas.getContext("2d");
    if (!bctx) return { canvas, w: maxW, h: totalH };
    bctx.scale(res, res);
    bctx.textBaseline = "alphabetic";

    const family = n.style.fontFamily ?? FONT_STACK;
    const baseWeight = n.style.fontWeight ?? 400;
    const strike = (n.style.strikethrough ?? false);
    const fontOf = (seg: { run: TextRun }): string => {
        const runSize = seg.run.fontSize ?? size;
        const bold = (seg.run.bold ?? false) || baseWeight >= 700;
        const italic = (seg.run.italic ?? false) || (n.style.italic ?? false);
        return `${italic ? "italic " : ""}${bold ? 700 : baseWeight} ${runSize}px ${family}`;
    };
    let yCursor = 0;
    for (const line of lines) {
        const lh = line.height ?? size * LINE_HEIGHT_FACTOR;
        yCursor += line.gapPx ?? 0;

        // The line box and its baseline come from wrapRunLines, which builds
        // them with the CSS rule (per-inline-box half-leading over every run
        // plus the strut). Recomputing them here – with ONE half-leading for the
        // whole line – is what used to drift from the editor on mixed sizes.
        const baselineY = yCursor + (line.baseline ?? lh * 0.8);

        // 2) draw the bullet in its own fixed-width column, then the text.
        // List items are ALWAYS left-aligned (as in the overlay's CSS):
        // centering a hanging indent is ill-defined and the two sides could
        // never agree on it.
        const indent = line.indent ?? 0;
        if (line.bullet) {
            bctx.font = fontOf({ run: line.bullet.run });
            bctx.fillStyle = line.bullet.run.color ?? color;
            bctx.fillText(line.bullet.char, line.bullet.x, baselineY);
        }
        const isList = indent > 0 || !!line.bullet;
        let x = n.style.align === "left" || isList ? indent : (maxW - line.width) / 2;
        for (const seg of line.segments) {
            const runSize = seg.run.fontSize ?? size;
            const bold = (seg.run.bold ?? false) || baseWeight >= 700;
            bctx.font = fontOf(seg);
            bctx.fillStyle = seg.run.color ?? color;
            const w = this.measurer.measure(seg.text, { fontSize: runSize, fontFamily: family, fontWeight: bold ? 700 : baseWeight, italic: (seg.run.italic
?? false) || (n.style.italic ?? false) }).width;
            bctx.fillText(seg.text, x, baselineY);
            const underline = (seg.run.underline ?? false) || (n.style.underline ?? false);
            if (underline || strike) {
                bctx.strokeStyle = seg.run.color ?? color;
                bctx.lineWidth = 1;
                bctx.beginPath();
                // underline ~0.1em below the baseline, strikethrough ~0.28em above
                const yy = underline ? baselineY + runSize * 0.1 : baselineY - runSize * 0.28;
                bctx.moveTo(x, yy);
                bctx.lineTo(x + w, yy);
                bctx.stroke();
            }
            x += w;
        }
        yCursor += lh;
    }
    return { canvas, w: maxW, h: totalH };
}

private drawRelationship(theme: RenderTheme, a: Placed, b: Placed, color: string, style: string, label?: string, bidirectional?: boolean, selected?:

```

```

boolean): void {
  const ctx = this.ctx;
  const ax = a.x + a.w / 2, ay = a.y + a.h / 2;
  const bx = b.x + b.w / 2, by = b.y + b.h / 2;
  const c1x = ax + (bx - ax) * 0.35, c1y = ay;
  const c2x = bx - (bx - ax) * 0.35, c2y = by;
  ctx.strokeStyle = color;
  ctx.lineWidth = selected ? 2.5 : 1.5;
  ctx.setLineDash(style === "dashed" ? [7, 5] : style === "dotted" ? [2, 4] : []);
  ctx.beginPath();
  ctx.moveTo(ax, ay);
  ctx.bezierCurveTo(c1x, c1y, c2x, c2y, bx, by);
  ctx.stroke();
  ctx.setLineDash([]);
  // Arrowheads: at the target, and at the source when bidirectional.
  const arrow = (fromX: number, fromY: number, toX: number, toY: number): void => {
    const ang = Math.atan2(toY - fromY, toX - fromX);
    const len = 9;
    ctx.fillStyle = color;
    ctx.beginPath();
    ctx.moveTo(toX, toY);
    ctx.lineTo(toX - len * Math.cos(ang - 0.42), toY - len * Math.sin(ang - 0.42));
    ctx.lineTo(toX - len * Math.cos(ang + 0.42), toY - len * Math.sin(ang + 0.42));
    ctx.closePath();
    ctx.fill();
  };
  // tangent at the end of the bezier ≈ direction from c2 to b
  arrow(c2x, c2y, bx, by);
  if (bidirectional) arrow(c1x, c1y, ax, ay);
  if (label) {
    const mx = (ax + bx) / 2, my = (ay + by) / 2;
    ctx.font = `600 12px system-ui, sans-serif`;
    ctx.textAlign = "center";
    ctx.textBaseline = "middle";
    ctx.fillStyle = theme.background;
    const w = ctx.measureText(label).width + 10;
    ctx.fillRect(mx - w / 2, my - 10, w, 20);
    ctx.fillStyle = color;
    ctx.fillText(label, mx, my + 0.5);
  }
}

// -----
// Groups & summaries
// -----

/** Union rect (padded) of the member topics, in world coords. */
private membersBounds(state: RenderState, memberIds: string[], byId?: Map<string, Placed>): { x: number; y: number; w: number; h: number } | null {
  const placed = byId ?? new Map(this.placedNodes(state).map((p) => [p.node.id, p]));
  let minX = Infinity, minY = Infinity, maxX = -Infinity, maxY = -Infinity;
  let found = false;
  for (const m of memberIds) {
    const p = placed.get(m);
    if (!p) continue;
    found = true;
    minX = Math.min(minX, p.x);
    minY = Math.min(minY, p.y);
    maxX = Math.max(maxX, p.x + p.w);
    maxY = Math.max(maxY, p.y + p.h);
  }
  if (!found) return null;
  return { x: minX, y: minY, w: maxX - minX, h: maxY - minY };
}

private drawGroupsAndSummaries(theme: RenderTheme, state: RenderState, byId: Map<string, Placed>): void {
  const s = state.camera.scale;
  for (const g of state.sheet.boundaries) {
    const b = this.membersBounds(state, g.memberIds, byId);
    if (!b) continue;
    // cull: skip if the whole box is off-screen
    if (b.x + b.w < state.camera.x - state.viewW / 2 / s - 40 || b.x > state.camera.x + state.viewW / 2 / s + 40) continue;
    if (b.y + b.h < state.camera.y - state.viewH / 2 / s - 40 || b.y > state.camera.y + state.viewH / 2 / s + 40) continue;
    const pad = 16;
    const selected = state.groupSel === g.id;
    this.drawGroup(theme, g, b, pad, selected, s);
  }
  for (const sum of state.sheet.summaries) {
    const b = this.membersBounds(state, sum.memberIds, byId);
    if (!b) continue;
    if (b.y + b.h < state.camera.y - state.viewH / 2 / s - 40 || b.y > state.camera.y + state.viewH / 2 / s + 40) continue;
    const selected = state.summarySel === sum.id;
    this.drawSummary(theme, sum, b, selected, s, this.summaryFacingLeft(state, b, byId));
  }
}

private drawGroup(theme: RenderTheme, _g: Group, b: { x: number; y: number; w: number; h: number }, pad: number, selected: boolean, s: number): void {
  // Just the dashed boundary – no fill, no label tag (per the XMind-style

```

```

// look: the box only "encapsulates" the topics visually).
const ctx = this.ctx;
const x = b.x - pad, y = b.y - pad, w = b.w + pad * 2, h = b.h + pad * 2;
ctx.strokeStyle = selected ? theme.selection : theme.textMuted;
ctx.lineWidth = selected ? 2.5 / s : 1.5 / s;
ctx.setLineDash([7 / s, 5 / s]);
ctx.beginPath();
ctx.roundRect(x, y, w, h, 10 / s);
ctx.stroke();
ctx.setLineDash([]);
}

/** Which side of the root the summary members sit on: a summary on a left
 * branch opens its brace to the LEFT (facing the tree direction), one on
 * the right opens to the RIGHT. */
private summaryFacingLeft(state: RenderState, b: { x: number; y: number; w: number; h: number }, byId: Map<string, Placed>): boolean {
  const root = byId.get(state.sheet.rootNodeId);
  if (!root) return false;
  return b.x + b.w / 2 < root.x + root.w / 2;
}

private drawSummary(theme: RenderTheme, sum: Summary, b: { x: number; y: number; w: number; h: number }, selected: boolean, s: number, facingLeft:
boolean): void {
  const ctx = this.ctx;
  const y0 = b.y, y1 = b.y + b.h;
  const gap = 22 / s;
  const braceX = facingLeft ? b.x - gap : b.x + b.w + gap;
  const label = sum.label ?? "Summary";
  ctx.font = `600 ${12 / s}px system-ui, sans-serif`;
  const tw = ctx.measureText(label).width;
  const labelW = tw + 16, labelH = 22 / s;
  const midY = (y0 + y1) / 2;
  // brace: ends at the member column, middle vertex pointing away from the
  // tree (right on a right branch, left on a left branch)
  ctx.strokeStyle = selected ? theme.selection : theme.textMuted;
  ctx.lineWidth = selected ? 2.5 / s : 1.5 / s;
  ctx.beginPath();
  this.bracePath(ctx, braceX, y0, y1, (facingLeft ? -1 : 1) * (9 / s));
  ctx.stroke();
  // label box on the far side of the brace
  const bx = facingLeft ? braceX - 16 / s - labelW : braceX + 16 / s;
  const by = midY - labelH / 2;
  ctx.fillStyle = theme.background;
  ctx.strokeStyle = selected ? theme.selection : theme.nodeBorder;
  ctx.lineWidth = 1.2 / s;
  ctx.beginPath();
  ctx.roundRect(bx, by, labelW, labelH, 5 / s);
  ctx.fill();
  ctx.stroke();
  ctx.fillStyle = selected ? theme.selection : theme.text;
  ctx.textAlign = "center";
  ctx.textBaseline = "middle";
  ctx.fillText(label, bx + labelW / 2, by + labelH / 2 + 0.5);
}

/** A curly-brace path: ends at (x, y0/y1) tucked left, vertex pointing right. */
private bracePath(ctx: CanvasRenderingContext2D, x: number, y0: number, y1: number, depth: number): void {
  const span = y1 - y0;
  const gap = Math.min(10, Math.max(4, span / 8));
  const d = depth;
  const mid = (y0 + y1) / 2;
  ctx.moveTo(x, y0);
  ctx.bezierCurveTo(x + d, y0, x + d, y0 + gap, x, y0 + gap);
  ctx.bezierCurveTo(x - d, y0 + gap * 2, x - d, mid - gap, x + d, mid);
  ctx.bezierCurveTo(x - d, mid + gap, x - d, y1 - gap * 2, x, y1 - gap);
  ctx.bezierCurveTo(x + d, y1 - gap, x + d, y1, x, y1);
}

// -----
// Overlay hit testing (relationships, groups, summaries)
// -----

/** Distance from a point to the relationship bezier, in world units. */
hitTestRelationship(state: RenderState, wx: number, wy: number): string | null {
  const byId = new Map(this.placedNodes(state).map((p) => [p.node.id, p]));
  const threshold = 8 / state.camera.scale;
  for (const rel of state.sheet.relationships) {
    const a = byId.get(rel.fromId);
    const b = byId.get(rel.toId);
    if (!a || !b) continue;
    const ax = a.x + a.w / 2, ay = a.y + a.h / 2;
    const bx = b.x + b.w / 2, by = b.y + b.h / 2;
    const c1x = ax + (bx - ax) * 0.35, c1y = ay;
    const c2x = bx - (bx - ax) * 0.35, c2y = by;
    let prevX = ax, prevY = ay;
    for (let t = 0.05; t <= 1.0; t += 0.05) {
      const mt = 1 - t;

```

```

    const px = mt * mt * mt * ax + 3 * mt * mt * t * c1x + 3 * mt * t * t * c2x + t * t * t * t * bx;
    const py = mt * mt * mt * ay + 3 * mt * mt * t * c1y + 3 * mt * t * t * c2y + t * t * t * t * by;
    const dx = px - prevX, dy = py - prevY;
    const len = Math.hypot(dx, dy) || 1e-6;
    const dist = Math.abs((wx - prevX) * dy - (wy - prevY) * dx) / len;
    if (dist < threshold) return rel.id;
    prevX = px;
    prevY = py;
  }
}
return null;
}

hitTestGroup(state: RenderState, wx: number, wy: number): string | null {
  const placed = new Map(this.placedNodes(state).map((p) => [p.node.id, p]));
  for (const g of state.sheet.boundaries) {
    const b = this.membersBounds(state, g.memberIds, placed);
    if (!b) continue;
    const pad = 16;
    if (wx >= b.x - pad && wx <= b.x + b.w + pad && wy >= b.y - pad && wy <= b.y + b.h + pad) return g.id;
  }
  return null;
}

hitTestSummary(state: RenderState, wx: number, wy: number): string | null {
  const placed = new Map(this.placedNodes(state).map((p) => [p.node.id, p]));
  for (const sum of state.sheet.summaries) {
    const b = this.membersBounds(state, sum.memberIds, placed);
    if (!b) continue;
    const s = state.camera.scale;
    const facingLeft = this.summaryFacingLeft(state, b, placed);
    const gap = 22 / s;
    const braceX = facingLeft ? b.x - gap : b.x + b.w + gap;
    // approximate: a box around the brace + label, on the correct side
    const lo = facingLeft ? braceX - 90 / s : braceX - 8 / s;
    const hi = facingLeft ? braceX + 8 / s : braceX + 90 / s;
    if (wx >= lo && wx <= hi && wy >= b.y - 6 && wy <= b.y + b.h + 6) return sum.id;
  }
  return null;
}

private drawDropIndicator(theme: RenderTheme, target: Placed, mode: string): void {
  const ctx = this.ctx;
  ctx.strokeStyle = theme.dropIndicator;
  ctx.fillStyle = theme.dropIndicator;
  ctx.lineWidth = 2.5;
  ctx.setLineDash([]);
  if (mode === "child") {
    ctx.strokeRect(target.x - 3, target.y - 3, target.w + 6, target.h + 6);
  } else if (mode === "before" || mode === "after") {
    const y = mode === "before" ? target.y : target.y + target.h;
    ctx.fillRect(target.x - 4, y - 2, target.w + 8, 4);
  }
}

// -----
// Hit testing
// -----

hitTest(state: RenderState, worldX: number, worldY: number): string | null {
  const placed = this.placedNodes(state).filter((p) => p.visible);
  for (let i = placed.length - 1; i >= 0; i--) {
    const p = placed[i];
    if (worldX >= p.x && worldX <= p.x + p.w && worldY >= p.y && worldY <= p.y + p.h) return p.node.id;
  }
  return null;
}

nodeScreenBounds(state: RenderState, id: string): { x: number; y: number; w: number; h: number } | null {
  const p = this.placedNodes(state).find((p) => p.node.id === id);
  if (!p) return null;
  const s = state.camera.scale;
  const ox = state.viewW / 2 - state.camera.x * s;
  const oy = state.viewH / 2 - state.camera.y * s;
  return { x: p.x * s + ox, y: p.y * s + oy, w: p.w * s, h: p.h * s };
}

/**
 * Hit test for the resize handles: small squares centered on the left and
 * right edges of any SELECTED node (the node being edited is excluded).
 * Returns { id, side } or null. Checked before the body hit test so the
 * handles win even where they overlap the node edge.
 */
/**
 * Hit test for the image resize handle (T14): a small square on the
 * bottom-right corner of the node's image, same grab-box style as
 * hitTestResize. Only when the node has an image and is selected.

```

```

*/
hitTestImageResize(state: RenderState, worldX: number, worldY: number): string | null {
  const hs = 12;
  const ids = new Set(state.selection);
  if (state.imageSel) ids.add(state.imageSel);
  for (const id of ids) {
    if (state.editingId === id) continue;
    const rect = this.imageWorldRect(state, id);
    if (!rect) continue;
    const hx = rect.x + rect.w - hs / 2;
    const hy = rect.y + rect.h - hs / 2;
    if (worldX >= hx && worldX <= hx + hs && worldY >= hy && worldY <= hy + hs) return id;
  }
  return null;
}

/** World-space rect of the node's image (same formula as drawImage), or null. */
imageWorldRect(state: RenderState, id: string): { x: number; y: number; w: number; h: number } | null {
  const p = this.placedNodes(state).find((p) => p.node.id === id);
  if (!p) return null;
  return this.imageRectForPlaced(p);
}

/** World-space rect of the image of an already-placed node, or null. */
private imageRectForPlaced(p: Placed): { x: number; y: number; w: number; h: number } | null {
  const n = p.node;
  const imageId = n.style.image;
  if (!imageId || !this.resolveImage) return null;
  const att = this.resolveImage(imageId);
  if (!att || att.w <= 0) return null;
  const imgW = n.style.imageWidth ?? Math.min(att.w, MAX_IMAGE_W);
  const imgH = (imgW * att.h) / att.w;
  return { x: p.x + (p.w - imgW) / 2, y: p.y + (n.style.padding ?? 10), w: imgW, h: imgH };
}

/**
 * Hit test for the image INSIDE a node: the image is a selectable target
 * of its own (select → Backspace deletes only it; drag moves it to another
 * node). Checked before the node-body hit test so the image wins inside
 * its own rect.
 */
hitTestImage(state: RenderState, worldX: number, worldY: number): string | null {
  const placed = this.placedNodes(state).filter((p) => p.visible);
  for (let i = placed.length - 1; i >= 0; i--) {
    const r = this.imageRectForPlaced(placed[i]);
    if (r && worldX >= r.x && worldX <= r.x + r.w && worldY >= r.y && worldY <= r.y + r.h) {
      return placed[i].node.id;
    }
  }
  return null;
}

hitTestResize(state: RenderState, worldX: number, worldY: number): { id: string; side: "left" | "right" } | null {
  const hs = 12; // grab box slightly larger than the drawn 9px handle
  for (const id of state.selection) {
    if (state.editingId === id) continue;
    const rect = this.nodeWorldRect(state, id);
    if (!rect) continue;
    const hy = rect.y + rect.h / 2 - hs / 2;
    for (const side of ["left", "right"] as const) {
      const hx = (side === "left" ? rect.x : rect.x + rect.w) - hs / 2;
      if (worldX >= hx && worldX <= hx + hs && worldY >= hy && worldY <= hy + hs) return { id, side };
    }
  }
  return null;
}

nodeWorldRect(state: RenderState, id: string): { x: number; y: number; w: number; h: number } | null {
  const p = this.placedNodes(state).find((p) => p.node.id === id);
  if (!p) return null;
  return { x: p.x, y: p.y, w: p.w, h: p.h };
}

/** Visible node ids whose box intersects the given world-space rectangle (marquee). */
nodesInRect(state: RenderState, wx0: number, wy0: number, wx1: number, wy1: number): string[] {
  const minX = Math.min(wx0, wx1), maxX = Math.max(wx0, wx1);
  const minY = Math.min(wy0, wy1), maxY = Math.max(wy0, wy1);
  return this.placedNodes(state)
    .filter((p) => p.visible && p.x < maxX && p.x + p.w > minX && p.y < maxY && p.y + p.h > minY)
    .map((p) => p.node.id);
}

// -----
// Export
// -----

exportPng(state: RenderState, transparent = false, maxSize = 8192): HTMLCanvasElement {

```

```

const bounds = this.exportBounds(state);
const bw = Math.max(1, Math.ceil(bounds.maxX - bounds.minX));
const bh = Math.max(1, Math.ceil(bounds.maxY - bounds.minY));
const scale = Math.min(1, maxSize / Math.max(bw, bh));
const canvas = document.createElement("canvas");
canvas.width = Math.max(1, Math.ceil(bw * scale));
canvas.height = Math.max(1, Math.ceil(bh * scale));
const ctx = canvas.getContext("2d");
const theme = THEMES[state.themeName];
if (!transparent) {
  ctx.fillStyle = theme.background;
  ctx.fillRect(0, 0, canvas.width, canvas.height);
}
this.curScale = scale;
ctx.setTransform(scale, 0, 0, scale, -bounds.minX * scale, -bounds.minY * scale);
this.paintFull(ctx, state, theme);
return canvas;
}

private exportBounds(state: RenderState): { minX: number; minY: number; maxX: number; maxY: number } {
  let minX = Infinity, minY = Infinity, maxX = -Infinity, maxY = -Infinity;
  for (const p of this.placedNodes({ ...state, showHidden: true })) {
    if (p.x < minX) minX = p.x;
    if (p.y < minY) minY = p.y;
    if (p.x + p.w > maxX) maxX = p.x + p.w;
    if (p.y + p.h > maxY) maxY = p.y + p.h;
  }
  if (!isFinite(minX)) return { minX: 0, minY: 0, maxX: 100, maxY: 100 };
  const pad = 30;
  return { minX: minX - pad, minY: minY - pad, maxX: maxX + pad, maxY: maxY + pad };
}

private paintFull(_ctx: CanvasRenderingContext2D, state: RenderState, theme: RenderTheme): void {
  const all = this.placedNodes({ ...state, showHidden: true });
  const byId = new Map(all.map((p) => [p.node.id, p]));
  for (const rel of state.sheet.relationships) {
    const a = byId.get(rel.fromId);
    const b = byId.get(rel.toId);
    if (a && b) this.drawRelationship(theme, a, b, rel.color ?? theme.selection, rel.lineStyle ?? "dashed", rel.label, rel.bidirectional);
  }
  for (const p of all) {
    if (p.node.parentId) {
      const parent = byId.get(p.node.parentId);
      if (parent)
        this.drawConnector(
          parent,
          p,
          state.sheet.structure.structureType,
          state.sheet.structure.orientation,
          this.branchColor(theme, p.node, state.sheet)
        );
    }
  }
  for (const p of all) this.drawNode(theme, p, { ...state, showHidden: true });
  // export with a fixed 1px-ish stroke scale
  this.drawGroupsAndSummaries(theme, { ...state, showHidden: true, camera: { ...state.camera, scale: 1 } }, byId);
}
}

```

## 71. src/ui/CanvasView.tsx

```

import { useCallback, useEffect, useRef, useState, useSyncExternalStore, type CSSProperties, type MouseEvent as RMouseEvent, type PointerEvent as RPointerEvent } from "react";
import { useStore } from "../editor/context";
import { viewSize } from "../editor/view";
import { setExportPngHandler } from "../editor/exportBridge";
import { Renderer, type RenderState } from "../render/renderer";
import { screenToWorld, worldToScreen } from "../render/viewport";
import { imageResolver, measureNode } from "../layout/mindmap";
import { createCanvasTextMeasurer, MAX_IMAGE_W, MIN_TOPIC_W } from "../layout/measure";
import { isDescendantOf } from "../core/tree";
import type { MindNode, Sheet } from "../core/types";
import { RichEditor } from "../RichEditor";
import { installTrace, trace } from "../dev/trace";
import { fetchImageAsFile, firstImageFile, firstUriFromList } from "../editor/externalImage";

/**
 * Accept OS file drags (Explorer) and browser image-URL drags (text/uri-list);
 * anything else (text, links) is left to the browser default.
 */
function acceptsExternalDrop(dt: DataTransfer): boolean {
  const types = [...dt.types];
  return types.includes("Files") || types.includes("text/uri-list");
}

```

```

/**
 * Paint synchronously on every store change. RAF is unreliable in embedded
 * webviews (it never fires when the view is not compositing), so we render
 * directly; culling keeps this cheap even for large maps. A RAF-coalescing
 * fast path can be reintroduced once the WebGPU renderer lands.
 */
function schedulePaint(fn: () => void): void {
  fn();
}

interface DragState {
  dragging: string | null;
  moved: boolean;
  panning: boolean;
  startX: number;
  startY: number;
  lastX: number;
  lastY: number;
  grabOffsetX: number;
  grabOffsetY: number;
  resizing: string | null;
  resizeSide: "left" | "right" | null;
  resizeStartWorldX: number;
  resizeStartWidth: number;
  resizeStartX: number;
  /** Image resize (T14): dragging the image corner handle. */
  imgResizing: string | null;
  imgResizeStartWorldX: number;
  imgResizeStartWidth: number;
  /** Image move: dragging a selected image onto another node. */
  imgDragging: string | null;
  /** Marquee selection: anchor of the box drag (screen coords, null = inactive). */
  marqueeStartX: number | null;
  marqueeStartY: number | null;
  marqueeActive: boolean;
}

/** Extra box width some shapes add beyond the text width (mirrors measure.ts). */
function shapeWidthAllowance(n: MindNode): number {
  const shape = n.style.shape ?? "rounded";
  const fs = n.style.fontSize ?? 14;
  return shape === "diamond" ? fs : shape === "hexagon" ? 14 : 0;
}

const overlayMeasurer = createCanvasTextMeasurer();

/** `imageResolver` memoized on the sheet it was built from. */
let lastResolverSheet: Sheet | null = null;
let lastResolver: ReturnType<typeof imageResolver> | null = null;
function overlayImageResolver(sheet: Sheet): ReturnType<typeof imageResolver> {
  if (lastResolverSheet !== sheet || !lastResolver) {
    lastResolverSheet = sheet;
    lastResolver = imageResolver(sheet);
  }
  return lastResolver;
}

export function CanvasView(): JSX.Element {
  const store = useStore();
  const state = useSyncExternalStore(store.subscribe, store.getSnapshot);
  const canvasRef = useRef<HTMLCanvasElement>(null);
  const rendererRef = useRef<Renderer | null>(null);
  const sizeRef = useRef({ w: viewSize.w, h: viewSize.h });
  const dragRef = useRef<DragState>({
    dragging: null,
    moved: false,
    panning: false,
    startX: 0,
    startY: 0,
    lastX: 0,
    lastY: 0,
    grabOffsetX: 0,
    grabOffsetY: 0,
    resizing: null,
    resizeSide: null,
    resizeStartWorldX: 0,
    resizeStartWidth: 0,
    resizeStartX: 0,
    imgResizing: null,
    imgResizeStartWorldX: 0,
    imgResizeStartWidth: 0,
    imgDragging: null,
    marqueeStartX: null,
    marqueeStartY: null,
    marqueeActive: false,
  });

```

```

// Right/middle-drag pan started over the Lexical overlay (the overlay is a
// sibling of the canvas, so the canvas pointer handlers never see it).
const overlayPanRef = useRef<{ pointerId: number } | null>(null);
// True when an overlay pan actually moved (so a right-drag doesn't leave a
// browser context menu behind; a plain right-click keeps the native menu).
const panMovedRef = useRef(false);
const [, force] = useState(0);
const [panning, setPanning] = useState(false);
const [resizing, setResizing] = useState(false);
const [resizeHover, setResizeHover] = useState(false);
const [imgResizeHover, setImgResizeHover] = useState(false);
const [marquee, setMarquee] = useState<{ x: number; y: number; w: number; h: number } | null>(null);
// Ghost preview of an image being dragged to another node (internal
// reassignment): world coords of the cursor + the image to draw. Read by
// paint/currentRenderState on every frame; the renderer draws the cached
// bitmap semi-transparent until the drop lands.
// True once the user has touched the canvas (pointer/wheel/click): the
// startup auto-fit must NOT override a view the user has already started
// to use – otherwise the fit fires right after a click and the camera
// "jumps" under the cursor.
const interactedRef = useRef(false);
const ghostRef = useRef<{ imageId: string; x: number; y: number; nodeId: string } | null>(null);
// Marquee preview: ids of the topics inside the drag box, live-updated on
// every move so the canvas can paint the "will be selected" rings (the
// box itself is a DOM div; the rings need canvas repaints).
const marqueeSelRef = useRef<Set<string> | null>(null);
// External drag (Explorer / browser): DOM ghost that follows the cursor.
// `src` is either an objectURL (revoke on cleanup) or the image URL from
// text/uri-list. Resolved once per drag, positioned via the el ref (no
// React re-render on every dragover).
const extGhostRef = useRef<{ src: string; revoke: boolean } | null>(null);
const extGhostElRef = useRef<HTMLImageElement | null>(null);
const [extGhostSrc, setExtGhostSrc] = useState<string | null>(null);

const paint = useCallback(() => {
  const renderer = rendererRef.current;
  const canvas = canvasRef.current;
  if (!renderer || !canvas) return;
  const { w, h } = sizeRef.current;
  const s = store.getSnapshot();
  const rs: RenderState = {
    sheet: store.sheet,
    camera: s.camera,
    selection: new Set(s.selection),
    editingId: s.editingId,
    hoverId: s.hoverId,
    drop: s.drop,
    themeName: s.theme,
    viewW: w,
    viewH: h,
    relSel: s.relSel,
    groupSel: s.groupSel,
    summarySel: s.summarySel,
    imageSel: s.imageSel,
    ghostImage: ghostRef.current,
    marqueeSel: marqueeSelRef.current,
  };
  renderer.render(rs);
}, [store]);

const schedule = useCallback(() => {
  schedulePaint(paint);
}, [paint]);

useEffect(() => {
  const canvas = canvasRef.current;
  if (!canvas) return;
  // Image decodes finish asynchronously; the repaint callback brings them
  // on screen without waiting for the next user gesture.
  const renderer = new Renderer(canvas, { onRepaint: schedule });
  rendererRef.current = renderer;

  let firstResize = true;
  const ro = new ResizeObserver(() => {
    const rect = canvas.parentElement!.getBoundingClientRect();
    const old = sizeRef.current;
    const nw = rect.width;
    const nh = rect.height;
    // Keep the map visually anchored when the canvas area resizes
    // (Inspector/panel toggle, window resize). Without this the map jumps
    // sideways by half the size delta: the camera stays centered on the
    // canvas center, so when the panel opens and the canvas narrows the
    // whole map shifts – the "GUI moves the canvas view" bug. The first
    // observation is skipped (sizeRef still holds the 1200x800 default).
    if (!firstResize && old.w > 0 && old.h > 0 && nw > 0 && nh > 0) {
      const dW = nw - old.w;
      const dH = nh - old.h;

```

```

    if (dW !== 0 || dH !== 0) store.panBy(-dW / 2, -dH / 2);
  }
  firstResize = false;
  sizeRef.current = { w: nw, h: nh };
  viewSize.w = nw;
  viewSize.h = nh;
  renderer.resize(canvas, nw, nh);
  schedule();
});
ro.observe(canvas.parentElement!);
renderer.resize(canvas, sizeRef.current.w, sizeRef.current.h);
schedule();
// fit the map into view once layout settles after mount – but only if
// the user hasn't already interacted with the canvas (a click/pan/zoom
// before the timer fires must not be overridden by the startup fit).
const fitTimer = setTimeout(() => {
  if (interactedRef.current) return;
  store.fitView(sizeRef.current.w, sizeRef.current.h);
}, 150);

setExportPngHandler(() => {
  const s = store.getSnapshot();
  const rs: RenderState = {
    sheet: store.sheet,
    camera: s.camera,
    selection: new Set(),
    editingId: null,
    hoverId: null,
    drop: null,
    themeName: s.theme,
    viewW: sizeRef.current.w,
    viewH: sizeRef.current.h,
  };
  const out = renderer.exportPng(rs, false);
  out.toBlob((blob) => {
    if (!blob) return;
    const url = URL.createObjectURL(blob);
    const a = document.createElement("a");
    a.href = url;
    a.download = `${store.doc.doc.title.replace(/[\w-]+/g, "_").png}`;
    a.click();
    setTimeout(() => URL.revokeObjectURL(url), 1000);
  }, "image/png");
  store.toast("Exported as PNG");
});

const unsub = store.subscribe(schedule);
const uninstallTrace = installTrace();

// wheel must be non-passive to preventDefault
const onWheel = (e: WheelEvent): void => {
  // ALWAYS swallow the event, whatever we then decide to do with it.
  // While editing, the pointer sits over the Lexical overlay – a sibling
  // div, not the canvas – so a listener bound to the canvas never saw the
  // event at all: the browser handled it instead and ctrl+wheel zoomed the
  // whole page. Hence binding to the wrapper, which contains both.
  // Wheel is NOT blocked while editing: the overlay's position is derived
  // from the store camera on every render (see editStyle below), so a
  // pan/zoom here keeps the overlay glued to the node.
  e.preventDefault();
  interactedRef.current = true;
  const { w, h } = sizeRef.current;
  const rect = canvas.getBoundingClientRect();
  const sx = e.clientX - rect.left;
  const sy = e.clientY - rect.top;
  const zoom = e.ctrlKey || e.metaKey;
  if (zoom) {
    // Wheel-up zooms in (deltaY < 0). Flipped alongside pan so ctrl+scroll
    // stays coherent for users with inverted/natural-scroll deltas.
    trace.applied("wheel:zoom", { deltaY: Math.round(e.deltaY) });
    store.zoomAt(sx, sy, e.deltaY < 0 ? 1.12 : 1 / 1.12, w, h);
  } else {
    // Scroll down → map content moves up (document-style pan). The sign is
    // flipped from the raw delta so the direction matches the OS-scroll
    // convention instead of appearing inverted.
    trace.applied("wheel:pan", { deltaX: Math.round(e.deltaX), deltaY: Math.round(e.deltaY) });
    store.panBy(-e.deltaX, -e.deltaY);
  }
};

const wheelTarget = canvas.parentElement ?? canvas;
wheelTarget.addEventListener("wheel", onWheel as EventListener, { passive: false });

return () => {
  clearTimeout(fitTimer);
  ro.disconnect();
  unsub();
  wheelTarget.removeEventListener("wheel", onWheel as EventListener);
};

```

```

    uninstallTrace();
    setExportPngHandler(null);
    clearExternalGhost(); // revoke a pending objectURL on unmount
  };
}, [store, schedule]);

// re-paint when state changes (camera, selection, edits, layout...)
useEffect(() => {
  schedule();
  force((n) => n + 1);
}, [state, schedule]);

// -----
// Pointer handling
// -----

const localPoint = (e: RPointerEvent): { x: number; y: number } => {
  const rect = canvasRef.current!.getBoundingClientRect();
  return { x: e.clientX - rect.left, y: e.clientY - rect.top };
};

const onPointerDown = (e: RPointerEvent): void => {
  interactedRef.current = true;
  // Best-effort: capture keeps move/up events coming if the pointer leaves
  // the element mid-gesture. Untrusted/synthetic events have no active
  // pointer and throw here – never let that kill the handler.
  try {
    canvasRef.current!.setPointerCapture(e.pointerId);
  } catch {
    /* capture unavailable (e.g. synthetic event) */
  }
  const s = store.getSnapshot();
  const { x, y } = localPoint(e);
  const drag = dragRef.current;
  drag.startX = x;
  drag.startY = y;
  drag.lastX = x;
  drag.lastY = y;
  drag.moved = false;
  ghostRef.current = null; // no stale ghost between gestures
  marqueeSelRef.current = null; // no stale marquee preview between gestures

  // Panning is right-drag (or middle-drag / pan mode): left-click never
  // pans, it only selects or drags topics. Works while editing too – over
  // the canvas here, over the Lexical overlay via the wrapper handlers
  // (onWrapPointerDown), which capture the pointer because the overlay is a
  // sibling of the canvas and its events never reach this handler.
  if (e.button === 2 || e.button === 1 || s.mode === "pan") {
    drag.panning = true;
    drag.dragging = null;
    setPanning(true);
    return;
  }
  if (e.button !== 0) return;
  drag.panning = false;

  const world = screenToWorld(s.camera, sizeRef.current.w, sizeRef.current.h, x, y);
  const renderer = rendererRef.current!;
  const rs = currentRenderState();

  // Image resize (T14): grab the corner handle on the node's image. Same
  // precedence as the width handle – before the body hit test.
  const imgResizeHit = s.relFrom ? null : renderer.hitTestImageResize(rs, world.x, world.y);
  if (imgResizeHit) {
    const n = store.doc.node(imgResizeHit)!;
    const att = n.style.image ? store.sheet.attachments.find((a) => a.id === n.style.image) : undefined;
    drag.imgResizing = imgResizeHit;
    drag.imgResizeStartWorldX = world.x;
    // imageWidth may be unset – resolve the natural width like the renderer.
    drag.imgResizeStartWidth = n.style.imageWidth ?? (att ? Math.min(att.w, MAX_IMAGE_W) : 0);
    store.beginImageResize(imgResizeHit);
    store.select(imgResizeHit, { additive: false });
    setResizing(true);
    setResizeHover(false);
    return;
  }

  // Xmind-style width resize: grab the handle on the selected node's edge.
  // Checked BEFORE the body hit test so the handle wins on the edge line
  // (skipped while a relationship target is being picked).
  const resizeHit = s.relFrom ? null : renderer.hitTestResize(rs, world.x, world.y);
  if (resizeHit) {
    const rect = renderer.nodeWorldRect(rs, resizeHit.id!);
    const n = store.doc.node(resizeHit.id!);
    drag.resizing = resizeHit.id;
    drag.resizeSide = resizeHit.side;
    drag.resizeStartWorldX = world.x;

```

```

    drag.resizeStartWidth = n.style.width ?? Math.max(MIN_TOPIC_W, rect.w - shapeWidthAllowance(n));
    drag.resizeStartX = rect.x;
    store.beginResize(resizeHit.id);
    store.select(resizeHit.id, { additive: false });
    setResizing(true);
    setResizeHover(false);
    return;
}

// Image body hit (before the node body hit test): the image inside a
// node is a selectable target of its own – a click selects it, a drag
// moves it onto another node. Skipped while picking a relationship
// target, like the resize handles.
const imgHit = s.relFrom ? null : rendererer.hitTestImage(rs, world.x, world.y);
if (imgHit) {
    const rect = rendererer.imageWorldRect(rs, imgHit);
    drag.grabOffsetX = rect ? world.x - rect.x : 0;
    drag.grabOffsetY = rect ? world.y - rect.y : 0;
    store.selectImage(imgHit);
    drag.imgDragging = imgHit;
    drag.dragging = null;
    drag.marqueeStartX = null;
    return;
}

const hit = rendererer.hitTest(rs, world.x, world.y);

if (s.relFrom) {
    if (hit) {
        store.createRelationship(s.relFrom, hit);
        store.clearRelFrom();
    } else {
        store.clearRelFrom();
    }
    return;
}

if (hit) {
    const rect = rendererer.nodeWorldRect(rs, hit);
    drag.grabOffsetX = rect ? world.x - rect.x : 0;
    drag.grabOffsetY = rect ? world.y - rect.y : 0;
    store.select(hit, { additive: e.shiftKey || e.ctrlKey || e.metaKey });
    drag.dragging = hit;
    // Capture the pre-drag position: the final op's `prev` must be this,
    // so undo restores the exact pre-drag state (see commitNodeDrag).
    store.beginNodeDrag(hit);
    drag.marqueeStartX = null;
} else {
    // Overlay objects first: a relationship line, a group box, a summary
    // brace/label – then, on plain empty canvas, begin a marquee box.
    const relId = rendererer.hitTestRelationship(rs, world.x, world.y);
    if (relId) {
        store.selectRelationship(relId);
        drag.marqueeStartX = null;
        return;
    }
    const grpId = rendererer.hitTestGroup(rs, world.x, world.y);
    if (grpId) {
        store.selectGroup(grpId);
        drag.marqueeStartX = null;
        return;
    }
    const sumId = rendererer.hitTestSummary(rs, world.x, world.y);
    if (sumId) {
        store.selectSummary(sumId);
        drag.marqueeStartX = null;
        return;
    }
    // A plain click on empty canvas clears the selection; with Shift held
    // the current selection is kept so the marquee can extend it.
    if (!e.shiftKey && !e.ctrlKey && !e.metaKey) store.clearSelection();
    drag.marqueeStartX = x;
    drag.marqueeStartY = y;
    drag.marqueeActive = false;
}
};

const onPointerMove = (e: RPointerEvent): void => {
    const s0 = store.getSnapshot();
    const drag = dragRef.current;
    const { x, y } = localPoint(e);
    const dx = x - drag.lastX;
    const dy = y - drag.lastY;
    drag.lastX = x;
    drag.lastY = y;

    if (drag.imgResizing) {

```

```

const s = store.getSnapshot();
const world = screenToWorld(s.camera, sizeRef.current.w, sizeRef.current.h, x, y);
const dx = world.x - drag.imgResizeStartWorldX;
store.setImageResizeDraft(drag.imgResizing, drag.imgResizeStartWidth + dx);
drag.moved = true;
return;
}
if (drag.resizing) {
const s = store.getSnapshot();
const world = screenToWorld(s.camera, sizeRef.current.w, sizeRef.current.h, x, y);
const dx = world.x - drag.resizeStartWorldX;
if (drag.resizeSide === "left") {
// Right edge stays anchored; the left edge (position.x) follows the cursor.
store.setResizeDraft(drag.resizing, drag.resizeStartWidth - dx, {
anchorRight: true,
x: drag.resizeStartX + dx,
});
} else {
store.setResizeDraft(drag.resizing, drag.resizeStartWidth + dx);
}
drag.moved = true;
return;
}
if (drag.panning) {
store.panBy(dx, dy);
return;
}
// Marquee: activate once the pointer moves 4px from the anchor, then
// track the box in screen coords (the overlay div is positioned in
// canvas-wrap space, same as localPoint).
const mx0 = drag.marqueeStartX, my0 = drag.marqueeStartY;
if (mx0 !== null && my0 !== null && !drag.marqueeActive) {
if (Math.hypot(x - mx0, y - my0) >= 4) {
drag.marqueeActive = true;
drag.dragging = null;
setMarquee({
x: mx0,
y: my0,
w: Math.abs(x - mx0),
h: Math.abs(y - my0),
});
}
}
if (drag.marqueeActive && mx0 !== null && my0 !== null) {
setMarquee({
x: Math.min(mx0, x),
y: Math.min(my0, y),
w: Math.abs(x - mx0),
h: Math.abs(y - my0),
});
// Live preview of what the release will select: same geometry as the
// commit path (world rect corners -> nodesInRect), repainted now so
// the purple rings follow the box instead of appearing on release.
const s = store.getSnapshot();
const w0 = screenToWorld(s.camera, sizeRef.current.w, sizeRef.current.h, mx0, my0);
const w1 = screenToWorld(s.camera, sizeRef.current.w, sizeRef.current.h, x, y);
const ids = rendererRef.current!.nodesInRect(currentRenderState(), w0.x, w0.y, w1.x, w1.y);
marqueeSelRef.current = ids.length > 0 ? new Set(ids) : null;
schedule();
return;
}
// Image move: the dragged image highlights the node under the pointer as
// its new home (the "child" drop indicator); the source node is never a
// target. Threshold before feedback, same as node dragging. The ghost
// preview follows the cursor (set BEFORE setDrop - the store change
// repaints synchronously and must already see it).
if (drag.imgDragging) {
if (!drag.moved && Math.hypot(x - drag.startX, y - drag.startY) < 4) return;
drag.moved = true;
const s = store.getSnapshot();
const world = screenToWorld(s.camera, sizeRef.current.w, sizeRef.current.h, x, y);
const srcNode = store.doc.node(drag.imgDragging);
ghostRef.current = srcNode?.style.image
? { imageId: srcNode.style.image, x: world.x, y: world.y, nodeId: drag.imgDragging }
: null;
const hit = rendererRef.current!.hitTest(currentRenderState(), world.x, world.y);
if (hit && hit !== drag.imgDragging) {
store.setDrop({ mode: "child", nodeId: hit });
} else {
store.setDrop({ mode: "none", nodeId: drag.imgDragging });
}
}
return;
}
if (!drag.dragging) {
const p = localPoint(e);
const w = screenToWorld(s0.camera, sizeRef.current.w, sizeRef.current.h, p.x, p.y);
const renderer = rendererRef.current;

```

```

const rs = currentRenderState();
const rh = renderer?.hitTestResize(rs, w.x, w.y) ?? null;
const ih = renderer?.hitTestImageResize(rs, w.x, w.y) ?? null;
setResizeHover(!rh);
setImgResizeHover(!ih);
store.setHover(rh || ih ? null : (renderer?.hitTest(rs, w.x, w.y) ?? null));
}
if (!drag.dragging) return;
if (!drag.moved && Math.hypot(x - drag.startX, y - drag.startY) < 4) return;
drag.moved = true;

const s = store.getSnapshot();
const world = screenToWorld(s.camera, sizeRef.current.w, sizeRef.current.h, x, y);
const renderer = rendererRef.current!;
const rs = currentRenderState();
const hit = renderer.hitTest(rs, world.x, world.y);

let dropMode: "child" | "before" | "after" | "floating" | "none" = "none";
if (hit && hit !== drag.dragging && !isDescendantOf(store.doc, drag.dragging, hit)) {
  const r = renderer.nodeWorldRect(rs, hit);
  if (r) {
    const relY = (world.y - r.y) / r.h;
    dropMode = relY < 0.28 ? "before" : relY > 0.72 ? "after" : "child";
    store.setDrop({ mode: dropMode, nodeId: hit });
  }
} else {
  // Only main topics (root children) and floating topics can be pinned to
  // a free position – deeper hierarchical nodes stay in the auto layout.
  const dragged = store.doc.node(drag.dragging);
  const canFloat = !dragged && (!dragged.parentId || dragged.parentId === store.sheet.rootNodeId);
  dropMode = canFloat ? "floating" : "none";
  store.setDrop({ mode: dropMode, nodeId: drag.dragging });
}
// Live follow (fluid drag): while the node is free-floating it follows
// the cursor and its subtree re-flows around it every move. Over a drop
// target it returns to its slot so the reorder indicator stays usable;
// deeper nodes never float, so nothing moves for them.
if (dropMode === "floating") {
  store.setNodeDragDraft(drag.dragging, world.x - drag.grabOffsetX, world.y - drag.grabOffsetY);
} else {
  store.resetNodeDragDraft(drag.dragging);
}
}
};

const onPointerUp = (e: RPointerEvent): void => {
  const drag = dragRef.current;
  // A plain click on empty canvas (marquee never activated): just reset.
  if (drag.marqueeStartX !== null && !drag.marqueeActive) {
    drag.marqueeStartX = null;
    drag.marqueeStartY = null;
  }
  if (drag.marqueeActive) {
    const s = store.getSnapshot();
    const { x, y } = localPoint(e);
    const w0 = screenToWorld(s.camera, sizeRef.current.w, sizeRef.current.h, drag.marqueeStartX!, drag.marqueeStartY!);
    const w1 = screenToWorld(s.camera, sizeRef.current.w, sizeRef.current.h, x, y);
    const ids = rendererRef.current!.nodesInRect(currentRenderState(), w0.x, w0.y, w1.x, w1.y);
    if (ids.length > 0) store.selectMany(ids, { additive: e.shiftKey || e.ctrlKey || e.metaKey });
    marqueeSelRef.current = null;
    drag.marqueeActive = false;
    drag.marqueeStartX = null;
    drag.marqueeStartY = null;
    setMarquee(null);
    schedule();
    return;
  }
  if (drag.imgResizing) {
    store.commitImageResize();
    drag.imgResizing = null;
    drag.moved = false;
    setResizing(false);
    return;
  }
  if (drag.resizing) {
    store.commitResize();
    drag.resizing = null;
    drag.resizeSide = null;
    drag.moved = false;
    setResizing(false);
    return;
  }
  if (drag.panning) {
    drag.panning = false;
    drag.dragging = null;
    setPanning(false);
    return;
  }
}

```

```

// Image move dropped: assign the image to the highlighted node. A plain
// click (no move) only leaves the image selected - never moves it.
if (drag.imgDragging) {
  if (drag.moved) {
    const drop = store.getSnapshot().drop;
    if (drop && drop.mode !== "none" && drop.nodeId !== drag.imgDragging) {
      store.assignImageToNode(drag.imgDragging, drop.nodeId);
    }
  }
  store.setDrop(null);
  ghostRef.current = null;
  drag.imgDragging = null;
  drag.moved = false;
  return;
}
if (drag.dragging && drag.moved) {
  const s = store.getSnapshot();
  const drop = s.drop;
  if (drop && drop.mode !== "none") {
    const { x, y } = localPoint(e);
    const world = screenToWorld(s.camera, sizeRef.current.w, sizeRef.current.h, x, y);
    const target = drop.mode === "floating" ? null : drop.nodeId;
    store.commitNodeDrag(
      drag.dragging,
      target,
      drop.mode,
      world.x - drag.grabOffsetX,
      world.y - drag.grabOffsetY
    );
  }
}
drag.dragging = null;
drag.moved = false;
};

const onDbClick = (e: RMouseEvent): void => {
  const s = store.getSnapshot();
  const { x, y } = localPoint(e as unknown as RPointerEvent);
  const world = screenToWorld(s.camera, sizeRef.current.w, sizeRef.current.h, x, y);
  const renderer = rendererRef.current!;
  const hit = renderer.hitTest(currentRenderState(), world.x, world.y);
  if (hit) {
    store.startEdit(hit);
  } else {
    store.createFloatingAt(world.x - 60, world.y - 18);
  }
};

const onContextMenu = (e: RMouseEvent): void => {
  e.preventDefault();
};

// -----
// Wrapper-level pan over the editing overlay
// -----
// Bound in CAPTURE phase on .canvas-wrap (see the main effect): capture runs
// before any stopPropagation an inner element could do, so right/middle-drag
// over the Lexical overlay pans the map even while editing. The left button
// is never touched here - it stays with the editor (text selection/caret).

const onWrapPointerDown = (e: RPointerEvent): void => {
  if (e.target === canvasRef.current) return; // the canvas path handles it
  if (e.button !== 2 && e.button !== 1) return; // never steal left-click
  if (!store.getSnapshot().editingId) return;
  e.preventDefault();
  const { x, y } = localPoint(e);
  const drag = dragRef.current;
  drag.panning = true;
  drag.dragging = null;
  drag.startX = x;
  drag.startY = y;
  drag.lastX = x;
  drag.lastY = y;
  overlayPanRef.current = { pointerId: e.pointerId };
  panMovedRef.current = false;
  setPanning(true);
  try {
    canvasRef.current!.parentElement!.setPointerCapture(e.pointerId);
  } catch {
    /* capture unavailable (e.g. synthetic event) */
  }
};

const onWrapPointerMove = (e: RPointerEvent): void => {
  if (!overlayPanRef.current) return;
  const drag = dragRef.current;
  const { x, y } = localPoint(e);

```

```

const dx = x - drag.lastX;
const dy = y - drag.lastY;
drag.lastX = x;
drag.lastY = y;
if (Math.hypot(x - drag.startX, y - drag.startY) > 3) panMovedRef.current = true;
store.panBy(dx, dy);
};

const onWrapPointerUp = (e: RPointerEvent): void => {
  if (!overlayPanRef.current) return;
  const drag = dragRef.current;
  drag.panning = false;
  drag.dragging = null;
  setPanning(false);
  const wrap = canvasRef.current?.parentElement;
  if (wrap?.hasPointerCapture(e.pointerId)) {
    try {
      wrap.releasePointerCapture(e.pointerId);
    } catch {
      /* already released */
    }
  }
  overlayPanRef.current = null;
};

const onWrapContextMenu = (e: RMouseEvent): void => {
  // A real right-drag pan must not leave a browser context menu behind;
  // a plain right-click on the editor keeps the native menu (paste etc.).
  if (panMovedRef.current) {
    e.preventDefault();
    panMovedRef.current = false;
  }
};

const currentRenderState = (): RenderState => {
  const s = store.getSnapshot();
  return {
    sheet: store.sheet,
    camera: s.camera,
    selection: new Set(s.selection),
    editingId: s.editingId,
    hoverId: s.hoverId,
    drop: s.drop,
    themeName: s.theme,
    viewW: sizeRef.current.w,
    viewH: sizeRef.current.h,
    relSel: s.relSel,
    groupSel: s.groupSel,
    summarySel: s.summarySel,
    ghostImage: ghostRef.current,
    marqueeSel: marqueeSelRef.current,
  };
};

// -----
// External drag ghost (Explorer / browser drags)
// -----

const clearExternalGhost = useCallback(() => {
  const g = extGhostRef.current;
  if (g?.revoke) URL.revokeObjectURL(g.src);
  extGhostRef.current = null;
  setExtGhostSrc(null);
}, []);

/** Resolve the dragged payload into a previewable src, once per drag. */
const ensureExternalGhost = useCallback((dt: DataTransfer) => {
  if (extGhostRef.current) return; // already resolved for this drag
  const file = firstImageFile(dt.files);
  let src: string | null = null;
  let revoke = false;
  if (file) {
    src = URL.createObjectURL(file);
    revoke = true;
  } else {
    src = firstUriFromList(dt.getData("text/uri-list"));
  }
  if (!src) return;
  extGhostRef.current = { src, revoke };
  setExtGhostSrc(src);
}, []);

// -----
// Inline editor overlay
// -----

let editStyle: CSSProperties | null = null;

```

```

// The overlay wears the node's painted colors; resolving them in the
// renderer keeps branch palettes in one place.
let editColors: { fill: string; text: string } | undefined;
if (state.editingId) {
  const n = store.doc.node(state.editingId);
  if (n) {
    // The overlay box must match the canvas box: same measurer, same image
    // resolver (invariant I9) – otherwise the node jumps on double-click.
    // Memoized on the sheet: building it inline reallocated the map on every
    // render, which is invisible at 20 nodes and is not at 5.000.
    const m = measureNode(n, overlayMeasurer, overlayImageResolver(store.sheet));
    const { x, y } = worldToScreen(state.camera, viewSize.w, viewSize.h, n.position.x, n.position.y);
    editStyle = {
      left: x,
      top: y,
      width: m.w * state.camera.scale,
      height: m.h * state.camera.scale,
    };
    editColors = rendererRef.current?.nodeColors(currentRenderState(), state.editingId) ?? undefined;
  }
}

return (
  <div
    className="canvas-wrap"
    onPointerDownCapture={onWrapPointerDown}
    onPointerMoveCapture={onWrapPointerMove}
    onPointerUpCapture={onWrapPointerUp}
    onContextMenuCapture={onWrapContextMenu}
  >
    <canvas
      ref={canvasRef}
      className="canvas"
      style={{
        cursor:
          imgResizeHover || dragRef.current.imgResizing
            ? "nwse-resize"
          : resizing || resizeHover
            ? "ew-resize"
          : panning
            ? "grabbing"
          : state.mode === "pan"
            ? "grab"
            : "default",
      }}
      onPointerDown={onPointerDown}
      onPointerMove={onPointerMove}
      onPointerUp={onPointerUp}
      onPointerLeave={() => {
        store.setHover(null);
        setResizeHover(false);
        setImgResizeHover(false);
      }}
      onDoubleClick={onDbClick}
      onContextMenu={onContextMenu}
      onDragOver={(e) => {
        // Accept the drop (T13-2): without preventDefault the browser would
        // navigate away on drop. Highlight the node under the cursor while
        // dragging so the user sees the target.
        if (!acceptsExternalDrop(e.dataTransfer)) return;
        e.preventDefault();
        e.dataTransfer.dropEffect = "copy";
        // Ghost preview: resolve the dragged payload once, then pin the
        // image under the cursor (direct DOM update – dragover fires at
        // ~10Hz and React re-renders are wasted here).
        ensureExternalGhost(e.dataTransfer);
        const el = extGhostElRef.current;
        if (el && extGhostRef.current) {
          const rect = canvasRef.current!.getBoundingClientRect();
          el.style.left = `${e.clientX - rect.left + 16}px`;
          el.style.top = `${e.clientY - rect.top + 12}px`;
        }
        const { x, y } = localPoint(e as unknown as RPointerEvent);
        const world = screenToWorld(store.getSnapshot().camera, sizeRef.current.w, sizeRef.current.h, x, y);
        const hit = rendererRef.current?.hitTest(currentRenderState(), world.x, world.y) ?? null;
        store.setHover(hit);
      }}
      onDragLeave={(e) => {
        if (!dragRef.current?.dragging) store.setHover(null);
        // The canvas has no children: relatedTarget is null only when the
        // drag actually leaves the canvas area – clear the ghost then.
        // Moving over the editing overlay (a sibling) keeps the preview.
        if (!e.relatedTarget) clearExternalGhost();
      }}
      onDrop={async (e) => {
        // External sources: an OS file (Explorer, or a browser image that
        // Chromium materializes as a temp file) OR a browser image URL

```

```

    // (text/uri-list, the only payload some browsers hand over).
    if (!acceptsExternalDrop(e.dataTransfer)) return;
    e.preventDefault();
    clearExternalGhost();
    const { x, y } = localPoint(e as unknown as RPointerEvent);
    const world = screenToWorld(store.getSnapshot().camera, sizeRef.current.w, sizeRef.current.h, x, y);
    const target = rendererRef.current?.hitTest(currentRenderState(), world.x, world.y) ?? null;
    store.setHover(null);
    if (!target) {
      store.toast("Drop the image on a topic");
      return;
    }
    let file = firstImageFile(e.dataTransfer.files);
    if (!file) {
      const url = firstUriFromList(e.dataTransfer.getData("text/uri-list"));
      if (!url) {
        store.toast("Drop an image file or a browser image on a topic");
        return;
      }
      file = await fetchImageAsFile(url);
      if (!file) {
        store.toast("Could not load the image from its URL");
        return;
      }
    }
    const res = await store.attachImageFile(target, file);
    if (!res.ok) store.toast(res.reason ?? "Could not import image");
  }}
/>
{state.relFrom && <div className="rel-pending">Click a target topic to link – Esc cancels</div>}
{extGhostSrc && <img ref={extGhostElRef} className="ext-drop-ghost" src={extGhostSrc} alt="" draggable={false} />}
{marquee && (
  <div
    className="marquee"
    style={{ left: marquee.x, top: marquee.y, width: marquee.w, height: marquee.h }}
  />
)}
{editStyle && state.editingId && (
  <RichEditor
    key={state.editingId}
    node={store.doc.node(state.editingId)!}
    style={editStyle}
    scale={state.camera.scale}
    colors={editColors}
  />
)}
</div>
);
}

```

## 72. src/editor/store.ts

```

/**
 * EditorStore – the editor interaction layer.
 *
 * Owns all editor state (documents, selection, camera, editing, drop target,
 * UI flags) and exposes commands. Every mutation that touches the document
 * goes through the operation system + history, so undo/redo and (later) the
 * collaboration layer get a single consistent stream of ops.
 *
 * The store is framework-free: React subscribes via useSyncExternalStore.
 */
import { DocumentModel, nowIso, uid } from "../core/doc";
import { History } from "../core/history";
import { applyWithInverse, makeOp, type Op } from "../core/ops";
import { trace } from "../dev/trace";
import { SCHEMA_VERSION, type AttachmentInfo, type Group, type MindNode, type NodeType, type Position, type Relationship, type RnodeDocument, type Sheet, type Style, type Summary, type TaskInfo, type TextRun } from "../core/types";
import { isEmptyRuns, nodeRuns, normalizeRuns, plainToRuns, runsEqual, runsToPlain, trimRuns } from "../core/text";
import { applyLayout, layoutSheet } from "../layout/mindmap";
import { createCanvasTextMeasurer, measureNode, MIN_TOPIC_W, type TextMeasurer } from "../layout/measure";
import { centerOn, fitBounds, panBy, zoomAt, type Camera } from "../render/viewport";
import { THEMES } from "../render/theme";
import type { DropIndicator } from "../render/renderer";
import { LocalStorageAdapter, TauriStorageAdapter, type StorageAdapter } from "../persist/storage";
import { getAssetStore, referencedAssetIds, TauriAssetStore } from "../persist/assets";
import { buildRnodeZip, estimateRnodeZip, importRnodeZip, type RnodeZipMode } from "../exportBridge";
import { importImageFile, validateImageSource, type ImportedImage } from "../imageImport";

declare global {
  interface Window {
    showSaveFilePicker?: (opts?: {
      suggestedName?: string;
      types?: { description?: string; accept: Record<string, string[]> }[];
    }) => Promise<File | null>;
  }
}

```

```

    }) => Promise<FileSystemFileHandle>;
  }
}

/** Human-readable size for the pre-export estimate toast. */
function formatBytes(bytes: number): string {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`;
  return `${(bytes / (1024 * 1024)).toFixed(1)} MB`;
}

export type PortableFormat = "json" | "zip";

/**
 * Storage key of the picked file handle for a document+format pair. The
 * portable format of a document CHANGES when images appear or disappear
 * (save switches between .rnode.json and .rnode.zip): a key on the docId
 * alone would silently write zip bytes into the user's .json file. One
 * handle per format means removing the images later goes back to the
 * original file, and the format switch merely re-asks where to save.
 */
export function portableFileKey(docId: string, format: PortableFormat): string {
  return `r-node.file-handle.${docId}.${format}`;
}

/**
 * Filesystem-safe base name for a document title. The name typed in the GUI
 * is the name of the real file (desktop .rnode and the portable saves): the
 * save dialog is pre-filled with it and a later rename renames the file.
 */
export function docFileBaseName(title: string): string {
  const base = title.replace(/^[^w-]+/g, "_").trim();
  return base || "Untitled map";
}

/** Directory of a file path (with trailing separator), or "" when none. */
function pathDirname(p: string): string {
  const i = Math.max(p.lastIndexOf("\\"), p.lastIndexOf("/"));
  return i < 0 ? "" : p.slice(0, i + 1);
}

/**
 * Validate/sanitize titleRuns coming from an imported document: only plain
 * text with bold/italic/underline/color survives; anything else is dropped.
 * Falls back to a single plain run of `title` when no valid runs are present.
 */
function sanitizeTitleRuns(raw: unknown): TextRun[] | undefined {
  if (!Array.isArray(raw)) return undefined;
  const runs: TextRun[] = [];
  for (const r of raw) {
    if (!r || typeof r !== "object") continue;
    const text = typeof (r as { text?: unknown }).text === "string" ? (r as { text: string }).text : "";
    if (text.length === 0) continue;
    const run: TextRun = { text };
    if ((r as { bold?: unknown }).bold === true) run.bold = true;
    if ((r as { italic?: unknown }).italic === true) run.italic = true;
    if ((r as { underline?: unknown }).underline === true) run.underline = true;
    const color = (r as { color?: unknown }).color;
    if (typeof color === "string" && /^#[0-9a-fA-F]{3,8}$/.test(color)) run.color = color;
    runs.push(run);
  }
  if (runs.length === 0) return undefined;
  return normalizeRuns(runs);
}

export type NavDir = "up" | "down" | "left" | "right";

export interface EditorState {
  docs: RnodeDocument[];
  activeDocId: string;
  selection: string[];
  camera: Camera;
  editingId: string | null;
  drop: DropIndicator | null;
  hoverId: string | null;
  showPalette: boolean;
  showOutliner: boolean;
  showInspector: boolean;
  search: string;
  searchResults: string[];
  searchIndex: number;
  /** Initial content for the inline editor: set by type-to-edit / paste-to-edit. */
  pendingInsert: string | null;
  sync: "saved" | "dirty";
  message: string | null;
  canUndo: boolean;
  canRedo: boolean;

```

```

docTitle: string;
theme: "light";
structureType: string;
mode: "select" | "pan";
zen: boolean;
relFrom: string | null;
/** Selected overlay object: a relationship, group or summary (not a node). */
relSel: string | null;
groupSel: string | null;
summarySel: string | null;
/** The node whose IMAGE is selected (image selection is exclusive: the
 * node itself is not in `selection` then). Null = no image selected. */
imageSel: string | null;
}

export class EditorStore {
  private state: EditorState;
  private snap: EditorState;
  private listeners = new Set<() => void>();
  private model: DocumentModel;
  private history = new History();
  private adapter: StorageAdapter;
  /**
   * File System Access handles (per document) so later saves silently
   * OVERWRITE the .rnode.json the user picked instead of re-downloading.
   * Handles are persisted in IndexedDB to survive reloads.
   */
  /**
   * Sanitized document title when the current desktop root was established
   * (open or save-as). A rename in the GUI changes the title; on save the
   * REAL file is renamed to match ONLY when the title moved away from this
   * baseline – a file opened with a different internal title is never
   * silently renamed on the first save. Null = no desktop root yet.
   */
  private rootTitleBase: string | null = null;
  private fileHandles = new Map<string, FileSystemFileHandle>();
  private layoutTimer: ReturnType<typeof setTimeout> | null = null;
  private msgTimer: ReturnType<typeof setTimeout> | null = null;
  /**
   * Live rich-text draft of the inline editor (TextRun[]), kept in sync by
   * the Lexical overlay on every change. The canvas pointer handler runs
   * BEFORE the editor blur, so selection changes commit the draft instead of
   * losing it. While editing, the node's title/titleRuns mirror this draft so
   * the layout engine measures the topic at its live size (30ms debounce).
   */
  private editingDraftRuns: TextRun[] | null = null;
  /**
   * The node's committed title/titleRuns when editing started – used as the
   * `prev` of the final setTitle op (undo restores exactly this) and to
   * restore the node when the edit is cancelled (Escape).
   */
  private editOriginal: { title: string; titleRuns?: TextRun[] } | null = null;
  /**
   * Canvas resize drag (Xmind-style handles on the selected node's edges).
   * setResizeDraft mutates the node's width (and, for a left-edge drag, its
   * position.x with a transient manual flag) ephemerally – no op, no history
   * – so the layout re-wraps the text live during the drag. commitResize
   * emits ONE batch op (setStyle + optional setPosition), so undo restores
   * the exact pre-drag state.
   */
  private resizeState: { nodeId: string; original: Style; origPos: Position } | null = null;
  private imageResizeState: { nodeId: string; original: Style } | null = null;
  /** Canvas-backed measurer: layout and renderer agree on every topic size. */
  private measurer: TextMeasurer = createCanvasTextMeasurer();

  constructor(adapter: StorageAdapter = new LocalStorageAdapter()) {
    this.adapter = adapter;
    this.model = new DocumentModel(DocumentModel.sample());
    this.state = this.makeState();
    this.snap = this.state;
    this.normalizeBranchColors(this.model.sheet);
  }

  // -----
  // React binding
  // -----

  subscribe = (cb: () => void): (() => void) => {
    this.listeners.add(cb);
    return () => this.listeners.delete(cb);
  };

  getSnapshot = (): EditorState => this.snap;

  get doc(): DocumentModel {
    return this.model;
  }
}

```

```

get sheet(): Sheet {
    return this.model.sheet;
}

private makeState(): EditorState {
    const d = this.model.doc;
    return {
        docs: [d],
        activeDocId: d.documentId,
        selection: [],
        camera: { x: 0, y: 0, scale: 1 },
        editingId: null,
        drop: null,
        hoverId: null,
        showPalette: false,
        showOutliner: false,
        showInspector: true,
        search: "",
        searchResults: [],
        searchIndex: 0,
        pendingInsert: null,
        sync: "saved",
        message: null,
        canUndo: false,
        canRedo: false,
        docTitle: d.title,
        theme: "light",
        structureType: d.sheets[0].structure.structureType,
        mode: "select",
        zen: false,
        relFrom: null,
        relSel: null,
        groupSel: null,
        summarySel: null,
        imageSel: null,
    };
}

private notify(): void {
    const d = this.model.doc;
    this.state = {
        ...this.state,
        docs: this.state.docs,
        canUndo: this.history.canUndo,
        canRedo: this.history.canRedo,
        docTitle: d.title,
        theme: "light",
        structureType: d.sheets[0].structure.structureType,
    };
    this.snap = { ...this.state, selection: [...this.state.selection], searchResults: [...this.state.searchResults] };
    for (const l of this.listeners) l();
}

toast(msg: string): void {
    this.state.message = msg;
    this.notify();
    if (this.msgTimer) clearTimeout(this.msgTimer);
    this.msgTimer = setTimeout(() => {
        this.state.message = null;
        this.notify();
    }, 2200);
}

// -----
// Init
// -----

async init(): Promise<void> {
    const docs = await this.adapter.load();
    if (docs.length > 0) {
        this.model = new DocumentModel(docs[0]);
    } else {
        // First run: start from the sample map. Nothing is persisted until the
        // user presses Save / Ctrl+S - the sample acts as an in-memory draft.
        this.model = new DocumentModel(DocumentModel.sample());
    }
    this.normalizeBranchColors(this.model.sheet);
    this.state = this.makeState();
    if (docs.length > 0) this.state.docs = docs;
    if (docs.length === 0) this.state.sync = "dirty";
    // Loading a document must respect positions explicitly placed by the
    // user. Forced layout is reserved for the explicit "Auto layout" command.
    this.scheduleLayout(false);
    this.notify();
}

```

```

// -----
// Op execution
// -----

/** Apply ops, record them in history, mark dirty, schedule layout. */
execOps(ops: Op[], opts?: { skipHistory?: boolean }): void {
  if (ops.length === 0) return;
  const t = typeof performance !== "undefined" ? performance.now() : 0;
  const inverses: Op[][] = [];
  for (const op of ops) inverses.push(applyWithInverse(this.sheet, op));
  if (!opts?.skipHistory) this.history.push(ops, inverses);
  trace.op(ops.map((o) => o.type).join(","), ops.length, (typeof performance !== "undefined" ? performance.now() : 0) - t);
  this.touch();
}

undo(): void {
  const ops = this.history.undo();
  if (!ops) return;
  for (const op of ops) applyWithInverse(this.sheet, op);
  this.clearSelection();
  this.touch();
}

redo(): void {
  const ops = this.history.redo();
  if (!ops) return;
  for (const op of ops) applyWithInverse(this.sheet, op);
  this.clearSelection();
  this.touch();
}

private touch(): void {
  this.model.doc.updatedAt = nowIso();
  this.scheduleLayout(false);
  this.state.sync = "dirty";
  this.notify();
}

// -----
// Layout (derived data – never in history)
// -----

private scheduleLayout(force: boolean, clearManual = false): void {
  if (this.layoutTimer) clearTimeout(this.layoutTimer);
  this.layoutTimer = setTimeout(() => {
    const t = typeof performance !== "undefined" ? performance.now() : 0;
    applyLayout(this.sheet, force, this.mesurer, clearManual);
    trace.layout(Object.keys(this.sheet.nodes).length, (typeof performance !== "undefined" ? performance.now() : 0) - t);
    this.notify();
  }, 30);
}

/**
 * Apply the pending layout IMMEDIATELY (no 30ms debounce). Used after
 * structural ops (node creation) so the result appears in its FINAL
 * position on the very first paint – a new topic must not flash at the
 * provisional estimated spot and then jump to its layout slot 30ms later.
 */
private settleLayoutNow(): void {
  if (this.layoutTimer) {
    clearTimeout(this.layoutTimer);
    this.layoutTimer = null;
  }
  applyLayout(this.sheet, false, this.mesurer);
  this.notify();
}

// -----
// Save / Load (manual – the user decides when to persist)
// -----

/**
 * Save the workspace: persist all docs to the storage adapter AND write the
 * portable .rnode.json. The first save lets the user pick the file location
 * (File System Access API); every later save silently OVERWRITES that same
 * file, so the file on disk is always the current version. Falls back to a
 * plain download (current content) where the API is unavailable.
 */
async saveNow(): Promise<void> {
  // A Ctrl+S pressed while the inline editor is open must save the text the
  // user is typing: commit the draft without closing the editor, so they
  // can keep going. The later blur/Enter commit finds the title up to date.
  this.commitDraftKeepEditing();
  try {
    if (this.adapter instanceof TauriStorageAdapter) {
      // Desktop: the document IS a single .rnode file – save writes the
      // document row only. The first save has no file yet: it becomes a

```

```

    // "Save as..." that picks one, copies the referenced assets into it
    // and switches both the adapter and the asset store to it. Every
    // later save just rewrites the document row (T20).
    if (!this.adapter.hasRoot) {
        const ok = await this.saveAsDesktop();
        if (!ok) {
            this.state.sync = "dirty";
            this.toast("Save cancelled – no file chosen");
            return;
        }
    } else {
        // A rename in the GUI becomes a rename of the REAL file: if the
        // title moved away from the file's name, save writes the document
        // under the new name (and drops the old file). Otherwise the file
        // is just rewritten in place.
        const renamedTo = await this.renameDesktopFile();
        if (!renamedTo) await this.adapter.save([this.model.doc]);
    }
    this.state.sync = "saved";
    this.toast("Saved");
    return;
}

await this.adapter.save(this.state.docs);
// Images live outside the document (T12a): a plain .rnode.json would
// arrive without them. With images the portable save is a .rnode.zip.
const hasImages = referencedAssetIds(this.sheet).size > 0;
const fileWritten = hasImages
    ? await this.writePortableZip()
    : await this.writePortableFile(JSON.stringify(this.model.doc, null, 2));
this.state.sync = "saved";
this.toast(fileWritten ? "Saved" : "Saved locally (no file chosen)");
} catch {
    this.state.sync = "dirty";
    this.toast("Save failed – check storage");
}
}

/**
 * Desktop "Save as...": native file picker → copy every referenced asset
 * into the chosen `.rnode` file (all three levels + meta – the same
 * per-asset data the zip exporter iterates, inside SQLite instead of an
 * archive), then write the document and switch the storage adapter AND the
 * asset store to the file. The Renderer holds the same asset store
 * instance, so it starts reading the new file without any re-instantiation
 * (the T19 trap, unchanged).
 */
async saveAsDesktop(): Promise<boolean> {
    // The save dialog already knows the name: pre-fill it with the document
    // title typed in the GUI, so the real file takes that name.
    const file = await this.pickDesktopFile("save", docFileName(this.model.doc.title));
    if (!file) return false;
    const assetStore = getAssetStore();
    if (!(assetStore instanceof TauriAssetStore) || !(this.adapter instanceof TauriStorageAdapter)) {
        return false;
    }
    // adoptFile reads from the current path FIRST, then switches: the assets
    // are never re-written into the same store before the file exists.
    await assetStore.adoptFile(file, [...referencedAssetIds(this.sheet)]);
    this.adapter.setRoot(file);
    this.rootTitleBase = docFileName(this.model.doc.title);
    // The ACTIVE document is saved, not docs[0]: on desktop the sidebar may
    // still carry the never-saved sample, and saving it over the chosen file
    // would corrupt the document on disk.
    await this.adapter.save([this.model.doc]);
    return true;
}

/**
 * Desktop rename-on-save: when the GUI title moved away from the title the
 * current file was opened/saved with, the REAL file is renamed to match
 * (assets adopted into the new file first, old file removed only after the
 * new one is fully written and the app switched to it). Returns the new
 * file name, or null when nothing was renamed (then the caller saves to
 * the current file as usual).
 */
private async renameDesktopFile(): Promise<string | null> {
    if (!(this.adapter instanceof TauriStorageAdapter)) return null;
    const current = this.adapter.currentPath;
    if (!current || this.rootTitleBase === null) return null;
    const base = docFileName(this.model.doc.title);
    if (base === this.rootTitleBase) return null; // title did not change
    const target = pathDirname(current) + base + ".rnode";
    if (target.toLowerCase() === current.toLowerCase()) return null; // same file
    const assetStore = getAssetStore();
    if (!(assetStore instanceof TauriAssetStore) || typeof window === "undefined" || !window.__TAURI__) {
        return null;
    }
}

```

```

try {
  const exists = (await window.__TAURI__.core.invoke("document_file_exists", { path: target })) as boolean;
  if (exists) {
    this.toast(`A file named "${base}.rnode" already exists – saved to the current file`);
    return null;
  }
  // Same order as save-as: read from the old file, write into the new,
  // only then switch. The old file is removed last, so a failure mid-way
  // leaves the document intact at its previous path.
  await assetStore.adaptFile(target, [...referencedAssetIds(this.sheet)]);
  this.adapter.setRoot(target);
  this.rootTitleBase = base;
  await this.adapter.save([this.model.doc]);
  await window.__TAURI__.core.invoke("remove_document", { path: current }).catch((e) => {
    console.warn("remove_document failed", e); // best-effort: the new file is already saved
  });
  return `${base}.rnode`;
} catch (e) {
  this.toast("Rename failed – saved to the current file");
  console.warn("renameDesktopFile failed", e);
  return null;
}
}

/**
 * Desktop "Open...": native file picker → read the document from the file →
 * switch both the adapter and the asset store to it. The path is switched
 * only after the document is known to be valid, so a wrong file never moves
 * the app away from the current document.
 */
async openDesktop(): Promise<boolean> {
  const file = await this.pickDesktopFile("open");
  if (!file) return false;
  const assetStore = getAssetStore();
  if (!(assetStore instanceof TauriAssetStore) || !(this.adapter instanceof TauriStorageAdapter)) {
    return false;
  }
  const doc = await this.adapter.readDocumentAt(file);
  if (!doc) {
    this.toast("Not a valid R-node document in that file");
    return false;
  }
  assetStore.setRoot(file);
  this.adapter.setRoot(file);
  this.importDocumentFromJson(JSON.stringify(doc));
  // Baseline for rename-on-save: the OPENED document's own title. Opening a
  // file whose internal title differs from its file name must never rename
  // the file on the first save – only a later GUI rename does.
  this.rootTitleBase = doc.fileName(doc.title);
  this.toast("Opened document file");
  return true;
}

/**
 * Re-entrancy guard: the native dialog is modal, so a second click while
 * one is open must not queue another dialog – stacked pickers were part of
 * the "Open keeps reopening" report. The flag is reset when the dialog
 * closes (picked or cancelled), never by a timeout.
 */
private pickBusy = false;

private async pickDesktopFile(mode: "open" | "save", suggestedName?: string): Promise<string | null> {
  if (typeof window === "undefined" || !window.__TAURI__) return null;
  if (this.pickBusy) return null;
  this.pickBusy = true;
  try {
    return (
      ((await window.__TAURI__.core.invoke("pick_document_file", {
        mode,
        suggestedName: mode === "save" ? suggestedName : undefined,
      })) as string | null) ?? null
    );
  } catch {
    return null;
  } finally {
    this.pickBusy = false;
  }
}

/**
 * Write a portable file (`.rnode.json` or `.rnode.zip`): stored file handle
 * → silent overwrite; File System Access picker → user chooses once, then
 * every later save overwrites the same file; download as fallback. Returns
 * true when a file was written or downloaded, false when the user cancelled
 * the picker (the document is still persisted to app storage).
 */
private async writePortableBytes(

```

```

blob: Blob,
fileName: string,
pickerType: { description: string; accept: Record<string, string[]> },
format: PortableFormat
): Promise<boolean> {
  const docId = this.model.doc.documentId;
  const key = portableFileKey(docId, format);
  const otherKey = portableFileKey(docId, format === "zip" ? "json" : "zip");

  // 1) Reuse the stored handle for THIS format → silent overwrite, no
  //     dialog, no download. The other format's handle is left alone: if the
  //     document loses its images later, the save goes back to that file.
  let handle = this.fileHandles.get(key) ?? null;
  if (!handle) {
    handle = await this.loadFileHandle(key);
    if (handle) this.fileHandles.set(key, handle);
  }
  if (handle) {
    try {
      const writable = await handle.createWritable();
      await writable.write(blob);
      await writable.close();
      return true;
    } catch {
      // Handle stale (file moved/deleted) → drop it and ask again.
      this.fileHandles.delete(key);
      await this.clearFileHandle(key);
    }
  }

  // 2) No handle but the API exists → let the user pick where to save.
  if (typeof window !== "undefined" && typeof window.showSaveFilePicker === "function") {
    // Reaching the picker with a handle for the OTHER format means the
    // portable format of this document just changed: say why, instead of
    // silently asking for a new location.
    const otherHandle = this.fileHandles.get(otherKey) ?? (await this.loadFileHandle(otherKey));
    if (otherHandle) {
      this.toast(
        format === "zip"
          ? "The document now contains images – choose where to save the .rnode.zip"
          : "The images were removed – choose where to save the .rnode.json"
      );
    }
    try {
      const picked = await window.showSaveFilePicker({ suggestedName: fileName, types: [pickerType] });
      const writable = await picked.createWritable();
      await writable.write(blob);
      await writable.close();
      this.fileHandles.set(key, picked);
      await this.storeFileHandle(key, picked);
      return true;
    } catch (e) {
      if (e instanceof DOMException && e.name === "AbortError") return false;
      // picker failed (permission/unsupported) → fall back to download
    }
  }

  // 3) Fallback: download the file with the CURRENT content.
  this.download(blob, fileName);
  return true;
}

/**
 * Referenced assets whose original bytes are gone (a compact .rnode.zip
 * import). A complete export of them would silently ship the resized
 * level, so the count is shown before generating (T18-B).
 */
private degradedAssetCount(): number {
  const referenced = referencedAssetIds(this.sheet);
  return this.sheet.attachments.filter((a) => a.originalLost && referenced.has(a.id)).length;
}

private async writePortableFile(json: string): Promise<boolean> {
  return this.writePortableBytes(
    new Blob([json], { type: "application/json" }),
    this.docFileName(),
    { description: "R-node document", accept: { "application/json": [".rnode.json", ".json"] } },
    "json"
  );
}

/**
 * Portable save with images. The size is estimated and shown BEFORE the
 * zip is built – generating first and telling the user after would be the
 * wrong order for a file that can weigh hundreds of MB.
 */
private async writePortableZip(): Promise<boolean> {

```

```

const store = getAssetStore();
const estimate = await estimateRnodeZip(this.model.doc, this.sheet, store, "complete");
const degraded = this.degradedAssetCount();
const est = formatBytes(estimate);
this.toast(
  degraded > 0
    ? `${degraded} asset${degraded === 1 ? "" : "s"} lost their original in a compact import – exporting display levels (~${est})`
    : `Exporting .rnode.zip (~${est})...`
);
const bytes = await buildRnodeZip(this.model.doc, this.sheet, store, "complete");
return this.writePortableBytes(
  new Blob([bytes.slice().buffer], { type: "application/zip" }),
  this.docZipName(),
  { description: "R-node document with images", accept: { "application/zip": [".rnode.zip"] } },
  "zip"
);
}

/**
 * Manual export in either mode (complete = originals, compact = display
 * levels only). Downloads the file directly: the save flow owns the file
 * handle, an explicit export asks the OS for a location every time.
 */
async exportRnodeZip(mode: RnodeZipMode): Promise<void> {
  const store = getAssetStore();
  const estimate = await estimateRnodeZip(this.model.doc, this.sheet, store, mode);
  const est = formatBytes(estimate);
  // Compact is explicitly lossy, so only complete mode must warn about
  // assets whose originals are gone (T18-B).
  const degraded = mode === "complete" ? this.degradedAssetCount() : 0;
  this.toast(
    degraded > 0
      ? `${degraded} asset${degraded === 1 ? "" : "s"} lost their original in a compact import – exporting display levels (~${est})`
      : `Exporting .rnode.zip (~${est})...`
  );
  const bytes = await buildRnodeZip(this.model.doc, this.sheet, store, mode);
  this.download(new Blob([bytes.slice().buffer], { type: "application/zip" }), this.docZipName());
}

private async storeFileHandle(key: string, handle: FileSystemFileHandle): Promise<void> {
  if (typeof indexedDB === "undefined") return;
  try {
    const db = await this.openIdb();
    await new Promise<void>((resolve, reject) => {
      const tx = db.transaction("file-handles", "readwrite");
      tx.objectStore("file-handles").put(handle, key);
      tx.oncomplete = () => resolve();
      tx.onerror = () => reject(tx.error);
    });
  } catch {
    /* handle persistence is best-effort */
  }
}

private async loadFileHandle(key: string): Promise<FileSystemFileHandle | null> {
  if (typeof indexedDB === "undefined") return null;
  try {
    const db = await this.openIdb();
    return await new Promise<FileSystemFileHandle | null>((resolve, reject) => {
      const tx = db.transaction("file-handles", "readonly");
      const req = tx.objectStore("file-handles").get(key);
      req.onsuccess = () => resolve((req.result as FileSystemFileHandle | undefined) ?? null);
      req.onerror = () => reject(req.error);
    });
  } catch {
    return null;
  }
}

private async clearFileHandle(key: string): Promise<void> {
  if (typeof indexedDB === "undefined") return;
  try {
    const db = await this.openIdb();
    await new Promise<void>((resolve, reject) => {
      const tx = db.transaction("file-handles", "readwrite");
      tx.objectStore("file-handles").delete(key);
      tx.oncomplete = () => resolve();
      tx.onerror = () => reject(tx.error);
    });
  } catch {
    /* best-effort */
  }
}

private openIdb(): Promise<IDBDatabase> {
  return new Promise((resolve, reject) => {
    const req = indexedDB.open("r-node", 1);

```

```

    req.onupgradeneeded = () => {
      const db = req.result;
      if (!db.objectStoreNames.contains("file-handles")) db.createObjectStore("file-handles");
    };
    req.onsuccess = () => resolve(req.result);
    req.onerror = () => reject(req.error);
  });
}

/**
 * Open a portable file (file picker): .rnode.zip (map + images), .rnode.json
 * and legacy .rmind.json (plain documents). The container is sniffed by its
 * PK\x03\x04 magic bytes, never by extension alone.
 */
async loadFile(): Promise<void> {
  // Desktop: "Open" picks the .rnode file itself. The web file-input flow
  // below stays for the browser.
  if (this.adapter instanceof TauriStorageAdapter) {
    await this.openDesktop();
    return;
  }
  const input = document.createElement("input");
  input.type = "file";
  input.accept = ".rnode.json,.rmind.json,.rnode.zip,application/json,application/zip";
  const file: File | null = await new Promise((resolve) => {
    input.onChange = (): void => resolve(input.files?.[0] ?? null);
    input.click();
  });
  if (!file) return;
  try {
    const buf = await file.arrayBuffer();
    const head = new Uint8Array(buf, 0, 4);
    const isZip = head.length === 4 && head[0] === 0x50 && head[1] === 0x4b && head[2] === 0x03 && head[3] === 0x04;
    if (isZip) {
      const doc = await importRnodeZip(new Uint8Array(buf), getAssetStore());
      if (!doc) {
        this.toast("Not a valid R-node file");
        return;
      }
      const id = this.importDocumentFromJson(JSON.stringify(doc));
      this.toast(id ? `Opened ${file.name}` : "Not a valid R-node file");
      return;
    }
    const text = new TextDecoder().decode(buf);
    const id = this.importDocumentFromJson(text);
    if (id) this.toast(`Opened ${file.name}`);
    else this.toast("Not a valid R-node file");
  } catch {
    this.toast("Could not read the file");
  }
}

/**
 * Parse, validate and open a .rnode.json document. Returns the imported
 * document id, or null if the text is not a valid R-node document.
 */
importDocumentFromJson(text: string): string | null {
  let raw: unknown;
  try {
    raw = JSON.parse(text);
  } catch {
    return null;
  }
  const doc = this.validateImportedDoc(raw);
  if (!doc) return null;
  const existing = this.state.docs.findIndex((d) => d.documentId === doc.documentId);
  if (existing >= 0) this.state.docs[existing] = doc;
  else this.state.docs = [...this.state.docs, doc];
  this.switchToDoc(doc.documentId);
  this.state.sync = "dirty"; // loaded from disk - not yet in app storage
  this.normalizeBranchColors(this.sheet);
  this.notify();
  return doc.documentId;
}

/** Structural validation + sanitization of an imported document. */
private validateImportedDoc(raw: unknown): RnodeDocument | null {
  if (!raw || typeof raw !== "object") return null;
  const d = raw as Partial<RnodeDocument>;
  if (typeof d.documentId !== "string" || d.documentId.length === 0) return null;
  if (!Array.isArray(d.sheets) || d.sheets.length === 0) return null;
  const s = d.sheets[0] as Partial<Sheet> | undefined;
  if (!s || typeof s.rootNodeId !== "string" || !s.nodes || typeof s.nodes !== "object") return null;
  const rawNodes = s.nodes as Record<string, unknown>;
  const root = rawNodes[s.rootNodeId];
  if (!root || typeof root !== "object") return null;

```

```

const nodes: Record<string, MindNode> = {};
for (const [id, n] of Object.entries(rawNodes)) {
  if (!n || typeof n !== "object") continue;
  const node = n as Partial<MindNode>;
  const position = (typeof node.position === "object" && node.position ? node.position : {}) as Partial<Position>;
  nodes[id] = {
    id,
    type: typeof node.type === "string" ? node.type : "subtopic",
    parentId: typeof node.parentId === "string" || node.parentId === null ? node.parentId : null,
    childrenIds: Array.isArray(node.childrenIds) ? node.childrenIds.filter((c): c is string => typeof c === "string") : [],
    title: typeof node.title === "string" ? node.title : "",
    titleRuns: sanitizeTitleRuns(node.titleRuns),
    position: {
      x: typeof position.x === "number" ? position.x : 0,
      y: typeof position.y === "number" ? position.y : 0,
      manual: !!position.manual,
    },
    style: (typeof node.style === "object" && node.style ? node.style : {}) as Style,
    collapsed: !!node.collapsed,
    labels: Array.isArray(node.labels) ? node.labels.filter((l): l is string => typeof l === "string") : [],
    markers: Array.isArray(node.markers) ? node.markers.filter((m): m is string => typeof m === "string") : [],
    notes: typeof node.notes === "string" ? node.notes : "",
    task: typeof node.task === "object" && node.task ? (node.task as TaskInfo) : null,
    metadata: {
      createdAt: typeof node.metadata?.createdAt === "string" ? node.metadata.createdAt : nowIso(),
      updatedAt: typeof node.metadata?.updatedAt === "string" ? node.metadata.updatedAt : nowIso(),
    },
  };
}

const sheet: Sheet = {
  sheetId: typeof s.sheetId === "string" ? s.sheetId : uid("s"),
  title: typeof s.title === "string" ? s.title : "Map 1",
  structure: {
    structureType: s.structure?.structureType ?? "mindmap",
    orientation: s.structure?.orientation ?? "horizontal",
    spacing: typeof s.structure?.spacing === "number" ? s.structure.spacing : 180,
    branchSpacing: typeof s.structure?.branchSpacing === "number" ? s.structure.branchSpacing : 14,
    padding: typeof s.structure?.padding === "number" ? s.structure.padding : 18,
    compactMode: !!s.structure?.compactMode,
    autoBalance: s.structure?.autoBalance ?? true,
    freePositioningBranches: !!s.structure?.freePositioningBranches,
    allowManualPositioning: s.structure?.allowManualPositioning ?? true,
    connectorStyle: s.structure?.connectorStyle ?? "curved",
  },
  rootNodeId: s.rootNodeId,
  nodes,
  relationships: Array.isArray(s.relationships) ? s.relationships : [],
  boundaries: Array.isArray(s.boundaries) ? s.boundaries : [],
  summaries: Array.isArray(s.summaries) ? s.summaries : [],
  callouts: Array.isArray(s.callouts) ? s.callouts : [],
  labels: Array.isArray(s.labels) ? s.labels : [],
  zones: Array.isArray(s.zones) ? s.zones : [],
  attachments: Array.isArray(s.attachments) ? s.attachments : [],
  comments: Array.isArray(s.comments) ? s.comments : [],
  presentation: typeof s.presentation === "object" && s.presentation ? s.presentation : {},
};

return {
  schemaVersion: typeof d.schemaVersion === "string" ? d.schemaVersion : SCHEMA_VERSION,
  documentId: d.documentId,
  title: typeof d.title === "string" ? d.title : "Imported map",
  createdAt: typeof d.createdAt === "string" ? d.createdAt : nowIso(),
  updatedAt: nowIso(),
  archived: !!d.archived,
  pinned: !!d.pinned,
  settings: {
    theme: "light",
    showOutliner: !!d.settings?.showOutliner,
    showInspector: d.settings?.showInspector ?? true,
  },
  themeId: typeof d.themeId === "string" ? d.themeId : "r-node-light",
  sheets: [sheet],
};
}

private docFileName(): string {
  return `${docFileNameBaseName(this.model.doc.title)}.rnode.json`;
}

private docZipName(): string {
  return `${docFileNameBaseName(this.model.doc.title)}.rnode.zip`;
}

// -----
// Selection
// -----

```

```

select(id: string, opts?: { additive?: boolean; center?: boolean }): void {
  // The canvas pointer handler runs before the textarea blur fires. Commit
  // the in-progress edit here so clicking away never loses typed text.
  this.commitDraftOnLeave();
  const additive = opts?.additive ?? false;
  let sel: string[];
  if (additive) {
    sel = this.state.selection.includes(id) ? this.state.selection.filter((s) => s !== id) : [...this.state.selection, id];
  } else {
    sel = [id];
  }
  this.state.selection = sel;
  this.state.editingId = null;
  this.state.pendingInsert = null;
  this.state.relSel = null;
  this.state.groupSel = null;
  this.state.summarySel = null;
  this.state.imageSel = null;
  if (opts?.center) this.centerOnNode(id);
  this.notify();
}

```

```

selectMany(ids: string[], opts?: { additive?: boolean }): void {
  this.commitDraftOnLeave();
  if (opts?.additive) {
    const set = new Set(this.state.selection);
    for (const id of ids) {
      if (set.has(id)) set.delete(id);
      else set.add(id);
    }
    this.state.selection = [...set];
  } else {
    this.state.selection = [...ids];
  }
  this.state.editingId = null;
  this.state.relSel = null;
  this.state.groupSel = null;
  this.state.summarySel = null;
  this.state.imageSel = null;
  this.notify();
}

```

```

clearSelection(): void {
  this.commitDraftOnLeave();
  if (
    this.state.selection.length === 0 &&
    !this.state.relSel &&
    !this.state.groupSel &&
    !this.state.summarySel &&
    !this.state.imageSel
  )
    return;
  this.state.selection = [];
  this.state.editingId = null;
  this.state.relSel = null;
  this.state.groupSel = null;
  this.state.summarySel = null;
  this.state.imageSel = null;
  this.notify();
}

```

```

/** Select the IMAGE of a node (exclusive: clears every other selection). */
selectImage(nodeId: string): void {
  this.commitDraftOnLeave();
  this.state.selection = [];
  this.state.editingId = null;
  this.state.pendingInsert = null;
  this.state.relSel = null;
  this.state.groupSel = null;
  this.state.summarySel = null;
  this.state.imageSel = nodeId;
  this.notify();
}

```

```

get selectionNode(): MindNode | null {
  const id = this.state.selection[this.state.selection.length - 1];
  return id ? this.model.node(id) ?? null : null;
}

```

```

// -----
// Text editing
// -----

```

```

startEdit(id: string): void {
  trace.edit("start", id);
  this.commitDraftOnLeave();
}

```

```

const node = this.model.node(id);
if (!node) return;
this.state.selection = [id];
this.state.editingId = id;
this.state.pendingInsert = null;
this.state.imageSel = null;
this.editOriginal = { title: node.title, titleRuns: node.titleRuns };
// Seed the draft with the current title so layout keeps measuring the
// node at its current size until the editor reports its first change.
this.applyDraftRuns(id, nodeRuns(node.title, node.titleRuns), false);
this.notify();
}

/**
 * XMind-style type/paste-to-edit: a printable character (or pasted text)
 * with a topic selected starts editing it with that content. The editor
 * consumes the pending insert on mount, replacing the previous title.
 */
typeToEdit(text: string): void {
  this.commitDraftOnLeave();
  const node = this.selectionNode;
  if (!node) return;
  this.state.selection = [node.id];
  this.state.editingId = node.id;
  this.state.pendingInsert = text;
  this.editOriginal = { title: node.title, titleRuns: node.titleRuns };
  this.applyDraftRuns(node.id, nodeRuns(node.title, node.titleRuns), false);
  this.notify();
}

/** The Lexical overlay calls this on mount to pick up the pending type/paste text. */
consumePendingInsert(): string | null {
  const v = this.state.pendingInsert;
  this.state.pendingInsert = null;
  return v;
}

/**
 * Buffer an extra character while the type-to-edit editor is still
 * mounting (pendingInsert not yet consumed). Without this, fast typists
 * lose the keystrokes between typeToEdit() and the editor appearing:
 * the shortcut handler sees editingId set and drops them.
 */
appendPendingInsert(ch: string): boolean {
  if (this.state.pendingInsert === null) return false;
  this.state.pendingInsert += ch;
  this.notify();
  return true;
}

/** Plain-text shim (Inspector/Outliner/tests): a single unstyled run. */
setEditingDraft(text: string | null): void {
  if (text === null) {
    this.editingDraftRuns = null;
    return;
  }
  this.setEditingDraftRuns(plainToRuns(text));
}

/**
 * The Lexical overlay reports its live content as TextRuns on every
 * change. The draft is applied to the node ephemerally (no op, no history)
 * so the debounced layout and the canvas repaint track the text live.
 */
setEditingDraftRuns(runs: TextRun[]): void {
  const id = this.state.editingId;
  if (!id) return;
  this.applyDraftRuns(id, runs, true);
}

/** Apply the draft runs to the node (ephemeral) and schedule live layout. */
private applyDraftRuns(id: string, runs: TextRun[], notify: boolean): void {
  const node = this.model.node(id);
  if (!node) return;
  const clean = normalizeRuns(runs);
  this.editingDraftRuns = clean;
  if (clean.length > 0) {
    node.title = runsToPlain(clean);
    node.titleRuns = clean;
  } else {
    node.title = "";
    node.titleRuns = [];
  }
  if (notify) {
    this.scheduleLayout(false);
    this.notify();
  }
}

```

```

}

private restoreOriginal(node: MindNode, original: { title: string; titleRuns?: TextRun[] } | null): void {
  if (!original) return;
  node.title = original.title;
  if (original.titleRuns) node.titleRuns = original.titleRuns.map((r) => ({ ...r }));
  else delete node.titleRuns;
}

```

```

/**
 * Commit the in-progress edit as a real setTitle op. Selection changes
 * commit the draft too, so clicking away (pointerdown runs before blur)
 * never loses typed text. Undo restores the pre-edit title exactly.
 */
commitEdit(): void {
  trace.edit("commit", this.state.editingId ?? undefined);
  const id = this.state.editingId;
  const runs = this.editingDraftRuns;
  const original = this.editOriginal;
  this.state.editingId = null;
  this.editingDraftRuns = null;
  this.editOriginal = null;
  this.state.pendingInsert = null;
  if (!id || runs === null || !original) return;
  const node = this.model.node(id);
  if (!node) return;
  const clean = trimRuns(runs);
  if (isEmptyRuns(clean)) {
    // empty new topic -> delete it; a topic with children keeps its title
    if (node.childrenIds.length === 0) this.deleteNodes([id]);
  } else {
    this.restoreOriginal(node, original);
    this.notify();
  }
  return;
}
const plain = runsToPlain(clean);
if (plain !== original.title || !runsEqual(clean, original.titleRuns)) {
  this.execOps([
    makeOp<Op & { type: "setTitle" }>("setTitle", {
      id,
      title: plain,
      prev: original.title,
      titleRuns: clean,
      prevRuns: original.titleRuns,
    }),
  ]);
} else {
  // unchanged -> drop the ephemeral patch, nothing to record
  this.restoreOriginal(node, original);
  this.notify();
}
}

```

```

/**
 * Apply the in-progress edit (setTitle) WITHOUT closing the editor. Ctrl+S
 * while typing therefore saves the current text and keeps editing.
 */
private commitDraftKeepEditing(): void {
  const id = this.state.editingId;
  const runs = this.editingDraftRuns;
  const original = this.editOriginal;
  if (!id || runs === null || !original) return;
  const node = this.model.node(id);
  if (!node) return;
  const clean = trimRuns(runs);
  if (isEmptyRuns(clean)) return;
  const plain = runsToPlain(clean);
  if (plain === original.title && runsEqual(clean, original.titleRuns)) return;
  this.execOps([
    makeOp<Op & { type: "setTitle" }>("setTitle", {
      id,
      title: plain,
      prev: original.title,
      titleRuns: clean,
      prevRuns: original.titleRuns,
    }),
  ]);
  // The committed state is now the baseline for any further Ctrl+S.
  this.editOriginal = { title: plain, titleRuns: clean };
}

```

```

/** Commit the draft when the selection/editing context is about to change. */
private commitDraftOnLeave(): void {
  if (this.state.editingId !== null) this.commitEdit();
  else {
    this.editingDraftRuns = null;
  }
}

```

```

        this.editOriginal = null;
        this.state.pendingInsert = null;
    }
}

cancelEdit(): void {
    trace.edit("cancel", this.state.editingId ?? undefined);
    const id = this.state.editingId;
    const original = this.editOriginal;
    this.state.editingId = null;
    this.editingDraftRuns = null;
    this.editOriginal = null;
    this.state.pendingInsert = null;
    if (id) {
        const node = this.model.node(id);
        if (!node) return;
        this.restoreOriginal(node, original);
        if (node.title === "" && node.childrenIds.length === 0) this.deleteNodes([id]);
        else this.notify();
    }
}

// -----
// Structure commands (Enter / Tab / Shift+Tab ...)
// -----

private defaultTopicTitle(type: NodeType, parentId: string | null): string {
    if (type === "central") return "Central Topic";
    if (type === "floating") return "New Idea";
    const label = type === "main" ? "Main Topic" : "Subtopic";
    const parent = parentId ? this.model.node(parentId) : undefined;
    const siblingCount = parent?.childrenIds.reduce((count, id) => {
        return count + (this.model.node(id)?.type === type ? 1 : 0);
    }, 0) ?? Object.values(this.sheet.nodes).filter((n) => n.type === type).length;
    return `${label} ${siblingCount + 1}`;
}

private normalizeBranchColors(sheet: Sheet): void {
    const root = sheet.nodes[sheet.rootNodeId];
    if (!root) return;
    for (const [index, childId] of root.childrenIds.entries()) {
        const child = sheet.nodes[childId];
        if (!child) continue;
        if (!child.style.fill) child.style.fill = this.defaultBranchFill(index);
        const softFill = this.defaultBranchSoftFill(childId);
        const stack = [childId];
        while (stack.length > 0) {
            const currentId = stack.pop();
            const current = sheet.nodes[currentId];
            if (!current) continue;
            if (currentId !== childId && !current.style.fill) current.style.fill = softFill;
            stack.push(...current.childrenIds);
        }
    }
}

private defaultBranchFill(index: number): string {
    return THEMES.light.branch[index % THEMES.light.branch.length];
}

private createNodePosition(parent: MindNode): { x: number; y: number; manual: boolean } {
    const base = parent.position;
    const st = this.sheet.structure;
    const pSize = measureNode(parent, this.measurer);
    const estimatedW = 140;
    const offsetX = parent.type === "central" ? pSize.w / 2 + st.spacing + estimatedW / 2 + 20 : pSize.w / 2 + st.spacing + estimatedW / 2 + 10;
    const offsetY = parent.type === "central" ? parent.childrenIds.length * (pSize.h + st.branchSpacing) - (parent.childrenIds.length > 0 ? pSize.h / 2 : 0) : parent.childrenIds.length * (pSize.h + st.branchSpacing);
    return { x: base.x + offsetX, y: base.y + offsetY, manual: false };
}

private branchRootId(parentId: string | null): string | null {
    if (!parentId) return null;
    let cur: string | null = parentId;
    let prev: string | null = parentId;
    while (cur && cur !== this.sheet.rootNodeId) {
        prev = cur;
        cur = this.sheet.nodes[cur]?.parentId ?? null;
    }
    return prev;
}

private defaultBranchSoftFill(rootId: string | null): string | undefined {
    if (!rootId) return undefined;
    const root = this.sheet.nodes[this.sheet.rootNodeId];
    if (!root) return undefined;
    const index = root.childrenIds.indexOf(rootId);

```

```

    return THEMES.light.branchSoft[(index >= 0 ? index : 0) % THEMES.light.branchSoft.length];
}

private createNodeStyle(type: NodeType, parentId: string | null, index: number): Style | undefined {
    if (type === "main" && parentId === this.sheet.rootNodeId) {
        return { fill: this.defaultBranchFill(index) };
    }
    if (type === "subtopic") {
        const branchRootId = this.branchRootId(parentId);
        return { fill: this.defaultBranchSoftFill(branchRootId) };
    }
    return undefined;
}

createSibling(): void {
    const node = this.selectionNode;
    if (!node) return;
    if (node.type === "central") return this.createChild();
    if (!node.parentId) {
        this.toast("Floating topics have no siblings");
        return;
    }
    const parent = this.model.requireNode(node.parentId);
    const index = parent.childrenIds.indexOf(node.id) + 1;
    const type: NodeType = parent.type === "central" ? "main" : "subtopic";
    const id = uid("n");
    const position = this.createNodePosition(type === "main" ? parent : node);
    const style = this.createNodeStyle(type, parent.id, index);
    this.execOps([makeOp<Op & { type: "createNode" }>("createNode", { id, nodeType: type, parentId: parent.id, index, title:
this.defaultTopicTitle(type, node.id), position, style })]);
    // The layout must settle BEFORE the editor opens, or the overlay would
    // mount on the provisional position and the topic would appear to jump.
    this.settleLayoutNow();
    this.startEdit(id);
}

createChild(): void {
    const node = this.selectionNode ?? this.model.rootNode;
    const type: NodeType = node.type === "central" ? "main" : "subtopic";
    const id = uid("n");
    const position = this.createNodePosition(node);
    const style = this.createNodeStyle(type, node.id, node.childrenIds.length);
    this.execOps([makeOp<Op & { type: "createNode" }>("createNode", { id, nodeType: type, parentId: node.id, index: node.childrenIds.length, title:
this.defaultTopicTitle(type, node.id), position, style })]);
    // Tab spawns the child without entering edit mode: the selection stays on
    // the source node, so repeated Tab keeps adding siblings under the same
    // parent instead of nesting (or stealing the selection).
    this.settleLayoutNow();
}

createParent(): void {
    const node = this.selectionNode;
    if (!node || !node.parentId) return;
    const parent = this.model.requireNode(node.parentId);
    const type: NodeType = parent.type === "central" ? "main" : "subtopic";
    const newId = uid("n");
    const idx = parent.childrenIds.indexOf(node.id);
    const style = this.createNodeStyle(type, parent.id, idx);
    const ops: Op[] = [
        makeOp<Op & { type: "createNode" }>("createNode", { id: newId, nodeType: type, parentId: parent.id, index: idx, title:
this.defaultTopicTitle(type, parent.id), style }),
        makeOp<Op & { type: "moveNode" }>("moveNode", {
            id: node.id,
            fromParentId: parent.id,
            fromIndex: idx + 1,
            toParentId: newId,
            toIndex: 0,
        })),
    ];
    if (node.position.manual) {
        ops.push(makeOp<Op & { type: "setPosition" }>("setPosition", {
            id: node.id,
            x: node.position.x,
            y: node.position.y,
            manual: false,
            prev: { ...node.position },
        }));
    }
    this.execOps(ops);
    this.settleLayoutNow();
    this.select(newId);
}

createFloatingAt(x: number, y: number): void {
    const id = uid("n");
    this.execOps([
        makeOp<Op & { type: "createNode" }>("createNode", {

```

```

        id,
        nodeType: "floating",
        parentId: null,
        index: 0,
        title: this.defaultTopicTitle("floating", null),
        position: { x, y, manual: true },
    })),
    []);
this.settleLayoutNow();
this.startEdit(id);
}

promote(): void {
    const node = this.selectionNode;
    if (!node || !node.parentId) return;
    const parent = this.model.requireNode(node.parentId);
    if (!parent.parentId) {
        this.toast("Already at top level");
        return;
    }
    const grand = this.model.requireNode(parent.parentId);
    const fromIndex = parent.childrenIds.indexOf(node.id);
    const toIndex = grand.childrenIds.indexOf(parent.id) + 1;
    const ops: Op[] = [makeOp<Op & { type: "moveNode" }>("moveNode", { id: node.id, fromParentId: parent.id, fromIndex, toParentId: grand.id, toIndex
    });];
    if (node.position.manual) {
        ops.push(makeOp<Op & { type: "setPosition" }>("setPosition", {
            id: node.id,
            x: node.position.x,
            y: node.position.y,
            manual: false,
            prev: { ...node.position },
        }));
    }
    this.execOps(ops);
}

demote(): void {
    const node = this.selectionNode;
    if (!node || !node.parentId) return;
    const parent = this.model.requireNode(node.parentId);
    const idx = parent.childrenIds.indexOf(node.id);
    if (idx <= 0) return;
    const prev = this.model.requireNode(parent.childrenIds[idx - 1]);
    const ops: Op[] = [makeOp<Op & { type: "moveNode" }>("moveNode", { id: node.id, fromParentId: parent.id, fromIndex: idx, toParentId: prev.id,
    toIndex: prev.childrenIds.length }));];
    if (node.position.manual) {
        ops.push(makeOp<Op & { type: "setPosition" }>("setPosition", {
            id: node.id,
            x: node.position.x,
            y: node.position.y,
            manual: false,
            prev: { ...node.position },
        }));
    }
    this.execOps(ops);
}

toggleCollapsed(id: string): void {
    const node = this.model.node(id);
    if (!node) return;
    this.execOps([makeOp<Op & { type: "setCollapsed" }>("setCollapsed", { id, collapsed: !node.collapsed, prev: node.collapsed })]);
}

expandAll(id: string): void {
    const ops: Op[] = [];
    for (const sid of this.model.subtreeIds(id)) {
        const n = this.model.node(sid);
        if (n?.collapsed) ops.push(makeOp<Op & { type: "setCollapsed" }>("setCollapsed", { id: sid, collapsed: false, prev: true }));
    }
    if (ops.length) this.execOps(ops);
}

deleteSelection(): void {
    this.deleteNodes([...this.state.selection]);
}

/** Delete nodes (and descendants); central topic is protected. */
deleteNodes(ids: string[]): void {
    const pruned = ids.filter((id) => id !== this.sheet.rootNodeId);
    const toDelete: string[] = [];
    for (const id of pruned) {
        if (toDelete.some((d) => this.isDescendant(d, id))) continue;
        toDelete.push(id);
    }
    const ops: Op[] = [];
    for (const id of toDelete) {

```

```

const node = this.model.node(id);
if (!node) continue;
const subtree = this.model.subtreeIds(id).map((sid => this.model.node(sid)!).filter(Boolean);
const subtreeSet = new Set(subtree.map((s) => s.id));
const removedRelationships = this.sheet.relationships.filter((r) => subtreeSet.has(r.fromId) || subtreeSet.has(r.toId));
const parentId = node.parentId;
const index = parentId ? this.model.requireNode(parentId).childrenIds.indexOf(id) : 0;
ops.push(makeOp<Op & { type: "deleteNode" }>("deleteNode", { id, parentId, index, subtree, removedRelationships }));
}
// Drop groups/summaries whose members are all gone (their boxes/braces
// would dangle otherwise). Undo of the batch restores them.
const deletedSet = new Set<string>();
for (const id of toDelete) for (const sid of this.model.subtreeIds(id)) deletedSet.add(sid);
for (const g of this.sheet.boundaries) {
  if (g.memberIds.length > 0 && g.memberIds.every((m) => deletedSet.has(m))) {
    ops.push(makeOp<Op & { type: "deleteGroup" }>("deleteGroup", { id: g.id, group: g }));
  }
}
for (const s of this.sheet.summaries) {
  if (s.memberIds.length > 0 && s.memberIds.every((m) => deletedSet.has(m))) {
    ops.push(makeOp<Op & { type: "deleteSummary" }>("deleteSummary", { id: s.id, summary: s }));
  }
}
if (ops.length) {
  this.execOps(ops);
  this.state.selection = [];
  this.state.relSel = null;
  this.state.groupSel = null;
  this.state.summarySel = null;
  this.state.imageSel = null;
  this.notify();
}
}

private isDescendant(ancestorId: string, id: string): boolean {
  let cur = this.model.node(id)?.parentId ?? null;
  while (cur) {
    if (cur === ancestorId) return true;
    cur = this.model.node(cur)?.parentId ?? null;
  }
  return false;
}

/** Descendant ids whose position is manually pinned (must follow their parent). */
private collectManualDescendants(id: string): string[] {
  const out: string[] = [];
  const stack = [...(this.model.node(id)?.childrenIds ?? [])];
  while (stack.length > 0) {
    const cur = stack.pop()!;
    const n = this.model.node(cur);
    if (!n) continue;
    if (n.position.manual) out.push(cur);
    stack.push(...n.childrenIds);
  }
  return out;
}

sortSiblings(parentId: string, compare: (a: MindNode, b: MindNode) => number): void {
  const parent = this.model.node(parentId);
  if (!parent || parent.childrenIds.length < 2) return;
  const prevOrder = [...parent.childrenIds];
  const order = [...prevOrder].sort((a, b) => compare(this.model.node(a)!, this.model.node(b)!));
  if (order.every((v, i) => v === prevOrder[i])) return;
  this.execOps([makeOp<Op & { type: "sortSiblings" }>("sortSiblings", { parentId, order, prevOrder })]);
}

// -----
// Drag & drop
// -----

setDrop(drop: DropIndicator | null): void {
  this.state.drop = drop;
  this.notify();
}

setHover(id: string | null): void {
  if (this.state.hoverId === id) return;
  this.state.hoverId = id;
  this.notify();
}

/** Reparenting hands the topic back to the automatic layout engine. */
private releaseManualPosition(node: MindNode): Op | null {
  if (!node.position.manual) return null;
  return makeOp<Op & { type: "setPosition" }>("setPosition", {
    id: node.id,
    x: node.position.x,

```

```

    y: node.position.y,
    manual: false,
    prev: { ...node.position },
  });
}

/** Resolve a drop of `draggedId` over `targetId`/empty space. */
dropAt(draggedId: string, targetId: string | null, mode: "child" | "before" | "after" | "floating", worldX: number, worldY: number): void {
  const node = this.model.node(draggedId);
  if (!node || draggedId === targetId) {
    this.setDrop(null);
    return;
  }
  if (targetId) {
    const target = this.model.node(targetId);
    if (!target || this.isDescendant(draggedId, targetId)) {
      this.setDrop(null);
      return;
    }
    if (mode === "child") {
      const ops: Op[] = [makeOp<Op & { type: "moveNode" }>("moveNode", {
        id: draggedId,
        fromParentId: node.parentId,
        fromIndex: node.parentId ? this.model.requireNode(node.parentId).childrenIds.indexOf(draggedId) : 0,
        toParentId: targetId,
        toIndex: target.childrenIds.length,
      })];
      const release = this.releaseManualPosition(node);
      if (release) ops.push(release);
      this.execOps(ops);
    } else if (mode === "before" || mode === "after") {
      if (!target.parentId) {
        this.setDrop(null);
        return;
      }
      const targetParent = this.model.requireNode(target.parentId);
      const targetIdx = targetParent.childrenIds.indexOf(targetId);
      const desired = mode === "after" ? targetIdx + 1 : targetIdx;
      // final index after removing the dragged node (if same parent)
      let finalIdx = desired;
      if (node.parentId === target.parentId) {
        const curIdx = targetParent.childrenIds.indexOf(draggedId);
        if (curIdx >= 0 && curIdx < desired) finalIdx = desired - 1;
      }
      const ops: Op[] = [makeOp<Op & { type: "moveNode" }>("moveNode", {
        id: draggedId,
        fromParentId: node.parentId,
        fromIndex: node.parentId ? this.model.requireNode(node.parentId).childrenIds.indexOf(draggedId) : 0,
        toParentId: target.parentId,
        toIndex: finalIdx,
      })];
      const release = node.parentId !== target.parentId ? this.releaseManualPosition(node) : null;
      if (release) ops.push(release);
      this.execOps(ops);
    }
  } else if (mode === "floating") {
    // allow floating drops generally; respect sheet setting if manual positioning disabled
    if (!this.sheet.structure.allowManualPositioning) {
      this.setDrop(null);
      return;
    }
    // Only main topics (direct children of root) and floating topics (no
    // parent) may be pinned to an absolute position. Deeper hierarchical
    // nodes stay in the auto layout and follow their parent – dropping one
    // on empty canvas does nothing.
    const isRootChild = node.parentId === this.sheet.rootNodeId;
    const isFloatingTopic = !node.parentId;
    if (!isRootChild && !isFloatingTopic) {
      this.setDrop(null);
      return;
    }
    // If dragging a direct child of root, allow swapping sides by reordering
    if (isRootChild) {
      const parent = this.model.requireNode(this.sheet.rootNodeId);
      const layout = layoutSheet(this.sheet, false, this.mesurer);
      const rootCx = layout.positions.get(this.sheet.rootNodeId)?.x ?? parent.position.x;
      let desiredIndex = worldX < (rootCx + measureNode(parent, this.mesurer).w / 2) ? parent.childrenIds.length : 0;
      // adjust for current index
      const curIdx = parent.childrenIds.indexOf(draggedId);
      if (curIdx >= 0 && curIdx < desiredIndex) desiredIndex = desiredIndex - 1;
      const ops: Op[] = [
        makeOp<Op & { type: "moveNode" }>("moveNode", {
          id: draggedId,
          fromParentId: node.parentId,
          fromIndex: curIdx >= 0 ? curIdx : 0,
          toParentId: parent.id,
          toIndex: desiredIndex,
        })
      ];
    }
  }
}

```

```

    }},
    makeOp<Op & { type: "setPosition" }>("setPosition", {
      id: draggedId,
      x: worldX,
      y: worldY,
      manual: true,
      prev: node.position,
    }},
  ]),
  // Children must follow the main keeping their relative layout: drop any
  // leftover manual pin on descendants so the whole subtree re-flows
  // around the main's new position.
  for (const descId of this.collectManualDescendants(draggedId)) {
    const d = this.model.node(descId);
    if (!d) continue;
    ops.push(makeOp<Op & { type: "setPosition" }>("setPosition", {
      id: descId,
      x: d.position.x,
      y: d.position.y,
      manual: false,
      prev: { ...d.position },
    }));
  }
  this.execOps(ops);
} else {
  this.execOps([
    makeOp<Op & { type: "setPosition" }>("setPosition", {
      id: draggedId,
      x: worldX,
      y: worldY,
      manual: true,
      prev: node.position,
    }},
  ]),
  ]),
  this.setDrop(null);
  this.select(draggedId);
}

/**
 * Main-topic drag (live repositioning): a movable node (root child or
 * floating topic) FOLLOWS the cursor during the drag – ephemeral manual
 * position, no op, no history – and its subtree re-flows around it on
 * every move, so the whole branch slides with the pointer. commitNodeDrag
 * emits ONE setPosition op whose `prev` is the PRE-drag position, so a
 * single undo restores the exact pre-drag state.
 */
private nodeDragState: { nodeId: string; origPos: Position } | null = null;

/** A node drag started: capture the pre-drag position for the final op. */
beginNodeDrag(nodeId: string): void {
  const node = this.model.node(nodeId);
  if (!node) return;
  this.nodeDragState = { nodeId, origPos: { ...node.position } };
}

/**
 * Live position during the drag. The layout runs SYNCHRONOUSLY (the 30ms
 * debounce of scheduleLayout would starve during a continuous drag – the
 * subtree would never follow until the pointer stops); manual is set
 * transiently so applyLayout does not snap the node back to its slot.
 */
setNodeDragDraft(nodeId: string, worldX: number, worldY: number): void {
  const node = this.model.node(nodeId);
  if (!node) return;
  node.position = { ...node.position, x: worldX, y: worldY, manual: true };
  applyLayout(this.sheet, false, this.measurer);
  this.notify();
}

/** Restore the pre-drag position (the drag left the free-position zone). */
resetNodeDragDraft(nodeId: string): void {
  const d = this.nodeDragState;
  if (!d || d.nodeId !== nodeId) return;
  const node = this.model.node(nodeId);
  if (!node) return;
  node.position = { ...d.origPos };
  applyLayout(this.sheet, false, this.measurer);
  this.notify();
}

/**
 * End of the drag: restore the model to the pre-drag state, then commit
 * through dropAt – its ops therefore carry `prev: origPos` (undo restores
 * exactly) and releaseManualPosition sees the ORIGINAL manual flag.
 */

```

```

* The tree is then settled SYNCHRONOUSLY: dropAt's execOps only schedules
* the 30ms-debounced layout, and waiting for it would (a) leave the
* released subtree detached from its new position for a few frames (the
* release glitch) and (b) let a late reflow fire while the user has
* already moved on – e.g. clicking another node that then visibly shifts.
*/
commitNodeDrag(draggedId: string, targetId: string | null, mode: "child" | "before" | "after" | "floating", worldX: number, worldY: number): void {
  const d = this.nodeDragState;
  this.nodeDragState = null;
  if (d) {
    const node = this.model.node(d.nodeId);
    if (node) node.position = { ...d.origPos };
  }
  this.dropAt(draggedId, targetId, mode, worldX, worldY);
  if (this.layoutTimer) {
    clearTimeout(this.layoutTimer);
    this.layoutTimer = null;
  }
  applyLayout(this.sheet, false, this.measurer);
  this.notify();
}

// -----
// Style / tasks
// -----

setNodeStyle(id: string, patch: Partial<Style>): void {
  const node = this.model.node(id);
  if (!node) return;
  this.execOps([makeOp<Op & { type: "setStyle" }>("setStyle", { id, style: { ...node.style, ...patch }, prev: node.style })]);
}

// -----
// Node image (T12-4) – the op carries only the id, never the bytes
// -----

/** Attach (or remove, with imageId = null) an image reference on a node. */
setNodeImage(nodeId: string, imageId: string | null): void {
  const node = this.model.node(nodeId);
  if (!node) return;
  const prevImageId = node.style.image ?? null;
  if (prevImageId === imageId) return;
  this.execOps([
    makeOp<Op & { type: "setNodeImage" }>("setNodeImage", { nodeId, imageId, prevImageId }),
  ]);
}

/**
 * Delete the SELECTED image (Backspace/Delete with the image selected):
 * removes only the reference from the node, never the node itself. The
 * attachment card stays (it may be shared; collectOrphans is the GC).
 */
deleteSelectedImage(): void {
  const nodeId = this.state.imageSel;
  if (!nodeId) return;
  const node = this.model.node(nodeId);
  this.state.imageSel = null;
  if (!node?.style.image) {
    this.notify();
    return;
  }
  this.setNodeImage(nodeId, null);
  this.notify();
}

/**
 * Move a node's image reference to another node (image drag & drop). Both
 * reference changes are ONE undoable batch: undo restores the image to its
 * original node. After the move the image on the TARGET stays selected.
 */
assignImageToNode(fromNodeId: string, toNodeId: string): void {
  if (fromNodeId === toNodeId) return;
  const from = this.model.node(fromNodeId);
  const to = this.model.node(toNodeId);
  if (!from || !to) return;
  const imageId = from.style.image;
  if (!imageId) return;
  const ops: Op[] = [];
  const prevFrom = from.style.image ?? null;
  const prevTo = to.style.image ?? null;
  if (prevFrom !== prevTo) {
    ops.push(
      makeOp<Op & { type: "setNodeImage" }>("setNodeImage", { nodeId: fromNodeId, imageId: null, prevImageId: prevFrom })
    );
    ops.push(
      makeOp<Op & { type: "setNodeImage" }>("setNodeImage", { nodeId: toNodeId, imageId, prevImageId: prevTo })
    );
  }
}

```

```

    this.execOps(ops);
  }
  this.selectImage(toNodeId);
}

/**
 * Attach an imported image: register its metadata card (idempotent,
 * content-addressed – the same image shared by N nodes has one card) and
 * reference it from the node in a single undoable op. The card is NOT
 * removed by undo: it may be shared, and collectOrphans is the GC.
 */
attachImage(nodeId: string, card: AttachmentInfo): void {
  if (!this.sheet.attachments.some((a) => a.id === card.id)) {
    this.sheet.attachments.push({ ...card });
  }
  this.setNodeImage(nodeId, card.id);
}

/**
 * Full import+attach flow (T13-2): validate, import in the worker, store
 * the three levels in the AssetStore, then reference the node. Every
 * rejection says why (rule §4bis of AGENT_GUIDE).
 */
async attachImageFile(nodeId: string, file: Blob & { name?: string }): Promise<{ ok: boolean; reason?: string }> {
  const v = validateImageSource(file.type, file.size);
  if (!v.ok) {
    trace.ignored(
      "drop",
      v.reason.startsWith("unsupported") ? "unsupported mime" : "too large",
      { mime: file.type, bytes: file.size }
    );
    return { ok: false, reason: v.reason };
  }
  let imported: ImportedImage;
  try {
    imported = await importImageFile(file);
  } catch (err) {
    trace.ignored("drop", "decode failed", { error: String(err) });
    return { ok: false, reason: String(err) };
  }
  const assetStore = getAssetStore();
  const id = await assetStore.put(imported.levels, imported.meta);
  this.attachImage(nodeId, {
    id,
    mime: imported.meta.mime,
    w: imported.meta.w,
    h: imported.meta.h,
    bytes: imported.meta.bytes,
    name: imported.meta.name,
  });
  trace.applied("drop:image", { bytes: imported.meta.bytes, w: imported.meta.w, h: imported.meta.h });
  return { ok: true };
}

// -----
// Canvas resize drag (Xmind-style handle) – ephemeral draft, single commit
// -----

/** A resize drag started on this node: capture the pre-drag style + position. */
beginResize(nodeId: string): void {
  const node = this.model.node(nodeId);
  if (!node) return;
  this.resizeState = { nodeId, original: { ...node.style }, origPos: { ...node.position } };
}

/**
 * Live width during the drag: ephemeral mutation, no op, no history.
 * For a left-edge drag the right edge is the anchor: position.x follows
 * the cursor with a TRANSIENT manual flag – applyLayout skips manual
 * nodes and would otherwise snap the position back every 30ms. The flag
 * is restored to its original value by commitResize.
 */
setResizeDraft(nodeId: string, width: number, opts?: { anchorRight?: boolean; x?: number }): void {
  const node = this.model.node(nodeId);
  if (!node) return;
  const w = Math.round(Math.min(640, Math.max(MIN_TOPIC_W, width)));
  node.style = { ...node.style, width: w };
  if (opts?.anchorRight && typeof opts.x === "number") {
    node.position = { ...node.position, x: opts.x, manual: true };
  }
  this.scheduleLayout(false);
  this.notify();
}

/** End of the drag: one batch op (setStyle + optional setPosition); undo restores it all. */
commitResize(): void {
  const r = this.resizeState;

```

```

this.resizeState = null;
if (!r) return;
const node = this.model.node(r.nodeId);
if (!node) return;
const w = node.style.width;
const xChanged = node.position.x !== r.origPos.x;
// A left-edge drag moves position.x. It persists only for floating or
// already-manual nodes; auto-layout nodes return to the layout's slot
// (the width is what survives there – the next applyLayout re-pins x).
const keepX = xChanged && (node.type === "floating" || r.origPos.manual);
const ops: Op[] = [];
if (w !== undefined && w !== r.original.width) {
  ops.push(
    makeOp<Op & { type: "setStyle" }>("setStyle", {
      id: r.nodeId,
      style: { ...node.style, width: w },
      prev: r.original,
    })
  );
}
if (keepX) {
  ops.push(
    makeOp<Op & { type: "setPosition" }>("setPosition", {
      id: r.nodeId,
      x: node.position.x,
      y: node.position.y,
      manual: true,
      prev: r.origPos,
    })
  );
} else if (xChanged) {
  node.position = { ...r.origPos };
}
if (ops.length > 0) {
  this.execOps(ops);
} else {
  // click without a real change – restore the pre-drag state, no op
  node.style = r.original;
  node.position = { ...r.origPos };
  this.scheduleLayout(false);
  this.notify();
}
}

// -----
// Image resize (T14) – same draft pattern as the width drag: live width is
// an ephemeral mutation (no op), one setStyle op commits the whole gesture.
// -----

/** A resize drag on the image handle: capture the pre-drag style. */
beginImageResize(nodeId: string): void {
  const node = this.model.node(nodeId);
  if (!node) return;
  this.imageResizeState = { nodeId, original: { ...node.style } };
}

/** Live image width during the drag/slider: ephemeral, no op, no history. */
setImageResizeDraft(nodeId: string, width: number): void {
  const node = this.model.node(nodeId);
  if (!node) return;
  const w = Math.max(48, Math.round(width));
  node.style = { ...node.style, imageWidth: w };
  this.scheduleLayout(false);
  this.notify();
}

/** End of the gesture: one batch op; undo restores the pre-drag width. */
commitImageResize(): void {
  const r = this.imageResizeState;
  this.imageResizeState = null;
  if (!r) return;
  const node = this.model.node(r.nodeId);
  if (!node) return;
  const w = node.style.imageWidth;
  if (w !== undefined && w !== r.original.imageWidth) {
    this.execOps([
      makeOp<Op & { type: "setStyle" }>("setStyle", {
        id: r.nodeId,
        style: { ...node.style, imageWidth: w },
        prev: r.original,
      })
    ],);
  } else {
    // click without a real change – restore, no op
    node.style = r.original;
    this.scheduleLayout(false);
    this.notify();
  }
}

```

```

    }
}

/** Remove the custom imageWidth (back to the natural display size). */
resetImageWidth(nodeId: string): void {
    const node = this.model.node(nodeId);
    if (!node) return;
    if (node.style.imageWidth === undefined) return;
    this.execOps([
        makeOp<Op & { type: "setStyle" }>("setStyle", {
            id: nodeId,
            style: { ...node.style, imageWidth: undefined },
            prev: node.style,
        }),
    ]);
}

setBranchFreePosition(id: string, enabled: boolean): void {
    const node = this.model.node(id);
    if (!node || node.type !== "main") return;
    this.execOps([
        makeOp<Op & { type: "setPosition" }>("setPosition", {
            id,
            x: node.position.x,
            y: node.position.y,
            manual: enabled,
            prev: { ...node.position },
        }),
    ]);
}

setTask(id: string, patch: Partial<TaskInfo>): void {
    const node = this.model.node(id);
    if (!node) return;
    const current = node.task ?? { status: "not-started", priority: "none", progress: 0 };
    this.execOps([makeOp<Op & { type: "setTask" }>("setTask", { id, task: { ...current, ...patch }, prev: node.task })]);
}

toggleTaskComplete(id: string): void {
    const node = this.model.node(id);
    if (!node) return;
    const current = node.task ?? { status: "not-started", priority: "none", progress: 0 };
    const next: TaskInfo = { ...current, status: current.status === "completed" ? "not-started" : "completed", progress: current.status === "completed"
? 0 : 100 };
    this.execOps([makeOp<Op & { type: "setTask" }>("setTask", { id, task: next, prev: node.task })]);
}

duplicateTopic(): void {
    const node = this.selectionNode;
    if (!node || !node.parentId) return;
    if (node.id === this.sheet.rootNodeId) {
        this.toast("Cannot duplicate the central topic");
        return;
    }
    const parent = this.model.requireNode(node.parentId);
    const index = parent.childrenIds.indexOf(node.id) + 1;
    const source = this.model.subtreeIds(node.id).map((id) => this.model.node(id!).filter(Boolean));
    const ops = this.remapOps(source, parent.id, index, node.type);
    if (ops.length) this.execOps(ops);
}

/** Indented text outline (XMind-style) of the pruned selection: each selected
 * topic is a root line, descendants follow indented, in tree order. */
private outlineText(pruned: string[]): string {
    const lines: string[] = [];
    const emit = (id: string, depth: number): void => {
        const n = this.model.node(id);
        if (!n) return;
        lines.push("  ".repeat(depth) + runsToPlain(nodeRuns(n.title, n.titleRuns)));
        for (const cid of n.childrenIds) emit(cid, depth + 1);
    };
    const walk = (id: string): void => {
        if (pruned.includes(id)) emit(id, 0);
        const n = this.model.node(id);
        if (!n) return;
        for (const cid of n.childrenIds) walk(cid);
    };
    walk(this.sheet.rootNodeId);
    return lines.join("\n");
}

async copySelection(): Promise<void> {
    const ids = this.state.selection;
    const pruned = ids.filter((id) => !ids.some((other) => other !== id && this.isDescendant(other, id)));
    const rootId = pruned[0];
    if (!rootId) return;
    const subtreeIds = this.model.subtreeIds(rootId);

```

```

const nodes = subtreeIds.map(id => this.model.node(id!)).filter(Boolean);
const rels = this.sheet.relationships.filter((r) => subtreeIds.includes(r.fromId) && subtreeIds.includes(r.toId));
const payload = JSON.stringify({ app: "r-node", payload: { rootId, nodes, relationships: rels } });
const text = this.outlineText(pruned);
try {
  // Two formats on the clipboard at once: the full map as the custom
  // `text/rnode` MIME (internal paste keeps every run/style), and the
  // plain-text outline as `text/plain` so pasting OUTSIDE R-node (chat,
  // Word, notes) yields readable text instead of raw JSON.
  await navigator.clipboard.write([
    new ClipboardItem({
      "text/rnode": new Blob([payload], { type: "text/rnode" }),
      "text/plain": new Blob([text], { type: "text/plain" }),
    }),
  ]);
} catch {
  // ClipboardItem with custom types unsupported (older engines / blocked) –
  // fall back to plain text only.
  try {
    await navigator.clipboard.writeText(text);
  } catch {
    /* clipboard may be blocked; copying still succeeded in-memory */
  }
}
this.toast(`Copied ${nodes.length} topic${nodes.length > 1 ? "s" : ""}`);
}

/** Copy the selected branches to the clipboard as an indented text outline
 * (XMind-style): each selected topic becomes a root line, its descendants
 * follow indented, in tree order. */
async copySelectionOutline(): Promise<void> {
  const ids = this.state.selection;
  const pruned = ids.filter((id) => !ids.some((other) => other !== id && this.isDescendant(other, id)));
  if (pruned.length === 0) return;
  const text = this.outlineText(pruned);
  if (!text) return;
  try {
    await navigator.clipboard.writeText(text);
  } catch {
    /* clipboard may be blocked */
  }
  this.toast(`Copied outline (${text.split("\n").length} lines)`);
}

async cutSelection(): Promise<void> {
  await this.copySelection();
  this.deleteSelection();
}

async paste(anchorId?: string | null): Promise<void> {
  // Prefer the full-fidelity `text/rnode` payload that copySelection writes;
  // fall back to whatever plain text is on the clipboard (external sources,
  // or older engines that can't read custom MIME types).
  let text: string | null = null;
  let items: ClipboardItems | null = null;
  try {
    items = await navigator.clipboard.read();
  } catch {
    /* clipboard read with MIME types blocked – fall through to readText */
  }
  if (items) {
    // An image on the clipboard beats text (T13-2): with a node selected, a
    // copied/screenshoted image attaches instead of inserting text.
    const imageItem = items.find((i) => i.types.some((t) => t.startsWith("image/")));
    if (imageItem) {
      const type = imageItem.types.find((t) => t.startsWith("image/"));
      const blob = await imageItem.getType(type);
      const node = this.selectionNode ?? (anchorId ? this.model.node(anchorId) : null);
      if (node) {
        await this.attachImageFile(node.id, new File([blob], "pasted-image", { type: blob.type || type }));
      } else {
        trace.ignored("paste:image", "no node selected");
      }
    }
    return;
  }
  for (const item of items) {
    if (item.types.includes("text/rnode")) {
      text = await item.getType("text/rnode").then((blob) => blob.text());
      break;
    }
  }
}
if (text === null) text = await navigator.clipboard.readText().catch(() => null);
if (!text) return;
let parsed: { app?: string; payload?: { rootId: string; nodes: MindNode[]; relationships: { fromId: string; toId: string; label?: string }[] } } |
null = null;
try {

```

```

    parsed = JSON.parse(text);
  } catch {
    parsed = null;
  }
  if (!parsed || parsed.app !== "r-node" || !parsed.payload) {
    // Plain text on the clipboard: XMind-style, paste it straight into the
    // selected topic instead of rejecting it as "not a map".
    const target = this.selectionNode;
    if (target) {
      this.typeToEdit(text);
      this.toast("Pasted into topic");
    } else {
      this.toast("Clipboard does not contain a map");
    }
    return;
  }
  const payload = parsed.payload;
  const anchor = anchorId ?? this.state.selection[this.state.selection.length - 1] ?? this.sheet.rootNodeId;
  const anchorNode = this.model.node(anchor);
  const srcRoot = payload.nodes.find((n) => n.id === payload.rootId);
  if (!anchorNode || !srcRoot) return;
  const type: NodeType = anchorNode.type === "central" ? "main" : "subtopic";
  const ops = this.remapOps(payload.nodes, anchor, anchorNode.childrenIds.length, type);
  if (ops.length === 0) return;
  for (const rel of payload.relationships ?? []) {
    const from = this.lastRemap.get(rel.fromId);
    const to = this.lastRemap.get(rel.toId);
    if (from && to) {
      ops.push(
        makeOp<Op & { type: "createRelationship" }>("createRelationship", {
          relationship: { id: uid("rel"), fromId: from, toId: to, label: rel.label },
        })
      );
    }
  }
  this.execOps(ops);
  this.select(this.lastRemap.get(srcRoot.id)!);
  this.toast("Pasted");
}

private lastRemap = new Map<string, string>();

/** Build createNode ops that clone a subtree under a new parent, remapping ids. */
private remapOps(source: MindNode[], parentId: string, index: number, rootType: NodeType): Op[] {
  const idMap = new Map<string, string>();
  this.lastRemap = idMap;
  const ops: Op[] = [];
  const srcRoot = source.find((n) => !source.some((o) => o.childrenIds.includes(n.id)));
  if (!srcRoot) return [];
  const newRoot = uid("n");
  idMap.set(srcRoot.id, newRoot);
  ops.push(
    makeOp<Op & { type: "createNode" }>("createNode", {
      id: newRoot,
      nodeType: rootType,
      parentId,
      index,
      title: srcRoot.title,
      titleRuns: srcRoot.titleRuns,
      style: srcRoot.style,
      task: srcRoot.task,
    })
  );
  const queue = [srcRoot];
  while (queue.length > 0) {
    const src = queue.shift()!;
    for (const cid of src.childrenIds) {
      const c = source.find((n) => n.id === cid);
      if (!c) continue;
      const newId = uid("n");
      idMap.set(cid, newId);
      ops.push(
        makeOp<Op & { type: "createNode" }>("createNode", {
          id: newId,
          nodeType: "subtopic",
          parentId: idMap.get(src.id)!,
          index: src.childrenIds.indexOf(cid),
          title: c.title,
          titleRuns: c.titleRuns,
          style: c.style,
          task: c.task,
        })
      );
      queue.push(c);
    }
  }
  return ops;
}

```

```

}

beginRelationship(fromId: string): void {
    this.state.relFrom = fromId;
    this.state.message = "Click another topic to link it – Escape to cancel";
    this.notify();
}

clearRelFrom(): void {
    this.state.relFrom = null;
    this.state.message = null;
    this.notify();
}

createRelationship(fromId: string, toId: string): void {
    if (fromId === toId) return;
    const exists = this.sheet.relationships.some((r) => (r.fromId === fromId && r.toId === toId) || (r.fromId === toId && r.toId === fromId));
    if (exists) {
        this.toast("Relationship already exists");
        return;
    }
    this.execOps([
        makeOp<Op & { type: "createRelationship" }>("createRelationship", {
            relationship: { id: uid("rel"), fromId, toId },
        }),
    ]);
}

// -----
// Overlay selection: relationships, groups, summaries
// -----

selectRelationship(id: string | null): void {
    this.state.selection = [];
    this.state.groupSel = null;
    this.state.summarySel = null;
    this.state.imageSel = null;
    this.state.relSel = id;
    this.notify();
}

selectGroup(id: string | null): void {
    this.state.selection = [];
    this.state.relSel = null;
    this.state.summarySel = null;
    this.state.imageSel = null;
    this.state.groupSel = id;
    this.notify();
}

selectSummary(id: string | null): void {
    this.state.selection = [];
    this.state.relSel = null;
    this.state.groupSel = null;
    this.state.imageSel = null;
    this.state.summarySel = id;
    this.notify();
}

setRelationship(id: string, patch: Partial<Relationship>): void {
    const rel = this.sheet.relationships.find((r) => r.id === id);
    if (!rel) return;
    this.execOps([makeOp<Op & { type: "setRelationship" }>("setRelationship", { id, relationship: { ...rel, ...patch }, prev: { ...rel } })]);
}

deleteSelectedRelationship(): void {
    const id = this.state.relSel;
    if (!id) return;
    const rel = this.sheet.relationships.find((r) => r.id === id);
    if (!rel) return;
    this.execOps([makeOp<Op & { type: "deleteRelationship" }>("deleteRelationship", { id, relationship: rel })]);
    this.state.relSel = null;
    this.notify();
}

// -----
// Groups & summaries
// -----

/** The selected nodes, iff they are at least two SIBLINGS (what a
 * group/brace can span). Returns null otherwise. */
private selectedSiblings(): MindNode[] | null {
    const ids = this.state.selection;
    if (ids.length < 2) return null;
    const nodes = ids.map((id) => this.model.node(id)).filter((n): n is MindNode => !!n);
    if (nodes.length < 2) return null;
    const parent = nodes[0].parentId;

```

```

    if (!parent || nodes.some((n) => n.parentId !== parent)) return null;
    return nodes;
}

createGroupFromSelection(): void {
    // A group can enclose ANY selected topics – siblings, cousins, nodes from
    // different branches (the box is just the union of their bounds). Only
    // the summary brace needs a sibling column.
    const ids = this.state.selection;
    if (ids.length < 2) {
        this.toast("Select at least two topics to group");
        return;
    }
    const nodes = ids.map((id) => this.model.node(id)).filter((n): n is MindNode => !!n);
    if (nodes.length < 2) {
        this.toast("Select at least two topics to group");
        return;
    }
    this.execOps([makeOp<Op & { type: "createGroup" }>("createGroup", { group: { id: uid("grp"), memberIds: nodes.map((n) => n.id), label: "group" }
    ]));
    this.toast("Group created");
}

createSummaryFromSelection(): void {
    const nodes = this.selectedSiblings();
    if (!nodes) {
        this.toast("Select at least two sibling topics to summarize");
        return;
    }
    this.execOps([makeOp<Op & { type: "createSummary" }>("createSummary", { summary: { id: uid("sum"), memberIds: nodes.map((n) => n.id), label:
    "Summary" } } ]));
    this.toast("Summary created");
}

setGroup(id: string, patch: Partial<Group>): void {
    const g = this.sheet.boundaries.find((x) => x.id === id);
    if (!g) return;
    this.execOps([makeOp<Op & { type: "setGroup" }>("setGroup", { id, group: { ...g, ...patch }, prev: { ...g, memberIds: [...g.memberIds] } } ]));
}

setSummary(id: string, patch: Partial<Summary>): void {
    const s = this.sheet.summaries.find((x) => x.id === id);
    if (!s) return;
    this.execOps([makeOp<Op & { type: "setSummary" }>("setSummary", { id, summary: { ...s, ...patch }, prev: { ...s, memberIds: [...s.memberIds] } } ]));
}

deleteGroup(id: string): void {
    const g = this.sheet.boundaries.find((x) => x.id === id);
    if (!g) return;
    this.execOps([makeOp<Op & { type: "deleteGroup" }>("deleteGroup", { id, group: g } ]));
    this.state.groupSel = null;
    this.notify();
}

deleteSummary(id: string): void {
    const s = this.sheet.summaries.find((x) => x.id === id);
    if (!s) return;
    this.execOps([makeOp<Op & { type: "deleteSummary" }>("deleteSummary", { id, summary: s } ]));
    this.state.summarySel = null;
    this.notify();
}

// -----
// Structure / layout controls
// -----

setStructure(patch: Partial<Sheet["structure"]>): void {
    const cur = this.sheet.structure;
    this.execOps([makeOp<Op & { type: "setStructure" }>("setStructure", { config: { ...cur, ...patch }, prev: cur } ]));
    // Changing the structure is an explicit global reflow. Clear manual
    // overrides so the new structure does not immediately fight old ones.
    this.scheduleLayout(true, true);
}

autoLayoutAll(): void {
    // Run immediately and clear manual flags so auto-layout takes over
    applyLayout(this.sheet, true, this.measurer, true);
    this.notify();
    this.toast("Layout recalculated");
}

// -----
// Camera
// -----

zoomAt(sx: number, sy: number, factor: number, vw: number, vh: number): void {
    this.state.camera = zoomAt(this.state.camera, vw, vh, sx, sy, factor);
}

```

```

    this.notify();
}

panBy(dx: number, dy: number): void {
    this.state.camera = panBy(this.state.camera, dx, dy);
    this.notify();
}

fitView(vw: number, vh: number): void {
    this.state.camera = fitBounds(this.state.camera, vw, vh, this.mapBounds());
    this.state.camera = { ...this.state.camera, scale: Math.max(this.state.camera.scale, 0.4) };
    this.notify();
}

centerOnNode(id: string): void {
    const node = this.model.node(id);
    if (!node) return;
    const m = measureNode(node, this.measurer);
    this.state.camera = centerOn(this.state.camera, node.position.x + m.w / 2, node.position.y + m.h / 2);
    this.notify();
}

zoomStep(factor: number, vw: number, vh: number): void {
    this.zoomAt(vw / 2, vh / 2, factor, vw, vh);
}

private mapBounds(): { minX: number; minY: number; maxX: number; maxY: number } {
    const res = layoutSheet(this.sheet, false, this.measurer);
    let minX = Infinity, minY = Infinity, maxX = -Infinity, maxY = -Infinity;
    for (const [id, p] of res.positions) {
        const n = this.model.node(id);
        if (!n) continue;
        const m = measureNode(n, this.measurer);
        if (p.x < minX) minX = p.x;
        if (p.y < minY) minY = p.y;
        if (p.x + m.w > maxX) maxX = p.x + m.w;
        if (p.y + m.h > maxY) maxY = p.y + m.h;
    }
    if (!isFinite(minX)) return { minX: -200, minY: -200, maxX: 200, maxY: 200 };
    return { minX, minY, maxX, maxY };
}

// -----
// Keyboard navigation
// -----

navigate(dir: NavDir): void {
    const node = this.selectionNode ?? this.model.rootNode;
    const sheet = this.sheet;
    const rootId = sheet.rootNodeId;

    // build a center X map from layout positions + manual positions
    const layout = layoutSheet(this.sheet, false, this.measurer);
    const centerX = (id: string): number => {
        const n = this.model.node(id);
        if (!n) return 0;
        const measured = measureNode(n, this.measurer);
        const p = layout.positions.get(id);
        if (p) return p.x + measured.w / 2;
        return n.position.x + measured.w / 2;
    };

    if (dir === "left" || dir === "right") {
        if (node.type === "central") {
            const kids = node.childrenIds.map((id) => this.model.node(id)).filter((k): k is any => !!k);
            const rootCx = centerX(rootId);
            const rightKids = kids.filter((k) => centerX(k.id) >= rootCx);
            const leftKids = kids.filter((k) => centerX(k.id) < rootCx);
            if (dir === "right" && rightKids.length) this.select(rightKids[0].id, { center: true });
            if (dir === "left" && leftKids.length) this.select(leftKids[0].id, { center: true });
            return;
        }

        const branchRootId = this.model.branchRootId(node.id);
        const rootCx = centerX(rootId);
        const branchCx = centerX(branchRootId);
        const isLeftWing = branchCx < rootCx;

        if (dir === "right") {
            if (isLeftWing) {
                if (node.parentId) this.select(node.parentId, { center: true });
            } else {
                if (node.childrenIds.length && !node.collapsed) this.select(node.childrenIds[0], { center: true });
            }
        }
        return;
    }
}

```

```

    if (dir === "left") {
      if (isLeftWing) {
        if (node.childrenIds.length && !node.collapsed) this.select(node.childrenIds[0], { center: true });
      } else {
        if (node.parentId) this.select(node.parentId, { center: true });
      }
      return;
    }
  }

  if (dir === "up") {
    if (!node.parentId) return;
    const parent = this.model.requireNode(node.parentId);
    const idx = parent.childrenIds.indexOf(node.id);
    if (idx > 0) this.select(parent.childrenIds[idx - 1], { center: true });
    else this.select(parent.id, { center: true });
    return;
  }

  // down
  if (!node.parentId) return;
  const parent = this.model.requireNode(node.parentId);
  const idx = parent.childrenIds.indexOf(node.id);
  if (idx >= 0 && idx < parent.childrenIds.length - 1) this.select(parent.childrenIds[idx + 1], { center: true });
  else if (node.parentId !== rootId) this.select(parent.id, { center: true });
}

// -----
// Search
// -----

setSearch(q: string): void {
  this.state.search = q;
  const query = q.trim().toLowerCase();
  let results: string[] = [];
  if (query) {
    results = this.model
      .visibleIds(this.sheet.rootNodeId)
      .filter((id) => {
        const n = this.model.node(id);
        return !!n && (n.title.toLowerCase().includes(query) || n.notes.toLowerCase().includes(query));
      });
  }
  this.state.searchResults = results;
  this.state.searchIndex = results.length > 0 ? 0 : 0;
  this.notify();
  if (results.length > 0) this.jumpToSearch(0);
}

jumpToSearch(offset: number): void {
  const results = this.state.searchResults;
  if (results.length === 0) return;
  const next = (this.state.searchIndex + offset + results.length) % results.length;
  this.state.searchIndex = next;
  this.select(results[next], { center: true });
  this.notify();
}

// -----
// Theme & UI toggles
// -----

togglePalette(): void {
  this.state.showPalette = !this.state.showPalette;
  this.notify();
}

toggleOutliner(): void {
  this.state.showOutliner = !this.state.showOutliner;
  this.notify();
}

toggleInspector(): void {
  this.state.showInspector = !this.state.showInspector;
  this.notify();
}

toggleZen(): void {
  this.state.zen = !this.state.zen;
  this.notify();
}

setMode(mode: "select" | "pan"): void {
  this.state.mode = mode;
  this.notify();
}

```

```

/**
 * Dev-only (perf spike): bulk-generate a balanced tree of `count` topics
 * under the root as a single op batch. Exposed via window.__rnode.
 */
debugGenerateBalancedTree(count: number): { opsMs: number; totalNodes: number } {
  const sheet = this.sheet;
  const ops: Op[] = [];
  const queue: string[] = [sheet.rootNodeId];
  const BRANCH = 8;
  let created = 0;
  const t0 = performance.now();
  while (created < count && queue.length > 0) {
    const parentId = queue.shift()!;
    const isRoot = parentId === sheet.rootNodeId;
    for (let i = 0; i < BRANCH && created < count; i++) {
      const id = uid("n");
      ops.push(
        makeOp<Op & { type: "createNode" }>("createNode", {
          id,
          nodeType: isRoot ? "main" : "subtopic",
          parentId,
          index: 0,
          title: `Topic ${created}`,
        })
      );
      queue.push(id);
      created++;
    }
  }
  const opsMs = performance.now() - t0;
  this.execOps(ops);
  return { opsMs, totalNodes: Object.keys(sheet.nodes).length };
}

// -----
// Document management
// -----

newDocument(): void {
  const doc = DocumentModel.blank("Untitled map");
  doc.settings.theme = "light";
  this.state.docs = [...this.state.docs, doc];
  this.state.activeDocId = doc.documentId;
  this.switchToDoc(doc.documentId);
  this.state.sync = "dirty"; // new doc not persisted until the user saves
  this.notify();
  this.toast("New document created");
}

/** Create a doc from the roadmap template; returns its id. */
duplicateSample(): string {
  const doc = DocumentModel.sample();
  doc.settings.theme = "light";
  this.state.docs = [...this.state.docs, doc];
  this.state.sync = "dirty";
  this.notify();
  return doc.documentId;
}

duplicateDocument(id: string): void {
  const src = this.state.docs.find((d) => d.documentId === id);
  if (!src) return;
  const copy: RnodeDocument = structuredClone(src);
  copy.documentId = uid("d");
  copy.title = src.title + " (copy)";
  copy.createdAt = nowIso();
  copy.updatedAt = nowIso();
  this.state.docs = [...this.state.docs, copy];
  this.state.sync = "dirty";
  this.notify();
  this.toast("Document duplicated");
}

renameDocument(id: string, title: string): void {
  const doc = this.state.docs.find((d) => d.documentId === id);
  if (!doc) return;
  doc.title = title;
  doc.updatedAt = nowIso();
  // The active document's model is what gets saved (and what the desktop
  // rename-on-save detection reads): keep the two titles in sync so the
  // GUI name becomes the real file's name.
  if (this.state.activeDocId === id) this.model.doc.title = title;
  this.state.sync = "dirty";
  this.notify();
}

toggleArchive(id: string): void {

```

```

const doc = this.state.docs.find((d) => d.documentId === id);
if (!doc) return;
doc.archived = !doc.archived;
this.state.sync = "dirty";
this.notify();
}

deleteDocument(id: string): void {
  this.state.docs = this.state.docs.filter((d) => d.documentId !== id);
  if (this.state.activeDocId === id) {
    const next = this.state.docs[0];
    if (next) this.switchToDoc(next.documentId);
  } else {
    this.model = new DocumentModel(DocumentModel.blank());
    this.state = this.makeState();
  }
}
this.state.sync = "dirty";
this.notify();
}

switchToDoc(id: string): void {
  this.commitDraftOnLeave();
  const doc = this.state.docs.find((d) => d.documentId === id);
  if (!doc) return;
  this.model = new DocumentModel(doc);
  this.history.clear();
  this.state.activeDocId = id;
  this.state.selection = [];
  this.state.editingId = null;
  this.state.search = "";
  this.state.searchResults = [];
  this.state.sync = "saved"; // the loaded doc is its persisted version
  this.notify();
  this.scheduleLayout(false);
}

setSheetTitle(title: string): void {
  const sheet = this.sheet;
  this.execOps([makeOp<Op & { type: "setSheetTitle" }>("setSheetTitle", { title, prev: sheet.title })]);
}

// -----
// Export
// -----

exportJson(): void {
  const json = JSON.stringify(this.model.doc, null, 2);
  const blob = new Blob([json], { type: "application/json" });
  this.download(blob, this.docFileName());
  this.toast("Document exported as JSON");
}

exportMarkdown(): void {
  const md = this.toMarkdown(this.sheet, this.sheet.rootNodeId, 0);
  const blob = new Blob([md], { type: "text/markdown" });
  this.download(blob, `${this.model.doc.title.replace(/^[^w-]+/g, "_").md}`);
  this.toast("Document exported as Markdown");
}

private toMarkdown(sheet: Sheet, id: string, depth: number): string {
  const node = sheet.nodes[id];
  if (!node) return "";
  const bullet = depth === 0 ? `# ${node.title}` : `${"-".repeat(1)} ${node.title}`;
  const prefix = depth === 0 ? "" : " ".repeat(depth - 1);
  let out = depth === 0 ? `${bullet}\n` : `${prefix}- ${node.title}\n`;
  for (const c of node.childrenIds) out += this.toMarkdown(sheet, c, depth + 1);
  return out;
}

private download(blob: Blob, filename: string): void {
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
  a.click();
  setTimeout(() => URL.revokeObjectURL(url), 1000);
}
}

```